# Vel Tech Group of Institutions

Name: NIKHIL  KUMAR
Email: vtu28682@veltech.edu.in
Roll no: vtu28682
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

## 2028_NeoColab_Design and Analysis of AlgorithmsS1L10

### VTU_DAA_Week 1_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 14

### Section 1 : MCQ

1. An algorithm must have a clear stopping point to ensure

***Answer***

Finiteness

***Status :*** Correct ***Marks : 1/1***

2. Which of the following is true about algorithmic determinism?

***Answer***

Algorithms must produce the same output for the same input

***Status :*** Correct ***Marks : 1/1***

vtu28682

3. An algorithm's effectiveness refers to

***Answer***

Each step contributes to solving the problem

***Status :*** Correct ***Marks : 1/1***

4. Which notation is used to describe both the upper and lower bounds of an algorithm's efficiency?

***Answer***

Θ (Big theta)

***Status :*** Correct ***Marks : 1/1***

5. What is the purpose of using asymptotic notations in the analysis of algorithm efficiency?

***Answer***

To take meaningful statements about an algorithm's efficiency based on its growth rate

***Status :*** Correct ***Marks : 1/1***

6. What does the Big Theta notation (Θ) indicate about an algorithm's efficiency?

***Answer***

It describes the tightest possible bound on the algorithm's running time.

***Status :*** Correct ***Marks : 1/1***

7. Which of the following statements is true regarding the relationship between Big Oh (O), Big Omega (Ω), and Big Theta (Θ) notations?

***Answer***

vtu28682

If f(n) = O(g(n)) and f(n) = Ω(g(n)), then f(n) = Θ(g(n))

***Status :*** Correct ***Marks : 1/1***

8. What does "Big O" notation represent in relation to algorithms?

***Answer***

A method to analyze and express the efficiency of an algorithm

***Status :*** Correct ***Marks : 1/1***

9. An algorithm is required to produce a definite and unambiguous outcome for every valid input. This characteristic is known as _____.

***Answer***

determinism

***Status :*** Correct ***Marks : 1/1***

10. Which approach is used to prove the correctness of algorithms by analyzing a sequence of steps?

***Answer***

Mathematical induction

***Status :*** Correct ***Marks : 1/1***

11. In the context of algorithm design, what does the term "in-place" refer to?

***Answer***

The algorithm modifies the input data directly without using additional memory

***Status :*** Correct ***Marks : 1/1***

12. What is the primary purpose of analyzing an algorithm's efficiency?

***Answer***

To optimize the algorithm's time and space efficiency

***Status :*** Correct ***Marks : 1/1***

13. In the context of algorithmic problem-solving, what does the "generality of an algorithm" refer to?

***Answer***

The ability of an algorithm to handle a wide range of problem types

***Status :*** Correct ***Marks : 1/1***

14. Which parameter is commonly used to measure an algorithm's input size?

***Answer***

Number of elements in the input array/list

***Status :*** Wrong ***Marks : 0/1***

15. The concept of "Orders of Growth" in algorithm analysis refers to:

***Answer***

The function that describes the algorithm's efficiency

***Status :*** Correct ***Marks : 1/1***

vtu28682

# Vel Tech Group of Institutions

Name: NIKHIL  KUMAR
Email: vtu28682@veltech.edu.in
Roll no: vtu28682
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

## 2028_NeoColab_Design and Analysis of AlgorithmsS1L10

### VTU_DAA_Week 2_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 15

### Section 1 : MCQ

1. What is a non-recursive algorithm?

***Answer***

An algorithm that does not use recursion

***Status :*** Correct ***Marks : 1/1***

2. Which of the following is an example of a non-recursive sorting algorithm?

***Answer***

Bubble Sort

***Status :*** Correct ***Marks : 1/1***

vtu28682

3. The time complexity of non-recursive algorithms is often expressed using:

***Answer***

All of the mentioned options

***Status :*** Correct ***Marks : 1/1***

4. Which non-recursive algorithm is commonly used for searching in a sorted array by repeatedly dividing the search interval in half?

***Answer***

Binary Search

***Status :*** Correct ***Marks : 1/1***

5. In non-recursive algorithms, iteration is commonly achieved through:

***Answer***

Loops

***Status :*** Correct ***Marks : 1/1***

6. What is the output of the code?

```
#include <stdio.h>

int main() {
    int num = 2;
    do {
        printf("%d ", num);
        num *= 2;
    } while (num <= 8);
    return 0;
}
```

***Answer***

2 4 8

***Status :*** Correct ***Marks : 1/1***

7. What is the output for the code?

```
#include <stdio.h>

void nonRecursiveAlgorithm(int n) {
    for (int i = 1; i <= n; i *= 2) {
        printf("%d ", i);
    }
}

int main() {
    nonRecursiveAlgorithm(8);
    return 0;
}
```

***Answer***

1 2 4 8

***Status :*** Correct ***Marks : 1/1***

8. What is the output for the code?

```
#include <stdio.h>

void nonRecursiveFunction1(int n) {
    while (n > 0) {
        printf("%d ", n);
        n /= 2;
    }
}

int main() {
    int number = 80;
    nonRecursiveFunction1(number);
    return 0;
```

```
}
```

***Answer***

80 40 20 10 5 2 1

***Status :*** Correct ***Marks : 1/1***

9. What will be the output for the following recursive code?

```
#include <stdio.h>

int power(int base, int exponent) {
    if (exponent == 0)
        return 1;
    else
        return base * power(base, exponent - 1);
}

int main() {
    int result = power(3, 2);
    printf("%d", result);
    return 0;
}
```

***Answer***

9

***Status :*** Correct ***Marks : 1/1***

10. What will be the output for the following recursive code?

```
#include <stdio.h>

int num(int n, int rev) {
    if (n == 0) {
        return rev;
    } else {
        int digit = n % 10;
        rev = rev * 10 + digit;
```

```
    return num(n / 10, rev);
  }
}

int main() {
  int d = 12345;
  int rev = num(d, 0);
  printf("%d", rev);
  return 0;
}
```

***Answer***

54321

***Status :*** Correct ***Marks : 1/1***

11. What will be the output for the following recursive code?

```
#include <stdio.h>

int function(int number) {
  if (number <= 0) {
    return 1;
  } else {
    return number * function(number - 1);
  }
}

int main() {
  int result;
  result = function(4);
  printf("%d", result);

  return 0;
}
```

***Answer***

24

***Status :*** Correct ***Marks : 1/1***

12. What will be the output for the following recursive code?

```
#include <iostream>
using namespace std;

void print(int n) {
    if (n == 0)
        return;
    print(n / 2);
    cout << n % 2;
}

int main() {
    int num = 12;
    print(num);
    return 0;
}
```

***Answer***

1100

***Status :*** Correct ***Marks : 1/1***

13. What will be the output for the following recursive code?

```
#include <stdio.h>

int function(int a, int b) {
    if (b == 1)
        return a;
    else
        return a + function(a, b - 1);
}

int main() {
    int result = function(2, 3);
    printf("%d", result);
    return 0;
}
```

*Answer*

6

*Status :* Correct *Marks : 1/1*

14. What will be the output for the following recursive code?

```
#include <stdio.h>

int function(int a, int b) {
    if (b == 0)
        return 0;
    if (b == 1)
        return a;
    return a + function(a, b - 1);
}

int main() {
    int result = function(3, 8);
    printf("%d", result);
    return 0;
}
```

*Answer*

24

*Status :* Correct *Marks : 1/1*

15. What will be the output for the following recursive code?

```
#include <iostream>
using namespace std;

int recursive(int n) {
    if (n <= 0)
        return 1;
    else
        return n + recursive(n - 1);
```

```
}

int main() {
    int result = recursive(5);
    cout << result;
    return 0;
}
```

***Answer***

16

***Status :*** Correct ***Marks : 1/1***

vtu28682

# Vel Tech Group of Institutions

Name: NIKHIL  KUMAR
Email: vtu28682@veltech.edu.in
Roll no: vtu28682
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

## 2028_NeoColab_Design and Analysis of AlgorithmsS1L10

### VTU_DAA_Week 2_COD

Attempt : 1
Total Mark : 40
Marks Obtained : 13.5

### Section 1 : Coding

1. Problem Statement

Given an integer n. The task is to find the first n Fibonacci Numbers.

***Input Format***

The input consists of a single integer n, representing how many Fibonacci terms to print.

***Output Format***

The output prints n space-separated Fibonacci numbers starting from 0.

Refer to the sample output for the formatting specifications.

vtu28682

***Sample Test Case***

Input: 3

Output: 0 1 1

***Answer***

```
#include <stdio.h>
int main(){
   int n,i;
   int a=0,b=1,next;

   scanf("%d" , &n);

   for(i=1;i<=n;i++){
      if(i == 1){
         printf("%d" , a);

      }
      else if(i == 2){
         printf("%d" , b);
      }
      else{
         next=a+b;
         printf("%d" , next);
         a=b;
         b=next;
      }
   }
   return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

2. Problem Statement

Given a string s, check if it is a palindrome using recursion. A palindrome is a word, phrase, or sequence that reads the same backward as forward.

***Input Format***

The input consists of a single string s containing no spaces.

***Output Format***

The output prints "The string is a palindrome" if the string is symmetric.

Otherwise, prints "The string is not a palindrome".

Refer to the sample output for the formatting specifications.

***Sample Test Case***

Input: abba

Output: The string is a palindrome

***Answer***

```
#include <stdio.h>
#include <string.h>

int palindrome(char s[], int start, int end) {
   if(start >= end)
   return 1;

   if (s[start] != s[end])
   return 0;

   return palindrome(s, start+1, end+1);
}
int main() {
   char s[50];

   scanf("%s" , s);
   int len = strlen(s);

   if(palindrome(s, 0, len-1)){
     printf("The string is a palindrome");
}else{
printf("The string is not a palindrome");
}
return 0;
}
```

***Status :*** Partially correct ***Marks : 3.5/10***

3. Problem Statement

Shilpa is learning about the Fibonacci sequence for her computer science class. She wants to generate the first N terms of the Fibonacci series.

Can you help Shilpa by writing a program that takes a number N and prints the first N terms of the Fibonacci series using recursion?

***Input Format***

The input consists of an integer N, representing the number of terms in the Fibonacci sequence that need to be printed.

***Output Format***

The output prints the first N Fibonacci numbers, separated by spaces.

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 6
Output: 0 1 1 2 3 5

***Answer***

```
import java.util.Scanner;

int fib(int n){
   if(n==0)
   return 0;
   else if(n==1)
   return 1;
   else
   return fib(n-1) = FIB(N-2);
}

public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      int N = sc.nextInt();
```

```
        for(int i = 0; i < N; i++) {
            System.out.print(fib(i));
            if(i != N - 1) System.out.print(" ");
        }
        System.out.println();
    }
}
```

***Status :*** Wrong ***Marks : 0/10***

4. Problem Statement

Ashish is preparing a mathematics assignment where he needs to calculate the factorial of a number. Since his teacher asked him to use recursion for this task, he plans to complete the method int factorial to solve it.

Help him to implement the task.

***Input Format***

The input consists of an integer num representing the number for which the factorial is to be calculated.

***Output Format***

The output prints the factorial value of num.

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 3
Output: 6

***Answer***

```
#include <stdio.h>

// You are using GCC
```

```
int main() {
    int num;
    scanf("%d", &num);
    printf("%d\n", factorial(num));
    return 0;
}
```

***Status :*** Wrong ***Marks : 0/10***

vtu28682

# Vel Tech Group of Institutions

Name: NIKHIL  KUMAR
Email: vtu28682@veltech.edu.in
Roll no: vtu28682
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

## 2028_NeoColab_Design and Analysis of AlgorithmsS1L10

### VTU_DAA_Week 2_PAH

Attempt : 1
Total Mark : 50
Marks Obtained : 50

### Section 1 : Coding

1. Problem Statement

Manoj loves puzzles and recently encountered an interesting mathematical challenge. He needs to find how many ways he can express a number x as the sum of distinct positive integers raised to the power of n. For example, for a given number x, he wants to find all possible ways to represent it as a sum of distinct powers of integers.

Help him implement this task.

Example 1

Input:

10
2

vtu28682

Output:

1

Explanation:

x = 10

n = 2

10 = 12 + 32, hence we have only 1 possibility.

Example 2

Input:

100

2

Output:

3

Explanation:

x = 100

n = 2

100 = 102 OR 62 + 82 OR 12 + 32 + 42 + 52 + 72, hence total 3 possibilities.

Function Specifications: getAllWays(int, int, int)

***Input Format***

The first line of input consists of an integer x, representing the target sum.

The second line of input consists of an integer n, representing the power to which the integers are raised.

***Output Format***

The first line of output prints: The total number of ways to express x as the sum of distinct powers.

vtu28682

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 100
2

Output: 3

***Answer***

```
#include <stdio.h>
#include <string.h>

#define MAX_X 100010

long long dp[MAX_X];
long long powers[MAX_X];

int main() {
    int x, n;
    scanf("%d", &x);
    scanf("%d", &n);
    int num_p = 0;
    for (long long k = 1;; k++) {
        long long p = 1;
        int i;
        for (i = 0; i < n; i++) {
            p *= k;
            if (p > x) break;
        }
        if (i < n || p > x) break;
        powers[num_p++] = p;
    }
    memset(dp, 0, sizeof(dp));
    dp[0] = 1LL;
    for (int idx = 0; idx < num_p; idx++) {
        long long p = powers[idx];
        int pp = (int)p;
        for (int j = x; j >= pp; j--) {
            dp[j] += dp[j - pp];
        }
    }
    printf("%lld\n", dp[x]);
```

```
    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

2. Problem Statement

Given an integer array of coins of size N representing different types of currency and an integer sum M. The task is to find the number of ways to make the sum by using different combinations from the coins array, using recursion.

Function Specifications: int countWays(int [], int , int)

***Input Format***

The first line of input consists of the number of coins N.

The second line consists of N integers denoting the coin denominations.

The third line consists of the integer sum M to be produced with those coins.

***Output Format***

The output displays the number of ways to make the sum by using different combinations from the coins array.

Refer to the sample output for the formatting specifications.

***Sample Test Case***

Input: 3
1 2 3
4
Output: 4

***Answer***

```
// You are using GCC
#include <stdio.h>
```

```
#include <string.h>

long long memo[11][101];
int coins[11];
int Nglob;
int Mglob;

long long countWays(int idx, int rem) {
    if (rem == 0) return 1;
    if (idx == Nglob) return 0;
    if (memo[idx][rem] != -1) return memo[idx][rem];
    long long res = countWays(idx + 1, rem);
    if (rem >= coins[idx]) {
        res += countWays(idx, rem - coins[idx]);
    }
    memo[idx][rem] = res;
    return res;
}

int main() {
    int N;
    scanf("%d", &N);
    for (int i = 0; i < N; i++) {
        scanf("%d", &coins[i]);
    }
    int M;
    scanf("%d", &M);
    Nglob = N;
    Mglob = M;
    memset(memo, -1, sizeof(memo));
    long long ans = countWays(0, M);
    printf("%lld\n", ans);
    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

3. Problem Statement

Meredith is a cashier at a local convenience store. To improve the efficiency of making change for customers, Meredith wants to develop a

small program that calculates the number of different ways she can give change using a given set of coin denominations. This will help her quickly figure out the number of ways to make change for a particular amount of money using the available coins.

Meredith has a list of coin denominations and a specific amount of money for which she needs to provide change. She wants to write a program to determine the number of possible ways to achieve the given amount using the available denominations. This will help her understand all possible combinations without having to manually calculate them each time.

***Input Format***

The first line contains an integer n, the number of different coin denominations.

The second line contains n space-separated integers representing the coin denominations.

The third line contains an integer m, the target amount of money for which Meredith needs to make change.

***Output Format***

The output displays the number of ways to make the sum by using different combinations from the coins array.

Refer to the sample output for the formatting specifications.

***Sample Test Case***

Input: 3
1 2 3
4

Output: 4

***Answer***

```
#include <stdio.h>
#include <string.h>

int main() {
```

```
    int n;
    scanf("%d", &n);
    int coins[101];
    for (int i = 0; i < n; i++) {
        scanf("%d", &coins[i]);
    }
    int m;
    scanf("%d", &m);
    long long dp[101];
    memset(dp, 0, sizeof(dp));
    dp[0] = 1;
    for (int i = 0; i < n; i++) {
        int c = coins[i];
        if (c > m || c <= 0) continue;
        for (int j = c; j <= m; j++) {
            dp[j] += dp[j - c];
        }
    }
    printf("%lld\n", dp[m]);
    return 0;
}// You are using GCC
```

***Status :*** Correct ***Marks : 10/10***

4. Problem Statement

In a small research lab, a programmer named Arin is analyzing binary data to detect system flags stored as set bits in an integer. One day Arin receives an integer and must determine how many bits are set to 1 in its binary representation.

Write a program to ensure that this task is done so Arin can continue validating the system's binary logs. The objective is to count all set bits in the given number using any valid approach.

Function specification: countSetBits(int n)

***Input Format***

The input consists of a single positive integer n.

***Output Format***

The output prints a single integer representing the count of set bits (1s) in the binary representation of n.

Refer to the sample output for the formatting specifications.

***Sample Test Case***

Input: 21

Output: 3

***Answer***

```
// You are using GCC
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);
    int count = 0;
    while (n) {
        count += n & 1;
        n >>= 1;
    }
    printf("%d\n", count);
    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

5. Problem Statement

Raju is a computer graphics designer. He works around matrices with ease. His friend challenged him to rotate an M x M matrix clockwise with an angle of 90 degrees.

However, Raju was given a constraint that he must not use any additional 2D matrix for doing the above-mentioned task. The rotation must be done in place, which means Raju has to modify the given input matrix directly.

Example: 1

Input:

[[1,2,3],[4,5,6],[7,8,9]]

Output:

[[7,4,1],[8,5,2],[9,6,3]]

Example: 2

Input:

matrix = [[5,1,9,11],[2,4,8,10],[13,3,6,7],[15,14,12,16]]

Output:

[[15,13,2,5],[14,3,4,1],[12,6,8,9],[16,7,10,11]]

***Input Format***

The first line of the input is a single integer value for the size of the row and column, M.

The next M lines of the input are the matrix elements, each containing M integers representing the elements of the matrix.

***Output Format***

The output displays the rotated matrix, with each row printed on a new line.

Refer to the sample input and output for formatting specifications.

***Sample Test Case***

Input: 3
1 2 3
4 5 6
7 8 9

Output: 7 4 1
8 5 2
9 6 3

***Answer***

```
#include <stdio.h>

int main() {
    int M;
    scanf("%d", &M);
    int matrix[7][7];
    for (int i = 0; i < M; i++) {
        for (int j = 0; j < M; j++) {
            scanf("%d", &matrix[i][j]);
        }
    }
    // Transpose in place
    for (int i = 0; i < M; i++) {
        for (int j = i + 1; j < M; j++) {
            int temp = matrix[i][j];
            matrix[i][j] = matrix[j][i];
            matrix[j][i] = temp;
        }
    }
    // Reverse each row
    for (int i = 0; i < M; i++) {
        for (int j = 0; j < M / 2; j++) {
            int temp = matrix[i][j];
            matrix[i][j] = matrix[i][M - 1 - j];
            matrix[i][M - 1 - j] = temp;
        }
    }
    // Print the rotated matrix
    for (int i = 0; i < M; i++) {
        for (int j = 0; j < M; j++) {
            printf("%d", matrix[i][j]);
            if (j < M - 1) {
                printf(" ");
            } else {
                printf("\n");
            }
        }
```

```
    }
    return 0;
}// You are using GCC
```

***Status :*** Correct ***Marks : 10/10***

vtu28682

# Vel Tech Group of Institutions

Name: NIKHIL  KUMAR
Email: vtu28682@veltech.edu.in
Roll no: vtu28682
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

## 2028_NeoColab_Design and Analysis of AlgorithmsS1L10

### VTU_DAA_Week 2_CY

Attempt : 1
Total Mark : 50
Marks Obtained : 44

### Section 1 : Coding

1. Problem Statement

In a kingdom far away, two types of soup are prepared daily — Soup A and Soup B. The royal chef always starts with n milliliters of each soup. Each time a servant is served, one of the following four serving operations is chosen at random (each with equal probability of 0.25):

Serve 100 ml of Soup A and 0 ml of Soup B

Serve 75 ml of Soup A and 25 ml of Soup B

Serve 50 ml of Soup A and 50 ml of Soup B

Serve 25 ml of Soup A and 75 ml of Soup B

The serving continues until either or both soups become empty. If the required soup amount for an operation exceeds the available quantity,

serve as much as possible (up to what remains).

You are to compute the probability that Soup A becomes empty first, plus half the probability that both soups become empty at the same time.

Special Note:

Since all operations serve soup in multiples of 25 ml, you must first scale n down to units of 25 ml before recursive processing:

Example

Sample Input 1

n = 50

Sample Output 2

0.625

Explanation

If we choose the first two operations, A will become empty first.

For the third operation, A and B will become empty at the same time.

For the fourth operation, B will become empty first.

So the total probability of A becoming empty first plus half the probability that A and B become empty at the same time, is 0.25 * (1 + 1 + 0.5 + 0) = 0.625.

***Input Format***

The input consists of an integer n, representing the total volume of the soup.

***Output Format***

The output prints the probability that soup A will be empty first, plus half the probability that A and B become empty at the same time.

Refer to the sample output for formatting specifications.

vtu28682

***Sample Test Case***

Input: 50

Output: 0.625

***Answer***

```
#include <stdio.h>
#include <string.h>

double memo[5][5];
int vis[5][5];

double get_prob(int a, int b);

int main() {
    int n;
    scanf("%d", &n);
    int u = n / 25;
    memset(vis, 0, sizeof(vis));
    double ans = get_prob(u, u);
    printf("%.3f\n", ans);
    return 0;
}

double get_prob(int a, int b) {
    if (a == 0) {
        return (b == 0 ? 0.5 : 1.0);
    }
    if (b == 0) {
        return 0.0;
    }
    if (vis[a][b]) {
        return memo[a][b];
    }
    vis[a][b] = 1;
    double contrib = 0.0;
    int das[4] = {4,3,2,1};
    int dbs[4] = {0,1,2,3};
    for (int i = 0; i < 4; i++) {
        int sa = (das[i] <= a ? das[i] : a);
        int sb = (dbs[i] <= b ? dbs[i] : b);
        int na = a - sa;
```

```
        int nb = b - sb;
        double p;
        if (na > 0 && nb > 0) {
            p = get_prob(na, nb);
        } else if (na == 0 && nb > 0) {
            p = 1.0;
        } else if (na > 0 && nb == 0) {
            p = 0.0;
        } else {
            p = 0.5;
        }
        contrib += 0.25 * p;
    }
    memo[a][b] = contrib;
    return contrib;
}// You are using GCC
```

***Status :*** Partially correct **Marks : 4/10**

2. Problem Statement

In a unique team-building exercise, a company has decided to distribute a certain number of resources among its departments based on mathematical powers. The company has x resources that need to be distributed such that each department receives a different power of an integer. Specifically, each department i receives n resources, where n is a given power.

George, the resource manager, needs to find all possible ways to distribute exactly x resources using the powers of integers starting from 1. Each integer's power can only be used once.

Write a program to help George determine the number of ways to distribute x resources in this manner using a recursive manner.

Example 1

Input:

10

2

Output:

1

Explanation:

x = 10

n = 2

10 = 12 + 32, hence we have only 1 possibility.

Example 2

Input:

100

2

Output:

3

Explanation:

x = 100

n = 2

100 = 102 OR 62 + 82 OR 12 + 32 + 42 + 52 + 72, hence total 3 possibilities.

***Input Format***

The first line of input consists of the integer x, representing the total number of resources.

The second line consists of the integer n, representing the power to which the integers are raised.

***Output Format***

The output should be a single integer representing the number of ways to distribute exactly x resources.

vtu28682

Refer to the sample output for the formatting specifications.

***Sample Test Case***

Input: 100
2

Output: 3

***Answer***

```
// You are using Java
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int x = sc.nextInt();
        int n = sc.nextInt();
        List<Integer> powers = new ArrayList<>();
        for (long k = 1; ; k++) {
            long p = 1;
            boolean overflow = false;
            for (int i = 0; i < n; i++) {
                if (p > Long.MAX_VALUE / k) {
                    overflow = true;
                    break;
                }
                p *= k;
                if (p > x) {
                    overflow = true;
                    break;
                }
            }
            if (overflow || p > x || p <= 0) {
                break;
            }
            powers.add((int) p);
        }
        long[] dp = new long[x + 1];
        dp[0] = 1;
        for (int p : powers) {
            for (int j = x; j >= p; j--) {
```

```
            dp[j] += dp[j - p];
        }
    }
    System.out.println(dp[x]);
  }
}
```

***Status :*** Correct ***Marks : 10/10***

3. Problem Statement

In a small research lab, a programmer named Arin is analyzing binary data to detect system flags stored as set bits in an integer. One day Arin receives an integer and must determine how many bits are set to 1 in its binary representation.

Write a program to ensure that this task is done so Arin can continue validating the system's binary logs. The objective is to count all set bits in the given number using any valid approach.

Function specification: countSetBits(int n)

***Input Format***

The input consists of a single positive integer n.

***Output Format***

The output prints a single integer representing the count of set bits (1s) in the binary representation of n.

Refer to the sample output for the formatting specifications.

***Sample Test Case***

Input: 21
Output: 3

***Answer***

```
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);
    int count = 0;
    while (n) {
        count += n & 1;
        n >>= 1;
    }
    printf("%d\n", count);
    return 0;
}// You are using GCC
```

***Status :*** Correct ***Marks : 10/10***

4. Problem Statement

Given a square matrix of size n×n, rotate the elements along each layer (or boundary) of the matrix by one position in a clockwise direction. The rotation should be performed layer by layer, starting from the outermost layer and proceeding inward.

Examples:

Input 2:

3 3

1 2 3

4 5 6

7 8 9

Output 2:

4 1 2

7 5 3

8 9 6

Explanation:

For a clockwise boundary rotation by one step:

We start with the outermost layer (there's only one layer in a 3×3 matrix)

We save the top-left element (1)

Move left edge upward: 4  1, 7  4

Move bottom edge leftward: 8  7, 9  8

Move right edge downward: 6  9, 3  6

Move top edge rightward: 2  3

Place the saved element (1) in the second position of the top row, so 1  2

***Input Format***

The first line contains two integers R and C — the number of rows and columns in the matrix.

The next R lines each contain C integers, representing the elements of the matrix.

***Output Format***

The output displays the rotated matrix after performing the clockwise boundary rotation.

Refer to the sample input and output for formatting specifications.

***Sample Test Case***

Input: 4 4
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 16

Output: 5 1 2 3
9 10 6 4
13 11 7 8
14 15 16 12

vtu28682

***Answer***

```
#include <stdio.h>

int main() {
    int r, c;
    scanf("%d %d", &r, &c);
    int n = r;
    int mat[6][6];
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &mat[i][j]);
        }
    }
    for (int layer = 0; layer < n / 2; layer++) {
        int first = layer;
        int last = n - 1 - layer;
        int ring[20];
        int idx = 0;
        // top row
        for (int j = first; j <= last; j++) {
            ring[idx++] = mat[first][j];
        }
        // right column excluding top
        for (int i = first + 1; i <= last; i++) {
            ring[idx++] = mat[i][last];
        }
        // bottom row excluding right
        for (int j = last - 1; j >= first; j--) {
            ring[idx++] = mat[last][j];
        }
        // left column excluding bottom and top
        for (int i = last - 1; i >= first + 1; i--) {
            ring[idx++] = mat[i][first];
        }
        int len = idx;
        int newring[20];
        for (int i = 0; i < len; i++) {
            newring[i] = ring[(i - 1 + len) % len];
        }
        // put back
        idx = 0;
        // top
```

```
        for (int j = first; j <= last; j++) {
            mat[first][j] = newring[idx++];
        }
        // right
        for (int i = first + 1; i <= last; i++) {
            mat[i][last] = newring[idx++];
        }
        // bottom
        for (int j = last - 1; j >= first; j--) {
            mat[last][j] = newring[idx++];
        }
        // left
        for (int i = last - 1; i >= first + 1; i--) {
            mat[i][first] = newring[idx++];
        }
    }
    // print
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            printf("%d", mat[i][j]);
            if (j < n - 1) {
                printf(" ");
            } else {
                printf("\n");
            }
        }
    }
    return 0;
}// You are using GCC
```

***Status :*** Correct **Marks : 10/10**

5. Problem Statement

Given a 2D matrix of size ROW × COL, print all elements of the matrix in diagonal order.

Diagonal Order Definition:

Start from the first element at position (0,0).Traverse all diagonals starting from the leftmost column going downward, then continue with diagonals

starting from the bottom row going rightward.Within each diagonal, print elements from the top-right corner to bottom-left corner.

Example:

Input matrix

5 4

1 2 3 4

5 6 7 8

9 10 11 12

13 14 15 16

17 18 19 20

Diagonal printing of the above matrix is

1

5 2

9 6 3

13 10 7 4

17 14 11 8

18 15 12

19 16

20

***Input Format***

The first line of input contains two integers, ROW and COL.

The next ROW lines each contain COL space-separated integers

***Output Format***

The output consists of multiple lines, one for each diagonal. Each line contains the matrix elements of that diagonal in the order from top-right to bottom-left, separated by spaces.

vtu28682

Refer to the sample input and output for formatting specifications.

***Sample Test Case***

Input: 5 4
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 16
17 18 19 20

Output: 1
5 2
9 6 3
13 10 7 4
17 14 11 8
18 15 12
19 16
20

***Answer***

```
#include <stdio.h>

int main() {
    int r, c;
    scanf("%d %d", &r, &c);
    int mat[11][11];
    for (int i = 0; i < r; i++) {
        for (int j = 0; j < c; j++) {
            scanf("%d", &mat[i][j]);
        }
    }
    for (int k = 0; k < r + c - 1; k++) {
        int ii = (k < r ? k : r - 1);
        int lowi = (k - c + 1 > 0 ? k - c + 1 : 0);
        int first = 1;
        for (int i = ii; i >= lowi; i--) {
            int j = k - i;
            if (!first) {
                printf(" ");
            }
```

```
            printf("%d", mat[i][j]);
            first = 0;
        }
        printf("\n");
    }
    return 0;
}// You are using GCC
```

***Status :*** Correct **Marks : 10/10**

vtu28682

# Vel Tech Group of Institutions

Name: NIKHIL  KUMAR
Email: vtu28682@veltech.edu.in
Roll no: vtu28682
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

## 2028_NeoColab_Design and Analysis of AlgorithmsS1L10

### VTU_DAA_Week 3_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 14

### Section 1 : MCQ

1. What is the primary characteristic of the Brute Force approach to solving the Closest-Pair problem?

***Answer***

Computing the distance between every pair of points and selecting the minimum

***Status :*** Correct ***Marks : 1/1***

2. In the brute force convex hull algorithm (Jarvis March/Gift Wrapping), what is the next point selected after identifying a point on the hull?

***Answer***

The point that creates the smallest counterclockwise turn relative to the current point

vtu28682

***Status :*** Correct ***Marks : 1/1***

3. In C, if you have a structure `Point { int x; int y; };`, what is the correct formula to calculate the squared distance between two points `p1` and `p2` to avoid unnecessary floating-point operations?

***Answer***

`(p1.x - p2.x)*(p1.x - p2.x) + (p1.y - p2.y)*(p1.y - p2.y)`

***Status :*** Correct ***Marks : 1/1***

4. Given an array of `n` points in a 2D plane, what is the exact time complexity of the Brute Force Closest-Pair algorithm?

***Answer***

O(n$^{2}$)

***Status :*** Correct ***Marks : 1/1***

5. Which geometric test is essential for determining if a point lies to the left or right of a directed line in the Convex Hull brute force algorithm?

***Answer***

The Orientation test (Cross Product)

***Status :*** Correct ***Marks : 1/1***

6. If two points are exactly the same in the input set for the Closest Pair problem, what will the brute force algorithm correctly return?

***Answer***

A distance of 0

***Status :*** Correct ***Marks : 1/1***

7. In the brute force algorithm to check if a set of points is in a convex

position, the sign of the cross product indicates:

***Answer***

The direction of the turn (Left or Right)

***Status :*** Correct ***Marks : 1/1***

8. Given 5 points in C where all x-coordinates are identical, what is the result of the brute force closest pair algorithm?

***Answer***

The distance between the two points with the closest y-coordinates

***Status :*** Correct ***Marks : 1/1***

9. What is the worst-case time complexity of the Brute Force algorithm for checking if given points form a convex polygon?

***Answer***

O(n<sup>2</sup>)

***Status :*** Correct ***Marks : 1/1***

10. In a C program solving the Closest Pair problem via brute force, which nested loop structure correctly avoids redundant calculations (i.e., doesn't calculate distance between A,B and B,A)?

***Answer***

`for(i=0; i

***Status :*** Correct ***Marks : 1/1***

11. When implementing the Brute Force Closest Pair algorithm in C, what is the minimal number of distance calculations required if there are exactly 3 points?

***Answer***

3

***Status :*** Correct ***Marks : 1/1***

12. In the brute force method for finding the convex hull (checking all edges), an edge (i, j) is part of the convex hull if:

***Answer***

All other points lie on one side of the line through i and j

***Status :*** Correct ***Marks : 1/1***

13. Which of the following best describes "collinearity" in the context of the Convex Hull problem when using brute force?

***Answer***

Points lying exactly on the hull edge

***Status :*** Correct ***Marks : 1/1***

14. What is a significant disadvantage of the Brute Force Convex Hull algorithm compared to Divide and Conquer algorithms?

***Answer***

It is O(n$^{2}$) in the worst case and O(n$^{2}$) in the best case (for Jarvis March)

***Status :*** Correct ***Marks : 1/1***

15. Which of the following correctly represents the Euclidean distance between two points (x1,y1) and (x2,y2) in C?

***Answer***

`sqrt(pow(x2 - x1, 2) + pow(y2 - y1, 2))`

***Status :*** Wrong ***Marks : 0/1***

# Vel Tech Group of Institutions

Name: NIKHIL  KUMAR
Email: vtu28682@veltech.edu.in
Roll no: vtu28682
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

## 2028_NeoColab_Design and Analysis of AlgorithmsS1L10

### VTU_DAA_Week 3_COD

Attempt : 1
Total Mark : 40
Marks Obtained : 40

### Section 1 : Coding

1. Problem Statement

You are given a set of N points on a 2D plane. Your task is to find the distance between the closest pair of points.

Write a program that takes as input the number of points N, followed by the x and y coordinates of each point. The program should output the distance between the closest pair of points, rounded to two decimal places.

***Input Format***

The first line of input consists of an integer N, representing the number of points.

The following N lines represent the coordinate points x and y, separated by space.

***Output Format***

The output prints a single floating-point number representing the distance between the closest pair of points, rounded to two decimal places.

***Sample Test Case***

Input: 3
2 3
12 30
40 50

Output: 28.79

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <math.h>

int main() {
    int n;
    scanf("%d", &n);

    int x[n], y[n];

    // Input points
    for(int i = 0; i < n; i++) {
        scanf("%d %d", &x[i], &y[i]);
    }

    double minDist = 1e9; // large value

    // Brute force: check all pairs
    for(int i = 0; i < n; i++) {
        for(int j = i + 1; j < n; j++) {
            double dx = x[i] - x[j];
            double dy = y[i] - y[j];
            double dist = sqrt(dx * dx + dy * dy);

            if(dist < minDist) {
                minDist = dist;
            }
        }
    }

    // Output with 2 decimal places
```

```
    printf("%.2lf", minDist);

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

2. Problem Statement

Ishita is working on a mapping application that helps locate the nearest landmarks on a 2D map. Each landmark is represented by its (x, y) coordinate. Her task is to find out the shortest distance between any two landmarks from the given list.

Help Ishita write a program that calculates the distance between the closest pair of points (landmarks) on the map.

***Input Format***

The first line of input contains an integer N representing the number of landmarks.

The next N lines contain two space-separated integers, representing the x and y coordinates of each landmark.

***Output Format***

The output prints a single floating-point number representing the distance between any two landmarks rounded to two decimal places.

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 4
5 7
12 32
21 23
43 65

Output: 12.73

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <math.h>

int main() {
    int n;
    scanf("%d", &n);

    int x[n], y[n];

    // Input coordinates
    for(int i = 0; i < n; i++) {
        scanf("%d %d", &x[i], &y[i]);
    }

    double minDist = 1e9;

    // Check all pairs
    for(int i = 0; i < n; i++) {
        for(int j = i + 1; j < n; j++) {
            double dx = x[i] - x[j];
            double dy = y[i] - y[j];
            double dist = sqrt(dx * dx + dy * dy);

            if(dist < minDist) {
                minDist = dist;
            }
        }
    }

    // Output rounded to 2 decimal places
    printf("%.2lf", minDist);

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

3. Problem Statement

Ragu, an aspiring computer scientist, is currently learning about the concept of finding the closest pair of points in a plane. He wants to test his skills by solving a programming challenge related to this concept.

Given a set of points in a 2D plane, Ragu needs to find the closest pair of points and calculate their Euclidean distance.

***Input Format***

The first line of input consists of an integer N, representing the number of points in the plane.

The following N lines, each consisting of two space-separated real numbers x and y, represent the coordinates of a point.

***Output Format***

The first line of output prints the coordinates of the two points forming the closest pair in the below format:

"Closest pair of points: (x1, y1) and (x2, y2)"

The second line prints "Distance: <distance>" representing the Euclidean distance between the closest pair of points, rounded off to two decimal places.

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 4
0 3
1 1
10 18
4 9

Output: Closest pair of points: (0, 3) and (1, 1)
Distance: 2.24

***Answer***

vtu28682

```
// You are using GCC
#include <stdio.h>
#include <math.h>

int main() {
    int n;
    scanf("%d", &n);

    double x[n], y[n];

    // Input points
    for(int i = 0; i < n; i++) {
        scanf("%lf %lf", &x[i], &y[i]);
    }

    double minDist = 1e9;
    int p1 = 0, p2 = 1;

    // Brute force check
    for(int i = 0; i < n; i++) {
        for(int j = i + 1; j < n; j++) {
            double dx = x[i] - x[j];
            double dy = y[i] - y[j];
            double dist = sqrt(dx * dx + dy * dy);

            if(dist < minDist) {
                minDist = dist;
                p1 = i;
                p2 = j;
            }
        }
    }

    // Output format exactly as required
    printf("Closest pair of points: (%.0lf, %.0lf) and (%.0lf, %.0lf)\n",
           x[p1], y[p1], x[p2], y[p2]);
    printf("Distance: %.2lf", minDist);

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

4. Problem Statement

Ragu, a passionate computer science student, is exploring geometry problems to sharpen his algorithmic thinking. Today, he challenges himself to determine the two closest points among a set of 2D coordinates.

Ragu must find the closest pair of points from the given set and compute the Euclidean distance between them with high precision. He also wants the output to display which two points are the closest, formatted clearly.

***Input Format***

The first line of input contains an integer N representing the number of points in the 2D plane.

The next N lines each contain two space-separated real numbers x and y representing the coordinates of each point.

***Output Format***

The first line of output prints the coordinates of the two points forming the closest pair in the below format:

"Closest pair of points: (x1, y1) and (x2, y2)"

The second line prints "Distance: <distance>" representing the Euclidean distance between the closest pair of points, rounded off to two decimal places.

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 5
1 2
3 4
1 0
6 8
2 3
Output: Closest pair of points: (1, 2) and (2, 3)
Distance: 1.41

vtu28682

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <math.h>

int main() {
    int n;
    scanf("%d", &n);

    double x[n], y[n];

    // Input points
    for(int i = 0; i < n; i++) {
        scanf("%lf %lf", &x[i], &y[i]);
    }

    double minDist = 1e9;
    int p1 = 0, p2 = 1;

    // Brute force approach
    for(int i = 0; i < n; i++) {
        for(int j = i + 1; j < n; j++) {
            double dx = x[i] - x[j];
            double dy = y[i] - y[j];
            double dist = sqrt(dx * dx + dy * dy);

            if(dist < minDist) {
                minDist = dist;
                p1 = i;
                p2 = j;
            }
        }
    }

    // Output (IMPORTANT: exact format)
    printf("Closest pair of points: (%.0lf, %.0lf) and (%.0lf, %.0lf)\n",
           x[p1], y[p1], x[p2], y[p2]);
    printf("Distance: %.2lf", minDist);

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

vtu28682

# Vel Tech Group of Institutions

Name: NIKHIL  KUMAR
Email: vtu28682@veltech.edu.in
Roll no: vtu28682
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

## 2028_NeoColab_Design and Analysis of AlgorithmsS1L10

### VTU_DAA_Week 3_CY

Attempt : 1
Total Mark : 50
Marks Obtained : 37

### Section 1 : Coding

1. Problem Statement

Adhi is learning about computational geometry and has come across the concept of a Convex Hull.

A Convex Hull is a convex polygon that encompasses a set of points in a two-dimensional plane, such that no point lies within the polygon's interior.

Adhi is particularly interested in determining whether a given point lies inside or outside the convex hull formed by a set of points.

Help Adhi by writing a program that checks whether a given point lies inside or outside the convex hull formed by a set of points.

Example

vtu28682

N = 7,

Points: {(1, 1), (2, 1), (3, 1), (4, 1), (4, 2), (4, 3), (4, 4)},

Query: X = 3, Y = 2

Below is the image of the plotting of the given points:

The query (3, 2) lies inside the Convex Hull.

***Input Format***

The first line of input consists of an integer N, representing the number of points in the set.

The following N lines, each containing two integers x and y, represent the coordinates of the points in the set.

The last line consists of two integers px and py, representing the coordinates of the test point.

***Output Format***

The output prints whether the given test points lie inside or outside the Convex Hull.

Refer to the sample output for the formatting specifications.

***Sample Test Case***

Input: 7
1 1
2 1
3 1
4 1
4 2
4 3
4 4
3 2

vtu28682

Output: The point (3, 2) lies inside the Convex Hull.

***Answer***

```
// You are using GCC
#include <stdio.h>

#define MAX 20

typedef struct {
   int x, y;
} Point;

// Function to find orientation
int orientation(Point p, Point q, Point r) {
   int val = (q.y - p.y) * (r.x - q.x) - (q.x - p.x) * (r.y - q.y);
   if (val == 0) return 0;      // collinear
   return (val > 0) ? 1 : 2;    // 1 = clockwise, 2 = counterclockwise
}

// Function to check if point is inside convex polygon
int isInside(Point hull[], int hSize, Point p) {
   int prev = 0;
   for (int i = 0; i < hSize; i++) {
      Point a = hull[i];
      Point b = hull[(i + 1) % hSize];
      int o = orientation(a, b, p);

      if (o == 0) continue; // on edge
      if (prev == 0) prev = o;
      else if (o != prev) return 0; // different direction   outside
   }
   return 1; // inside
}

int main() {
   int n;
   scanf("%d", &n);

   Point pts[MAX];

   for (int i = 0; i < n; i++) {
      scanf("%d %d", &pts[i].x, &pts[i].y);
```

```
    }
    Point query;
    scanf("%d %d", &query.x, &query.y);

    // Convex Hull (Jarvis March)
    Point hull[MAX];
    int hSize = 0;

    if (n < 3) {
        printf("The point (%d, %d) lies outside the Convex Hull.", query.x, query.y);
        return 0;
    }

    int left = 0;
    for (int i = 1; i < n; i++)
        if (pts[i].x < pts[left].x)
            left = i;

    int p = left, q;
    do {
        hull[hSize++] = pts[p];
        q = (p + 1) % n;

        for (int i = 0; i < n; i++) {
            if (orientation(pts[p], pts[i], pts[q]) == 2)
                q = i;
        }

        p = q;

    } while (p != left);

    // Check if point is inside
    if (isInside(hull, hSize, query))
        printf("The point (%d, %d) lies inside the Convex Hull.", query.x, query.y);
    else
        printf("The point (%d, %d) lies outside the Convex Hull.", query.x, query.y);

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

2. Problem Statement:

Olivia, a city planner, is designing a recreational area surrounded by specific landmarks. To ensure balance in the design, Olivia needs to calculate the weighted centroid of the boundary formed by these landmarks using the convex hull.

Your task is to compute the convex hull and determine the weighted centroid of its vertices.

Note:

MAXN (100000): The maximum capacity for both the points and hull arrays, allowing storage of up to 100,000 points each.1e-9 is the threshold for floating-point comparisons.

Example:

Input:

6

0 0

4 0

4 3

0 3

2 1

3 2

Output:

Weighted Centroid: 2.00 1.50

Explanation:

Input Points: (0, 0), (2, 0), (2, 2), (0, 2), (1, 1), (3, 1).

Pivot Selection: The point with the lowest y-coordinate (and leftmost in

case of a tie) is selected as the pivot: (0, 0).

Sorting by Polar Angle: Points are sorted by the polar angle relative to the pivot: (0, 0), (2, 0), (3, 1), (2, 2), (0, 2).

Convex Hull Construction: Using counterclockwise turns, the points forming the convex hull are: (0, 0), (2, 0), (3, 1), (2, 2), (0, 2).

***Input Format***

The first line contains an integer N, representing the number of points.

The next N lines each contain two floating-point numbers x and y, representing the coordinates of the points.

***Output Format***

The output prints the coordinates of the weighted centroid of the convex hull, formatted to two decimal places.

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 5
0 0
2 0
2 2
0 2
1 1
Output: Weighted Centroid: 1.00 1.00

***Answer***

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

#define MAXN 100000
#define EPS 1e-9

typedef struct {
    double x, y;
```

```
} Point;

Point p0;

// Distance squared
double distSq(Point a, Point b) {
    return (a.x - b.x)*(a.x - b.x) + (a.y - b.y)*(a.y - b.y);
}

// Orientation
int orientation(Point p, Point q, Point r) {
    double val = (q.y - p.y)*(r.x - q.x) - (q.x - p.x)*(r.y - q.y);
    if (fabs(val) < EPS) return 0;
    return (val > 0) ? 1 : 2;
}

// Comparator
int compare(const void *vp1, const void *vp2) {
    Point *p1 = (Point *)vp1;
    Point *p2 = (Point *)vp2;

    int o = orientation(p0, *p1, *p2);

    if (o == 0) {
        return (distSq(p0, *p2) >= distSq(p0, *p1)) ? -1 : 1;
    }

    return (o == 2) ? -1 : 1;
}

int main() {
    int n;
    scanf("%d", &n);

    Point points[MAXN];

    for (int i = 0; i < n; i++) {
        scanf("%lf %lf", &points[i].x, &points[i].y);
    }

    // Step 1: Find pivot
    int min = 0;
```

```
for (int i = 1; i < n; i++) {
    if (points[i].y < points[min].y ||
        (fabs(points[i].y - points[min].y) < EPS && points[i].x < points[min].x)) {
        min = i;
    }
}

// Swap pivot
Point temp = points[0];
points[0] = points[min];
points[min] = temp;

p0 = points[0];

// Step 2: Sort
qsort(&points[1], n - 1, sizeof(Point), compare);

// Step 3: Remove collinear duplicates
Point filtered[MAXN];
int m = 0;

filtered[m++] = points[0];

for (int i = 1; i < n; i++) {
    while (i < n - 1 && orientation(p0, points[i], points[i+1]) == 0)
        i++;
    filtered[m++] = points[i];
}

// Step 4: Build hull
Point hull[MAXN];
int h = 0;

for (int i = 0; i < m; i++) {
    while (h >= 2 && orientation(hull[h-2], hull[h-1], filtered[i]) != 2)
        h--;
    hull[h++] = filtered[i];
}

// Step 5: Compute centroid
double cx = 0, cy = 0;
for (int i = 0; i < h; i++) {
```

```
    cx += hull[i].x;
    cy += hull[i].y;
  }

  cx /= h;
  cy /= h;

  // Output
  printf("Weighted Centroid: %.2lf %.2lf", cx, cy);

  return 0;
}
```

***Status :*** Partially correct ***Marks : 6/10***

3. Problem Statement:

Gwen, a city planner, is designing a new park and needs to calculate the perimeter of the park's boundary. She has the coordinates of key boundary points and wants to find the minimum perimeter that encloses all these points.

Using the convex hull, Gwen needs help in computing both the boundary and its perimeter.

Note:

points[100000]: An array that can store up to 100,000 input points.hull[100000]: An array that can store up to 100,000 points on the convex hull.1e-9: A threshold for floating-point comparisons to handle precision issues.

Example

Input:

6

0.0 0.0

3.0 0.0

3.0 2.0

1.0 2.0

0.0 2.0

2.0 1.0

Output:

Convex Hull:

0 0

3 0

3 2

0 2

Perimeter: 10.00

Explanation:

Input Points: (0.0, 0.0), (3.0, 0.0), (3.0, 2.0), (1.0, 2.0), (0.0, 2.0), (2.0, 1.0).

Convex Hull Construction:

Pivot: Start with the lowest point, (0, 0).Sorted by Polar Angle: The points are sorted, and counterclockwise turns are included in the hull.Convex Hull Points: (0, 0), (3, 0), (3, 2), (0, 2).Perimeter Calculation:

Distances between consecutive hull points:(0, 0)-(3, 0): 3.00, (3, 0)-(3, 2): 2.00, (3, 2)-(0, 2): 3.00, (0, 2)-(0, 0): 2.00.Total Perimeter: 10.00.

***Input Format***

The first line contains an integer N, representing the number of points.

The next N lines each contain two space separated decimal values x and y, representing the coordinates of the points.

***Output Format***

The first line prints "Convex Hull:" followed by the coordinates of the points on the convex hull, each on a new line, in counterclockwise order.

The last line prints the "Perimeter: ", followed by the total perimeter of the convex hull, formatted to two decimal places.

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 6
0.0 0.0
3.0 0.0
3.0 2.0
1.0 2.0
0.0 2.0
2.0 1.0
Output: Convex Hull:
0 0
3 0
3 2
0 2
Perimeter: 10.00

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

#define MAXN 100000
#define EPS 1e-9

typedef struct {
    double x, y;
} Point;

Point p0;

// Distance
double dist(Point a, Point b) {
    return sqrt((a.x - b.x)*(a.x - b.x) + (a.y - b.y)*(a.y - b.y));
}

// Orientation
int orientation(Point p, Point q, Point r) {
```

```
    double val = (q.y - p.y)*(r.x - q.x) - (q.x - p.x)*(r.y - q.y);
    if (fabs(val) < EPS) return 0;
    return (val > 0) ? 1 : 2;
}

// Comparator
int compare(const void *vp1, const void *vp2) {
    Point *p1 = (Point *)vp1;
    Point *p2 = (Point *)vp2;

    int o = orientation(p0, *p1, *p2);

    if (o == 0)
        return (dist(p0, *p2) >= dist(p0, *p1)) ? -1 : 1;

    return (o == 2) ? -1 : 1;
}

int main() {
    int n;
    scanf("%d", &n);

    Point points[MAXN];

    for (int i = 0; i < n; i++) {
        scanf("%lf %lf", &points[i].x, &points[i].y);
    }

    // Step 1: Find pivot
    int min = 0;
    for (int i = 1; i < n; i++) {
        if (points[i].y < points[min].y ||
            (fabs(points[i].y - points[min].y) < EPS && points[i].x < points[min].x))
             min = i;
    }

    // Swap
    Point temp = points[0];
    points[0] = points[min];
    points[min] = temp;

    p0 = points[0];
```

```
    // Step 2: Sort
    qsort(&points[1], n - 1, sizeof(Point), compare);

    // Step 3: Remove collinear
    Point filtered[MAXN];
    int m = 0;
    filtered[m++] = points[0];

    for (int i = 1; i < n; i++) {
        while (i < n - 1 && orientation(p0, points[i], points[i+1]) == 0)
            i++;
        filtered[m++] = points[i];
    }

    // Step 4: Build hull
    Point hull[MAXN];
    int h = 0;

    for (int i = 0; i < m; i++) {
        while (h >= 2 && orientation(hull[h-2], hull[h-1], filtered[i]) != 2)
            h--;
        hull[h++] = filtered[i];
    }

    // Step 5: Print hull
    printf("Convex Hull:\n");
    for (int i = 0; i < h; i++) {
        printf("%.0lf %.0lf\n", hull[i].x, hull[i].y);
    }

    // Step 6: Perimeter
    double perimeter = 0;
    for (int i = 0; i < h; i++) {
        perimeter += dist(hull[i], hull[(i + 1) % h]);
    }

    printf("Perimeter: %.2lf", perimeter);

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

4. Problem Statement:

Maria, a wildlife researcher, is studying the distribution of endangered species in a forest. She has mapped several points that represent observation locations inside the forest.

Maria wants to identify the K points that are the farthest from the forest's outer boundary, represented by a convex hull formed by a set of points. These points represent key observations, and she needs to rank them by their distance to the outer boundary of the forest.

Help Maria compute the convex hull and find the K farthest points from the forest boundary.

Note:

MAXN (100000): Maximum number of points for points and hull arrays.1e-9:Threshold for floating-point equality checks to handle precision errors.1e-12: Threshold to detect degenerate (zero-length) segments.1e30:Initial large value for minimum distance calculations.

Example:

Input:

7 3

0 0

4 0

4 4

0 4

2 2

1 2

3 2

Output:

2 2

3 2

1 2

Explanation:

Convex Hull Points: Using Graham's Scan, the convex hull is: (0, 0), (4, 0), (4, 4), (0, 4).

Distance to Hull: Distances of all points to the convex hull boundary are computed.

Top 3 Farthest Points: Sorted by descending distance, the top 3 points are (2, 2), (3, 2), (1, 2).

***Input Format***

The first line contains two integers N and K, representing the number of points and the number of farthest points to find, respectively.

The next N lines each contain two integers x and y, representing the coordinates of the points.

***Output Format***

The output prints K lines, each containing two integers x and y, representing the coordinates of the K farthest points from the convex hull boundary. Print the points in descending order of their distances to the boundary.

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 7 3
0 0
4 0
4 4
0 4
2 2
1 2
3 2

Output: 2 2
3 2
1 2

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

#define MAXN 100000
#define EPS 1e-9

typedef struct {
    double x, y;
} Point;

typedef struct {
    Point p;
    double dist;
} Node;

Point p0;

// Distance between two points
double dist(Point a, Point b) {
    return sqrt((a.x - b.x)*(a.x - b.x) + (a.y - b.y)*(a.y - b.y));
}

// Orientation
int orientation(Point p, Point q, Point r) {
    double val = (q.y - p.y)*(r.x - q.x) - (q.x - p.x)*(r.y - q.y);
    if (fabs(val) < EPS) return 0;
    return (val > 0) ? 1 : 2;
}

// Distance squared
double distSq(Point a, Point b) {
    return (a.x - b.x)*(a.x - b.x) + (a.y - b.y)*(a.y - b.y);
}

// Comparator for sorting points (polar angle)
```

```
int compare(const void *a, const void *b) {
    Point *p1 = (Point *)a;
    Point *p2 = (Point *)b;

    int o = orientation(p0, *p1, *p2);
    if (o == 0)
        return (distSq(p0, *p2) >= distSq(p0, *p1)) ? -1 : 1;

    return (o == 2) ? -1 : 1;
}

// Distance from point P to line segment AB
double pointToSegment(Point A, Point B, Point P) {
    double dx = B.x - A.x;
    double dy = B.y - A.y;

    if (fabs(dx) < EPS && fabs(dy) < EPS)
        return dist(P, A);

    double t = ((P.x - A.x)*dx + (P.y - A.y)*dy) / (dx*dx + dy*dy);

    if (t < 0) return dist(P, A);
    if (t > 1) return dist(P, B);

    Point proj;
    proj.x = A.x + t * dx;
    proj.y = A.y + t * dy;

    return dist(P, proj);
}

// Comparator for sorting by distance (descending)
int cmpNode(const void *a, const void *b) {
    Node *n1 = (Node *)a;
    Node *n2 = (Node *)b;

    if (fabs(n2->dist - n1->dist) < EPS) return 0;
    return (n2->dist > n1->dist) ? 1 : -1;
}

int main() {
    int n, k;
```

```
    scanf("%d %d", &n, &k);

    Point points[MAXN];

    for (int i = 0; i < n; i++) {
        scanf("%lf %lf", &points[i].x, &points[i].y);
    }

    // Step 1: Find pivot
    int min = 0;
    for (int i = 1; i < n; i++) {
        if (points[i].y < points[min].y ||
            (fabs(points[i].y - points[min].y) < EPS && points[i].x < points[min].x))
            min = i;
    }

    Point temp = points[0];
    points[0] = points[min];
    points[min] = temp;

    p0 = points[0];

    // Step 2: Sort
    qsort(&points[1], n - 1, sizeof(Point), compare);

    // Step 3: Remove collinear
    Point filtered[MAXN];
    int m = 0;
    filtered[m++] = points[0];

    for (int i = 1; i < n; i++) {
        while (i < n - 1 && orientation(p0, points[i], points[i+1]) == 0)
            i++;
        filtered[m++] = points[i];
    }

    // Step 4: Build hull
    Point hull[MAXN];
    int h = 0;

    for (int i = 0; i < m; i++) {
        while (h >= 2 && orientation(hull[h-2], hull[h-1], filtered[i]) != 2)
```

```
        h--;
    hull[h++] = filtered[i];
}

// Step 5: Compute distance of each point to hull
Node arr[MAXN];

for (int i = 0; i < n; i++) {
    double minDist = 1e30;

    for (int j = 0; j < h; j++) {
        Point A = hull[j];
        Point B = hull[(j + 1) % h];

        double d = pointToSegment(A, B, points[i]);
        if (d < minDist) minDist = d;
    }

    arr[i].p = points[i];
    arr[i].dist = minDist;
}

// Step 6: Sort by distance descending
qsort(arr, n, sizeof(Node), cmpNode);

// Step 7: Print top K
for (int i = 0; i < k; i++) {
    printf("%.0lf %.0lf\n", arr[i].p.x, arr[i].p.y);
}

return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

5. Problem Statement:

Henry, a wildlife researcher, is studying the distribution of endangered species in a forest. He has mapped several points that represent observation locations inside the forest.

Henry wants to identify the K points that are the farthest from the forest's outer boundary, represented by a convex hull formed by a set of points. These points represent key observations, and he needs to rank them by their distance to the outer boundary of the forest.

Help Henry compute the convex hull and find the K farthest points from the forest boundary.

Note:

MAXN (100000): Maximum number of points for points and hull arrays.1e-9:Threshold for floating-point equality checks to handle precision errors.1e-12: Threshold to detect degenerate (zero-length) segments.1e30:Initial large value for minimum distance calculations.

Example:

Input:

7 3

0 0

4 0

4 4

0 4

2 2

1 2

3 2

Output:

2 2

3 2

1 2

Explanation:

Convex Hull Points: Using Graham's Scan, the convex hull is: (0, 0), (4, 0),

(4, 4), (0, 4).

Distance to Hull: Distances of all points to the convex hull boundary are computed.

Top 3 Farthest Points: Sorted by descending distance, the top 3 points are (2, 2), (3, 2), (1, 2).

***Input Format***

The first line contains two integers N and K, representing the number of points and the number of farthest points to find, respectively.

The next N lines each contain two integers x and y, representing the coordinates of the points.

***Output Format***

The output prints K lines, each containing two integers x and y, representing the coordinates of the K farthest points from the convex hull boundary. Print the points in descending order of their distances to the boundary.

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 7 3
0 0
4 0
4 4
0 4
2 2
1 2
3 2

Output: 2 2
3 2
1 2

***Answer***

```
#include <stdio.h>
#include <math.h>
```

```
#define MAX 20
#define INF 1e30

typedef struct {
   double x, y;
} Point;

Point hull[MAX];
int hSize = 0;

// Orientation
int orientation(Point p, Point q, Point r) {
   double val = (q.y - p.y)*(r.x - q.x) - (q.x - p.x)*(r.y - q.y);
   if (fabs(val) < 1e-9) return 0;
   return (val > 0) ? 1 : 2;
}

// Distance
double dist(Point a, Point b) {
   return sqrt((a.x - b.x)*(a.x - b.x) + (a.y - b.y)*(a.y - b.y));
}

// Distance from point to segment
double pointToSegment(Point A, Point B, Point P) {
   double dx = B.x - A.x;
   double dy = B.y - A.y;

   if (fabs(dx) < 1e-12 && fabs(dy) < 1e-12)
      return dist(P, A);

   double t = ((P.x - A.x)*dx + (P.y - A.y)*dy) / (dx*dx + dy*dy);

   if (t < 0) return dist(P, A);
   if (t > 1) return dist(P, B);

   Point proj;
   proj.x = A.x + t * dx;
   proj.y = A.y + t * dy;

   return dist(P, proj);
}
```

```
// Check if point is on hull
int isHullPoint(Point p) {
    for (int i = 0; i < hSize; i++) {
        if (fabs(p.x - hull[i].x) < 1e-9 && fabs(p.y - hull[i].y) < 1e-9)
            return 1;
    }
    return 0;
}

int main() {
    int n, k;
    scanf("%d %d", &n, &k);

    Point pts[MAX];

    for (int i = 0; i < n; i++) {
        scanf("%lf %lf", &pts[i].x, &pts[i].y);
    }

    // --- Convex Hull (Jarvis March) ---
    int left = 0;
    for (int i = 1; i < n; i++)
        if (pts[i].x < pts[left].x)
            left = i;

    int p = left, q;

    do {
        hull[hSize++] = pts[p];
        q = (p + 1) % n;

        for (int i = 0; i < n; i++) {
            if (orientation(pts[p], pts[i], pts[q]) == 2)
                q = i;
        }

        p = q;

    } while (p != left);

    // --- Compute distances for non-hull points ---
    Point result[MAX];
```

```
double d[MAX];
int cnt = 0;

for (int i = 0; i < n; i++) {

    if (isHullPoint(pts[i])) continue; // skip hull points

    double minDist = INF;

    for (int j = 0; j < hSize; j++) {
        Point A = hull[j];
        Point B = hull[(j + 1) % hSize];

        double val = pointToSegment(A, B, pts[i]);
        if (val < minDist)
            minDist = val;
    }

    result[cnt] = pts[i];
    d[cnt] = minDist;
    cnt++;
}

// --- Sort with tie-breaking ---
for (int i = 0; i < cnt - 1; i++) {
    for (int j = i + 1; j < cnt; j++) {

        if (d[j] > d[i] ||
           (fabs(d[j] - d[i]) < 1e-9 &&
            (result[j].x > result[i].x ||
            (result[j].x == result[i].x && result[j].y > result[i].y)))) {

            // swap distance
            double td = d[i]; d[i] = d[j]; d[j] = td;

            // swap points
            Point tp = result[i];
            result[i] = result[j];
            result[j] = tp;
        }
    }
}
```

```
    // --- Output top K ---
    for (int i = 0; i < k && i < cnt; i++) {
        printf("%.0lf %.0lf\n", result[i].x, result[i].y);
    }

    return 0;
}
```

***Status :*** Partially correct ***Marks : 1/10***

vtu28682

# Vel Tech Group of Institutions

Name: NIKHIL  KUMAR
Email: vtu28682@veltech.edu.in
Roll no: vtu28682
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

## 2028_NeoColab_Design and Analysis of AlgorithmsS1L10

### VTU_DAA_Week 3_PAH

Attempt : 1
Total Mark : 50
Marks Obtained : 48.5

### Section 1 : Coding

1. Problem Statement:

Javier, a landscape architect, is designing a new botanical garden and needs to determine the outer boundary that will enclose all the selected plant species.

He has gathered coordinates of key points where the plants are located and needs to calculate the convex hull to create a protective garden boundary. To determine this boundary, Javier decides to use Graham's Scan algorithm.

Help Javier compute the convex hull by implementing the algorithm to design the garden's boundary.

***Input Format***

vtu28682

The first line of input consists of an integer n, representing the number of plant species locations.

The next n lines contain two integers x and y, representing the coordinates of each location.

***Output Format***

The output prints "Convex Hull:" followed by the coordinates of the points forming the convex hull in counterclockwise order, with each point on a new line.

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 4
0 0
0 1
1 0
1 1

Output: Convex Hull:
(0, 0)
(1, 0)
(1, 1)
(0, 1)

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

#define MAX 20

typedef struct {
    int x, y;
} Point;

Point p0;

// Find orientation
int orientation(Point p, Point q, Point r) {
```

```
    int val = (q.y - p.y)*(r.x - q.x) - (q.x - p.x)*(r.y - q.y);
    if (val == 0) return 0;
    return (val > 0) ? 1 : 2; // 1 = clockwise, 2 = counterclockwise
}

// Distance squared (for sorting)
int distSq(Point p1, Point p2) {
    return (p1.x - p2.x)*(p1.x - p2.x) +
           (p1.y - p2.y)*(p1.y - p2.y);
}

// Comparator for sorting
int compare(const void *vp1, const void *vp2) {
    Point *p1 = (Point *)vp1;
    Point *p2 = (Point *)vp2;

    int o = orientation(p0, *p1, *p2);

    if (o == 0)
        return (distSq(p0, *p2) >= distSq(p0, *p1)) ? -1 : 1;

    return (o == 2) ? -1 : 1;
}

// Swap
void swap(Point *p1, Point *p2) {
    Point temp = *p1;
    *p1 = *p2;
    *p2 = temp;
}

int main() {
    int n;
    scanf("%d", &n);

    Point points[MAX];

    for (int i = 0; i < n; i++) {
        scanf("%d %d", &points[i].x, &points[i].y);
    }

    // Step 1: Find bottom-most point
```

```
int ymin = points[0].y, min = 0;
for (int i = 1; i < n; i++) {
    if ((points[i].y < ymin) ||
        (points[i].y == ymin && points[i].x < points[min].x)) {
        ymin = points[i].y;
        min = i;
    }
}

// Place pivot at first
swap(&points[0], &points[min]);
p0 = points[0];

// Step 2: Sort by polar angle
qsort(&points[1], n - 1, sizeof(Point), compare);

// Step 3: Remove collinear points (keep farthest)
Point filtered[MAX];
int m = 1;

for (int i = 1; i < n; i++) {
    while (i < n - 1 &&
           orientation(p0, points[i], points[i + 1]) == 0)
        i++;
    filtered[m++] = points[i];
}

if (m < 3) {
    printf("Convex Hull:\n");
    for (int i = 0; i < m; i++)
        printf("(%d, %d)\n", filtered[i].x, filtered[i].y);
    return 0;
}

// Step 4: Graham Scan using stack
Point stack[MAX];
int top = 0;

stack[top++] = p0;
stack[top++] = filtered[1];
stack[top++] = filtered[2];
```

```
    for (int i = 3; i < m; i++) {
        while (top >= 2 &&
                orientation(stack[top-2], stack[top-1], filtered[i]) != 2)
            top--;

        stack[top++] = filtered[i];
    }

    // Output
    printf("Convex Hull:\n");
    for (int i = 0; i < top; i++) {
        printf("(%d, %d)\n", stack[i].x, stack[i].y);
    }

    return 0;
}
```

***Status :*** Correct **Marks : 10/10**

2. Problem Statement:

Eva, a botanist, is surveying a nature reserve to protect rare plant species. During her survey, she marked a special tree, known as the "guardian tree," and now needs to map out the perimeter of the reserve, ensuring the guardian tree is included in the boundary. To do this, Eva needs to compute the convex hull of all the points, including the coordinates of the guardian tree.

Your task is to compute the convex hull, ensuring that the special guardian tree is included in the boundary.

Example

Input:

5

0.0 0.0

2.0 0.0

2.0 2.0

0.0 2.0

1.0 1.0

3.0 1.0

Output:

0 0

2 0

3 1

2 2

0 2

Explanation:

Input Points: (0.0, 0.0), (2.0, 0.0), (2.0, 2.0), (0.0, 2.0), (1.0, 1.0), (3.0, 1.0).

Pivot Selection: The point with the lowest y-coordinate (and leftmost in case of a tie) is selected as the pivot: (0, 0).

Sorting by Polar Angle: Points are sorted by the polar angle relative to the pivot: (0, 0), (2, 0), (3, 1), (2, 2), (0, 2).

Convex Hull Construction: Using counterclockwise turns, the points forming the convex hull are: (0, 0), (2, 0), (3, 1), (2, 2), (0, 2).

***Input Format***

The first line contains an integer N, representing the number of trees originally surveyed.

The next N lines each contain two double values x and y, representing the coordinates of the surveyed trees.

The last line contains two double values x and y, representing the coordinates of the guardian tree.

***Output Format***

The output prints the coordinates of the points forming the convex hull in counterclockwise order, ensuring the special guardian tree is included. Each coordinate should be printed on a new line, with the values of x and y separated

by a space.

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 5
0.0 0.0
2.0 0.0
2.0 2.0
0.0 2.0
1.0 1.0
3.0 1.0
Output: 0 0
2 0
3 1
2 2
0 2

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

#define MAX 50

typedef struct {
    double x, y;
} Point;

Point p0;

// Orientation
int orientation(Point p, Point q, Point r) {
    double val = (q.y - p.y)*(r.x - q.x) - (q.x - p.x)*(r.y - q.y);
    if (fabs(val) < 1e-9) return 0;
    return (val > 0) ? 1 : 2; // 1 = CW, 2 = CCW
}
```

```
// Distance squared
double distSq(Point p1, Point p2) {
    return (p1.x - p2.x)*(p1.x - p2.x) +
           (p1.y - p2.y)*(p1.y - p2.y);
}

// Comparator for sorting
int compare(const void *a, const void *b) {
    Point *p1 = (Point *)a;
    Point *p2 = (Point *)b;

    int o = orientation(p0, *p1, *p2);

    if (o == 0)
        return (distSq(p0, *p2) >= distSq(p0, *p1)) ? -1 : 1;

    return (o == 2) ? -1 : 1;
}

// Swap
void swap(Point *a, Point *b) {
    Point temp = *a;
    *a = *b;
    *b = temp;
}

int main() {
    int n;
    scanf("%d", &n);

    Point points[MAX];

    for (int i = 0; i < n; i++) {
        scanf("%lf %lf", &points[i].x, &points[i].y);
    }

    // Read guardian tree
    scanf("%lf %lf", &points[n].x, &points[n].y);
    n++; // include it

    // Step 1: find lowest point
    int ymin = points[0].y, min = 0;
```

```
for (int i = 1; i < n; i++) {
    if (points[i].y < ymin ||
        (points[i].y == ymin && points[i].x < points[min].x)) {
        ymin = points[i].y;
        min = i;
    }
}

swap(&points[0], &points[min]);
p0 = points[0];

// Step 2: sort
qsort(&points[1], n - 1, sizeof(Point), compare);

// Step 3: remove collinear
Point filtered[MAX];
int m = 1;

for (int i = 1; i < n; i++) {
    while (i < n - 1 &&
            orientation(p0, points[i], points[i+1]) == 0)
        i++;

    filtered[m++] = points[i];
}

// Step 4: stack
Point stack[MAX];
int top = 0;

stack[top++] = p0;
stack[top++] = filtered[1];
stack[top++] = filtered[2];

for (int i = 3; i < m; i++) {
    while (top >= 2 &&
            orientation(stack[top-2], stack[top-1], filtered[i]) != 2)
        top--;

    stack[top++] = filtered[i];
}
```

```
    // Output
    for (int i = 0; i < top; i++) {
        printf("%.0lf %.0lf\n", stack[i].x, stack[i].y);
    }

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

3. Problem Statement

Nathan is learning about computational geometry and has come across the concept of a Convex Hull.

A Convex Hull is a convex polygon that encompasses a set of points in a two-dimensional plane, such that no point lies within the polygon's interior.

Nathan is particularly interested in determining whether a given point lies inside or outside the convex hull formed by a set of points.

Help Nathan by writing a program that checks whether a given point lies inside or outside the convex hull formed by a set of points.

Example

N = 7,

Points: {(1, 1), (2, 1), (3, 1), (4, 1), (4, 2), (4, 3), (4, 4)},

Query: X = 3, Y = 2

Below is the image of the plotting of the given points:

The query (3, 2) lies inside the Convex Hull.

***Input Format***

The first line of input consists of an integer N, representing the number of points in the set.

The following N lines, each containing two integers x and y, represent the coordinates of the points in the set.

The last line consists of two integers px and py, representing the coordinates of the test point.

***Output Format***

The output prints whether the given test points lie inside or outside the Convex Hull.

Refer to the sample output for the formatting specifications.

***Sample Test Case***

Input: 7
1 1
2 1
3 1
4 1
4 2
4 3
4 4
3 2
Output: The point (3, 2) lies inside the Convex Hull.

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

#define MAX 20

typedef struct {
    int x, y;
} Point;

Point p0;
```

```
// Orientation
int orientation(Point p, Point q, Point r) {
    int val = (q.y - p.y)*(r.x - q.x) - (q.x - p.x)*(r.y - q.y);
    if (val == 0) return 0;
    return (val > 0) ? 1 : 2; // 1 = CW, 2 = CCW
}

// Distance squared
int distSq(Point p1, Point p2) {
    return (p1.x - p2.x)*(p1.x - p2.x) +
           (p1.y - p2.y)*(p1.y - p2.y);
}

// Comparator for sorting
int compare(const void *a, const void *b) {
    Point *p1 = (Point *)a;
    Point *p2 = (Point *)b;

    int o = orientation(p0, *p1, *p2);

    if (o == 0)
        return (distSq(p0, *p2) >= distSq(p0, *p1)) ? -1 : 1;

    return (o == 2) ? -1 : 1;
}

// Swap
void swap(Point *a, Point *b) {
    Point temp = *a;
    *a = *b;
    *b = temp;
}

// Check if point inside convex polygon
int isInside(Point hull[], int n, Point p) {
    int prev = 0;

    for (int i = 0; i < n; i++) {
        int next = (i + 1) % n;
        int o = orientation(hull[i], hull[next], p);
```

```
        if (o == 0) continue;

        if (prev == 0)
            prev = o;
        else if (o != prev)
            return 0; // outside
    }
    return 1; // inside
}

int main() {
    int n;
    scanf("%d", &n);

    Point points[MAX];

    for (int i = 0; i < n; i++) {
        scanf("%d %d", &points[i].x, &points[i].y);
    }

    Point query;
    scanf("%d %d", &query.x, &query.y);

    // --- Convex Hull (Graham Scan) ---
    int ymin = points[0].y, min = 0;

    for (int i = 1; i < n; i++) {
        if (points[i].y < ymin ||
            (points[i].y == ymin && points[i].x < points[min].x)) {
            ymin = points[i].y;
            min = i;
        }
    }

    swap(&points[0], &points[min]);
    p0 = points[0];

    qsort(&points[1], n - 1, sizeof(Point), compare);

    Point filtered[MAX];
    int m = 1;
```

```
    for (int i = 1; i < n; i++) {
        while (i < n - 1 &&
                orientation(p0, points[i], points[i+1]) == 0)
            i++;
        filtered[m++] = points[i];
    }

    Point stack[MAX];
    int top = 0;

    stack[top++] = p0;
    stack[top++] = filtered[1];
    stack[top++] = filtered[2];

    for (int i = 3; i < m; i++) {
        while (top >= 2 &&
                orientation(stack[top-2], stack[top-1], filtered[i]) != 2)
            top--;
        stack[top++] = filtered[i];
    }

    // --- Check inside/outside ---
    if (isInside(stack, top, query)) {
        printf("The point (%d, %d) lies inside the Convex Hull.", query.x, query.y);
    } else {
        printf("The point (%d, %d) lies outside the Convex Hull.", query.x, query.y);
    }

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

4. Problem Statement

Ragu, an aspiring computer scientist, is currently learning about the concept of finding the closest pair of points in a plane. He wants to test his skills by solving a programming challenge related to this concept.

Given a set of points in a 2D plane, Ragu needs to find the closest pair of

points and calculate their Euclidean distance.

***Input Format***

The first line of input consists of an integer N, representing the number of points in the plane.

The following N lines, each consisting of two space-separated real numbers x and y, represent the coordinates of a point.

***Output Format***

The first line of output prints the coordinates of the two points forming the closest pair in the below format:

"Closest pair of points: (x1, y1) and (x2, y2)"

The second line prints "Distance: <distance>" representing the Euclidean distance between the closest pair of points, rounded off to two decimal places.

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 4
0 3
1 1
10 18
4 9
Output: Closest pair of points: (0, 3) and (1, 1)
Distance: 2.24

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <math.h>

#define MAX 20

typedef struct {
    double x, y;
```

```
} Point;

// Distance function
double distance(Point a, Point b) {
    return sqrt((a.x - b.x)*(a.x - b.x) +
                (a.y - b.y)*(a.y - b.y));
}

int main() {
    int n;
    scanf("%d", &n);

    Point p[MAX];

    for (int i = 0; i < n; i++) {
        scanf("%lf %lf", &p[i].x, &p[i].y);
    }

    double minDist = 1e9;
    Point p1, p2;

    // Brute force: check all pairs
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {

            double d = distance(p[i], p[j]);

            if (d < minDist) {
                minDist = d;
                p1 = p[i];
                p2 = p[j];
            }
        }
    }

    // Output
    printf("Closest pair of points: (%.0lf, %.0lf) and (%.0lf, %.0lf)\n",
           p1.x, p1.y, p2.x, p2.y);

    printf("Distance: %.2lf", minDist);

    return 0;
```

}

***Status :*** Correct ***Marks : 10/10***

5. Problem Statement:

Lena, a geologist, is studying the boundaries of a newly discovered forest reserve. To define the outer edges of the reserve, Lena needs to identify the convex hull from a set of points marking different trees within the area.

Using Graham's Scan algorithm, help Lena compute the convex hull to outline the forest reserve's perimeter.

Example1

Input:

6

0 0

1 1

2 2

4 4

0 4

4 0

Output:

Convex Hull:

(0, 0)

(4, 0)

(4, 4)

(0, 4)

Explanation:

Pivot Selection: The bottom-most point is (0, 0).

Sorting by Polar Angle: Points sorted as (0, 0), (4, 0), (4, 4), (0, 4), (1, 1), (2, 2).

Convex Hull Construction: Start with (0, 0), and include points forming counterclockwise turns.

The final hull is: (0, 0), (4, 0), (4, 4), (0, 4).

Example2

Input:

3

0 0

0 3

3 0

Output:

Convex Hull:

(0, 0)

(3, 0)

(0, 3)

Explanation:

Pivot Selection: The bottom-most point is (0, 0).

Sorting by Polar Angle: Points sorted as (0, 0), (3, 0), (0, 3).

Convex Hull Construction: Start with (0, 0), and include points forming counterclockwise turns.

The final hull is: (0, 0), (3, 0), (0, 3).

***Input Format***

The first line of input contains an integer n, representing the number of trees in the forest reserve.

vtu28682

The next n lines contain two integers x and y, representing the coordinates of each tree.

***Output Format***

The output prints "Convex Hull:" followed by the coordinates of the trees forming the convex hull in counterclockwise order in new line.

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 6
0 0
1 1
2 2
4 4
0 4
4 0

Output: Convex Hull:
(0, 0)
(4, 0)
(4, 4)
(0, 4)

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

#define MAX 25

typedef struct {
    int x, y;
} Point;

Point p0;

// Orientation
int orientation(Point p, Point q, Point r) {
    int val = (q.y - p.y)*(r.x - q.x) - (q.x - p.x)*(r.y - q.y);
```

```
    if (val == 0) return 0;
    return (val > 0) ? 1 : 2; // 1 = CW, 2 = CCW
}

// Distance squared
int distSq(Point p1, Point p2) {
    return (p1.x - p2.x)*(p1.x - p2.x) +
           (p1.y - p2.y)*(p1.y - p2.y);
}

// Comparator
int compare(const void *a, const void *b) {
    Point *p1 = (Point *)a;
    Point *p2 = (Point *)b;

    int o = orientation(p0, *p1, *p2);

    if (o == 0)
        return (distSq(p0, *p2) >= distSq(p0, *p1)) ? -1 : 1;

    return (o == 2) ? -1 : 1;
}

// Swap
void swap(Point *a, Point *b) {
    Point temp = *a;
    *a = *b;
    *b = temp;
}

int main() {
    int n;
    scanf("%d", &n);

    Point points[MAX];

    for (int i = 0; i < n; i++) {
        scanf("%d %d", &points[i].x, &points[i].y);
    }

    // Step 1: Find bottom-most point
    int ymin = points[0].y, min = 0;
```

```
for (int i = 1; i < n; i++) {
    if (points[i].y < ymin ||
        (points[i].y == ymin && points[i].x < points[min].x)) {
        ymin = points[i].y;
        min = i;
    }
}

swap(&points[0], &points[min]);
p0 = points[0];

// Step 2: Sort
qsort(&points[1], n - 1, sizeof(Point), compare);

// Step 3: Remove collinear points
Point filtered[MAX];
int m = 1;

for (int i = 1; i < n; i++) {
    while (i < n - 1 &&
            orientation(p0, points[i], points[i+1]) == 0)
        i++;
    filtered[m++] = points[i];
}

// Step 4: Graham Scan
Point stack[MAX];
int top = 0;

stack[top++] = p0;
if (m > 1) stack[top++] = filtered[1];
if (m > 2) stack[top++] = filtered[2];

for (int i = 3; i < m; i++) {
    while (top >= 2 &&
            orientation(stack[top-2], stack[top-1], filtered[i]) != 2)
        top--;
    stack[top++] = filtered[i];
}

// Output
```

```
    printf("Convex Hull:\n");
    for (int i = 0; i < top; i++) {
        printf("(%d, %d)\n", stack[i].x, stack[i].y);
    }

    return 0;
}
```

***Status :*** Partially correct ***Marks : 8.5/10***

# Vel Tech Group of Institutions

Name: NIKHIL  KUMAR
Email: vtu28682@veltech.edu.in
Roll no: vtu28682
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

## 2028_NeoColab_Design and Analysis of AlgorithmsS1L10

### VTU_DAA_Week 4_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 15

### Section 1 : MCQ

1. Which of the following methods can be used to solve the Knapsack problem?

***Answer***

All the mentioned options

***Status :*** Correct ***Marks : 1/1***

2. 0/1 knapsack is based on _____method.

***Answer***

Dynamic programming

***Status :*** Correct ***Marks : 1/1***

vtu28682

3. What is the primary advantage of using the Branch and Bound approach for the Knapsack problem?

***Answer***

It systematically explores possible solutions while pruning non-promising ones.

***Status :*** Correct ***Marks : 1/1***

4. Which data structure is commonly used to implement the Branch and Bound approach for solving the Knapsack problem?

***Answer***

Priority Queue

***Status :*** Correct ***Marks : 1/1***

5. In the Branch and Bound approach, what does the term “bounding function” refer to?

***Answer***

A function that estimates the upper bound on the maximum value that can be achieved from a node.

***Status :*** Correct ***Marks : 1/1***

6. Which strategy is commonly used in the Branch and Bound method to decide which node to explore next?

***Answer***

Best-first search

***Status :*** Correct ***Marks : 1/1***

7. Which of the following methods is used to prune the search space in the Branch and Bound method for the Knapsack problem?

***Answer***

Ignoring nodes with a weight exceeding the capacity.

***Status :*** Correct ***Marks : 1/1***

8. The travelling salesman problem can be solved using _____.

***Answer***

a minimum spanning tree

***Status :*** Correct ***Marks : 1/1***

9. Travelling salesman problem is an example of _____.

***Answer***

None of the mentioned options

***Status :*** Correct ***Marks : 1/1***

10. How does the practical Travelling salesman problem differ from the classical Travelling salesman problem?

***Answer***

In the practical travelling salesman problem, each vertex can be visited more than once.

***Status :*** Correct ***Marks : 1/1***

11. The traveling salesman problem involves n cities with paths connecting the cities. The time taken for traversing through all the cities without knowing in advance the length of a minimum tour is

***Answer***

O(n!)

***Status :*** Correct ***Marks : 1/1***

12. In which search problem, to find the shortest path, each city must be

visited only once?

***Answer***

Travelling Salesman problem

***Status :*** Correct ***Marks : 1/1***

13. What is the main goal of using Branch and Bound in TSP?

***Answer***

To minimize search space by pruning unpromising paths

***Status :*** Correct ***Marks : 1/1***

14. Which value helps in deciding whether a node should be pruned in Branch and Bound?

***Answer***

Bound (lower bound of cost)

***Status :*** Correct ***Marks : 1/1***

15. Given items with weights [10, 20, 30] and values [60, 100, 120], what is the value-to-weight ratio for the second item?

***Answer***

5

***Status :*** Correct ***Marks : 1/1***

vtu28682

# Vel Tech Group of Institutions

Name: NIKHIL  KUMAR
Email: vtu28682@veltech.edu.in
Roll no: vtu28682
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

## 2028_NeoColab_Design and Analysis of AlgorithmsS1L10

### VTU_DAA_Week 4_COD

Attempt : 1
Total Mark : 40
Marks Obtained : 24

### Section 1 : Coding

1. Problem Statement

You are a delivery manager who needs to plan the delivery route for a fleet of trucks to a set of distribution centers in a city. Each distribution center is at a different location, and the cost of traveling between centers is known.

Your goal is to find the shortest route that allows a truck to visit each distribution center exactly once and return to the starting point, using the Traveling Salesman Problem (TSP). Write a program to calculate the minimum cost of this route and output the optimal path.

Note: Solve using Branch and Bound approach.

***Input Format***

The first line contains an integer n, the number of distribution centers.

The next n lines contain n space-separated integers each, representing the adjacency matrix.

The ith integer on the jth line represents the cost of traveling from distribution center i to distribution center j.

***Output Format***

The first line prints:

Minimum Cost is: X

The second line prints:

Optimal Path: followed by the sequence of visited centers ending at the start node

Refer to the sample output for the formatting specifications.

***Sample Test Case***

Input: 4
0 3 9 5
3 0 6 4
9 6 0 7
5 4 7 0

Output: Minimum Cost is: 21
Optimal Path: 0 1 2 3 0

***Answer***

```
#include <stdio.h>
#include <limits.h>

#define MAX 10
```

vtu28682

```
int n;
int cost[MAX][MAX];
int visited[MAX];

int final_path[MAX];
int curr_path[MAX];

int min_cost = INT_MAX;

void copy_path() {
    for (int i = 0; i < n; i++) {
        final_path[i] = curr_path[i];
    }
    final_path[n] = curr_path[0];
}

int firstMin(int i) {
    int min = INT_MAX;
    for (int k = 0; k < n; k++)
        if (cost[i][k] < min && i != k)
            min = cost[i][k];
    return min;
}

int secondMin(int i) {
    int first = INT_MAX, second = INT_MAX;
    for (int j = 0; j < n; j++) {
        if (i == j) continue;
        if (cost[i][j] <= first) {
            second = first;
            first = cost[i][j];
        } else if (cost[i][j] <= second && cost[i][j] != first) {
            second = cost[i][j];
        }
    }
    return second;
}

void TSPRec(int curr_bound, int curr_weight, int level) {

    if (level == n) {
```

```
    if (cost[curr_path[level - 1]][curr_path[0]] != 0) {
        int total_cost = curr_weight +
                    cost[curr_path[level - 1]][curr_path[0]];

        if (total_cost < min_cost) {
            min_cost = total_cost;
            copy_path();
        }
    }
    return;
}

for (int i = 0; i < n; i++) {

    if (cost[curr_path[level - 1]][i] != 0 && visited[i] == 0) {

        int temp = curr_bound;

        curr_weight += cost[curr_path[level - 1]][i];

        if (level == 1)
            curr_bound -= (firstMin(curr_path[level - 1]) +
                    firstMin(i)) / 2;
        else
            curr_bound -= (secondMin(curr_path[level - 1]) +
                    firstMin(i)) / 2;

        if (curr_bound + curr_weight < min_cost) {
            curr_path[level] = i;
            visited[i] = 1;

            TSPRec(curr_bound, curr_weight, level + 1);
        }

        curr_weight -= cost[curr_path[level - 1]][i];
        curr_bound = temp;

        for (int j = 0; j < n; j++)
            visited[j] = 0;

        for (int j = 0; j < level; j++)
            visited[curr_path[j]] = 1;
```

```
        }
    }
}

void TSP() {

    int curr_bound = 0;

    for (int i = 0; i < n; i++)
        curr_bound += (firstMin(i) + secondMin(i));

    curr_bound = (curr_bound & 1) ? curr_bound / 2 + 1 : curr_bound / 2;

    visited[0] = 1;
    curr_path[0] = 0;

    TSPRec(curr_bound, 0, 1);

    printf("Minimum Cost is: %d\n", min_cost);
    printf("Optimal Path: ");

    for (int i = 0; i <= n; i++) {
        printf("%d", final_path[i]);
        if (i < n) printf(" ");
    }
}

int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &cost[i][j]);
        }
    }

    TSP();

    return 0;
}
```

***Status :*** Partially correct ***Marks : 2/10***

2. Problem Statement

You are an art collector who wants to visit a set of art galleries in a city. Each gallery is at a different location, and the cost of traveling between galleries is known.

Your goal is to find the shortest route that allows you to visit each gallery exactly once and return to your starting point, using the traveling salesman problem (TSP). Write a program to calculate the minimum cost of this route.

Note: Solve it using Branch and Bound technique.

***Input Format***

The first line contains an integer n, the number of galleries.

The next n lines contain n integers each, representing the adjacency matrix.

The ith integer on the jth line represents the cost of traveling from gallery i to gallery j.

***Output Format***

The output displays "Minimum Cost is: " followed by the minimum cost of the route that visits each gallery exactly once and returns to the starting gallery.

Refer to the sample output for the formatting specifications.

***Sample Test Case***

Input: 4
0 10 15 20
10 0 35 25
15 35 0 30
20 25 30 0

Output: Minimum Cost is: 80

***Answer***

// You are using GCC

vtu28682

```
#include <stdio.h>
#include <limits.h>

#define MAX 10

int n;
int cost[MAX][MAX];
int visited[MAX];

int final_cost = INT_MAX;

int firstMin(int i) {
    int min = INT_MAX;
    for (int j = 0; j < n; j++)
        if (i != j && cost[i][j] < min)
            min = cost[i][j];
    return min;
}

int secondMin(int i) {
    int first = INT_MAX, second = INT_MAX;

    for (int j = 0; j < n; j++) {
        if (i == j) continue;

        if (cost[i][j] <= first) {
            second = first;
            first = cost[i][j];
        } else if (cost[i][j] <= second) {
            second = cost[i][j];
        }
    }
    return second;
}

void TSPRec(int curr_bound, int curr_weight, int level, int curr_path[]) {

    if (level == n) {
        if (cost[curr_path[level - 1]][curr_path[0]] != 0) {
            int total_cost = curr_weight +
                            cost[curr_path[level - 1]][curr_path[0]];
```

```
        if (total_cost < final_cost)
            final_cost = total_cost;
    }
    return;
  }

  for (int i = 0; i < n; i++) {

    if (cost[curr_path[level - 1]][i] != 0 && visited[i] == 0) {

        int temp_bound = curr_bound;
        int temp_weight = curr_weight;

        curr_weight += cost[curr_path[level - 1]][i];

        if (level == 1)
            curr_bound -= (firstMin(curr_path[level - 1]) + firstMin(i)) / 2;
        else
            curr_bound -= (secondMin(curr_path[level - 1]) + firstMin(i)) / 2;

        if (curr_bound + curr_weight < final_cost) {

            curr_path[level] = i;
            visited[i] = 1;

            TSPRec(curr_bound, curr_weight, level + 1, curr_path);
        }

        curr_weight = temp_weight;
        curr_bound = temp_bound;

        for (int j = 0; j < n; j++)
            visited[j] = 0;

        for (int j = 0; j < level; j++)
            visited[curr_path[j]] = 1;
    }
  }
}

void TSP() {
```

```
    int curr_path[MAX];

    int curr_bound = 0;

    for (int i = 0; i < n; i++)
        curr_bound += (firstMin(i) + secondMin(i));

    curr_bound = (curr_bound & 1) ? curr_bound / 2 + 1 : curr_bound / 2;

    visited[0] = 1;
    curr_path[0] = 0;

    TSPRec(curr_bound, 0, 1, curr_path);

    printf("Minimum Cost is: %d", final_cost);
}

int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &cost[i][j]);

    TSP();

    return 0;
}
```

***Status :*** Partially correct ***Marks : 2/10***

3. Problem Statement

Sarah, a young entrepreneur starting her e-commerce business, has a limited budget for storing products in her warehouse. Each product has a specific weight and value, and Sarah aims to maximize the total value of the products in her warehouse without exceeding the weight capacity.

To achieve this, Sarah decided to apply the dynamic programming technique, which helps explore potential product combinations and avoid

unnecessary calculations.

***Input Format***

The first line contains a single integer N, representing the number of products.

The next N lines each contain two integers: weight[i] and value[i], where weight[i] is the weight of the product and value[i] is the value (profit) of the product.

The next line contains a single integer W, representing the weight capacity of the warehouse.

***Output Format***

The output prints a single integer, representing the maximum total profit.

Refer to the sample input and output for formatting specifications.

***Sample Test Case***

Input: 5
2 40
3 50
1 100
5 95
3 30
10
Output: 245

***Answer***

```
// You are using GCC
#include <stdio.h>

int max(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    int n;
    scanf("%d", &n);
```

```
    int wt[n], val[n];

    for (int i = 0; i < n; i++) {
        scanf("%d %d", &wt[i], &val[i]);
    }

    int W;
    scanf("%d", &W);

    int dp[n + 1][W + 1];

    for (int i = 0; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            if (i == 0 || w == 0)
                dp[i][w] = 0;

            else if (wt[i - 1] <= w)
                dp[i][w] = max(val[i - 1] + dp[i - 1][w - wt[i - 1]],
                               dp[i - 1][w]);

            else
                dp[i][w] = dp[i - 1][w];
        }
    }

    printf("%d", dp[n][W]);

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

4. Problem Statement

You are a savvy thief preparing to rob a jewelry store. You have a knapsack with a limited capacity and a list of items, each with a certain weight and a corresponding profit.

Your objective is to maximize the total profit you can obtain by selecting items to place in your knapsack while staying within its weight capacity.

Write a program to help you calculate the maximum profit you can achieve.

Note: Use the 0/1 knapsack method to solve the program.

Example

Input:

3

60 10

100 20

120 30

50

Output:

220

Explanation:

By selecting the second and third items, you achieve a maximum profit of Rs. 220. The sum of their profits (100 + 120) gives you the highest total profit while not exceeding the knapsack's weight capacity of 50.

***Input Format***

The first line contains an integer N, representing the number of items available in the store.

Each of the next N lines contains two space-separated integers P and W, where P is the profit that can be obtained by stealing the item and W is the weight of the item.

The last line contains an integer C representing the weight capacity of your knapsack.

***Output Format***

The output displays an integer representing the maximum total profit that you can achieve by selecting items for your knapsack.

vtu28682

Refer to the sample output for the formatting specifications

***Sample Test Case***

Input: 3
60 10
100 20
120 30
50

Output: 220

***Answer***

```
// You are using GCC
#include <stdio.h>

int max(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    int n;
    scanf("%d", &n);

    int profit[n], weight[n];

    for (int i = 0; i < n; i++) {
        scanf("%d %d", &profit[i], &weight[i]);
    }

    int C;
    scanf("%d", &C);

    int dp[n + 1][C + 1];

    for (int i = 0; i <= n; i++) {
        for (int w = 0; w <= C; w++) {

            if (i == 0 || w == 0)
                dp[i][w] = 0;

            else if (weight[i - 1] <= w)
```

```
            dp[i][w] = max(
                profit[i - 1] + dp[i - 1][w - weight[i - 1]],
                dp[i - 1][w]
            );

        else
            dp[i][w] = dp[i - 1][w];
    }
}

printf("%d", dp[n][C]);

return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

# Vel Tech Group of Institutions

Name: NIKHIL  KUMAR
Email: vtu28682@veltech.edu.in
Roll no: vtu28682
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

## 2028_NeoColab_Design and Analysis of AlgorithmsS1L10

### VTU_DAA_Week 4_CY

Attempt : 1
Total Mark : 50
Marks Obtained : 40

### Section 1 : Coding

1. Problem Statement

Meera is preparing for a science exhibition and needs to select a group of items whose total weight is exactly equal to a required target value. Each item can be selected at most once.

Given a list of item weights and a target weight K, determine whether it is possible to select a subset of items such that their sum is exactly K.

If such a subset exists, print any one valid subset.If no such subset exists, print -1.

This problem is derived from the Subset Sum problem, which is NP-Complete, and the solution must be implemented using Dynamic Programming.

***Input Format***

vtu28682

The first line contains two integers n and K

The second line contains n space-separated integers

***Output Format***

Print elements of one valid subset whose sum equals K

If no such subset exists, print -1

Refer to the sample output for the formatting specifications.

***Sample Test Case***

Input: 5 9
3 34 4 12 5

Output: 5 4

***Answer***

```
// You are using GCC
#include <stdio.h>

int main() {
    int n, K;
    scanf("%d %d", &n, &K);

    int arr[n];

    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    int dp[n + 1][K + 1];

    // DP table
    for (int i = 0; i <= n; i++) {
        for (int j = 0; j <= K; j++) {

            if (j == 0)
                dp[i][j] = 1;
            else if (i == 0)
                dp[i][j] = 0;
```

```
        else if (arr[i - 1] <= j)
            dp[i][j] = dp[i - 1][j] || dp[i - 1][j - arr[i - 1]];
        else
            dp[i][j] = dp[i - 1][j];
    }
}

if (!dp[n][K]) {
    printf("-1");
    return 0;
}

// Backtracking to find subset
int i = n, j = K;
int result[n], idx = 0;

while (i > 0 && j > 0) {
    if (dp[i][j] && !dp[i - 1][j]) {
        result[idx++] = arr[i - 1];
        j -= arr[i - 1];
    }
    i--;
}

for (int k = 0; k < idx; k++)
    printf("%d ", result[k]);

return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

2. Problem Statement

Rohit is a travel planner who must organize a business trip covering multiple cities.

The travel cost between each pair of cities is given in a matrix.

He must:

Start from city 0Visit every city exactly onceReturn back to city 0Your task is to compute:

The minimum possible travel costThe sequence of cities visitedThe average cost per step (excluding the final return step)

This problem is a classic Travelling Salesman Problem (TSP) solved using

Dynamic Programming with Bitmasking (Held–Karp Algorithm).

***Input Format***

- The first line contains an integer n, the number of cities.
- The next n lines contain n space-separated integers, representing the cost matrix.

cost[i][j] is the cost of traveling from city i to city j.

***Output Format***

Print the path:

The Path is: 0--->...--->0

Print the minimum cost:

Minimum cost is X

Print the average cost per step:

Average cost per step is Y

Rounded to 2 decimal places.

vtu28682

Refer to the sample output for the formatting specifications.

***Sample Test Case***

Input: 4
0 10 15 20
10 0 35 25
15 35 0 30
20 25 30 0

Output: The Path is: 0--->2--->3--->1--->0
Minimum cost is 80
Average cost per step is 26.67

***Answer***

```
#include <stdio.h>
#include <limits.h>
#include <string.h>

int n;
int cost[20][20];
int dp[20][1 << 20];
int parent[20][1 << 20];

int tsp(int pos, int mask) {
    if (mask == (1 << n) - 1) {
        return cost[pos][0];
    }

    if (dp[pos][mask] != -1)
        return dp[pos][mask];

    int ans = INT_MAX;
    int bestCity = -1;

    for (int city = 0; city < n; city++) {
        if (!(mask & (1 << city))) {
            int newCost = cost[pos][city] + tsp(city, mask | (1 << city));

            if (newCost < ans) {
                ans = newCost;
                bestCity = city;
```

```
        }
    }
    }

    parent[pos][mask] = bestCity;
    return dp[pos][mask] = ans;
}

int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &cost[i][j]);

    memset(dp, -1, sizeof(dp));

    int minCost = tsp(0, 1);

    printf("The Path is: 0");

    int mask = 1, pos = 0;

    for (int i = 0; i < n - 1; i++) {
        int next = parent[pos][mask];
        printf("--->%d", next);
        mask |= (1 << next);
        pos = next;
    }

    printf("--->0\n");

    printf("Minimum cost is %d\n", minCost);

    double avg = (double)minCost / n;
    printf("Average cost per step is %.2f\n", avg);

    return 0;
}
```

***Status :*** Wrong ***Marks : 0/10***

3. Problem Statement

Captain Aidan, a daring pirate, has discovered an ancient treasure cave filled with valuable relics. However, the cave is protected by an ancient spell that limits the weight of the treasure he can carry. Aidan has a magic knapsack with a maximum weight capacity of W. Inside the cave, he finds n different treasures, each with a specific weight and value.

Since he can only take each treasure once, Aidan must carefully choose which items to take to maximize the total value without exceeding the weight limit of his knapsack.

Help Captain Aidan pick the most valuable treasures while staying within the weight limit using branch and bound technique

***Input Format***

The first line contains two integers n (the number of items) and W (the capacity of the knapsack), separated by a space.

The second line contains n integers, where the i-th integer represents the weight of the i-th item.

The third line contains n integers, where the i-th integer represents the value of the i-th item

***Output Format***

The output prints the single integer representing the maximum value obtained by filling the knapsack with the given set of items.

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 5 60
10 20 30 40 50
60 100 120 140 180
Output: 280

vtu28682

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int weight;
    int value;
    float ratio;
} Item;

typedef struct {
    int level;
    int profit;
    int weight;
    float bound;
} Node;

int n, W;
Item items[105];

int cmp(const void *a, const void *b) {
    Item *i1 = (Item *)a;
    Item *i2 = (Item *)b;
    if (i2->ratio > i1->ratio) return 1;
    return -1;
}

float bound(Node u) {
    if (u.weight >= W)
        return 0;

    float profit_bound = u.profit;
    int j = u.level + 1;
    int totalWeight = u.weight;

    while (j < n && totalWeight + items[j].weight <= W) {
        totalWeight += items[j].weight;
        profit_bound += items[j].value;
        j++;
    }
```

```
    if (j < n) {
        profit_bound += (W - totalWeight) * items[j].ratio;
    }

    return profit_bound;
}

int max(int a, int b) {
    return (a > b) ? a : b;
}

int knapsack() {
    Node Q[10000];
    int front = 0, rear = 0;

    Node u, v;

    v.level = -1;
    v.profit = 0;
    v.weight = 0;
    v.bound = bound(v);

    Q[rear++] = v;

    int maxProfit = 0;

    while (front < rear) {
        v = Q[front++];

        if (v.bound > maxProfit) {
            u.level = v.level + 1;

            if (u.level < n) {
                // TAKE ITEM
                u.weight = v.weight + items[u.level].weight;
                u.profit = v.profit + items[u.level].value;

                if (u.weight <= W && u.profit > maxProfit)
                    maxProfit = u.profit;

                u.bound = bound(u);
                if (u.bound > maxProfit)
```

```
                Q[rear++] = u;

            // NOT TAKE ITEM
            u.weight = v.weight;
            u.profit = v.profit;
            u.bound = bound(u);

            if (u.bound > maxProfit)
                Q[rear++] = u;
        }
    }
}

    return maxProfit;
}

int main() {
    scanf("%d %d", &n, &W);

    int w[105], val[105];

    for (int i = 0; i < n; i++)
        scanf("%d", &w[i]);

    for (int i = 0; i < n; i++)
        scanf("%d", &val[i]);

    for (int i = 0; i < n; i++) {
        items[i].weight = w[i];
        items[i].value = val[i];
        items[i].ratio = (float)val[i] / w[i];
    }

    qsort(items, n, sizeof(Item), cmp);

    printf("%d", knapsack());

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

4. Problem Statement

Josh is learning about the Fractional Knapsack problem, where a knapsack has a fixed maximum capacity. Given a set of items, each with a value and a weight, Josh wants to maximize the total value placed into the knapsack.

Unlike the 0/1 Knapsack problem, fractions of items are allowed. Items must be selected using a Greedy strategy based on their value-to-weight ratio.

Write a program to determine the fraction of each item that should be included in the knapsack to achieve the maximum total value.

Formula

Value-to-Weight Ratio

ratio = value / weight

Fraction Calculation

fraction = remainingCapacity / weight

Notes

Items must be selected using the Greedy approach based on value-to-weight ratio.Sorting is required to determine the optimal order.Output fractions must be printed in the original item order.Use double datatype only for calculations.

***Input Format***

The first line contains an integer n, representing the number of items.

The second line contains n space-separated integers representing item values.

The third line contains n space-separated integers representing item weights.

The fourth line contains an integer capacity, representing knapsack capacity.

***Output Format***

Print n space-separated double values, representing the fraction of each item selected.

vtu28682

Output must be printed rounded to exactly two decimal places.

Refer to the sample output for the formatting specifications.

***Sample Test Case***

Input: 5
3 5 1 2 4
40 50 20 10 30
75
Output: 0.00 0.70 0.00 1.00 1.00

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int index;
    double value;
    double weight;
    double ratio;
} Item;

int cmp(const void *a, const void *b) {
    Item *i1 = (Item *)a;
    Item *i2 = (Item *)b;

    if (i2->ratio > i1->ratio) return 1;
    else if (i2->ratio < i1->ratio) return -1;
    return 0;
}

int main() {
    int n;
    scanf("%d", &n);

    double value[10], weight[10];
    Item items[10];
```

```
double fraction[10];

for (int i = 0; i < n; i++)
    scanf("%lf", &value[i]);

for (int i = 0; i < n; i++)
    scanf("%lf", &weight[i]);

double capacity;
scanf("%lf", &capacity);

// Build items with ratio
for (int i = 0; i < n; i++) {
    items[i].index = i;
    items[i].value = value[i];
    items[i].weight = weight[i];
    items[i].ratio = value[i] / weight[i];
}

// Sort by ratio descending
qsort(items, n, sizeof(Item), cmp);

for (int i = 0; i < n; i++)
    fraction[i] = 0.0;

double remaining = capacity;

for (int i = 0; i < n; i++) {
    if (remaining <= 0)
        break;

    if (items[i].weight <= remaining) {
        fraction[items[i].index] = 1.0;
        remaining -= items[i].weight;
    } else {
        fraction[items[i].index] = remaining / items[i].weight;
        remaining = 0;
    }
}

for (int i = 0; i < n; i++) {
    printf("%.2f", fraction[i]);
```

```
    if (i != n - 1)
        printf(" ");
  }

  return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

5. Problem Statement

Ananya is preparing for a long trek and has a bag with limited capacity. She has several items, each with a weight and value, and she wants to maximize the total value she can carry. She may take whole items or fractions of them to fit in her bag.

The maximum value is calculated using the formula:

Maximum Value = Σ (value[i] * fraction_taken[i])

where fraction_taken[i] = min(1, remaining_capacity / weight[i])

Write a program to help Ananya determine the maximum achievable value.

***Input Format***

The input consists of several lines, each representing an item.

Each line (until -1) contains two integers: weight and value of an item.

The input terminates with -1.

The final line contains one integer representing the bag's capacity.

***Output Format***

The output prints "The maximum value of the current list is:"

On the next line, print the maximum value achievable with exactly two decimal places

Refer to the sample outputs for the formatting specifications.

***Sample Test Case***

Input: 10 60
20 100
30 120
-1
50

Output: The maximum value of the current list is:
240.00

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    double weight;
    double value;
    double ratio;
} Item;

int cmp(const void *a, const void *b) {
    Item *i1 = (Item *)a;
    Item *i2 = (Item *)b;

    if (i2->ratio > i1->ratio) return 1;
    else if (i2->ratio < i1->ratio) return -1;
    return 0;
}

int main() {
    Item items[1000];
    int n = 0;

    while (1) {
        double w;
        scanf("%lf", &w);

        if (w == -1)
```

```
        break;

        double v;
        scanf("%lf", &v);

        items[n].weight = w;
        items[n].value = v;
        items[n].ratio = v / w;
        n++;
    }

    double capacity;
    scanf("%lf", &capacity);

    qsort(items, n, sizeof(Item), cmp);

    double totalValue = 0.0;

    for (int i = 0; i < n && capacity > 0; i++) {
        if (items[i].weight <= capacity) {
            totalValue += items[i].value;
            capacity -= items[i].weight;
        } else {
            totalValue += items[i].ratio * capacity;
            capacity = 0;
        }
    }

    printf("The maximum value of the current list is:\n");
    printf("%.2f", totalValue);

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

# Vel Tech Group of Institutions

Name: NIKHIL  KUMAR
Email: vtu28682@veltech.edu.in
Roll no: vtu28682
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

## 2028_NeoColab_Design and Analysis of AlgorithmsS1L10

### VTU_DAA_Week 4_PAH

Attempt : 1
Total Mark : 50
Marks Obtained : 50

### Section 1 : Coding

1. Problem Statement

Janu is tasked with creating a program that solves the Traveling Salesman Problem (TSP). The goal is to find the shortest possible route that visits a set of cities and returns to the starting city.

You are required to implement a program that takes input regarding the number of cities and the distance matrix between cities and outputs the optimal route and its associated minimum cost.

The diagonal values will be 0, as the distance from a city to itself is 0.

Note: Solve it using Branch and Bound approach.

***Input Format***

vtu28682

The first line of input consists of an integer n, representing the number of cities.

The next n lines contain n space-separated integers each, representing the distance matrix between the cities.

The cell (i, j) represents the distance from city i to city j.

***Output Format***

The first line of output displays "The Path is: " followed by the path of the route using ---> in between.

The second line displays "Minimum cost is " followed by the total minimum cost associated with the shortest route.

Refer to the sample output for the formatting specifications.

***Sample Test Case***

Input: 4
0 4 1 3
4 0 2 1
1 2 0 5
3 1 5 0
Output: The Path is: 1--->3--->2--->4--->1
Minimum cost is 7

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <limits.h>

#define MAX 20

int n;
int cost[MAX][MAX];

int bestCost = INT_MAX;
int bestPath[MAX];

int currPath[MAX];
```

```
int visited[MAX];

int firstMin(int i) {
    int min = INT_MAX;
    for (int j = 0; j < n; j++) {
        if (i != j && cost[i][j] < min)
            min = cost[i][j];
    }
    return min;
}

int secondMin(int i) {
    int first = INT_MAX, second = INT_MAX;

    for (int j = 0; j < n; j++) {
        if (i == j) continue;

        if (cost[i][j] <= first) {
            second = first;
            first = cost[i][j];
        } else if (cost[i][j] <= second) {
            second = cost[i][j];
        }
    }
    return second;
}

void tsp(int level, int currCost, int bound) {
    if (level == n) {
        if (cost[currPath[level - 1]][currPath[0]] != 0) {
            int totalCost = currCost + cost[currPath[level - 1]][currPath[0]];

            if (totalCost < bestCost) {
                bestCost = totalCost;
                for (int i = 0; i < n; i++)
                    bestPath[i] = currPath[i];
            }
        }
        return;
    }

    for (int i = 0; i < n; i++) {
```

```
        if (!visited[i] && cost[currPath[level - 1]][i]) {

            int temp = bound;
            currCost += cost[currPath[level - 1]][i];

            if (level == 1)
                bound -= (firstMin(currPath[level - 1]) + firstMin(i)) / 2;
            else
                bound -= (secondMin(currPath[level - 1]) + firstMin(i)) / 2;

            if (bound + currCost < bestCost) {
                currPath[level] = i;
                visited[i] = 1;

                tsp(level + 1, currCost, bound);
            }

            currCost -= cost[currPath[level - 1]][i];
            bound = temp;

            for (int j = 0; j < n; j++)
                visited[j] = 0;

            for (int j = 0; j < level; j++)
                visited[currPath[j]] = 1;
        }
    }
}

int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &cost[i][j]);

    for (int i = 0; i < n; i++)
        visited[i] = 0;

    int bound = 0;

    for (int i = 0; i < n; i++)
```

```
        bound += (firstMin(i) + secondMin(i));

    bound = (bound % 2 == 0) ? bound / 2 : bound / 2 + 1;

    visited[0] = 1;
    currPath[0] = 0;

    tsp(1, 0, bound);

    printf("The Path is: ");

    for (int i = 0; i < n; i++)
        printf("%d--->", bestPath[i] + 1);

    printf("1\n");

    printf("Minimum cost is %d", bestCost);

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

2. Problem Statement

Karan is planning a delivery route that starts from city 0, visits every other city exactly once, and returns back to city 0.

The travel costs between cities are given as a cost matrix.

Your task is to find the minimum possible travel cost and print the optimal route.

Cost Formula

For a route:

0 a b ... 0

Total Cost = cost[ 0 ] [ a ] + cost[ a ] [ b ] + ... + cost[ last ] [ 0 ]

***Input Format***

The first line contains an integer n, representing the number of cities.

The next n lines contain n space-separated integers representing the cost matrix.

***Output Format***

The output prints multiple lines showing path exploration in the format:

The first line prints the minimum travel cost.

The second line prints the optimal path starting from city 0 and ending at city 0.

Refer to the sample output for the formatting specifications.

***Sample Test Case***

Input: 4
0 10 15 20
10 0 35 25
15 35 0 30
20 25 30 0

Output: 80
0 1 3 2 0

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <limits.h>

int n;
int cost[10][10];
int visited[10];
```

```
int bestCost = INT_MAX;
int bestPath[10];
int currPath[10];

void tsp(int level, int currCost) {
    if (level == n) {
        int totalCost = currCost + cost[currPath[level - 1]][0];

        if (totalCost < bestCost) {
            bestCost = totalCost;
            for (int i = 0; i < n; i++)
                bestPath[i] = currPath[i];
        }
        return;
    }

    for (int i = 1; i < n; i++) {
        if (!visited[i]) {
            visited[i] = 1;
            currPath[level] = i;

            tsp(level + 1, currCost + cost[currPath[level - 1]][i]);

            visited[i] = 0;
        }
    }
}

int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &cost[i][j]);

    visited[0] = 1;
    currPath[0] = 0;

    tsp(1, 0);

    printf("%d\n", bestCost);
```

```
    for (int i = 0; i < n; i++)
        printf("%d ", bestPath[i]);

    printf("0");

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

3. Problem Statement

You are a logistics manager responsible for planning delivery routes for a fleet of vehicles. Each vehicle needs to visit a set of locations to deliver goods, and the cost of traveling between locations is known.

Your goal is to find the shortest route for each vehicle, ensuring that each location is visited exactly once before returning to the starting point.

Note: Solve it using Branch and Bound approach.

***Input Format***

The first line contains an integer n, the number of locations.

The next n lines contain n integers each, representing the cost matrix of traveling between locations.

The ith integer on the jth line represents the cost of traveling from location i to location j.

***Output Format***

The first line of output displays "Shortest Path: " followed by the shortest path that visits all cities.

The second line of output displays "Minimum cost is " followed by the minimum cost.

Refer to the sample output for the formatting specifications.

***Sample Test Case***

Input: 5
0 10 8 9 7
10 0 10 5 6
8 10 0 8 9
9 5 8 0 6
7 6 9 6 0

Output: Shortest Path: 1 3 4 2 5 1
Minimum cost is 34

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <limits.h>
#include <string.h>

#define MAX 20

int n;
int cost[MAX][MAX];

int bestCost = INT_MAX;
int finalPath[MAX];

int currPath[MAX];
int visited[MAX];

int firstMin(int i) {
    int min = INT_MAX;
    for (int j = 0; j < n; j++) {
        if (i != j && cost[i][j] < min)
            min = cost[i][j];
    }
    return min;
}

int secondMin(int i) {
    int first = INT_MAX, second = INT_MAX;

    for (int j = 0; j < n; j++) {
```

```
        if (i == j) continue;

        if (cost[i][j] <= first) {
            second = first;
            first = cost[i][j];
        } else if (cost[i][j] <= second) {
            second = cost[i][j];
        }
    }
    return second;
}

void TSP(int level, int currCost, int bound) {
    if (level == n) {
        int totalCost = currCost + cost[currPath[level - 1]][currPath[0]];

        if (totalCost < bestCost) {
            bestCost = totalCost;
            for (int i = 0; i < n; i++)
                finalPath[i] = currPath[i];
        }
        return;
    }

    for (int i = 0; i < n; i++) {
        if (!visited[i]) {

            int tempBound = bound;

            currPath[level] = i;
            currCost += cost[currPath[level - 1]][i];

            if (level == 1)
                bound -= (firstMin(currPath[level - 1]) + firstMin(i)) / 2;
            else
                bound -= (secondMin(currPath[level - 1]) + firstMin(i)) / 2;

            if (bound + currCost < bestCost) {
                visited[i] = 1;
                TSP(level + 1, currCost, bound);
            }
```

```
            currCost -= cost[currPath[level - 1]][i];
            bound = tempBound;

            memset(visited, 0, sizeof(visited));
            for (int j = 0; j < level; j++)
                visited[currPath[j]] = 1;
        }
    }
}

int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &cost[i][j]);

    int bound = 0;

    for (int i = 0; i < n; i++)
        bound += (firstMin(i) + secondMin(i));

    bound = (bound % 2 == 0) ? bound / 2 : bound / 2 + 1;

    visited[0] = 1;
    currPath[0] = 0;

    TSP(1, 0, bound);

    printf("Shortest Path: ");

    for (int i = 0; i < n; i++)
        printf("%d ", finalPath[i] + 1);

    printf("1\n");

    printf("Minimum cost is %d", bestCost);

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

4. Problem Statement

Michael has been given an important task—developing a program to solve the classic 0/1 Knapsack Problem. His goal is to help users select the most valuable combination of items while staying within a specified weight limit.

In this problem, each item has a value and a weight, and the objective is to maximize the total value without exceeding the given weight capacity.

Michael's program should determine the maximum possible value that can be obtained by selecting the optimal set of items.

***Input Format***

The first line of input consists of an integer n, representing the number of items available for selection.

The second line of input consists of n space-separated integers, P, which represent the values of the items.

The third line of input consists of n space-separated integers, C, which represent the weights of the corresponding items.

The fourth line of input consists of an integer W, representing the maximum weight limit of the knapsack.

***Output Format***

The output displays the maximum value that can be obtained using the knapsack algorithm in the following format: "Maximum value: [maximum value]"

Refer to the sample outputs for the exact format.

***Sample Test Case***

Input: 6
300 150 120 100 90 80
6 3 3 2 2 2
10

vtu28682

Output: Maximum value: 490

***Answer***

```
// You are using GCC
#include <stdio.h>

int max(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    int n;
    scanf("%d", &n);

    int val[20], wt[20];
    for (int i = 0; i < n; i++)
        scanf("%d", &val[i]);

    for (int i = 0; i < n; i++)
        scanf("%d", &wt[i]);

    int W;
    scanf("%d", &W);

    int dp[1005] = {0};

    for (int i = 0; i < n; i++) {
        for (int w = W; w >= wt[i]; w--) {
            dp[w] = max(dp[w], val[i] + dp[w - wt[i]]);
        }
    }

    printf("Maximum value: %d", dp[W]);

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

5. Problem Statement

Ragul is interested in solving the knapsack problem. He has a list of n items, each with a weight and a value. Ragul wants to determine the maximum value he can obtain by selecting a combination of items to fit into his knapsack, which has a maximum weight capacity.

Help Ragul write a program that calculates the following:

Calculate and print the sum of all n values.Calculate and print the average value of all n values with exactly 2 decimal places.Calculate and print the maximum value that can be obtained by selecting a combination of items to fit into the knapsack without exceeding its capacity.

Note: Use the 0/1 knapsack method to solve the problem.

***Input Format***

The first line of input contains an integer n, representing the number of items.

The next line contains n space-separated integers, representing the weights of the items.

The next line contains n space-separated integers, representing the values of the items.

The last line contains an integer C, representing the maximum weight capacity of the knapsack.

***Output Format***

The output consists of the following in each line:

1. The sum of all n values as an integer.
2. The average value of all n values with exactly 2 decimal places is a double value.
3. The maximum value that can be obtained as an integer.

Refer to the sample output for the formatting specifications.

***Sample Test Case***

vtu28682

Input: 3
10 15 20
45 65 70
30

Output: Sum of values: 180
Average of values: 60.00
Maximum amount: 115

***Answer***

```
// You are using GCC
#include <stdio.h>

int max(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    int n;
    scanf("%d", &n);

    int wt[15], val[15];

    for(int i = 0; i < n; i++)
        scanf("%d", &wt[i]);

    int sum = 0;
    for(int i = 0; i < n; i++) {
        scanf("%d", &val[i]);
        sum += val[i];
    }

    int C;
    scanf("%d", &C);

    // average
    double avg = (double)sum / n;

    // DP knapsack
    int dp[31] = {0};

    for(int i = 0; i < n; i++) {
        for(int w = C; w >= wt[i]; w--) {
```

```
            dp[w] = max(dp[w], val[i] + dp[w - wt[i]]);
        }
    }

    printf("Sum of values: %d\n", sum);
    printf("Average of values: %.2f\n", avg);
    printf("Maximum amount: %d", dp[C]);

    return 0;
}
```

***Status :*** Correct **Marks : 10/10**

vtu28682

# Vel Tech Group of Institutions

Name: NIKHIL  KUMAR
Email: vtu28682@veltech.edu.in
Roll no: vtu28682
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

## 2028_NeoColab_Design and Analysis of AlgorithmsS1L10

### VTU_DAA_Week 5_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 15

### Section 1 : MCQ

1. In heap sort, when does the actual sorting take place?

***Answer***

During the swap and heapify phase

***Status :*** Correct ***Marks : 1/1***

2. In heap sort, which of the following heap operations is performed to build a valid heap from an unsorted array?

***Answer***

Heapify

***Status :*** Correct ***Marks : 1/1***

vtu28682

3. In heap sort, the element that is swapped with the final element of the heap is _____, in the case of a max-heap, sorted in descending order.

***Answer***

Largest element

***Status :*** Correct ***Marks : 1/1***

4. What is the worst-case time complexity of the heap sort algorithm?

***Answer***

O(n log n)

***Status :*** Correct ***Marks : 1/1***

5. In a quick sort algorithm, where are the smaller elements placed to the pivot during the partition process, assuming we are sorting in increasing order?

***Answer***

To the left of the pivot

***Status :*** Correct ***Marks : 1/1***

6. Which of the following is true about Quicksort?

***Answer***

It is an in-place sorting algorithm

***Status :*** Correct ***Marks : 1/1***

7. What is the worst-case time complexity of the Quicksort algorithm?

***Answer***

$O(n^2)$

***Status :*** Correct ***Marks : 1/1***

vtu28682

8. Given a sorted array of 1000 unique integers, how many maximum comparisons does Binary Search make in the worst case?

***Answer***

10

***Status :*** Correct ***Marks : 1/1***

9. Suppose you are using Binary Search on a sorted array of even length (e.g., 8 elements). At each step, you calculate the middle as:

mid = (low + high) // 2

Which of the following statements is always true?

***Answer***

The mid index may not divide the array into equal halves in all steps.

***Status :*** Correct ***Marks : 1/1***

10. You apply Binary Search on the sorted array [3, 7, 10, 15, 21, 30] to find the element 13. Which sequence of middle elements will Binary Search compare with before declaring that the element is not present?

***Answer***

10 21 15

***Status :*** Correct ***Marks : 1/1***

11. You are using Binary Search to find the first occurrence of a value x in a sorted array that may contain duplicates. Which of the following modifications is necessary?

***Answer***

When x == arr[mid], move high to mid - 1.

***Status :*** Correct ***Marks : 1/1***

vtu28682

12. A student implements Binary Search and calculates the middle index as:

mid = (low + high) // 2

But when the input values of low and high are both large (e.g., low = 2_000_000_000, high = 2_000_000_010), the program crashes due to integer overflow. What is the best fix?

***Answer***

Use mid = low + (high - low) // 2

***Status :*** Correct ***Marks : 1/1***

13. Is Merge Sort a stable sorting algorithm?

***Answer***

Yes, always stable.

***Status :*** Correct ***Marks : 1/1***

14. Merge sort is _____.

***Answer***

Comparison-based sorting algorithm

***Status :*** Correct ***Marks : 1/1***

15. Which of the following methods is used for sorting in merge sort?

***Answer***

merging

***Status :*** Correct ***Marks : 1/1***

vtu28682

# Vel Tech Group of Institutions

Name: NIKHIL  KUMAR
Email: vtu28682@veltech.edu.in
Roll no: vtu28682
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

## 2028_NeoColab_Design and Analysis of AlgorithmsS1L10

### VTU_DAA_Week 5_COD

Attempt : 1
Total Mark : 40
Marks Obtained : 40

### Section 1 : Coding

1. Problem Statement

Sam needs to sort an array of integers using the divide-and-conquer method. He wants to implement the merge sort algorithm, displaying the array after each iteration to track the sorting process.

Help him write a program that divides the array, merges it back, and prints the array.

***Input Format***

The first line of input consists of an integer n, denoting the array size.

The second line consists of n space-separated integers, representing the array of elements.

***Output Format***

vtu28682

The first line of output prints "Given Array" followed by a (String), indicating the start of the initial array display.

The second line of output prints the array elements (integers) separated by spaces.

After each merge iteration, the output prints "After iteration X", where X is the iteration number (starting from 1). This is followed by the (String) "After iteration X" to indicate the iteration count.

On the next line, the current state of the array after that iteration is printed, showing the array elements (integers) separated by spaces.

Finally, after all iterations are completed, the output prints "Sorted Array" followed by a (String), indicating the sorted array.

The final line prints the sorted array (integers), with each element separated by spaces.

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 6
4 1 5 3 2 6

Output: Given Array
4 1 5 3 2 6
After iteration 1
1 4 5 3 2 6
After iteration 2
1 4 5 3 2 6
After iteration 3
1 4 5 2 3 6
After iteration 4

vtu28682

1 4 5 2 3 6
After iteration 5
1 2 3 4 5 6
Sorted Array
1 2 3 4 5 6

***Answer***

```
// You are using GCC
#include <stdio.h>

int arr[20];
int n;
int iteration = 1;

// Function to print array
void printArray() {
    for(int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

// Merge function
void merge(int left, int mid, int right) {
    int temp[20];
    int i = left, j = mid + 1, k = left;

    while(i <= mid && j <= right) {
        if(arr[i] <= arr[j]) {
            temp[k++] = arr[i++];
        } else {
            temp[k++] = arr[j++];
        }
    }

    while(i <= mid) {
        temp[k++] = arr[i++];
    }

    while(j <= right) {
        temp[k++] = arr[j++];
    }
```

```
    // Copy back to original array
    for(int p = left; p <= right; p++) {
        arr[p] = temp[p];
    }

    // Print after each merge
    printf("After iteration %d\n", iteration++);
    printArray();
}

// Merge Sort function
void mergeSort(int left, int right) {
    if(left < right) {
        int mid = (left + right) / 2;

        mergeSort(left, mid);
        mergeSort(mid + 1, right);

        merge(left, mid, right);
    }
}

int main() {
    scanf("%d", &n);

    for(int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    printf("Given Array\n");
    printArray();

    mergeSort(0, n - 1);

    printf("Sorted Array\n");
    printArray();

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

2. Problem Statement

Micheal is tasked with developing a system to manage and sort student records for a university. Each student record contains the student's name, GPA, age, and major. The goal is to implement a program that allows the user to input multiple student records and then sort these records based on a chosen criterion: GPA, age, or major.

Can you guide Micheal in developing the program using a quick sort algorithm?

***Input Format***

The first line of input consists of an integer n, representing the number of student records.

For each student record, the following lines consist of:

- Name (string)
- GPA (float)
- Age (integer)
- Major (string)

The last line of input consists of the sorting criterion, which can be one of the following strings:

- "gpa"
- "age"
- "major"

***Output Format***

The first line of output will print:

Sorted Student Records:

The output should display a sorted list of student records in a tabular format based on the chosen criterion. The GPA value should be rounded off to two decimal places.

Table Header

"Name\t\tGPA\t\tAge\t\tMajor"

vtu28682

Table Content

For each student, the format of the output will be:

"Name\tGPA\tAge\t\tMajor"

- The values should be aligned as follows:
- Name: Left-aligned, up to 10 characters.
- GPA: Displayed with two decimal places.
- Age: Left-aligned, up to 3 digits.
- Major: Left-aligned, up to 15 characters.

Note: "\t" - It creates a horizontal tab space

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 4
Harish
7.8
23
ece
Trell
8
21
cse
Kamalesh
7
25
it
Maaran
5.6
20
civil
age

vtu28682

Output: Sorted Student Records:
Name GPA Age Major
Maaran 5.60 20 civil
Trell 8.00 21 cse
Harish 7.80 23 ece
Kamalesh 7.00 25 it

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <string.h>

#define MAX 10

struct Student {
    char name[50];
    float gpa;
    int age;
    char major[50];
};

struct Student s[MAX];
char criterion[10];

// Swap function
void swap(struct Student *a, struct Student *b) {
    struct Student temp = *a;
    *a = *b;
    *b = temp;
}

// Comparison function
int compare(struct Student a, struct Student b) {
    if (strcmp(criterion, "gpa") == 0) {
        return a.gpa > b.gpa;
    }
    else if (strcmp(criterion, "age") == 0) {
        return a.age > b.age;
    }
    else if (strcmp(criterion, "major") == 0) {
        return strcmp(a.major, b.major) > 0;
    }
```

```
    return 0;
}

// Partition function
int partition(int low, int high) {
    struct Student pivot = s[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (!compare(s[j], pivot)) {
            i++;
            swap(&s[i], &s[j]);
        }
    }
    swap(&s[i + 1], &s[high]);
    return i + 1;
}

// Quick Sort
void quickSort(int low, int high) {
    if (low < high) {
        int pi = partition(low, high);
        quickSort(low, pi - 1);
        quickSort(pi + 1, high);
    }
}

int main() {
    int n;
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf("%s", s[i].name);
        scanf("%f", &s[i].gpa);
        scanf("%d", &s[i].age);
        scanf("%s", s[i].major);
    }

    scanf("%s", criterion);

    quickSort(0, n - 1);
```

```
    // Output
    printf("Sorted Student Records:\n");
    printf("Name\t\tGPA\t\tAge\t\tMajor\n");

    for (int i = 0; i < n; i++) {
        printf("%-10s\t%.2f\t%d \t\t%-15s\n",
               s[i].name,
               s[i].gpa,
               s[i].age,
               s[i].major);
    }

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

3. Problem Statement

Ragavi, a computer scientist, is working on efficient text processing for search engines. She has a list of words in random order and needs to sort them in lexicographic order using Heap Sort to improve search performance.

Help Ragavi implement Heap Sort to arrange the words in lexicographic order efficiently.

***Input Format***

The first line consists of an integer N denoting the number of strings.

The next line consists of N space-separated strings.

***Output Format***

The output displays N space-separated sorted strings in lexicographic order.

Refer to the sample output for the formatting specifications.

***Sample Test Case***

Input: 10
banana pineapple mango chikoo jack star guava lemon tomato orange
Output: banana chikoo guava jack lemon mango orange pineapple star tomato

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <string.h>

#define MAX 20
#define LEN 50

char arr[MAX][LEN];
int n;

// Swap two strings
void swap(char a[], char b[]) {
    char temp[LEN];
    strcpy(temp, a);
    strcpy(a, b);
    strcpy(b, temp);
}

// Heapify function
void heapify(int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    // Compare left child
    if (left < n && strcmp(arr[left], arr[largest]) > 0)
        largest = left;

    // Compare right child
    if (right < n && strcmp(arr[right], arr[largest]) > 0)
        largest = right;

    // Swap and continue heapifying
    if (largest != i) {
        swap(arr[i], arr[largest]);
```

```
        heapify(n, largest);
    }
}

// Heap Sort
void heapSort() {
    // Build max heap
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(n, i);

    // Extract elements
    for (int i = n - 1; i > 0; i--) {
        swap(arr[0], arr[i]);
        heapify(i, 0);
    }
}

int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf("%s", arr[i]);
    }

    heapSort();

    // Print sorted strings
    for (int i = 0; i < n; i++) {
        printf("%s ", arr[i]);
    }

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

4. Problem Statement

Imagine you are working as a librarian in a large library. The library uses an automated system to keep track of book positions on the shelves. Each book is assigned a unique ID, but some IDs are more common than others

due to the popularity of certain editions.

The system logs the positions of books in a sorted list based on their IDs. You need to find the first and last positions of a specific book ID in this sorted list using recursive binary search.

Example

Input:

10

1 2 2 2 2 3 4 8 8 8

8

Output:

7 9

Explanation:

The first and last positions of book ID 8 are [7, 9] since the index is 0-based.

***Input Format***

The first line of input consists of an integer n, representing the total number of book IDs.

The second line consists of a sorted list of n integers representing the book IDs.

The third line consists of an integer x, representing the specific book ID to find.

***Output Format***

The output displays indexes of the first and last occurrence of x, as space-separated integers.

If book ID x is not present in the shelves, print "NO OCCURRENCES".

Refer to the sample output for formatting specifications.

vtu28682

***Sample Test Case***

Input: 10
1 2 2 2 2 3 4 8 8 8
8

Output: 7 9

***Answer***

```
// You are using GCC
#include <stdio.h>

int arr[20];

// Find first occurrence
int firstOcc(int low, int high, int x) {
    if (low > high)
        return -1;

    int mid = low + (high - low) / 2;

    if ((mid == 0 || arr[mid - 1] < x) && arr[mid] == x)
        return mid;
    else if (arr[mid] >= x)
        return firstOcc(low, mid - 1, x);
    else
        return firstOcc(mid + 1, high, x);
}

// Find last occurrence
int lastOcc(int low, int high, int x, int n) {
    if (low > high)
        return -1;

    int mid = low + (high - low) / 2;

    if ((mid == n - 1 || arr[mid + 1] > x) && arr[mid] == x)
        return mid;
    else if (arr[mid] > x)
        return lastOcc(low, mid - 1, x, n);
    else
        return lastOcc(mid + 1, high, x, n);
}
```

```
int main() {
    int n, x;
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    scanf("%d", &x);

    int first = firstOcc(0, n - 1, x);
    int last = lastOcc(0, n - 1, x, n);

    if (first == -1 || last == -1)
        printf("NO OCCURRENCES");
    else
        printf("%d %d", first, last);

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

# Vel Tech Group of Institutions

Name: NIKHIL  KUMAR
Email: vtu28682@veltech.edu.in
Roll no: vtu28682
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

## 2028_NeoColab_Design and Analysis of AlgorithmsS1L10

### VTU_DAA_Week 5_CY

Attempt : 1
Total Mark : 50
Marks Obtained : 50

### Section 1 : Coding

1. Problem Statement

Given an array of integers, count the number of smaller elements that appear to the right of each element. Implement a solution using merge sort to count these elements efficiently. Output the result as an array of counts corresponding to each element in the original array.

Example

Input:

4

5 2 6 1

Output:

2 1 1 0

vtu28682

Explanation:

To the right of 5, there are 2 smaller elements (2 and 1).

To the right of 2, there is only 1 smaller element (1).

To the right of 6, there is 1 smaller element (1).

To the right of 1, there is 0 smaller element.

***Input Format***

The first line of input contains a single integer n, representing the length of the array.

The second line contains n space-separated integers, representing the elements of the array.

***Output Format***

The output prints a single line containing the counts of smaller elements for each value in the array, separated by a space.

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 4
5 2 6 1
Output: 2 1 1 0

***Answer***

```
// You are using GCC
#include <stdio.h>

#define MAX 20

struct Node {
    int value;
    int index;
};
```

vtu28682

```
struct Node arr[MAX], temp[MAX];
int count[MAX];
int n;

// Merge function
void merge(int left, int mid, int right) {
    int i = left, j = mid + 1, k = left;
    int rightCount = 0;

    while (i <= mid && j <= right) {
        if (arr[j].value < arr[i].value) {
            temp[k++] = arr[j++];
            rightCount++;
        } else {
            count[arr[i].index] += rightCount;
            temp[k++] = arr[i++];
        }
    }

    while (i <= mid) {
        count[arr[i].index] += rightCount;
        temp[k++] = arr[i++];
    }

    while (j <= right) {
        temp[k++] = arr[j++];
    }

    for (int p = left; p <= right; p++) {
        arr[p] = temp[p];
    }
}

// Merge Sort
void mergeSort(int left, int right) {
    if (left < right) {
        int mid = (left + right) / 2;

        mergeSort(left, mid);
        mergeSort(mid + 1, right);

        merge(left, mid, right);
```

```
    }
}

int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i].value);
        arr[i].index = i;
        count[i] = 0;
    }

    mergeSort(0, n - 1);

    // Print result
    for (int i = 0; i < n; i++) {
        printf("%d ", count[i]);
    }

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

2. Problem Statement

Ravi, a mathematics teacher, is helping his students understand quadratic equations. He wants to apply the quadratic formula [A * x2 + B * x + C] to each number in a sorted array. Once the formula is applied, the modified numbers need to be sorted again. Ravi plans to use QuickSort for efficient sorting. Help Ravi by implementing a program that performs these operations.

***Input Format***

The first line contains an integer n, representing the size of the array.

The second line contains n space-separated integers, representing the elements of the array in ascending order.

The third line contains three integers A, B, and C, the coefficients of the quadratic

formula.

***Output Format***

The first line of output prints: Array after applying the equation and sorting

The subsequent lines print: <sorted_value>

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 6
-1 0 1 2 3 4
-1 2 -1
Output: Array after applying the equation and sorting:
-9 -4 -4 -1 -1 0

***Answer***

```
// You are using GCC
#include <stdio.h>

#define MAX 100

int arr[MAX];
int n;

// Swap function
void swap(int *a, int *b) {
   int temp = *a;
   *a = *b;
   *b = temp;
}

// Partition function (Quick Sort)
int partition(int low, int high) {
   int pivot = arr[high];
   int i = low - 1;

   for (int j = low; j < high; j++) {
      if (arr[j] <= pivot) {
```

```
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}

// Quick Sort
void quickSort(int low, int high) {
    if (low < high) {
        int pi = partition(low, high);
        quickSort(low, pi - 1);
        quickSort(pi + 1, high);
    }
}

int main() {
    int A, B, C;

    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    scanf("%d %d %d", &A, &B, &C);

    // Apply quadratic formula
    for (int i = 0; i < n; i++) {
        int x = arr[i];
        arr[i] = A * x * x + B * x + C;
    }

    // Sort using Quick Sort
    quickSort(0, n - 1);

    // Output
    printf("Array after applying the equation and sorting:\n");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
```

```
    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

3. Problem Statement

Amit, an e-commerce manager, is working on an inventory management system to track the top-selling products. Given the sales data of various products, he wants to quickly determine the Kth best-selling product to make informed restocking decisions.

Help Amit implement Heap Sort to efficiently find the Kth best-selling product from the given sales data.

***Input Format***

The first line of input consists of an integer n, representing the number of products.

The second line consists of n space-separated integers, representing the sales data of each product.

The third line contains an integer K, representing the rank of the product to retrieve.

***Output Format***

The output prints the Kth best-selling product based on the given sales data.

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 5
10 20 15 5 25
2
Output: 20

vtu28682

***Answer***

```
// You are using GCC
#include <stdio.h>

#define MAX 100000

int arr[MAX];
int n;

// Swap function
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// Heapify (Max Heap)
void heapify(int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        swap(&arr[i], &arr[largest]);
        heapify(n, largest);
    }
}

// Build Max Heap
void buildHeap() {
    for (int i = n / 2 - 1; i >= 0; i--) {
        heapify(n, i);
    }
}

int main() {
```

```
    int k;
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    scanf("%d", &k);

    buildHeap();

    int size = n;
    int result;

    // Extract max k times
    for (int i = 0; i < k; i++) {
        result = arr[0];              // max element
        swap(&arr[0], &arr[size - 1]);  // move to end
        size--;                       // reduce heap size
        heapify(size, 0);             // restore heap
    }

    printf("%d", result);

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

4. Problem Statement

Tom is working in a warehouse inventory system, where the item IDs are assigned sequentially in ascending order.

He wants to develop a program using recursive binary search to efficiently determine the closest item ID that is less than or equal to a given target ID.

The program takes the total number of items and their sorted IDs as input, assisting warehouse staff in quickly identifying the closest available item less than or equal to the target ID. Help Tom in this program.

***Input Format***

The first line of input consists of an integer N, representing the number of items in the warehouse.

The second line consists of N space-separated integers, representing the sorted list of item IDs.

The third line consists of an integer representing the target item ID.

***Output Format***

The output prints "The closest item ID less than or equal to X is Y", where X is the target ID and Y is the closest item ID less than or equal to X.

If no such element exists, print "No closest item with an ID less than or equal to X exists in the warehouse", where X is the target ID.

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 7
17 25 38 47 51 62 79
50
Output: The closest item ID less than or equal to 50 is 47

***Answer***

```
// You are using GCC
#include <stdio.h>

int arr[20];
int result = -1;  // stores closest value

void binarySearch(int low, int high, int x) {
   if (low > high)
      return;

   int mid = low + (high - low) / 2;

   if (arr[mid] <= x) {
```

```
        result = arr[mid];            // possible answer
        binarySearch(mid + 1, high, x); // search right for closer
    } else {
        binarySearch(low, mid - 1, x);  // search left
    }
}

int main() {
    int n, x;
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    scanf("%d", &x);

    binarySearch(0, n - 1, x);

    if (result == -1) {
        printf("No closest item with an ID less than or equal to %d exists in the
warehouse", x);
    } else {
        printf("The closest item ID less than or equal to %d is %d", x, result);
    }

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

5. Problem Statement

Rahul owns a bookstore and maintains a sorted list of ISBN numbers of books in his inventory. When a customer asks for a specific book, Rahul wants to quickly check whether the book is available in his inventory.

Write a program that implements a non-recursive binary search to determine if a given ISBN exists in the sorted list of ISBN numbers.

***Input Format***

The first line contains an integer N, representing the number of ISBN numbers in the inventory.

The second line contains N space-separated integers representing the sorted list of ISBN numbers.

The third line contains a single integer X, the ISBN number to be searched.

***Output Format***

If the ISBN X is found in the inventory, print: "Book with ISBN X is available at index Y" where Y is the zero-based index of the ISBN in the list.

If the ISBN X is not found, print: "Book with ISBN X is not available in the inventory"

Refer to the sample output for the formatting specifications.

***Sample Test Case***

Input: 5
2 3 40 50 20
3
Output: Book with ISBN 3 is available at index 1

***Answer***

```
// You are using GCC
#include <stdio.h>

int main() {
    int n, arr[30], x;

    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    scanf("%d", &x);
```

```
    int low = 0, high = n - 1;
    int foundIndex = -1;

    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (arr[mid] == x) {
            foundIndex = mid;
            break;
        }
        else if (arr[mid] < x) {
            low = mid + 1;
        }
        else {
            high = mid - 1;
        }
    }

    if (foundIndex != -1) {
        printf("Book with ISBN %d is available at index %d", x, foundIndex);
    } else {
        printf("Book with ISBN %d is not available in the inventory", x);
    }

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

# Vel Tech Group of Institutions

Name: NIKHIL  KUMAR
Email: vtu28682@veltech.edu.in
Roll no: vtu28682
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

## 2028_NeoColab_Design and Analysis of AlgorithmsS1L10

### VTU_DAA_Week 5_PAH

Attempt : 1
Total Mark : 50
Marks Obtained : 48

### Section 1 : Coding

1. Problem Statement

Suppose you are working on a project that involves sorting a large dataset of student records based on a single criterion, such as their CGPA, age, or major. You have decided to use a modified quicksort algorithm to efficiently sort the dataset while ensuring that the sorting is stable, i.e., that records with equal values maintain their original relative order.

To achieve this, you will implement a modified version of the quicksort algorithm that maintains stability during the sorting process. Specifically, you will use a three-way partitioning scheme that sorts the records into three groups: those with values less than the pivot, those with values equal to the pivot, and those with values greater than the pivot. When the algorithm processes subarrays with equal pivot values, it uses insertion sort to maintain stability.

vtu28682

Write a function that implements this modified version of the quicksort algorithm and uses it to sort the student records according to one specified criterion (CGPA, age, or major). Your function should take as input a list of student records and return the sorted list in ascending order based on the chosen criterion. If two records have the same values for the sorting criterion, their relative order should be maintained in the sorted list.

***Input Format***

The first line contains an integer n, representing the number of student records to be sorted.

The next n groups of 4 lines each contain the details of each student:

- Line 1: Name (string)
- Line 2: CGPA (float)
- Line 3: Age (integer)
- Line 4: Major (string)

The last line contains the sorting criteria (one of three options: "CGPA", "age", or "major").

***Output Format***

The output displays the sorted student records in a table format with the following specifications:

Header row: "Name CGPA Age Major"

Each student record is displayed in one row with the following format:

- Name: Left-aligned, 10 characters wide
- CGPA: Left-aligned, 8 characters wide, displayed with 2 decimal places
- Age: Left-aligned, 5 characters wide
- Major: Left-aligned, 20 characters wide

The table is sorted in ascending order according to the chosen sorting criteria.

Refer to the sample outputs for the formatting specifications.

***Sample Test Case***

Input: 4

vtu28682

harish
7.8
23
ece
trello
8
21
cse
kamalesh
7
25
it
maaran
5.6
20
civil
age

Output: Name CGPA Age Major
maaran 5.60 20 civil
trello 8.00 21 cse
harish 7.80 23 ece
kamalesh 7.00 25 it

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <string.h>

#define MAX 1000

struct Student {
    char name[50];
    float cgpa;
    int age;
    char major[50];
};

struct Student arr[MAX], temp[MAX];
char criterion[10];

// Compare function
int compare(struct Student a, struct Student b) {
```

```
    if (strcmp(criterion, "age") == 0)
        return a.age - b.age;
    else if (strcmp(criterion, "major") == 0)
        return strcmp(a.major, b.major);
    else // CGPA
        return (a.cgpa > b.cgpa) - (a.cgpa < b.cgpa);
}

// Stable Quick Sort
void stableQuickSort(int left, int right) {
    if (left >= right) return;

    struct Student pivot = arr[left];

    struct Student less[MAX], equal[MAX], greater[MAX];
    int l = 0, e = 0, g = 0;

    // Partition into 3 groups
    for (int i = left; i <= right; i++) {
        int cmp = compare(arr[i], pivot);
        if (cmp < 0)
            less[l++] = arr[i];
        else if (cmp == 0)
            equal[e++] = arr[i];
        else
            greater[g++] = arr[i];
    }

    // Copy back (stable order preserved)
    int idx = left;

    for (int i = 0; i < l; i++) arr[idx++] = less[i];
    for (int i = 0; i < e; i++) arr[idx++] = equal[i];
    for (int i = 0; i < g; i++) arr[idx++] = greater[i];

    // Recursively sort
    stableQuickSort(left, left + l - 1);
    stableQuickSort(left + l + e, right);
}

int main() {
    int n;
```

```
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf("%s", arr[i].name);
        scanf("%f", &arr[i].cgpa);
        scanf("%d", &arr[i].age);
        scanf("%s", arr[i].major);
    }

    scanf("%s", criterion);

    // Normalize criterion (case-insensitive)
    if (strcmp(criterion, "gpa") == 0) strcpy(criterion, "CGPA");

    stableQuickSort(0, n - 1);

    // Output
    printf("Name      CGPA     Age   Major             \n");

    for (int i = 0; i < n; i++) {
        printf("%-10s %-8.2f %-5d %-20s\n",
               arr[i].name,
               arr[i].cgpa,
               arr[i].age,
               arr[i].major);
    }

    return 0;
}
```

***Status :*** Partially correct ***Marks : 8/10***

2. Problem Statement

Vishnu, a math enthusiast, is given a task to explore the magic of numbers. He has an array of positive integers, and his goal is to find the integer with the highest digit sum in the sorted array using the merge sort algorithm.

You have to assist Vishnu in implementing the merge sort algorithm.

Example

Input:

6

789 123 456 876 234 567

Output:

The sorted array is: 123 234 456 567 789 876

The integer with the highest digit sum is: 789

Explanation

Sorted Array Output: 123 234 456 567 789 876

The digit sum of an integer is the sum of its individual digits.

In this case, the integer 789 has the highest digit sum (7 + 8 + 9 = 24).

***Input Format***

The first line of input consists of an integer N, representing the number of elements in the array.

The second line consists of N space-separated integers, representing the array elements.

***Output Format***

The first line prints "The sorted array is: " followed by the sorted array in ascending order.

The second line prints "The integer with the highest digit sum is: " followed by the integer with the highest-digit sum.

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 5
123 456 789 321 654

vtu28682

Output: The sorted array is: 123 321 456 654 789
The integer with the highest digit sum is: 789

***Answer***

```
// You are using GCC
#include <stdio.h>

#define MAX 20

int arr[MAX];

// Function to calculate digit sum
int digitSum(int x) {
    int sum = 0;
    while (x > 0) {
        sum += x % 10;
        x /= 10;
    }
    return sum;
}

// Merge function
void merge(int left, int mid, int right) {
    int temp[MAX];
    int i = left, j = mid + 1, k = left;

    while (i <= mid && j <= right) {
        if (arr[i] <= arr[j])
            temp[k++] = arr[i++];
        else
            temp[k++] = arr[j++];
    }

    while (i <= mid)
        temp[k++] = arr[i++];

    while (j <= right)
        temp[k++] = arr[j++];

    for (int p = left; p <= right; p++)
        arr[p] = temp[p];
}
```

```
// Merge Sort
void mergeSort(int left, int right) {
    if (left < right) {
        int mid = (left + right) / 2;
        mergeSort(left, mid);
        mergeSort(mid + 1, right);
        merge(left, mid, right);
    }
}

int main() {
    int n;
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    // Sort array
    mergeSort(0, n - 1);

    // Print sorted array
    printf("The sorted array is: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    // Find element with highest digit sum
    int maxSum = -1, result = arr[0];

    for (int i = 0; i < n; i++) {
        int sum = digitSum(arr[i]);
        if (sum > maxSum) {
            maxSum = sum;
            result = arr[i];
        }
    }

    // Print result
    printf("The integer with the highest digit sum is: %d", result);
```

```
    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

3. Problem Statement

Pavi is working on a project to help a library manage its collection of books efficiently. As part of this project, you need to develop a program to assist the library staff in finding specific books in their extensive catalog.

The library's book collection is organized in such a way that each book is assigned a unique numerical identifier. The library staff has provided you with a list of these identifiers, and you need to write a program to help them find a book's identifier efficiently.

Therefore, you decide to implement a recursive binary search algorithm to quickly locate books based on their unique identifiers.

***Input Format***

The first line of input contains an integer, n, representing the number of books in the library's collection.

The second line of input consists of n space-separated integers, where each integer represents the unique identifier of a book in the library's collection.

The third line of input contains a single integer t, representing the unique identifier of the book the library staff is searching for.

***Output Format***

If the book with the identifier target is found in the library's collection, the output prints the index (0-based) of the book in the collection.

If the book is not found, print "t not present", where the t is the unique identifier of a book.

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 5
12 13 14 15 16
14

Output: 2

***Answer***

```
// You are using GCC
#include <stdio.h>

int arr[20];

// Recursive Binary Search
int binarySearch(int low, int high, int target) {
    if (low > high)
        return -1;

    int mid = low + (high - low) / 2;

    if (arr[mid] == target)
        return mid;
    else if (arr[mid] > target)
        return binarySearch(low, mid - 1, target);
    else
        return binarySearch(mid + 1, high, target);
}

int main() {
    int n, t;
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    scanf("%d", &t);

    int result = binarySearch(0, n - 1, t);
```

```
    if (result != -1)
        printf("%d", result);
    else
        printf("%d not present", t);

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

4. Problem Statement

Jack is building a number analyzer tool that identifies numbers based on the sum of their digits. He is given a list of integers and wants to find the smallest number in the list whose digit sum equals a target value d.

To do this efficiently, Jack first sorts the list based on each number's digit sum and then applies binary search to find a match. Write a program to ensure that Jack can determine the correct number using this approach. If no number in the list matches the digit sum d, print -1.

***Input Format***

The first line of input consists of an integer n, representing the number of elements in the list.

The second line contains n space-separated integers, representing the sorted list.

The third line contains an integer d, representing the target digit sum.

***Output Format***

The output prints a single integer representing the smallest number from the list whose digit sum equals d, based on sorted digit sum order.

If no such number exists, print -1.

Refer to the sample output for formatting specifications.

vtu28682

***Sample Test Case***

Input: 5
12 23 30 48 56
3

Output: 12

***Answer***

```
// You are using GCC
#include <stdio.h>

#define MAX 20

struct Node {
   int value;
   int sum;
};

struct Node arr[MAX];

// Function to calculate digit sum
int digitSum(int x) {
   int sum = 0;
   while (x > 0) {
      sum += x % 10;
      x /= 10;
   }
   return sum;
}

// Swap
void swap(struct Node *a, struct Node *b) {
   struct Node temp = *a;
   *a = *b;
   *b = temp;
}

// Simple sort based on digit sum, then value
void sort(int n) {
   for (int i = 0; i < n - 1; i++) {
      for (int j = 0; j < n - i - 1; j++) {
         if (arr[j].sum > arr[j + 1].sum ||
```

```
            (arr[j].sum == arr[j + 1].sum && arr[j].value > arr[j + 1].value)) {
             swap(&arr[j], &arr[j + 1]);
          }
       }
    }
}

// Binary Search for first occurrence of digit sum = d
int binarySearch(int n, int d) {
    int low = 0, high = n - 1, ans = -1;

    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (arr[mid].sum == d) {
            ans = mid;
            high = mid - 1; // move left for smallest
        }
        else if (arr[mid].sum < d)
            low = mid + 1;
        else
            high = mid - 1;
    }
    return ans;
}

int main() {
    int n, d;
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i].value);
        arr[i].sum = digitSum(arr[i].value);
    }

    scanf("%d", &d);

    // Sort based on digit sum
    sort(n);

    int index = binarySearch(n, d);
```

```
    if (index == -1)
        printf("-1");
    else
        printf("%d", arr[index].value);

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

5. Problem Statement

Riya, an HR manager, needs a quick way to identify the Kth highest salary among employees for bonus distribution. To efficiently retrieve this information, she wants to use the max-heap technique.

Help Riya implement a solution that finds the Kth highest salary from the given list of salaries.

***Input Format***

The first line of input consists of an integer N, representing the number of employees.

The second line consists of N space-separated integers, representing the salaries of N employees.

The third line consists of an integer K, indicating the rank of the desired Kth highest salary.

***Output Format***

The output prints: <k>th largest Salary: <result>

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 7
30000 25000 40000 35000 20000 28000 29500

vtu28682

5
Output: 5th largest Salary: 28000

***Answer***

```
// You are using GCC
#include <stdio.h>

#define MAX 105

int arr[MAX];
int n;

// Swap function
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// Heapify (Max Heap)
void heapify(int size, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < size && arr[left] > arr[largest])
        largest = left;

    if (right < size && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        swap(&arr[i], &arr[largest]);
        heapify(size, largest);
    }
}

// Build Max Heap
void buildHeap() {
    for (int i = n / 2 - 1; i >= 0; i--) {
        heapify(n, i);
    }
```

```
}

int main() {
    int k;
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    scanf("%d", &k);

    buildHeap();

    int size = n;
    int result;

    // Extract max k times
    for (int i = 0; i < k; i++) {
        result = arr[0];
        swap(&arr[0], &arr[size - 1]);
        size--;
        heapify(size, 0);
    }

    printf("%dth largest Salary: %d", k, result);

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

# Vel Tech Group of Institutions

Name: NIKHIL  KUMAR
Email: vtu28682@veltech.edu.in
Roll no: vtu28682
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

## 2028_NeoColab_Design and Analysis of AlgorithmsS1L10

### VTU_DAA_Week 6_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 15

### Section 1 : MCQ

1. Which of the following is/are property/properties of a dynamic programming problem?

***Answer***

Both optimal substructure and overlapping subproblems

***Status :*** Correct ***Marks : 1/1***

2. If an optimal solution can be created for a problem by constructing optimal solutions for its subproblems, the problem possesses _____ property.

***Answer***

Optimal substructure

vtu28682

***Status :*** Correct ***Marks : 1/1***

3. In dynamic programming, the technique of storing the previously calculated values is called _____.

***Answer***

Memoization

***Status :*** Correct ***Marks : 1/1***

4. The following paradigm can be used to find the solution to the problem in minimum time:

Given a set of non-negative integers, and a value K, determine if there is a subset of the given set with a sum equal to K.

***Answer***

Dynamic Programming

***Status :*** Correct ***Marks : 1/1***

5. Which of the following is an example of a problem that can be solved using memoization in dynamic programming?

***Answer***

Computing the edit distance between two strings

***Status :*** Correct ***Marks : 1/1***

6. The Floyd-Warshall algorithm for all-pairs shortest paths computation is based on

***Answer***

Dynamic Programming paradigm

***Status :*** Correct ***Marks : 1/1***

vtu28682

7. Floyd-Warshall algorithm utilizes _____ to solve the all-pairs shortest paths problem on a directed graph in _____ time.

***Answer***

Dynamic programming, θ (V3)

***Status :*** Correct ***Marks : 1/1***

8. Floyd Warshall's Algorithm can be applied to _____.

***Answer***

Directed graphs

***Status :*** Correct ***Marks : 1/1***

9. In the Floyd Warshall algorithm, how many iterations are required to find the shortest path between all pairs of vertices in a graph with N vertices?

***Answer***

N

***Status :*** Correct ***Marks : 1/1***

10. Consider a directed weighted graph with V vertices and E edges, where each edge weight is represented as a 32-bit signed integer. What will be the maximum possible value for the edge weight to guarantee the correct shortest path calculation using the Floyd Warshall algorithm?

***Answer***

2^31 - 1

***Status :*** Correct ***Marks : 1/1***

11. 0/1 knapsack is based on _____method.

***Answer***

Dynamic programming

***Status :*** Correct ***Marks : 1/1***

12. Fractional knapsack problem differs from 0/1 knapsack because:

***Answer***

Items can be divided into fractions

***Status :*** Correct ***Marks : 1/1***

13. Which of the following methods can be used to solve the Knapsack problem?

***Answer***

All the mentioned options

***Status :*** Correct ***Marks : 1/1***

14. What is the primary advantage of using the Branch and Bound approach for the Knapsack problem?

***Answer***

It systematically explores possible solutions while pruning non-promising ones.

***Status :*** Correct ***Marks : 1/1***

15. Which of the following problems cannot be solved optimally by greedy approach?

***Answer***

0/1 Knapsack

***Status :*** Correct ***Marks : 1/1***

vtu28682

# Vel Tech Group of Institutions

Name: NIKHIL  KUMAR
Email: vtu28682@veltech.edu.in
Roll no: vtu28682
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

## 2028_NeoColab_Design and Analysis of AlgorithmsS1L10

### VTU_DAA_Week 6_COD

Attempt : 1
Total Mark : 40
Marks Obtained : 40

### Section 1 : Coding

1. Problem Statement

Ankit wants to share the job among his employees so that maximum profit can be obtained. The condition is that no two jobs for a particular employee overlap.

Given N jobs, where every job is represented by the following three elements:

Start Time

Finish Time

Profit or Value Associated (>= 0)

Find the maximum-profit subset of jobs such that no two jobs in the subset overlap.

vtu28682

***Input Format***

The first line of the input consists of the value of n.

The next n lines of the inputs consist of the start, end, and profit.

***Output Format***

The output prints the optimal profit.

Refer to the sample input and output for format specifications.

***Sample Test Case***

Input: 4
3 10 20
1 2 50
6 19 100
2 100 200
Output: The optimal profit is 250

***Answer***

```
// You are using GCC
#include <stdio.h>

#define MAX 100

struct Job {
    int start, finish, profit;
};

struct Job jobs[MAX];

// Swap
void swap(struct Job *a, struct Job *b) {
    struct Job temp = *a;
    *a = *b;
    *b = temp;
}

// Sort jobs by finish time
```

```
void sortJobs(int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (jobs[j].finish > jobs[j + 1].finish) {
                swap(&jobs[j], &jobs[j + 1]);
            }
        }
    }
}

// Find last non-conflicting job
int latestNonConflict(int i) {
    for (int j = i - 1; j >= 0; j--) {
        if (jobs[j].finish <= jobs[i].start)
            return j;
    }
    return -1;
}

int main() {
    int n;
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf("%d %d %d", &jobs[i].start, &jobs[i].finish, &jobs[i].profit);
    }

    // Step 1: Sort jobs
    sortJobs(n);

    int dp[MAX];
    dp[0] = jobs[0].profit;

    // Step 2: Fill DP table
    for (int i = 1; i < n; i++) {
        int includeProfit = jobs[i].profit;
        int l = latestNonConflict(i);

        if (l != -1)
            includeProfit += dp[l];

        dp[i] = (includeProfit > dp[i - 1]) ? includeProfit : dp[i - 1];
```

```
    }

    // Output
    printf("The optimal profit is %d", dp[n - 1]);

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

2. Problem Statement

In a city, there are several locations connected by roads. Each road has a certain distance. You are tasked with finding the shortest distances between all pairs of locations in the city using the Floyd-Warshall algorithm.

Write a program that calculates the shortest path between all pairs of locations in the city.

Example

Input:

4

4

0 1 3

1 2 1

2 3 1

0 2 5

Output:

0 3 4 5

3 0 1 2

4 1 0 1

5 2 1 0

Explanation:

The city has 4 locations (0 to 3).

There are 4 roads connecting the locations.

The roads and their distances are:

0 1 with distance 3

1 2 with distance 1

2 3 with distance 1

0 2 with distance 5

Initially, the distance matrix is set to infinity (INF) except for direct roads and diagonal values (0).

The Floyd-Warshall algorithm updates the matrix by checking intermediate paths.

The shortest path from 0 to 2 (via 1) is updated to 4 (0 1 2) instead of 5.

The shortest path from 0 to 3 is 5 (0 1 2 3).

The final distance matrix is printed, showing the shortest distances.

0 3 4 5

3 0 1 2

4 1 0 1

5 2 1 0

***Input Format***

The first line contains an integer N, representing the number of locations in the city.

The second line contains an integer M, representing the number of roads in the city.

The next M lines each contain three space-separated integers: u, v, w where:

- u and v represent two locations connected by a road.
- w represents the distance between u and v.

***Output Format***

The output prints a matrix, where the value at the i-th row and j-th column represents the shortest distance between location i and location j.

Refer to the sample output for the formatting specifications.

***Sample Test Case***

Input: 4
4
0 1 3
1 2 1
2 3 1
0 2 5

Output: 0 3 4 5
3 0 1 2
4 1 0 1
5 2 1 0

***Answer***

```
// You are using GCC
#include <stdio.h>

#define MAX 10
#define INF 99999

int main() {
    int n, m;
    scanf("%d", &n);
    scanf("%d", &m);

    int dist[MAX][MAX];

    // Step 1: Initialize matrix
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (i == j)
                dist[i][j] = 0;
            else
```

```
            dist[i][j] = INF;
        }
    }

    // Step 2: Read edges
    for (int i = 0; i < m; i++) {
        int u, v, w;
        scanf("%d %d %d", &u, &v, &w);
        dist[u][v] = w;
        dist[v][u] = w; // undirected graph
    }

    // Step 3: Floyd-Warshall
    for (int k = 0; k < n; k++) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (dist[i][k] + dist[k][j] < dist[i][j]) {
                    dist[i][j] = dist[i][k] + dist[k][j];
                }
            }
        }
    }

    // Step 4: Print result
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            printf("%d ", dist[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

***Status :*** Correct **Marks : 10/10**

3. Problem Statement

Sharon is developing a program that analyzes the reachability of nodes in a graph. Given an adjacency matrix representing the graph and its size, her goal is to determine whether there exists a path from one vertex to another

for all pairs of vertices.

Write a program to help Sharon perform this reachability analysis using Warshall's Algorithm.

***Input Format***

The first line of input consists of an integer N, representing the number of vertices in the graph.

The following N lines consist of N space-separated integers, forming the adjacency matrix graph, where graph[i][j] is either 0 or 1.

***Output Format***

The output prints an NxN matrix representing the reachability matrix for all pairs of vertices.

Each element in the matrix should be 1 if there is a path from the corresponding row vertex to the column vertex, and 0 otherwise.

***Sample Test Case***

Input: 4
0 1 0 0
0 0 1 0
0 0 0 1
0 0 0 0
Output: 0 1 1 1
0 0 1 1
0 0 0 1
0 0 0 0

***Answer***

```
// You are using GCC
#include <stdio.h>

#define MAX 10

int main() {
    int n;
    scanf("%d", &n);

    int graph[MAX][MAX];
```

```
    // Input adjacency matrix
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    // Warshall's Algorithm
    for (int k = 0; k < n; k++) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                graph[i][j] = graph[i][j] || (graph[i][k] && graph[k][j]);
            }
        }
    }

    // Output reachability matrix
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            printf("%d ", graph[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

4. Problem Statement

You and your group of adventurous friends are planning a once-in-a-lifetime outdoor adventure trip to an exotic location. To ensure that your adventure goes smoothly, you need to carefully choose what to carry with you, as you have a limited carrying capacity. Your goal is to maximize the overall value of the items you bring while staying within your weight limit.

Write a program that helps you and your friends determine the optimal selection of items to bring on your adventure trip. The program should

implement the 0–1 Knapsack Problem algorithm to find the best combination of items to maximize their total value while respecting the weight constraint.

***Input Format***

The first line contains an integer n, representing the number of items available for selection.

The second line contains an integer limit, representing the maximum weight limit your group can carry in a bag.

The next n lines contain space-separated information for each item:

- The name of the item is a string (without spaces).
- The weight of the item.
- The value of the item.

***Output Format***

The first line of output displays "Total value: " followed by an integer representing the total value of the selected items.

The second line of output displays "Total weight: " followed by an integer representing the total weight of the selected items.

Refer to the sample outputs for the exact format.

***Sample Test Case***

Input: 2
25
Bag 10 250
Note 15 430

Output: Total value: 680
Total weight: 25

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <string.h>
```

```
#define MAXN 10
#define MAXW 1005

int main() {
    int n, W;
    scanf("%d", &n);
    scanf("%d", &W);

    char name[MAXN][50];
    int weight[MAXN], value[MAXN];

    for (int i = 0; i < n; i++) {
        scanf("%s %d %d", name[i], &weight[i], &value[i]);
    }

    int dp[MAXN][MAXW];

    // Initialize DP
    for (int i = 0; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            if (i == 0 || w == 0)
                dp[i][w] = 0;
            else if (weight[i - 1] <= w) {
                int include = value[i - 1] + dp[i - 1][w - weight[i - 1]];
                int exclude = dp[i - 1][w];
                dp[i][w] = (include > exclude) ? include : exclude;
            } else {
                dp[i][w] = dp[i - 1][w];
            }
        }
    }

    // Backtrack to find total weight used
    int res = dp[n][W];
    int w = W;
    int totalWeight = 0;

    for (int i = n; i > 0 && res > 0; i--) {
        if (res != dp[i - 1][w]) {
            totalWeight += weight[i - 1];
            res -= value[i - 1];
```

```
        w -= weight[i - 1];
    }
  }

  // Output
  printf("Total value: %d\n", dp[n][W]);
  printf("Total weight: %d", totalWeight);

  return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

# Vel Tech Group of Institutions

Name: NIKHIL  KUMAR
Email: vtu28682@veltech.edu.in
Roll no: vtu28682
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

## 2028_NeoColab_Design and Analysis of AlgorithmsS1L10

### VTU_DAA_Week 6_CY

Attempt : 1
Total Mark : 50
Marks Obtained : 44.5

### Section 1 : Coding

1. Problem Statement

Virat is given a group of individuals, and he wants to identify if there is an influential individual within the group. An influential individual is defined as someone who is followed by more people than they follow.

He is provided with a list of follow-up relationships among the individuals in the form of a matrix. Your task is to help Virat determine if there is an influential individual and, if so, find their index within the group.

Write a program to solve this problem using Warshall's Algorithm.

***Input Format***

The first line of input consists of an integer N, representing the number of individuals in the group.

vtu28682

The following N lines will contain N space-separated integers, representing the following relationship matrix for the individuals.

***Output Format***

If an influential individual is found, print "Influential Individual: X," where X is the index (0-based) of the influential individual.

Otherwise, print "There is no influential individual in the group".

Refer to the sample output for the formatting specifications.

***Sample Test Case***

Input: 4
0 1 1 0
0 0 1 0
0 0 0 0
0 0 1 0

Output: Influential Individual: 2

***Answer***

```
// You are using GCC
#include <stdio.h>

#define MAX 10

int main() {
    int n;
    scanf("%d", &n);

    int graph[MAX][MAX];

    // Input matrix
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &graph[i][j]);
        }
    }
```

```
    // Warshall's Algorithm (Transitive Closure)
    for (int k = 0; k < n; k++) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                graph[i][j] = graph[i][j] || (graph[i][k] && graph[k][j]);
            }
        }
    }

    // Find influential individual
    for (int i = 0; i < n; i++) {
        int followers = 0, following = 0;

        for (int j = 0; j < n; j++) {
            if (graph[j][i]) followers++;   // people who reach i
            if (graph[i][j]) following++;   // people i reaches
        }

        if (followers > following) {
            printf("Influential Individual: %d", i);
            return 0;
        }
    }

    printf("There is no influential individual in the group");

    return 0;
}
```

***Status :*** Partially correct ***Marks : 4.5/10***

2. Problem statement

Sarah has been tasked with developing a program to determine the shortest paths among nodes in a directed network represented as a graph containing nodes and weighted edges.

Her goal is to implement the Floyd-Warshall algorithm to compute the shortest path matrix for this directed network.

***Input Format***

The first line of input consists of an integer n, representing the number of nodes in the network.

The next n lines contain a space-separated integer, representing the adjacency matrix 'graph' of the network. Each integer is the weight of the edge from node i to node j.

***Output Format***

The output prints the shortest path matrix as 'n x n' integers, where each element graph[i][j] represents the shortest distance from node 'i' to node 'j'.

Print each row of the matrix on a new line, with elements separated by a space, and if no path exists, 999 will be displayed.

Refer to the sample output for the formatting specifications.

***Sample Test Case***

Input: 4
0 3 999 4
8 0 2 999
5 999 0 1
2 999 999 0
Output: 0 3 5 4
5 0 2 3
3 6 0 1
2 5 7 0

***Answer***

```
// You are using GCC
#include <stdio.h>

#define MAX 100
#define INF 999

int main() {
    int n;
    scanf("%d", &n);
```

```
    int dist[MAX][MAX];

    // Input adjacency matrix
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &dist[i][j]);
        }
    }

    // Floyd-Warshall Algorithm
    for (int k = 0; k < n; k++) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (dist[i][k] != INF && dist[k][j] != INF) {
                    if (dist[i][k] + dist[k][j] < dist[i][j]) {
                        dist[i][j] = dist[i][k] + dist[k][j];
                    }
                }
            }
        }
    }

    // Output result
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            printf("%d ", dist[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

***Status :*** Correct **Marks : 10/10**

3. Problem Statement

You are working as a backend engineer for a drone delivery logistics company that operates across multiple cities. Each city is represented as a node, and the drone flight paths between cities are represented as directed

edges with weights indicating the time (in minutes) it takes for a drone to travel between them.

Due to varying air traffic, regulations, and geography, the direct travel time between cities may not always be the fastest. Sometimes, going through intermediate cities results in shorter delivery times.

Your task is to build a system that calculates the shortest possible delivery time between every pair of cities, considering all possible intermediate routes.

To efficiently compute this, you're required to implement the Floyd-Warshall Algorithm.

***Input Format***

The first line of input contains an integer N representing the total number of cities.

The next N lines contains N space-separated integers, representing the delivery time matrix:

time[i][j] is the time (in minutes) taken to fly directly from City i to City j.

A value of 0 means the same city (i.e., i == j).

A value of 999 indicates no direct route between City i and City j.

***Output Format***

The output prints the shortest distance between every pair of cities.

If there is no shortest path between two cities, then print INF.

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 4
0 5 999 10

vtu28682

999 0 3 999
999 999 0 1
999 999 999 0

Output: 0 5 8 9
INF 0 3 4
INF INF 0 1
INF INF INF 0

***Answer***

```
// You are using GCC
#include <stdio.h>

#define INF 1000000

int main() {
    int n;
    scanf("%d", &n);

    int dist[100][100];

    // Input
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &dist[i][j]);

            if (dist[i][j] == 999)
                dist[i][j] = INF; // Replace 999 with INF
        }
    }

    // Floyd-Warshall Algorithm
    for (int k = 0; k < n; k++) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (dist[i][k] + dist[k][j] < dist[i][j]) {
                    dist[i][j] = dist[i][k] + dist[k][j];
                }
            }
        }
    }

    // Output
```

```
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (dist[i][j] == INF)
                printf("INF ");
            else
                printf("%d ", dist[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

4. Problem Statement

Given the weights and values of n items, put these items in a knapsack of capacity W to get the maximum total value in the knapsack. In other words, given two integer arrays, val[0..n-1] and wt[0..n-1] which represent values and weights associated with n items, respectively. Also given an integer W which represents knapsack capacity, find out the maximum value subset of val[] such that the sum of the weights of this subset is smaller than or equal to W. You cannot break an item, either pick the complete item or don't pick it.

Solve this problem using dynamic programming.

***Input Format***

The first line of input represents the number of items.

The second line of input consists of the values of each item separated by spaces.

The third line of input consists of the weights of each item separated by spaces.

The fourth line of input consists of an integer, which represents the maximum capacity of the knapsack.

***Output Format***

The output prints the maximum value of items that can be held by the knapsack.

***Sample Test Case***

Input: 3
60 100 120
10 20 30
50

Output: 220

***Answer***

```
// You are using GCC
#include <stdio.h>

int max(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    int n, W;

    scanf("%d", &n);

    int val[n], wt[n];

    // Input values
    for (int i = 0; i < n; i++) {
        scanf("%d", &val[i]);
    }

    // Input weights
    for (int i = 0; i < n; i++) {
        scanf("%d", &wt[i]);
    }

    // Capacity
    scanf("%d", &W);

    int dp[n + 1][W + 1];

    // Initialize DP table
    for (int i = 0; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
```

```
            if (i == 0 || w == 0)
                dp[i][w] = 0;
            else if (wt[i - 1] <= w)
                dp[i][w] = max(dp[i - 1][w],
                               val[i - 1] + dp[i - 1][w - wt[i - 1]]);
            else
                dp[i][w] = dp[i - 1][w];
        }
    }

    // Output result
    printf("%d", dp[n][W]);

    return 0;
}
```

***Status :*** Correct **Marks : 10/10**

5. Problem Statement

Sarah loves challenges and has recently taken up a new programming problem. She is fascinated by the classic "0/1 Knapsack Problem" and wants to create a program that can efficiently solve it.

The 0/1 Knapsack Problem involves selecting a combination of items with given weights and values to maximize the total value, while not exceeding a given weight capacity.

Write a program that takes input for the 0/1 Knapsack Problem and outputs the maximum value that can be achieved, as well as the selected weights that contribute to this maximum value.

Note: The weights are printed in reverse order of the input, meaning the algorithm starts checking from the last item and prints items that are included first.

***Input Format***

The first line contains an integer n, representing the number of items available.

The second line contains n integers separated by space, representing the values of the items. The ith integer corresponds to the value of the ith item.

The third line contains n integers separated by space, representing the weights of the items. The ith integer corresponds to the weight of the ith item.

The fourth line contains an integer W, representing the maximum weight capacity of the knapsack.

***Output Format***

The first line output displays "Maximum value: " followed by an integer, representing the maximum value that can be achieved.

The second line output displays "Selected weights: " followed by space-separated integers, representing the selected weights contributing to the maximum value.

Refer to the sample outputs for the exact format.

***Sample Test Case***

Input: 3
60 100 120
10 20 30
50
Output: Maximum value: 220
Selected weights: 30 20

***Answer***

```
// You are using GCC
#include <stdio.h>

int max(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    int n, W;
    scanf("%d", &n);
```

```
int val[n], wt[n];

// Input values
for (int i = 0; i < n; i++)
    scanf("%d", &val[i]);

// Input weights
for (int i = 0; i < n; i++)
    scanf("%d", &wt[i]);

// Capacity
scanf("%d", &W);

int dp[n + 1][W + 1];

// Build DP table
for (int i = 0; i <= n; i++) {
    for (int w = 0; w <= W; w++) {
        if (i == 0 || w == 0)
            dp[i][w] = 0;
        else if (wt[i - 1] <= w)
            dp[i][w] = max(dp[i - 1][w],
                           val[i - 1] + dp[i - 1][w - wt[i - 1]]);
        else
            dp[i][w] = dp[i - 1][w];
    }
}

// Output maximum value
printf("Maximum value: %d\n", dp[n][W]);

// Backtrack to find selected weights
printf("Selected weights: ");

int w = W;
for (int i = n; i > 0; i--) {
    if (dp[i][w] != dp[i - 1][w]) {
        printf("%d ", wt[i - 1]);
        w -= wt[i - 1];
    }
}
```

```
    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

vtu28682

# Vel Tech Group of Institutions

Name: NIKHIL  KUMAR
Email: vtu28682@veltech.edu.in
Roll no: vtu28682
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

## 2028_NeoColab_Design and Analysis of AlgorithmsS1L10

### VTU_DAA_Week 6_PAH

Attempt : 2
Total Mark : 50
Marks Obtained : 50

### Section 1 : Coding

1. Problem Statement

Anitha is working on packing expensive artifacts into a container that has a limited weight capacity. Each artifact has a weight and a value, and Anitha wants to maximize the total worth of artifacts she can carry, even if it means taking only a fraction of an artifact when the container is nearly full.

The maximum worth is calculated as:

Maximum Worth = Σ (value[i] * fraction_taken[i]),

where fraction_taken[i] = min(1, remaining_capacity / weight[i])

Write a program to ensure that Anitha can calculate the maximum worth of objects she can carry in the container.

vtu28682

***Input Format***

The first line contains an integer N, representing the number of items.

The second line contains N space-separated integers representing the weights of the items.

The third line contains N space-separated integers representing the values of the items.

The fourth line contains an integer representing the capacity of the container.

***Output Format***

The output prints a single line showing the total maximum worth of objects carried as a double with exactly two decimal places, in the following format:

"Objects worth Rs.X.YY"

Refer to the sample output for the formatting specifications.

***Sample Test Case***

Input: 3
10 20 30
60 100 120
50
Output: Objects worth Rs.240.00

***Answer***

```
// You are using GCC
#include <stdio.h>

typedef struct {
   double weight;
   double value;
   double ratio;
} Item;
```

vtu28682

```
void swap(Item *a, Item *b) {
    Item t = *a;
    *a = *b;
    *b = t;
}

int main() {
    int n;
    scanf("%d", &n);

    double wt[10], val[10];
    Item items[10];

    for (int i = 0; i < n; i++)
        scanf("%lf", &wt[i]);

    for (int i = 0; i < n; i++)
        scanf("%lf", &val[i]);

    double capacity;
    scanf("%lf", &capacity);

    for (int i = 0; i < n; i++) {
        items[i].weight = wt[i];
        items[i].value = val[i];
        items[i].ratio = val[i] / wt[i];
    }

    // Sort by ratio descending
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (items[j].ratio < items[j + 1].ratio) {
                swap(&items[j], &items[j + 1]);
            }
        }
    }

    double totalValue = 0.0;

    for (int i = 0; i < n; i++) {
        if (capacity == 0)
```

```
            break;

        if (items[i].weight <= capacity) {
            capacity -= items[i].weight;
            totalValue += items[i].value;
        } else {
            totalValue += items[i].ratio * capacity;
            capacity = 0;
        }
    }

    printf("Objects worth Rs.%.2f", totalValue);

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

2. Problem Statement

Raju is a professional hiker preparing for a long trekking expedition. He has a backpack that can hold a maximum weight of W kilograms. There are N essential items you want to carry, each with a specific weight and importance value (utility score). His goal is to maximize the total importance value of the items you take while ensuring that the total weight does not exceed the backpack's capacity.

He can either include or exclude each item—meaning he cannot take fractions of an item (e.g., you cannot take half of a tent). Given the weights and importance values of the items, determine the maximum possible importance value he can carry in his backpack. Help him to achieve his task.

***Input Format***

The first line contains two integers, N (number of items) and W (maximum weight the backpack can carry).

The second line contains N integers, representing the weights of the items.

The third line contains N integers, representing the importance values of the items.

***Output Format***

The output prints a single integer, the maximum importance value that can be achieved without exceeding the weight limit.

Refer to the sample input and output for formatting specifications.

***Sample Test Case***

Input: 4 5
2 3 4 5
3 4 5 6
Output: 7

***Answer***

```
// You are using GCC
#include <stdio.h>

int max(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    int n, W;
    scanf("%d %d", &n, &W);

    int wt[n], val[n];

    for (int i = 0; i < n; i++)
        scanf("%d", &wt[i]);

    for (int i = 0; i < n; i++)
        scanf("%d", &val[i]);

    int dp[n + 1][W + 1];

    // initialize DP table
    for (int i = 0; i <= n; i++) {
```

```
        for (int w = 0; w <= W; w++) {
            if (i == 0 || w == 0)
                dp[i][w] = 0;
        }
    }

    // build table
    for (int i = 1; i <= n; i++) {
        for (int w = 1; w <= W; w++) {
            if (wt[i - 1] <= w) {
                dp[i][w] = max(dp[i - 1][w],
                               val[i - 1] + dp[i - 1][w - wt[i - 1]]);
            } else {
                dp[i][w] = dp[i - 1][w];
            }
        }
    }

    printf("%d", dp[n][W]);

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

3. Problem Statement

Rohit is responsible for scheduling flights for an airline company. He has a list of N flights numbered from 0 to N-1. Some flights have specific constraints that require them to depart after other flights.

Write a program to help Rohit determine if it's possible to create a flight schedule that adheres to these constraints using Warshall's algorithm.

Example 1

Input:

5
0 1

1 2

2 3

3 4

4 5

Output:

Yes

Explanation:

Flight 0 must depart after Flight 1.

Flight 1 must depart after Flight 2.

Flight 2 must depart after Flight 3.

Flight 3 must depart after Flight 4.

Flight 4 must depart after Flight 5.

There are no circular dependencies or conflicts in the schedule, so it is possible to create a flight schedule that adheres to the constraints.

Example 2

Input:

6

0 1

1 2

2 3

3 4

4 5

5 0

Output:

No

vtu28682

Explanation:

Flight 0 must depart after Flight 1.

Flight 1 must depart after Flight 2.

Flight 2 must depart after Flight 3.

Flight 3 must depart after Flight 4.

Flight 4 must depart after Flight 5.

Flight 5 must depart after Flight 0.

There is a circular dependency in the flight constraints. Therefore, it is not possible to create a flight schedule that follows the constraints.

***Input Format***

The first line of input consists of an integer N, representing the number of flights.

The following N lines consist of two space-separated integers each: the flight number and the flight number that must depart before it.

***Output Format***

If it is possible to create a flight schedule that adheres to the constraints, print "Yes".

Otherwise, print "No".

Refer to the sample output for the formatting specifications.

***Sample Test Case***

Input: 5
0 1
1 2
2 3
3 4
4 5
Output: Yes

***Answer***

vtu28682

```
// You are using GCC
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int graph[15][15] = {0};

    // Read edges
    for (int i = 0; i < n; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        graph[u][v] = 1;
    }

    // Warshall's algorithm
    for (int k = 0; k < n; k++) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (graph[i][k] && graph[k][j]) {
                    graph[i][j] = 1;
                }
            }
        }
    }

    // Check cycle (self reachability)
    for (int i = 0; i < n; i++) {
        if (graph[i][i] == 1) {
            printf("No");
            return 0;
        }
    }

    printf("Yes");
    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

4. Problem Statement

A traveler needs to find the shortest path between every pair of cities in a road network. The cities are connected by roads, each with a specific travel cost. To do this, the traveler uses the Floyd-Warshall Algorithm to calculate the shortest distances between every pair of cities in a given edge-weighted undirected graph.

Example1

Input:

4

3

0 1 2

1 2 3

2 3 4

Output:

Original matrix

0 2 INF INF

2 0 3 INF

INF 3 0 4

INF INF 4 0

Shortest path matrix

0 2 5 9

2 0 3 7

5 3 0 4

9 7 4 0

Explanation:

Given: 4 nodes (0, 1, 2, 3) and 3 edges: 0 1 (weight 2), 1 2 (weight 3), 2 3

vtu28682

(weight 4)

Any node pair without a direct edge is considered INF (unreachable initially).

Step 1: Construct Initial Graph (Original Matrix)

0 2 INF INF

2 0 3 INF

INF 3 0 4

INF INF 4 0

Diagonal elements (self to self) are 0.If no direct path exists, it's INF.Step 2: Running Floyd-Warshall Algorithm

Using node 0 as an intermediate: No updates since it only connects to node 1.

Using node 1 as an intermediate: 0 2 via 1: 0 1 (2) + 1 2 (3) = 5

2 0 also updates to 5.

Using node 2 as an intermediate: 0 3 via 2: 0 2 (5) + 2 3 (4) = 9

1 3 via 2: 1 2 (3) + 2 3 (4) = 7

Using node 3 as an intermediate: No further updates.

Step 3: Final Shortest Path Matrix

0 2 5 9

2 0 3 7

5 3 0 4

9 7 4 0

***Input Format***

The first line contains an integer V representing the number of cities (vertices) in the network.

The second line contains an integer E representing the number of roads (edges) in the network.

The next E lines each contain three space-separated integers u, v, w, where u and v represent the cities connected by a road, and w represents the travel cost (weight) of the road.

***Output Format***

The first line of output prints the original matrix, where each entry represents the direct travel cost between two cities. If there is no direct road between two cities, represent the cost as INF.

Then, print the shortest path matrix, where each entry represents the shortest travel cost between every pair of cities, computed using the Floyd-Warshall algorithm.

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 4
3
0 1 2
1 2 3
2 3 4
Output: Original matrix
0 2 INF INF
2 0 3 INF
INF 3 0 4
INF INF 4 0

Shortest path matrix
0 2 5 9
2 0 3 7
5 3 0 4
9 7 4 0

***Answer***

```
// You are using GCC
#include <stdio.h>

#define INF 999999
```

```
int min(int a, int b) {
    return (a < b) ? a : b;
}

int main() {
    int V, E;
    scanf("%d", &V);
    scanf("%d", &E);

    int dist[V][V];

    // initialize matrix
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            if (i == j)
                dist[i][j] = 0;
            else
                dist[i][j] = INF;
        }
    }

    // read edges (undirected graph)
    for (int i = 0; i < E; i++) {
        int u, v, w;
        scanf("%d %d %d", &u, &v, &w);
        dist[u][v] = w;
        dist[v][u] = w;
    }

    // print original matrix
    printf("Original matrix\n");
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            if (dist[i][j] == INF)
                printf("INF ");
            else
                printf("%d ", dist[i][j]);
        }
        printf("\n");
    }

    // Floyd-Warshall
```

```
    for (int k = 0; k < V; k++) {
        for (int i = 0; i < V; i++) {
            for (int j = 0; j < V; j++) {
                if (dist[i][k] != INF && dist[k][j] != INF) {
                    dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
                }
            }
        }
    }

    // print shortest path matrix
    printf("\nShortest path matrix\n");
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            if (dist[i][j] == INF)
                printf("INF ");
            else
                printf("%d ", dist[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

5. Problem Statement

A person starts walking from position X = 0, find the probability of reaching exactly on X = N if she can only take either 2 steps or 3 steps. The probability for step length 2 is given i.e. P, and the probability for step length 3 is 1 – P.

Write a program for the same.

Note: Use Dynamic Programming

Example

Input: N = 5, P = 0.20

Output: 0.32

Explanation:

There are two ways to reach 5.

2+3 with probability = 0.2 * 0.8 = 0.16

3+2 with probability = 0.8 * 0.2 = 0.16

So, total probability = 0.32.

***Input Format***

The input consists of an integer N representing the target position and a double-point number P representing the probability of taking a step of length 2, separated by a space.

***Output Format***

The output prints a double-point number representing the probability of reaching the exact position N, formatted to two decimal places.

Refer to the sample output for the formatting specifications.

***Sample Test Case***

Input: 5 0.20
Output: 0.32

***Answer***

```
// You are using GCC
#include <stdio.h>

int main() {
    int N;
    double P;

    scanf("%d %lf", &N, &P);

    double dp[25] = {0};
```

```
    dp[0] = 1.0;

    for (int i = 1; i <= N; i++) {
        if (i - 2 >= 0)
            dp[i] += dp[i - 2] * P;

        if (i - 3 >= 0)
            dp[i] += dp[i - 3] * (1.0 - P);
    }

    printf("%.2f", dp[N]);

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

# Vel Tech Group of Institutions

Name: NIKHIL  KUMAR
Email: vtu28682@veltech.edu.in
Roll no: vtu28682
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

## 2028_NeoColab_Design and Analysis of AlgorithmsS1L10

### VTU_DAA_Week 7_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 13

### Section 1 : MCQ

1. The frequencies of letters in a file are:

s = 17, k = 24, y = 37

A Huffman tree is constructed, and we assign 0 to the left branch and 1 to the right branch at each node.

Which of the following is the Huffman code for the letter 's'?

***Answer***

10

***Status :*** Correct ***Marks : 1/1***

2. In Huffman coding, does data in a tree always occur?

***Answer***

Leaves

***Status :*** Correct ***Marks : 1/1***

3. Which of the following algorithms is the best approach for solving Huffman codes?

***Answer***

Greedy algorithm

***Status :*** Correct ***Marks : 1/1***

4. From the following given tree, what is the computed codeword for ‘c’?

</a>

***Answer***

110

***Status :*** Correct ***Marks : 1/1***

5. From the following given tree, what is the code word for the character 'a'?

</a>

***Answer***

011

***Status :*** Correct ***Marks : 1/1***

6. Kruskal’s algorithm is used to _____.

***Answer***

Find minimum spanning tree

***Status :*** Correct ***Marks : 1/1***

7. Consider the following graph:

Which edges would be included in the minimum spanning tree using Prim's algorithm starting from vertex A?

***Answer***

AB, BD, DE, EF, FC

***Status :*** Correct ***Marks : 1/1***

8. Which of the following is true?

***Answer***

Prim’s algorithm initializes with a vertex which has smallest edge

***Status :*** Wrong ***Marks : 0/1***

9. Prim’s algorithm can be efficiently implemented using _____ for graphs with greater density.

***Answer***

fibonacci heap

***Status :*** Wrong ***Marks : 0/1***

10. Which of the following is false about Prim’s algorithm?

***Answer***

It constructs MST by selecting edges in increasing order of their weights

***Status :*** Correct ***Marks : 1/1***

vtu28682

11. What is the worst-case time complexity of Prim's algorithm if the adjacency matrix is used?

***Answer***

O(V2)

***Status :*** Correct ***Marks : 1/1***

12. Consider a graph G = (V, E), where V = { v1,v2,...,v100 }, E = { (vi, vj) 1≤ i < j ≤ 100} and the weight of the edge (vi, vj) is i–j . The weight of the minimum spanning tree of G is _____.

***Answer***

99

***Status :*** Correct ***Marks : 1/1***

13. An undirected graph G(V, E) contains n ( n > 2 ) nodes named v1 , v2 , ....vn. Two nodes vi , vj are connected if and only if 0 < |i – j| <= 2. Each edge (vi, vj ) is assigned a weight i + j. A sample graph with n = 4 is shown below. What will be the cost of the minimum spanning tree (MST) of such a graph with n nodes?

</a>

***Answer***

n^2 – n + 1

***Status :*** Correct ***Marks : 1/1***

14. An undirected graph G has n nodes. Its adjacency matrix is given by an n × n square matrix whose (i) diagonal elements are 0's and (ii) non-diagonal elements are 1's. which one of the following is TRUE?

***Answer***

Graph G has multiple distinct MSTs, each of cost n-1

***Status :*** Correct ***Marks : 1/1***

15. Consider a weighted complete graph G on the vertex set {v1,v2 ,v} such that the weight of the edge (v,,v) is 2|i-j|. The weight of a minimum spanning tree of G is:

***Answer***

2n-2

***Status :*** Correct ***Marks : 1/1***

vtu28682

# Vel Tech Group of Institutions

Name: NIKHIL  KUMAR
Email: vtu28682@veltech.edu.in
Roll no: vtu28682
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

## 2028_NeoColab_Design and Analysis of AlgorithmsS1L10

### VTU_DAA_Week 7_COD

Attempt : 1
Total Mark : 30
Marks Obtained : 27.5

### Section 1 : Coding

1. Problem Statement

Prim’s Algorithm for Minimum Spanning Tree (MST):

Sita is planning to lay roads to connect 5 locations in her city. The distances between locations are given as an adjacency matrix, where a value of 0 indicates no direct road between two locations (locations are numbered 0 to 4).

Help her find the Minimum Spanning Tree (MST) using Prim's Algorithm so that all locations are connected with the minimum total road length. Print each MST edge and its weight in the order they are added to the MST.

***Input Format***

The input consists of 5 lines, each containing 5 space-separated integers,

vtu28682

representing the adjacency matrix of the graph. A value of 0 indicates no direct connection between two locations.

***Output Format***

The first line of output prints the header: 'Edge \tWeight'

Each of the next V-1 lines prints one MST edge in the format:

'u - v\tw'

where u is the parent location, v is the child location, and w is the integer edge weight, printed in the order edges are added to the MST (MST discovery order).

Refer to the sample output for formatting specifications.

Refer to the sample output for format specifications.

***Sample Test Case***

Input: 0 2 0 6 0
2 0 3 8 5
0 3 0 8 7
6 8 0 0 9
0 5 7 9 0

Output: Edge  Weight
0 - 1  2
1 - 2  3
1 - 4  5
0 - 3  6

***Answer***

```
#include <stdio.h>
#include <limits.h>

#define V 5

int minKey(int key[], int mstSet[]) {
```

```
    int min = INT_MAX;
    int min_index = -1;

    for (int v = 0; v < V; v++) {
        if (mstSet[v] == 0) {
            if (key[v] < min ||
                (key[v] == min && v < min_index)) {
                min = key[v];
                min_index = v;
            }
        }
    }
    return min_index;
}

int main() {
    int graph[V][V];

    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    int parent[V], key[V], mstSet[V];

    for (int i = 0; i < V; i++) {
        key[i] = INT_MAX;
        mstSet[i] = 0;
    }

    key[0] = 0;
    parent[0] = -1;

    for (int count = 0; count < V - 1; count++) {

        int u = minKey(key, mstSet);
        mstSet[u] = 1;

        for (int v = 0; v < V; v++) {
            if (graph[u][v] && mstSet[v] == 0 &&
                graph[u][v] < key[v]) {
```

```
            key[v] = graph[u][v];
            parent[v] = u;
        }
    }
  }

  printf("Edge \tWeight\n");
  for (int i = 1; i < V; i++) {
    printf("%d - %d\t%d\n", parent[i], i, graph[i][parent[i]]);
  }

  return 0;
}
```

***Status :*** Partially correct ***Marks : 7.5/10***

2. Problem statement

Alex is a network engineer tasked with laying cables to connect multiple offices in the city. Each possible cable connection between two offices has an associated cost. To minimize the total cabling cost while ensuring all offices are connected, Alex decides to use Kruskal's Algorithm to find the Minimum Spanning Tree (MST) of the office network.

Given a connected, undirected, weighted graph with V vertices (numbered 0 to V-1) and E edges, find the MST and print each edge included along with the total minimum cost.

***Input Format***

The first line contains an integer V, representing the number of vertices (numbered 0 to V-1).

The second line contains an integer E, representing the number of edges.

Each of the next E lines contains three space-separated integers u, v, and w, representing an undirected edge between vertex u and vertex v with weight w

***Output Format***

The first line of output prints: 'Edges in the constructed MST:'

Each of the next V-1 lines prints one MST edge in the format:

'u -- v == weight'

where u and v are vertex numbers (integers) and weight is the edge cost (integer), printed in the order edges are added to the MST (ascending weight order).

The last line prints: 'Minimum Spanning Tree: totalCost'

where totalCost is an integer representing the total MST weight.

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 3
3
0 1 2
1 2 2
0 2 2
Output: Edges in the constructed MST:
0 -- 1 == 2
1 -- 2 == 2
Minimum Spanning Tree: 4

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int u, v, w;
} Edge;

int parent[100];
```

```
// Find with path compression
int find(int i) {
    if (parent[i] == i)
        return i;
    return parent[i] = find(parent[i]);
}

// Union
void unionSet(int a, int b) {
    parent[a] = b;
}

// Sort edges by weight (ascending)
int cmp(const void *a, const void *b) {
    Edge *e1 = (Edge *)a;
    Edge *e2 = (Edge *)b;
    return e1->w - e2->w;
}

int main() {
    int V, E;
    scanf("%d", &V);
    scanf("%d", &E);

    Edge edges[E];

    for (int i = 0; i < E; i++) {
        scanf("%d %d %d", &edges[i].u, &edges[i].v, &edges[i].w);
    }

    // Sort edges by weight
    qsort(edges, E, sizeof(edges[0]), cmp);

    // Initialize parent
    for (int i = 0; i < V; i++)
        parent[i] = i;

    int totalCost = 0;

    printf("Edges in the constructed MST:\n");
```

```
    int count = 0;

    for (int i = 0; i < E && count < V - 1; i++) {
        int u = edges[i].u;
        int v = edges[i].v;
        int w = edges[i].w;

        int pu = find(u);
        int pv = find(v);

        if (pu != pv) {
            printf("%d -- %d == %d\n", u, v, w);
            totalCost += w;
            unionSet(pu, pv);
            count++;
        }
    }

    printf("Minimum Spanning Tree: %d\n", totalCost);

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

3. Problem Statement

You are designing a data compression tool that uses Huffman Coding to encode data efficiently.

Write a program to read a set of characters along with their corresponding Huffman variable-length binary codes, and use them to generate the encoded binary string for a given input message.

It is guaranteed that every character in the input message has a corresponding Huffman code in the provided code table.

***Input Format***

The first line contains an integer N (integer), representing the number of characters in the code table.

Each of the next N lines contains a character C and its corresponding Huffman code (a binary string of 0s and 1s), separated by a space.

The last line contains the input message (a string of characters) that needs to be encoded. Every character in the message is guaranteed to have a corresponding entry in the code table.

***Output Format***

The output prints 'Coded Data: ' followed by a binary string (a sequence of 0s and 1s) representing the encoded message, obtained by concatenating the Huffman code of each character in the input message in order.

Refer to the sample output for the formatting specifications.

***Sample Test Case***

Input: 10
D 0100
U 0000
F 0101
M 0001
A 0010
N 0011
C 1100
O 1101
E 1110
H 1111
HUFF

Output: Coded Data: 1111000001010101

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <string.h>

int main() {
    int N;
    scanf("%d", &N);
```

```
    char ch[26];
    char code[26][25];

    // Read code table
    for (int i = 0; i < N; i++) {
        scanf(" %c %s", &ch[i], code[i]);
    }

    char msg[105];
    scanf("%s", msg);

    printf("Coded Data: ");

    // Encode message
    for (int i = 0; msg[i] != '\0'; i++) {
        for (int j = 0; j < N; j++) {
            if (msg[i] == ch[j]) {
                printf("%s", code[j]);
                break;
            }
        }
    }

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

# Vel Tech Group of Institutions

Name: NIKHIL  KUMAR
Email: vtu28682@veltech.edu.in
Roll no: vtu28682
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

## 2028_NeoColab_Design and Analysis of AlgorithmsS1L10

### VTU_DAA_Week 7_PAH

Attempt : 1
Total Mark : 50
Marks Obtained : 50

### Section 1 : Coding

1. Problem Statement

Bob is working on a project involving network optimization. He needs a program to find the Minimum Spanning Tree (MST) of a given undirected weighted graph with 5 nodes (numbered 0 to 4).

The graph is represented as a 5×5 symmetric adjacency matrix, where a value of 0 indicates no direct connection between two nodes. Implement Prim's Algorithm to find the MST and print each edge with its weight.

Example

Input

0 3 0 5 0

4 0 2 3 5

vtu28682

0 3 0 8 7

3 8 0 0 9

0 5 7 2 0

Output

Edge  Weight

0 - 1  4

1 - 2  3

1 - 3  8

1 - 4  5

Explanation

In the given 5x5 matrix representing an undirected graph, Bob has specified the weights of edges. The Prim's algorithm is applied to find the Minimum Spanning Tree (MST). The output displays the edges along with their corresponding weights, forming the optimal tree connecting all vertices. In this case, the MST includes edges (0-1), (1-2), (1-3), and (1-4) with weights 4, 3, 8, and 5 respectively, ensuring minimal total weight for network optimization.

***Input Format***

The input consists of 5 lines, each containing 5 space-separated integers, representing the adjacency matrix of the graph. A value of 0 indicates no direct connection between two nodes. The matrix is symmetric (graph[i][j] = graph[j][i]) since the graph is undirected.

***Output Format***

The first line of output prints the header: 'Edge \tWeight'

Each of the next 4 lines prints one MST edge in the format:

'u - v \tw'

vtu28682

where u is the parent node, v is the child node, and w is the integer edge weight, printed in ascending order of child node index (node-index order).

Refer to the sample output for formatting specifications.

Refer to the sample output for the formatting specifications.

***Sample Test Case***

Input: 0 2 0 6 0
2 0 3 8 5
0 3 0 8 7
6 8 0 0 9
0 5 7 9 0

Output: Edge  Weight
0 - 1  2
1 - 2  3
0 - 3  6
1 - 4  5

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <limits.h>

#define V 5

int minKey(int key[], int mstSet[]) {
    int min = INT_MAX, min_index = -1;

    for (int v = 0; v < V; v++) {
        if (!mstSet[v] && key[v] < min) {
            min = key[v];
            min_index = v;
        }
    }
    return min_index;
}
```

```
int main() {
    int graph[V][V];

    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    int parent[V], key[V], mstSet[V];

    for (int i = 0; i < V; i++) {
        key[i] = INT_MAX;
        mstSet[i] = 0;
    }

    key[0] = 0;
    parent[0] = -1;

    for (int count = 0; count < V - 1; count++) {

        int u = minKey(key, mstSet);
        mstSet[u] = 1;

        for (int v = 0; v < V; v++) {
            if (graph[u][v] && !mstSet[v] && graph[u][v] < key[v]) {
                key[v] = graph[u][v];
                parent[v] = u;
            }
        }
    }

    // Output header
    printf("Edge \tWeight\n");

    // Print in ascending order of child node index
    for (int v = 1; v < V; v++) {
        printf("%d - %d \t%d\n", parent[v], v, graph[v][parent[v]]);
    }

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

2. Problem Statement

You are given a set of cities in a country and the cost of building railway track segments to connect them. Your task is to find the minimum cost required to build the railway tracks and connect all the cities. The cost of each track segment is given, along with the source city and destination city to which it connects.

Write a program that takes the number of cities, the number of track segments, and the details of each track segment as input. Implement Kruskal's Algorithm to calculate the minimum cost of building the railway tracks.

Cities are numbered 0 to N-1.

Example

Input:

3

3

0 1 4

1 2 5

0 2 3

Output:

7

Explanation:

The input specifies 3 cities and 3 track segments with their corresponding weights.

The track segments are:

Track Segment 1: Connects city 0 and city 1, with a weight of 4.Track

Segment 2: Connects city 1 and city 2 with a weight of 5.Track Segment 3: Connects city 0 and city 2 with a weight of 3.The algorithm finds the minimum cost to build railway tracks connecting all cities. In this case, the minimum spanning tree includes track segment 3 (0-2 with weight 3) and track segment 1 (0-1 with weight 4). The total cost is 3 + 4 = 7.

***Input Format***

The first line of input contains an integer N, representing the number of cities (numbered 0 to N-1).

The second line contains an integer M, representing the number of track segments.

Each of the next M lines contains three space-separated integers: the source city, the destination city, and the weight (cost) of the track segment

***Output Format***

The first line of input contains an integer N, representing the number of cities (numbered 0 to N-1).

The second line contains an integer M, representing the number of track segments.

Each of the next M lines contains three space-separated integers: the source city, the destination city, and the weight (cost) of the track segment

Refer to the sample output for the formatting specifications.

***Sample Test Case***

Input: 4
5
0 1 1
0 2 4
1 2 2
1 3 5
2 3 3
Output: 6

vtu28682

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int u, v, w;
} Edge;

int parent[10];
int rankArr[10];

int find(int x) {
    if (parent[x] != x)
        parent[x] = find(parent[x]);
    return parent[x];
}

void unionSet(int a, int b) {
    int ra = find(a);
    int rb = find(b);

    if (ra != rb) {
        if (rankArr[ra] < rankArr[rb])
            parent[ra] = rb;
        else if (rankArr[ra] > rankArr[rb])
            parent[rb] = ra;
        else {
            parent[rb] = ra;
            rankArr[ra]++;
        }
    }
}

int cmp(const void *a, const void *b) {
    Edge *e1 = (Edge *)a;
    Edge *e2 = (Edge *)b;
    return e1->w - e2->w;
}

int main() {
    int N, M;
```

```
    scanf("%d", &N);
    scanf("%d", &M);

    Edge edges[M];

    for (int i = 0; i < M; i++) {
        scanf("%d %d %d", &edges[i].u, &edges[i].v, &edges[i].w);
    }

    qsort(edges, M, sizeof(Edge), cmp);

    for (int i = 0; i < N; i++) {
        parent[i] = i;
        rankArr[i] = 0;
    }

    int cost = 0;
    int count = 0;

    for (int i = 0; i < M; i++) {
        int u = edges[i].u;
        int v = edges[i].v;

        int pu = find(u);
        int pv = find(v);

        if (pu != pv) {
            unionSet(pu, pv);
            cost += edges[i].w;
            count++;
        }
    }

    // If MST does not include N-1 edges    disconnected
    if (count != N - 1) {
        printf("It is not possible to connect all cities.");
    } else {
        printf("%d", cost);
    }

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

3. Problem Statement

Raja is given a map of cities connected by bidirectional roads, where each road has an associated construction cost (weight). His task is to find the Minimum Spanning Tree (MST) of this network using Prim's Algorithm, starting from city 0, so that all cities are connected with the minimum total construction cost.

Prim's Algorithm works as follows:

Start from the source city (city 0). Mark it as included in the MST.At each step, select the edge with the minimum weight that connects a city already in the MST to a city not yet in the MST.Repeat until all cities are included in the MST.

The program should display each MST edge (the city included and the cost to connect it) and the total MST cost.

Note: The given graph is a connected, undirected, weighted graph. Self-loops and parallel edges are not present.

***Input Format***

The first line of input consists of two space-separated integers, n and m, where n represents the number of cities (vertices) and m represents the number of roads (edges).

The next m lines each consist of three space-separated integers u, v, and w, where u and v are the two cities connected by the road (0-indexed) and w is the construction cost (weight) of the road.

***Output Format***

The output consists of n lines.

The first n-1 lines each print the MST edge added, in the format:

"Edge: u - v, Cost: w", where u is the city already in the MST, v is the newly added city, and w is the integer edge weight. Edges are printed in the order they are added by Prim's algorithm starting from city 0.

The last line prints "Total MST Cost: t", where t is the total integer cost of the minimum spanning tree.

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 4 5
0 1 10
0 2 6
0 3 5
1 3 15
2 3 4

Output: Edge: 0 - 1, Cost: 10
Edge: 3 - 2, Cost: 4
Edge: 0 - 3, Cost: 5
Total MST Cost: 19

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <limits.h>

#define MAX 100

int minKey(int key[], int mstSet[], int n) {
   int min = INT_MAX, min_index = -1;

   for (int v = 0; v < n; v++) {
      if (!mstSet[v] && key[v] < min) {
         min = key[v];
         min_index = v;
      }
```

```
    }
    return min_index;
}

int main() {
    int n, m;
    scanf("%d %d", &n, &m);

    int graph[MAX][MAX];

    // initialize matrix
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            graph[i][j] = 0;

    // read edges (undirected)
    for (int i = 0; i < m; i++) {
        int u, v, w;
        scanf("%d %d %d", &u, &v, &w);
        graph[u][v] = w;
        graph[v][u] = w;
    }

    int parent[MAX];
    int key[MAX];
    int mstSet[MAX];

    for (int i = 0; i < n; i++) {
        key[i] = INT_MAX;
        mstSet[i] = 0;
    }

    key[0] = 0;
    parent[0] = -1;

    int totalCost = 0;

    printf("");

    for (int count = 0; count < n - 1; count++) {

        int u = minKey(key, mstSet, n);
```

```
        mstSet[u] = 1;

        for (int v = 0; v < n; v++) {
            if (graph[u][v] && !mstSet[v] && graph[u][v] < key[v]) {
                parent[v] = u;
                key[v] = graph[u][v];
            }
        }
    }

    // Print edges in Prim order (as required: order added)
    for (int i = 1; i < n; i++) {
        printf("Edge: %d - %d, Cost: %d\n",
               parent[i], i, graph[i][parent[i]]);
        totalCost += graph[i][parent[i]];
    }

    printf("Total MST Cost: %d", totalCost);

    return 0;
}
```

***Status :*** Correct **Marks : 10/10**

4. Problem Statement

Riya is working as a data scientist in a company that deals with large text data. Due to storage and bandwidth limitations, she needs a way to compress text while maintaining its readability.

She decides to use Huffman Encoding, a well-known algorithm that assigns shorter binary codes to frequently occurring characters and longer binary codes to less common characters, thus reducing the overall message size.

Given a set of characters with their respective frequencies, help Riya generate Huffman Codes for each character. Then, using these codes, encode a given message.

Assign '0' to left branches and '1' to right branches, merge the two nodes

with the lowest frequency at each step. Clarify that the message will only contain characters from the given input set. Specify that Huffman codes are printed in the order they appear during in-order/pre-order traversal of the Huffman tree (matching sample output)

The message will only contain characters from the provided character set.

***Input Format***

The first line of input consists of an integer n, representing the number of unique characters.

The next n lines each consist of a character (an uppercase or lowercase English letter) and a positive integer representing its frequency, separated by a space.

The last line of input consists of a string representing the message to be encoded. The message will only contain characters from the provided character set.

***Output Format***

The first n lines of output print the Huffman code for each character in the format:

"character: code"

where the characters are printed in the order they appear as leaf nodes during the Huffman tree traversal (left child before right child, assigning '0' to left and '1' to right).

All n input characters and their codes are printed, regardless of whether they appear in the message.

The last line of output prints the encoded message as a sequence of 0s and 1s obtained by replacing each character in the message with its corresponding Huffman code.

Refer to the sample output for formatting specifications.

***Sample Test Case***

vtu28682

Input: 3
A 1
B 2
C 3
ABCCBAABBC
Output: C: 0
A: 10
B: 11
10110011101011110

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX 1000

typedef struct Node {
    char ch;
    int freq;
    struct Node *left, *right;
} Node;

typedef struct {
    Node* arr[MAX];
    int size;
} MinHeap;

// Create node
Node* createNode(char ch, int freq) {
    Node* node = (Node*)malloc(sizeof(Node));
    node->ch = ch;
    node->freq = freq;
    node->left = node->right = NULL;
    return node;
}

// Heap swap
void swap(Node** a, Node** b) {
    Node* t = *a;
    *a = *b;
```

```
    *b = t;
}

// Heapify
void heapify(MinHeap* h, int i) {
    int smallest = i;
    int l = 2*i + 1;
    int r = 2*i + 2;

    if (l < h->size && h->arr[l]->freq < h->arr[smallest]->freq)
        smallest = l;

    if (r < h->size && h->arr[r]->freq < h->arr[smallest]->freq)
        smallest = r;

    if (smallest != i) {
        swap(&h->arr[i], &h->arr[smallest]);
        heapify(h, smallest);
    }
}

// Extract min
Node* extractMin(MinHeap* h) {
    Node* temp = h->arr[0];
    h->arr[0] = h->arr[h->size - 1];
    h->size--;
    heapify(h, 0);
    return temp;
}

// Insert heap
void insertHeap(MinHeap* h, Node* node) {
    h->arr[h->size] = node;
    int i = h->size;
    h->size++;

    while (i && h->arr[i]->freq < h->arr[(i-1)/2]->freq) {
        swap(&h->arr[i], &h->arr[(i-1)/2]);
        i = (i-1)/2;
    }
}
```

```
// Build Huffman tree
Node* buildTree(char chars[], int freq[], int n) {
    MinHeap h;
    h.size = 0;

    for (int i = 0; i < n; i++)
        insertHeap(&h, createNode(chars[i], freq[i]));

    while (h.size > 1) {
        Node* l = extractMin(&h);
        Node* r = extractMin(&h);

        Node* merged = createNode('$', l->freq + r->freq);
        merged->left = l;
        merged->right = r;

        insertHeap(&h, merged);
    }

    return extractMin(&h);
}

// Store codes
char codes[256][100];

void generateCodes(Node* root, char* path, int depth) {
    if (!root) return;

    if (!root->left && !root->right) {
        path[depth] = '\0';
        strcpy(codes[(int)root->ch], path);
        return;
    }

    path[depth] = '0';
    generateCodes(root->left, path, depth + 1);

    path[depth] = '1';
    generateCodes(root->right, path, depth + 1);
}

// Print codes in traversal order
```

```
void printCodes(Node* root) {
    if (!root) return;

    if (!root->left && !root->right) {
        printf("%c: %s\n", root->ch, codes[(int)root->ch]);
        return;
    }

    printCodes(root->left);
    printCodes(root->right);
}

int main() {
    int n;
    scanf("%d", &n);

    char chars[n];
    int freq[n];

    for (int i = 0; i < n; i++) {
        scanf(" %c %d", &chars[i], &freq[i]);
    }

    char msg[300];
    scanf("%s", msg);

    Node* root = buildTree(chars, freq, n);

    char path[100];
    generateCodes(root, path, 0);

    // Print Huffman codes
    printCodes(root);

    // Encode message
    for (int i = 0; msg[i] != '\0'; i++) {
        printf("%s", codes[(int)msg[i]]);
    }

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

5. Problem Statement

In a clandestine world of spies and covert operations, a group of operatives uses Huffman coding to encode and decode their classified messages securely.

Huffman coding assigns shorter binary codes to more frequently occurring characters and longer codes to less frequent ones. The algorithm works as follows:

Each character and its frequency is inserted into a min-heap (priority queue).At each step, extract the two nodes with the lowest frequencies, merge them into a new internal node whose frequency is the sum of the two, and reinsert it into the heap.Repeat until only one node (the root) remains — this is the Huffman tree.Assign '0' to every left branch and '1' to every right branch. The binary string from root to each leaf node is that character's Huffman code.

Given a set of characters with their frequencies, build the Huffman tree, print the codes for all characters, encode the given message using these codes, and decode the encoded message back to the original.

Note: The message will only contain characters from the provided input set. All character frequencies are unique.

***Input Format***

The first line of input consists of a positive integer N, representing the number of unique characters.

The next N lines each consist of a character (an uppercase or lowercase English letter) and a positive integer representing its frequency, separated by a space.

The last line of input consists of a string representing the secret message to be encoded. The message contains only characters from the provided character set

***Output Format***

The first N lines of output each print the Huffman code for a character in the format:

"character: code"

where characters are printed in the order they appear as leaf nodes during Huffman tree traversal (left child before right child, with '0' assigned to left and '1' to right).

The next line prints the encoded message — a sequence of 0s and 1s obtained by replacing each character in the message with its Huffman code.

The last line prints the decoded message obtained by traversing the Huffman tree using the encoded bits, which must match the original input message.

Refer to the sample outputs for better understanding and formatting.

***Sample Test Case***

Input: 5
A 5
B 9
C 12
D 13
E 16
ABCDE

Output: C: 00
D: 01
A: 100
B: 101
E: 11
100101000111
ABCDE

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```
#define MAX 100

typedef struct Node {
    char ch;
    int freq;
    struct Node *left, *right;
} Node;

typedef struct {
    Node* arr[MAX];
    int size;
} MinHeap;

// Create node
Node* createNode(char ch, int freq) {
    Node* node = (Node*)malloc(sizeof(Node));
    node->ch = ch;
    node->freq = freq;
    node->left = node->right = NULL;
    return node;
}

// Swap
void swap(Node** a, Node** b) {
    Node* t = *a;
    *a = *b;
    *b = t;
}

// Heapify
void heapify(MinHeap* h, int i) {
    int smallest = i;
    int l = 2*i + 1;
    int r = 2*i + 2;

    if (l < h->size && h->arr[l]->freq < h->arr[smallest]->freq)
        smallest = l;

    if (r < h->size && h->arr[r]->freq < h->arr[smallest]->freq)
        smallest = r;
```

```
    if (smallest != i) {
        swap(&h->arr[i], &h->arr[smallest]);
        heapify(h, smallest);
    }
}

// Extract min
Node* extractMin(MinHeap* h) {
    Node* temp = h->arr[0];
    h->arr[0] = h->arr[h->size - 1];
    h->size--;
    heapify(h, 0);
    return temp;
}

// Insert heap
void insertHeap(MinHeap* h, Node* node) {
    h->arr[h->size] = node;
    int i = h->size;
    h->size++;

    while (i && h->arr[i]->freq < h->arr[(i-1)/2]->freq) {
        swap(&h->arr[i], &h->arr[(i-1)/2]);
        i = (i-1)/2;
    }
}

// Build Huffman Tree
Node* buildTree(char ch[], int freq[], int n) {
    MinHeap h;
    h.size = 0;

    for (int i = 0; i < n; i++)
        insertHeap(&h, createNode(ch[i], freq[i]));

    while (h.size > 1) {
        Node* l = extractMin(&h);
        Node* r = extractMin(&h);

        Node* newNode = createNode('$', l->freq + r->freq);
        newNode->left = l;
        newNode->right = r;
```

```
        insertHeap(&h, newNode);
    }

    return extractMin(&h);
}

// Store codes
char codes[256][100];

// Generate Huffman codes
void generateCodes(Node* root, char* path, int depth) {
    if (!root) return;

    if (!root->left && !root->right) {
        path[depth] = '\0';
        codes[(int)root->ch][0] = '\0';
        strcpy(codes[(int)root->ch], path);
        return;
    }

    path[depth] = '0';
    generateCodes(root->left, path, depth + 1);

    path[depth] = '1';
    generateCodes(root->right, path, depth + 1);
}

// Print codes in leaf order
void printCodes(Node* root) {
    if (!root) return;

    if (!root->left && !root->right) {
        printf("%c: %s\n", root->ch, codes[(int)root->ch]);
        return;
    }

    printCodes(root->left);
    printCodes(root->right);
}

// Decode function
```

```
void decode(Node* root, char* encoded, char* decoded) {
    Node* temp = root;
    int k = 0;

    for (int i = 0; encoded[i] != '\0'; i++) {
        if (encoded[i] == '0')
            temp = temp->left;
        else
            temp = temp->right;

        if (!temp->left && !temp->right) {
            decoded[k++] = temp->ch;
            temp = root;
        }
    }
    decoded[k] = '\0';
}

int main() {
    int n;
    scanf("%d", &n);

    char ch[n];
    int freq[n];

    for (int i = 0; i < n; i++) {
        scanf(" %c %d", &ch[i], &freq[i]);
    }

    char msg[100];
    scanf("%s", msg);

    Node* root = buildTree(ch, freq, n);

    char path[100];
    generateCodes(root, path, 0);

    // Print Huffman codes
    printCodes(root);

    // Encode message
    char encoded[1000] = "";
```

```
    for (int i = 0; msg[i] != '\0'; i++) {
        strcat(encoded, codes[(int)msg[i]]);
    }

    printf("%s\n", encoded);

    // Decode message
    char decoded[100];
    decode(root, encoded, decoded);

    printf("%s", decoded);

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

# Vel Tech Group of Institutions

Name: NIKHIL  KUMAR
Email: vtu28682@veltech.edu.in
Roll no: vtu28682
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

## 2028_NeoColab_Design and Analysis of AlgorithmsS1L10

### VTU_DAA_Week 7_CY

Attempt : 1
Total Mark : 50
Marks Obtained : 30

### Section 1 : Coding

1. Problem Statement

Mary is planning to build a railway network to connect cities efficiently. She needs a program that, given the number of cities and the segments between them, determines the minimum cost to connect all cities using an undirected weighted graph.

Implement a solution using Kruskal's Algorithm to find the Minimum Spanning Tree (MST) of the city network. If it is not possible to connect all cities, output an appropriate message

Example

Input

3

vtu28682

3

0 1 4

1 2 5

0 2 3

Output

7

Explanation

The input specifies 3 cities and 3 track segments with their corresponding weights.

The track segments are:

Track Segment 1: Connects city 0 and city 1, with a weight of 4.

Track Segment 2: Connects city 1 and city 2 with a weight of 5.

Track Segment 3: Connects city 0 and city 2 with a weight of 3.

The algorithm finds the minimum cost to build railway tracks connecting all cities. In this case, the minimum spanning tree includes track segment 3 (0-2 with weight 3) and track segment 1 (0-1 with weight 4). The total cost is 3 + 4 = 7.

***Input Format***

The first line of input contains an integer num_cities, representing the number of cities (numbered 0 to num_cities - 1).

The second line contains an integer num_segments, representing the number of undirected segments (edges) between cities.

The next num_segments lines each contain three space-separated integers: source, destination, and weight, representing an undirected segment between city source and city destination with the given cost.

***Output Format***

The first line of output prints an integer representing the minimum cost to connect all cities.

vtu28682

If it is not possible to connect all the cities, the output prints 'It is not possible to connect all cities' (without a period).

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 3
3
0 1 4
1 2 5
0 2 3
Output: 7

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int u, v, w;
} Edge;

int parent[10];
int rankArr[10];

int find(int x) {
    if (parent[x] != x)
        parent[x] = find(parent[x]);
    return parent[x];
}

void unionSet(int a, int b) {
    int ra = find(a);
    int rb = find(b);

    if (ra != rb) {
        if (rankArr[ra] < rankArr[rb])
            parent[ra] = rb;
        else if (rankArr[ra] > rankArr[rb])
```

```
            parent[rb] = ra;
        else {
            parent[rb] = ra;
            rankArr[ra]++;
        }
    }
}

int cmp(const void *a, const void *b) {
    Edge *e1 = (Edge *)a;
    Edge *e2 = (Edge *)b;
    return e1->w - e2->w;
}

int main() {
    int n, m;
    scanf("%d", &n);
    scanf("%d", &m);

    Edge edges[m];

    for (int i = 0; i < m; i++) {
        scanf("%d %d %d", &edges[i].u, &edges[i].v, &edges[i].w);
    }

    qsort(edges, m, sizeof(Edge), cmp);

    for (int i = 0; i < n; i++) {
        parent[i] = i;
        rankArr[i] = 0;
    }

    int cost = 0;
    int edgesUsed = 0;

    for (int i = 0; i < m; i++) {
        int u = edges[i].u;
        int v = edges[i].v;

        int pu = find(u);
        int pv = find(v);
```

```
        if (pu != pv) {
            unionSet(pu, pv);
            cost += edges[i].w;
            edgesUsed++;
        }
    }

    // Check if MST is complete
    if (edgesUsed == n - 1)
        printf("%d", cost);
    else
        printf("It is not possible to connect all cities");

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

2. Problem Statement

Maya, a project planner, must connect multiple office branches using network cables while keeping the total cost within a given budget. Each possible connection between two branches has an associated cost.

Write a program to find the Minimum Spanning Tree (MST) of the branch network using Kruskal's Algorithm on an undirected weighted graph. If the total cost of the MST is within the given budget, display the cost. Otherwise, indicate that it is not possible.

It is guaranteed that the graph is connected in all test cases

***Input Format***

The first line contains three space-separated integers V, E, and budget, representing the number of branches (vertices, numbered 1 to V), the number of possible connections (edges), and the total budget respectively.

Each of the next E lines contains three space-separated integers src, dest, and weight, representing an undirected connection between branch src and branch dest with cost weight.

***Output Format***

If the MST cost is within the budget, the output prints:

'Minimum Cost Spanning Tree within the budget: X'

where X is an integer representing the total minimum cost.

Otherwise, the output prints:

'A valid MST within the budget is not possible.'

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 3 3 5
1 2 2
2 3 3
3 1 4
Output: Minimum Cost Spanning Tree within the budget: 5

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int u, v, w;
} Edge;

int parent[105];
int rankArr[105];

int find(int x) {
    if (parent[x] != x)
        parent[x] = find(parent[x]);
    return parent[x];
}

void unionSet(int a, int b) {
```

```
    int ra = find(a);
    int rb = find(b);

    if (ra != rb) {
        if (rankArr[ra] < rankArr[rb])
            parent[ra] = rb;
        else if (rankArr[ra] > rankArr[rb])
            parent[rb] = ra;
        else {
            parent[rb] = ra;
            rankArr[ra]++;
        }
    }
}

int cmp(const void *a, const void *b) {
    Edge *e1 = (Edge *)a;
    Edge *e2 = (Edge *)b;
    return e1->w - e2->w;
}

int main() {
    int V, E;
    long long budget;

    scanf("%d %d %lld", &V, &E, &budget);

    Edge edges[E];

    for (int i = 0; i < E; i++) {
        scanf("%d %d %d", &edges[i].u, &edges[i].v, &edges[i].w);
    }

    qsort(edges, E, sizeof(Edge), cmp);

    for (int i = 1; i <= V; i++) {
        parent[i] = i;
        rankArr[i] = 0;
    }

    long long cost = 0;
    int edgesUsed = 0;
```

```
    for (int i = 0; i < E && edgesUsed < V - 1; i++) {
        int u = edges[i].u;
        int v = edges[i].v;

        int pu = find(u);
        int pv = find(v);

        if (pu != pv) {
            unionSet(pu, pv);
            cost += edges[i].w;
            edgesUsed++;
        }
    }

    if (cost <= budget)
        printf("Minimum Cost Spanning Tree within the budget: %lld", cost);
    else
        printf("A valid MST within the budget is not possible.");

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

3. Problem Statement

Sita is planning to connect multiple locations in her city with roads. Each possible road between two locations has a construction distance (in kilometres). The city is represented as an undirected weighted graph using an adjacency matrix, where a value of 0 indicates no direct road between two locations (locations are numbered 0 to numLocations - 1).

To minimise the overall construction distance, she decides to use Prim's Algorithm to construct the Minimum Spanning Tree (MST). Help her find the MST edges and the total minimum distance.

***Input Format***

The first line of input contains an integer numLocations, representing the number of locations (numbered 0 to numLocations - 1).

The next numLocations lines each contain numLocations space-separated integers representing the adjacency matrix, where graph[i][j] is the distance between location i and location j. A value of 0 indicates no direct road between those two locations.

***Output Format***

The first line of output prints: 'Location\tDistance'

Each of the next numLocations - 1 lines prints the MST edge in the format:

'parent -> child \t\t\t dist km'

where parent is the parent location number, child is the child location number, and dist is the edge weight (integer) in kilometres.

The last line of output prints: 'Minimum total distance: totalCost km'

where totalCost is an integer.

Refer to the sample output for the formatting specifications.

***Sample Test Case***

Input: 2
0 5
5 0
Output: Location  Distance
0 -> 1  5 km
Minimum total distance: 5 km

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <limits.h>

#define MAX 10
```

```
int minKey(int key[], int mstSet[], int n) {
    int min = INT_MAX, min_index = -1;

    for (int v = 0; v < n; v++) {
        if (!mstSet[v] && key[v] < min) {
            min = key[v];
            min_index = v;
        }
    }
    return min_index;
}

int main() {
    int n;
    scanf("%d", &n);

    int graph[MAX][MAX];

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    int parent[MAX];
    int key[MAX];
    int mstSet[MAX];

    for (int i = 0; i < n; i++) {
        key[i] = INT_MAX;
        mstSet[i] = 0;
    }

    key[0] = 0;
    parent[0] = -1;

    for (int count = 0; count < n - 1; count++) {

        int u = minKey(key, mstSet, n);
        mstSet[u] = 1;
```

```
        for (int v = 0; v < n; v++) {
            if (graph[u][v] && !mstSet[v] && graph[u][v] < key[v]) {
                key[v] = graph[u][v];
                parent[v] = u;
            }
        }
    }

    // Output
    printf("Location\tDistance\n");

    int totalCost = 0;

    for (int i = 1; i < n; i++) {
        printf("%d -> %d\t\t\t%d km\n", parent[i], i, graph[i][parent[i]]);
        totalCost += graph[i][parent[i]];
    }

    printf("Minimum total distance: %d km", totalCost);

    return 0;
}
```

***Status :*** Wrong ***Marks : 0/10***

4. Problem Statement

Riya works at a data tools company that needs a compact representation of frequently used characters for fast transmission. She decides to use Huffman Coding, a lossless compression technique that assigns shorter bit-strings to more frequent symbols and longer bit-strings to less frequent ones.

The Huffman codes should be printed in the order that leaf nodes are encountered during a left-to-right traversal of the constructed Huffman tree.

Write a program that builds a Huffman tree from a list of symbols with their frequencies and prints the Huffman codes for each symbol.

***Input Format***

The first line contains an integer n, representing the number of distinct symbols.

Each of the next n lines contains a single printable character (the symbol) followed by a space and a positive integer (the frequency), representing the symbol and its occurrence count.

***Output Format***

The output prints n lines, one per symbol, in the order that leaf nodes are encountered during a left-to-right traversal of the Huffman tree (left child before right child at every internal node).

Each line prints the symbol followed by a colon, a space, and its Huffman code (a binary string of 0s and 1s):

'<symbol>: <binary_code>'

Refer to the sample output for the formatting specifications.

***Sample Test Case***

Input: 2
a 1
b 2

Output: a: 0
b: 1

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    char ch;
    int freq;
    struct Node *left, *right;
```

```
} Node;

typedef struct {
    Node* arr[300];
    int size;
} MinHeap;

// Create node
Node* createNode(char ch, int freq) {
    Node* n = (Node*)malloc(sizeof(Node));
    n->ch = ch;
    n->freq = freq;
    n->left = n->right = NULL;
    return n;
}

// Swap
void swap(Node** a, Node** b) {
    Node* t = *a;
    *a = *b;
    *b = t;
}

// Heapify
void heapify(MinHeap* h, int i) {
    int smallest = i;
    int l = 2*i + 1;
    int r = 2*i + 2;

    if (l < h->size && h->arr[l]->freq < h->arr[smallest]->freq)
        smallest = l;

    if (r < h->size && h->arr[r]->freq < h->arr[smallest]->freq)
        smallest = r;

    if (smallest != i) {
        swap(&h->arr[i], &h->arr[smallest]);
        heapify(h, smallest);
    }
}

// Extract min
```

```
Node* extractMin(MinHeap* h) {
    Node* temp = h->arr[0];
    h->arr[0] = h->arr[h->size - 1];
    h->size--;
    heapify(h, 0);
    return temp;
}

// Insert heap
void insertHeap(MinHeap* h, Node* node) {
    h->arr[h->size] = node;
    int i = h->size;
    h->size++;

    while (i && h->arr[i]->freq < h->arr[(i-1)/2]->freq) {
        swap(&h->arr[i], &h->arr[(i-1)/2]);
        i = (i-1)/2;
    }
}

// Build Huffman tree
Node* buildTree(char ch[], int freq[], int n) {
    MinHeap h;
    h.size = 0;

    for (int i = 0; i < n; i++)
        insertHeap(&h, createNode(ch[i], freq[i]));

    while (h.size > 1) {
        Node* l = extractMin(&h);
        Node* r = extractMin(&h);

        Node* newNode = createNode('$', l->freq + r->freq);
        newNode->left = l;
        newNode->right = r;

        insertHeap(&h, newNode);
    }

    return extractMin(&h);
}
```

```
// Store codes
char codes[256][100];

// Generate codes
void generate(Node* root, char* path, int depth) {
    if (!root) return;

    if (!root->left && !root->right) {
        path[depth] = '\0';
        codes[(int)root->ch][0] = '\0';
        for (int i = 0; i <= depth; i++)
            codes[(int)root->ch][i] = path[i];
        return;
    }

    path[depth] = '0';
    generate(root->left, path, depth + 1);

    path[depth] = '1';
    generate(root->right, path, depth + 1);
}

// Print in left-to-right leaf order
void printLeaves(Node* root) {
    if (!root) return;

    if (!root->left && !root->right) {
        printf("%c: %s\n", root->ch, codes[(int)root->ch]);
        return;
    }

    printLeaves(root->left);
    printLeaves(root->right);
}

int main() {
    int n;
    scanf("%d", &n);

    char ch[n];
    int freq[n];
```

```
    for (int i = 0; i < n; i++) {
        scanf(" %c %d", &ch[i], &freq[i]);
    }

    Node* root = buildTree(ch, freq, n);

    char path[100];
    generate(root, path, 0);

    printLeaves(root);

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

5. Problem Statement

In a data science class, Professor Huffman demonstrates how encoded messages can be decoded using a pre-built Huffman tree.

Given a Huffman code table (mapping each character to its binary code) and an encoded binary message, decode the message back to its original text by traversing the Huffman tree.

The code table is guaranteed to be a valid prefix-free Huffman code.

The encoded message consists only of '0' and '1' characters, and every bit maps to exactly one symbol in the Huffman tree

The tree is defined as follows:

Internal nodes are represented by "*".The left edge (0) leads to the symbol "A".The right edge (1) leads to the symbol "B".

This means:

"0"  A"1"  B

Your task is to decode a given encoded message (a string of 0s and 1s) back into its original form consisting of "A" and "B".

***Input Format***

The first line contains an integer n, representing the number of symbols in the code table.

Each of the next n lines contains a character and its Huffman code (binary string), separated by a space.

The last line contains the encoded binary message (a string of 0s and 1s) to be decoded

***Output Format***

The output prints the decoded message as a sequence of characters ('A' or 'B'), followed by a newline.

Refer to the sample output for formatting specifications

***Sample Test Case***

Input: 11100011100

Output: BBBAAABBBAA

***Answer***

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Node {
    char ch;
    struct Node *left, *right;
} Node;

Node* createNode() {
    Node* n = (Node*)malloc(sizeof(Node));
    n->ch = '\0';
    n->left = n->right = NULL;
    return n;
}

// Insert code into tree
void insert(Node* root, char ch, char* code) {
    Node* temp = root;
```

```
    for (int i = 0; code[i] != '\0'; i++) {
        if (code[i] == '0') {
            if (!temp->left) temp->left = createNode();
            temp = temp->left;
        } else {
            if (!temp->right) temp->right = createNode();
            temp = temp->right;
        }
    }
    temp->ch = ch;
}

// Decode message
void decode(Node* root, char* msg) {
    Node* temp = root;

    for (int i = 0; msg[i] != '\0'; i++) {
        if (msg[i] == '0')
            temp = temp->left;
        else
            temp = temp->right;

        if (temp->ch != '\0') {
            printf("%c", temp->ch);
            temp = root;
        }
    }
    printf("\n");
}

int main() {
    int n;
    scanf("%d", &n);

    Node* root = createNode();

    for (int i = 0; i < n; i++) {
        char ch, code[105];
        scanf(" %c %s", &ch, code);
        insert(root, ch, code);
    }
```

```
    char msg[105];
    scanf("%s", msg);

    decode(root, msg);

    return 0;
}
```

***Status :*** Wrong ***Marks : 0/10***

# Vel Tech Group of Institutions

Name: NIKHIL  KUMAR
Email: vtu28682@veltech.edu.in
Roll no: vtu28682
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

## 2028_NeoColab_Design and Analysis of AlgorithmsS1L10

### VTU_DAA_Week 7_CY

Attempt : 1
Total Mark : 50
Marks Obtained : 30

### Section 1 : Coding

1. Problem Statement

Mary is planning to build a railway network to connect cities efficiently. She needs a program that, given the number of cities and the segments between them, determines the minimum cost to connect all cities using an undirected weighted graph.

Implement a solution using Kruskal's Algorithm to find the Minimum Spanning Tree (MST) of the city network. If it is not possible to connect all cities, output an appropriate message

Example

Input

3

vtu28682

3

0 1 4

1 2 5

0 2 3

Output

7

Explanation

The input specifies 3 cities and 3 track segments with their corresponding weights.

The track segments are:

Track Segment 1: Connects city 0 and city 1, with a weight of 4.

Track Segment 2: Connects city 1 and city 2 with a weight of 5.

Track Segment 3: Connects city 0 and city 2 with a weight of 3.

The algorithm finds the minimum cost to build railway tracks connecting all cities. In this case, the minimum spanning tree includes track segment 3 (0-2 with weight 3) and track segment 1 (0-1 with weight 4). The total cost is 3 + 4 = 7.

***Input Format***

The first line of input contains an integer num_cities, representing the number of cities (numbered 0 to num_cities - 1).

The second line contains an integer num_segments, representing the number of undirected segments (edges) between cities.

The next num_segments lines each contain three space-separated integers: source, destination, and weight, representing an undirected segment between city source and city destination with the given cost.

***Output Format***

The first line of output prints an integer representing the minimum cost to connect all cities.

vtu28682

If it is not possible to connect all the cities, the output prints 'It is not possible to connect all cities' (without a period).

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 3
3
0 1 4
1 2 5
0 2 3
Output: 7

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int u, v, w;
} Edge;

int parent[10];
int rankArr[10];

int find(int x) {
    if (parent[x] != x)
        parent[x] = find(parent[x]);
    return parent[x];
}

void unionSet(int a, int b) {
    int ra = find(a);
    int rb = find(b);

    if (ra != rb) {
        if (rankArr[ra] < rankArr[rb])
            parent[ra] = rb;
        else if (rankArr[ra] > rankArr[rb])
```

```
            parent[rb] = ra;
        else {
            parent[rb] = ra;
            rankArr[ra]++;
        }
    }
}

int cmp(const void *a, const void *b) {
    Edge *e1 = (Edge *)a;
    Edge *e2 = (Edge *)b;
    return e1->w - e2->w;
}

int main() {
    int n, m;
    scanf("%d", &n);
    scanf("%d", &m);

    Edge edges[m];

    for (int i = 0; i < m; i++) {
        scanf("%d %d %d", &edges[i].u, &edges[i].v, &edges[i].w);
    }

    qsort(edges, m, sizeof(Edge), cmp);

    for (int i = 0; i < n; i++) {
        parent[i] = i;
        rankArr[i] = 0;
    }

    int cost = 0;
    int edgesUsed = 0;

    for (int i = 0; i < m; i++) {
        int u = edges[i].u;
        int v = edges[i].v;

        int pu = find(u);
        int pv = find(v);
```

```
        if (pu != pv) {
            unionSet(pu, pv);
            cost += edges[i].w;
            edgesUsed++;
        }
    }

    // Check if MST is complete
    if (edgesUsed == n - 1)
        printf("%d", cost);
    else
        printf("It is not possible to connect all cities");

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

2. Problem Statement

Maya, a project planner, must connect multiple office branches using network cables while keeping the total cost within a given budget. Each possible connection between two branches has an associated cost.

Write a program to find the Minimum Spanning Tree (MST) of the branch network using Kruskal's Algorithm on an undirected weighted graph. If the total cost of the MST is within the given budget, display the cost. Otherwise, indicate that it is not possible.

It is guaranteed that the graph is connected in all test cases

***Input Format***

The first line contains three space-separated integers V, E, and budget, representing the number of branches (vertices, numbered 1 to V), the number of possible connections (edges), and the total budget respectively.

Each of the next E lines contains three space-separated integers src, dest, and weight, representing an undirected connection between branch src and branch dest with cost weight.

***Output Format***

If the MST cost is within the budget, the output prints:

'Minimum Cost Spanning Tree within the budget: X'

where X is an integer representing the total minimum cost.

Otherwise, the output prints:

'A valid MST within the budget is not possible.'

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 3 3 5
1 2 2
2 3 3
3 1 4
Output: Minimum Cost Spanning Tree within the budget: 5

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int u, v, w;
} Edge;

int parent[105];
int rankArr[105];

int find(int x) {
    if (parent[x] != x)
        parent[x] = find(parent[x]);
    return parent[x];
}

void unionSet(int a, int b) {
```

```
    int ra = find(a);
    int rb = find(b);

    if (ra != rb) {
        if (rankArr[ra] < rankArr[rb])
            parent[ra] = rb;
        else if (rankArr[ra] > rankArr[rb])
            parent[rb] = ra;
        else {
            parent[rb] = ra;
            rankArr[ra]++;
        }
    }
}

int cmp(const void *a, const void *b) {
    Edge *e1 = (Edge *)a;
    Edge *e2 = (Edge *)b;
    return e1->w - e2->w;
}

int main() {
    int V, E;
    long long budget;

    scanf("%d %d %lld", &V, &E, &budget);

    Edge edges[E];

    for (int i = 0; i < E; i++) {
        scanf("%d %d %d", &edges[i].u, &edges[i].v, &edges[i].w);
    }

    qsort(edges, E, sizeof(Edge), cmp);

    for (int i = 1; i <= V; i++) {
        parent[i] = i;
        rankArr[i] = 0;
    }

    long long cost = 0;
    int edgesUsed = 0;
```

```
    for (int i = 0; i < E && edgesUsed < V - 1; i++) {
        int u = edges[i].u;
        int v = edges[i].v;

        int pu = find(u);
        int pv = find(v);

        if (pu != pv) {
            unionSet(pu, pv);
            cost += edges[i].w;
            edgesUsed++;
        }
    }

    if (cost <= budget)
        printf("Minimum Cost Spanning Tree within the budget: %lld", cost);
    else
        printf("A valid MST within the budget is not possible.");

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

3. Problem Statement

Sita is planning to connect multiple locations in her city with roads. Each possible road between two locations has a construction distance (in kilometres). The city is represented as an undirected weighted graph using an adjacency matrix, where a value of 0 indicates no direct road between two locations (locations are numbered 0 to numLocations - 1).

To minimise the overall construction distance, she decides to use Prim's Algorithm to construct the Minimum Spanning Tree (MST). Help her find the MST edges and the total minimum distance.

***Input Format***

The first line of input contains an integer numLocations, representing the number of locations (numbered 0 to numLocations - 1).

The next numLocations lines each contain numLocations space-separated integers representing the adjacency matrix, where graph[i][j] is the distance between location i and location j. A value of 0 indicates no direct road between those two locations.

***Output Format***

The first line of output prints: 'Location\tDistance'

Each of the next numLocations - 1 lines prints the MST edge in the format:

'parent -> child \t\t\t dist km'

where parent is the parent location number, child is the child location number, and dist is the edge weight (integer) in kilometres.

The last line of output prints: 'Minimum total distance: totalCost km'

where totalCost is an integer.

Refer to the sample output for the formatting specifications.

***Sample Test Case***

Input: 2
0 5
5 0
Output: Location  Distance
0 -> 1  5 km
Minimum total distance: 5 km

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <limits.h>

#define MAX 10
```

```
int minKey(int key[], int mstSet[], int n) {
    int min = INT_MAX, min_index = -1;

    for (int v = 0; v < n; v++) {
        if (!mstSet[v] && key[v] < min) {
            min = key[v];
            min_index = v;
        }
    }
    return min_index;
}

int main() {
    int n;
    scanf("%d", &n);

    int graph[MAX][MAX];

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    int parent[MAX];
    int key[MAX];
    int mstSet[MAX];

    for (int i = 0; i < n; i++) {
        key[i] = INT_MAX;
        mstSet[i] = 0;
    }

    key[0] = 0;
    parent[0] = -1;

    for (int count = 0; count < n - 1; count++) {

        int u = minKey(key, mstSet, n);
        mstSet[u] = 1;
```

```
        for (int v = 0; v < n; v++) {
            if (graph[u][v] && !mstSet[v] && graph[u][v] < key[v]) {
                key[v] = graph[u][v];
                parent[v] = u;
            }
        }
    }

    // Output
    printf("Location\tDistance\n");

    int totalCost = 0;

    for (int i = 1; i < n; i++) {
        printf("%d -> %d\t\t\t%d km\n", parent[i], i, graph[i][parent[i]]);
        totalCost += graph[i][parent[i]];
    }

    printf("Minimum total distance: %d km", totalCost);

    return 0;
}
```

***Status :*** Wrong ***Marks : 0/10***

4. Problem Statement

Riya works at a data tools company that needs a compact representation of frequently used characters for fast transmission. She decides to use Huffman Coding, a lossless compression technique that assigns shorter bit-strings to more frequent symbols and longer bit-strings to less frequent ones.

The Huffman codes should be printed in the order that leaf nodes are encountered during a left-to-right traversal of the constructed Huffman tree.

Write a program that builds a Huffman tree from a list of symbols with their frequencies and prints the Huffman codes for each symbol.

***Input Format***

The first line contains an integer n, representing the number of distinct symbols.

Each of the next n lines contains a single printable character (the symbol) followed by a space and a positive integer (the frequency), representing the symbol and its occurrence count.

***Output Format***

The output prints n lines, one per symbol, in the order that leaf nodes are encountered during a left-to-right traversal of the Huffman tree (left child before right child at every internal node).

Each line prints the symbol followed by a colon, a space, and its Huffman code (a binary string of 0s and 1s):

'<symbol>: <binary_code>'

Refer to the sample output for the formatting specifications.

***Sample Test Case***

Input: 2
a 1
b 2

Output: a: 0
b: 1

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    char ch;
    int freq;
    struct Node *left, *right;
```

```
} Node;

typedef struct {
    Node* arr[300];
    int size;
} MinHeap;

// Create node
Node* createNode(char ch, int freq) {
    Node* n = (Node*)malloc(sizeof(Node));
    n->ch = ch;
    n->freq = freq;
    n->left = n->right = NULL;
    return n;
}

// Swap
void swap(Node** a, Node** b) {
    Node* t = *a;
    *a = *b;
    *b = t;
}

// Heapify
void heapify(MinHeap* h, int i) {
    int smallest = i;
    int l = 2*i + 1;
    int r = 2*i + 2;

    if (l < h->size && h->arr[l]->freq < h->arr[smallest]->freq)
        smallest = l;

    if (r < h->size && h->arr[r]->freq < h->arr[smallest]->freq)
        smallest = r;

    if (smallest != i) {
        swap(&h->arr[i], &h->arr[smallest]);
        heapify(h, smallest);
    }
}

// Extract min
```

```
Node* extractMin(MinHeap* h) {
    Node* temp = h->arr[0];
    h->arr[0] = h->arr[h->size - 1];
    h->size--;
    heapify(h, 0);
    return temp;
}

// Insert heap
void insertHeap(MinHeap* h, Node* node) {
    h->arr[h->size] = node;
    int i = h->size;
    h->size++;

    while (i && h->arr[i]->freq < h->arr[(i-1)/2]->freq) {
        swap(&h->arr[i], &h->arr[(i-1)/2]);
        i = (i-1)/2;
    }
}

// Build Huffman tree
Node* buildTree(char ch[], int freq[], int n) {
    MinHeap h;
    h.size = 0;

    for (int i = 0; i < n; i++)
        insertHeap(&h, createNode(ch[i], freq[i]));

    while (h.size > 1) {
        Node* l = extractMin(&h);
        Node* r = extractMin(&h);

        Node* newNode = createNode('$', l->freq + r->freq);
        newNode->left = l;
        newNode->right = r;

        insertHeap(&h, newNode);
    }

    return extractMin(&h);
}
```

```
// Store codes
char codes[256][100];

// Generate codes
void generate(Node* root, char* path, int depth) {
    if (!root) return;

    if (!root->left && !root->right) {
        path[depth] = '\0';
        codes[(int)root->ch][0] = '\0';
        for (int i = 0; i <= depth; i++)
            codes[(int)root->ch][i] = path[i];
        return;
    }

    path[depth] = '0';
    generate(root->left, path, depth + 1);

    path[depth] = '1';
    generate(root->right, path, depth + 1);
}

// Print in left-to-right leaf order
void printLeaves(Node* root) {
    if (!root) return;

    if (!root->left && !root->right) {
        printf("%c: %s\n", root->ch, codes[(int)root->ch]);
        return;
    }

    printLeaves(root->left);
    printLeaves(root->right);
}

int main() {
    int n;
    scanf("%d", &n);

    char ch[n];
    int freq[n];
```

```
    for (int i = 0; i < n; i++) {
        scanf(" %c %d", &ch[i], &freq[i]);
    }

    Node* root = buildTree(ch, freq, n);

    char path[100];
    generate(root, path, 0);

    printLeaves(root);

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

5. Problem Statement

In a data science class, Professor Huffman demonstrates how encoded messages can be decoded using a pre-built Huffman tree.

Given a Huffman code table (mapping each character to its binary code) and an encoded binary message, decode the message back to its original text by traversing the Huffman tree.

The code table is guaranteed to be a valid prefix-free Huffman code.

The encoded message consists only of '0' and '1' characters, and every bit maps to exactly one symbol in the Huffman tree

The tree is defined as follows:

Internal nodes are represented by "*".The left edge (0) leads to the symbol "A".The right edge (1) leads to the symbol "B".

This means:

"0"  A"1"  B

Your task is to decode a given encoded message (a string of 0s and 1s) back into its original form consisting of "A" and "B".

***Input Format***

The first line contains an integer n, representing the number of symbols in the code table.

Each of the next n lines contains a character and its Huffman code (binary string), separated by a space.

The last line contains the encoded binary message (a string of 0s and 1s) to be decoded

***Output Format***

The output prints the decoded message as a sequence of characters ('A' or 'B'), followed by a newline.

Refer to the sample output for formatting specifications

***Sample Test Case***

Input: 11100011100

Output: BBBAAABBBAA

***Answer***

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Node {
    char ch;
    struct Node *left, *right;
} Node;

Node* createNode() {
    Node* n = (Node*)malloc(sizeof(Node));
    n->ch = '\0';
    n->left = n->right = NULL;
    return n;
}

// Insert code into tree
void insert(Node* root, char ch, char* code) {
    Node* temp = root;
```

```
    for (int i = 0; code[i] != '\0'; i++) {
        if (code[i] == '0') {
            if (!temp->left) temp->left = createNode();
            temp = temp->left;
        } else {
            if (!temp->right) temp->right = createNode();
            temp = temp->right;
        }
    }
    temp->ch = ch;
}

// Decode message
void decode(Node* root, char* msg) {
    Node* temp = root;

    for (int i = 0; msg[i] != '\0'; i++) {
        if (msg[i] == '0')
            temp = temp->left;
        else
            temp = temp->right;

        if (temp->ch != '\0') {
            printf("%c", temp->ch);
            temp = root;
        }
    }
    printf("\n");
}

int main() {
    int n;
    scanf("%d", &n);

    Node* root = createNode();

    for (int i = 0; i < n; i++) {
        char ch, code[105];
        scanf(" %c %s", &ch, code);
        insert(root, ch, code);
    }
```

```
    char msg[105];
    scanf("%s", msg);

    decode(root, msg);

    return 0;
}
```

***Status :*** Wrong ***Marks : 0/10***

# Vel Tech Group of Institutions

Name: NIKHIL  KUMAR
Email: vtu28682@veltech.edu.in
Roll no: vtu28682
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

## 2028_NeoColab_Design and Analysis of AlgorithmsS1L10

### VTU_DAA_Week 8_COD

Attempt : 1
Total Mark : 40
Marks Obtained : 30

### Section 1 : Coding

1. Problem Statement

Ram is tasked with finding the shortest distances from a source router to all other routers in a network. The network is represented as a graph of N routers connected by M bidirectional weighted links.

The program uses Dijkstra's Algorithm: starting from the source router, it repeatedly selects the unvisited router with the smallest known distance, then updates distances to its neighbors if a shorter path is found through it. This continues until all routers have been visited.

Given the network details and a source router, find and print the shortest distance from the source router to every router in the network.

***Input Format***

The first line of input consists of an integer N, representing the number of routers.

The second line of input consists of an integer M, representing the number of links.

The next M lines each consist of three space-separated integers: r1, r2, and w, representing a bidirectional link between router r1 and router r2 with weight w.

The last line of input consists of an integer s, representing the source router.

***Output Format***

The output consists of N lines printed in ascending order of router index (from 0 to N-1).

Each line prints two space-separated integers: the router index and its shortest distance from the source router.

The source router itself is printed with distance 0.

Refer to the sample output for the formatting specifications.

***Sample Test Case***

Input: 4
5
0 1 1
0 2 3
1 2 1
1 3 4
2 3 1
0
Output: 0 0
1 1
2 2
3 3

***Answer***

```
#include <stdio.h>
#include <limits.h>

// You are using GCC
```

```
#include <stdio.h>
#include <limits.h>

#define MAX_ROUTERS 10

void dijkstra(int graph[MAX_ROUTERS][MAX_ROUTERS],
              int adj[MAX_ROUTERS][MAX_ROUTERS],
              int source,
              int numRouters) {

    int dist[MAX_ROUTERS];
    int visited[MAX_ROUTERS];

    for (int i = 0; i < numRouters; i++) {
        dist[i] = INT_MAX;
        visited[i] = 0;
    }

    dist[source] = 0;

    for (int count = 0; count < numRouters - 1; count++) {

        int min = INT_MAX;
        int u = -1;

        // pick minimum distance unvisited node
        for (int i = 0; i < numRouters; i++) {
            if (!visited[i] && dist[i] <= min) {
                min = dist[i];
                u = i;
            }
        }

        visited[u] = 1;

        // relax neighbors
        for (int v = 0; v < numRouters; v++) {
            if (adj[u][v] &&
                !visited[v] &&
                dist[u] != INT_MAX &&
                dist[u] + graph[u][v] < dist[v]) {
```

```
            dist[v] = dist[u] + graph[u][v];
        }
    }
  }

  // print result
  for (int i = 0; i < numRouters; i++) {
    printf("%d %d\n", i, dist[i]);
  }
}

int main() {
  int numRouters, numLinks;
  scanf("%d", &numRouters);
  scanf("%d", &numLinks);
  int graph[MAX_ROUTERS][MAX_ROUTERS] = {0};
  int adj[MAX_ROUTERS][MAX_ROUTERS] = {0};
  for (int i = 0; i < numLinks; i++) {
    int router1, router2, weight;
    scanf("%d %d %d", &router1, &router2, &weight);
    graph[router1][router2] = weight;
    graph[router2][router1] = weight;
    adj[router1][router2] = 1;
    adj[router2][router1] = 1;
  }
  int source;
  scanf("%d", &source);
  dijkstra(graph, adj, source, numRouters);
  return 0;
}
```

***Status :*** Correct **Marks : 10/10**

2. Problem Statement

Alice is working on a delivery system for an e-commerce platform. The system uses a network of delivery hubs connected by roads, each with a certain travel time in minutes.

Alice needs to find the fastest delivery time from a source hub to every other hub in the network. Given the network details and a source hub, the

program uses Dijkstra's Algorithm to find and print the shortest delivery time from the source hub to all hubs in the network.

Dijkstra's Algorithm works by starting from the source hub with distance 0, then repeatedly selecting the unvisited hub with the smallest known time and updating the times to its neighbors if a shorter path is found through it. This continues until all hubs have been visited.

***Input Format***

The first line of input consists of an integer N, representing the number of delivery hubs.

The second line of input consists of an integer M, representing the number of roads between hubs.

The next M lines each consist of three space-separated integers: r1, r2, and t, representing a bidirectional road between hub r1 and hub r2 with a delivery time of t minutes.

The last line of input consists of an integer s, representing the source delivery hub.

***Output Format***

The output prints N lines representing the shortest delivery times from the source hub to all hubs (including the source) in ascending order of hub index.

Each line prints two space-separated integers: the hub index and its shortest delivery time from the source hub.

The delivery time of the source hub itself is 0.

Refer to the sample output for the formatting specifications.

***Sample Test Case***

Input: 3
3
0 1 2
0 2 4

vtu28682

1 2 1
0
Output: 0 0
1 2
2 3

***Answer***

```
#include <stdio.h>
#include <limits.h>

#define MAX_NODES 10
```


```
#include <limits.h>

#define MAX_NODES 10
```


```
int graph[MAX_NODES][MAX_NODES];
int dist[MAX_NODES];

void dijkstra(int N, int s) {

    int visited[MAX_NODES];

    for (int i = 0; i < N; i++) {
        dist[i] = INT_MAX;
        visited[i] = 0;
    }

    dist[s] = 0;

    for (int count = 0; count < N - 1; count++) {

        int min = INT_MAX, u = -1;

        for (int i = 0; i < N; i++) {
            if (!visited[i] && dist[i] <= min) {
                min = dist[i];
                u = i;
            }
        }

        visited[u] = 1;
```

```
        for (int v = 0; v < N; v++) {
            if (graph[u][v] != -1 &&
                !visited[v] &&
                dist[u] != INT_MAX &&
                dist[u] + graph[u][v] < dist[v]) {

                dist[v] = dist[u] + graph[u][v];
            }
        }
    }
}

int main() {
    int N, M, s;
    scanf("%d", &N);
    scanf("%d", &M);
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            graph[i][j] = -1;
        }
    }
    for (int i = 0; i < M; i++) {
        int r1, r2, w;
        scanf("%d %d %d", &r1, &r2, &w);
        graph[r1][r2] = w;
        graph[r2][r1] = w;
    }
    scanf("%d", &s);
    dijkstra(N, s);
    for (int i = 0; i < N; i++) {
        printf("%d %d\n", i, dist[i]);
    }
    return 0;
}
```

***Status :*** Correct **Marks : 10/10**

3. Problem statement

Create a program to calculate and display the topological order of a

Directed Acyclic Graph (DAG) with n vertices. The graph is represented as a lower triangular adjacency matrix, where for every vertex i and vertex j with j < i, the edge from i to j exists (value 1), and no edge exists from j to i (value 0). All diagonal and upper triangular values are 0.

The program should:

Construct and display the adjacency matrix of the graph.Perform a Depth First Search (DFS) traversal on the graph.Display the topological order of the vertices in reverse order of their DFS end times.

***Input Format***

The first line of input consists of an integer n, representing the number of vertices in the graph. No further input is required; the adjacency matrix is constructed internally by the program.

***Output Format***

he first line of output prints: Adjacency Matrix of the graph

The next n lines print the adjacency matrix row by row, where each value is preceded by a tab character (\t).

A blank line is printed after the matrix.

The next line prints: Topological order:

The final line prints the topological order of the vertices in the format: -->v1-->v2-->...-->vn, where vertices appear in reverse order of their DFS end times.

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 3

Output: Adjacency Matrix of the graph

```
	0	0	0
	1	0	0
	1	1	0
```

Topological order:

-->2-->1-->0

***Answer***

```
#include<stdio.h>

int s[100], res[100], top;

// You are using GCC
#include <stdio.h>

int s[100], res[100], top;

void buildMatrix(int a[100][100], int n) {

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {

            if (i > j)
                a[i][j] = 1;
            else
                a[i][j] = 0;
        }
    }
}

// DFS
void dfs(int v, int n, int a[100][100], int visited[100]) {

    visited[v] = 1;

    for (int i = 0; i < n; i++) {
        if (a[v][i] == 1 && !visited[i]) {
            dfs(i, n, a, visited);
        }
    }

    res[top++] = v;
}

void topological_order(int n, int a[100][100]) {

    int visited[100];
    top = 0;
```

```
    for (int i = 0; i < n; i++)
        visited[i] = 0;

    // run DFS for all nodes
    for (int i = 0; i < n; i++) {
        if (!visited[i]) {
            dfs(i, n, a, visited);
        }
    }
}

int main() {
    int a[100][100], n, i, j;
    scanf("%d", &n);
    buildMatrix(a, n);
    printf("Adjacency Matrix of the graph\n");
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++)
            printf("\t%d", a[i][j]);
        printf("\n");
    }
    printf("\nTopological order:\n");
    topological_order(n, a);
    for (i = top - 1; i >= 0; i--)
        printf("-->%d", res[i]);
    printf("\n");
    return 0;
}
```

***Status :*** Correct **Marks : 10/10**

4. Problem Statement

Write a program to implement and print the topological order of a directed acyclic graph (DAG). The program should take the number of vertices and the adjacency matrix of the graph as input and then output the topological order.

When multiple vertices have zero indegree simultaneously, process them in ascending order of their vertex number (1-indexed). The output vertices are

numbered starting from 1

***Input Format***

The first line of input consists of an integer N, representing the number of vertices in the graph.

The following N lines contain the adjacency matrix of the graph. Each line contains N space-separated integers (0 or 1), where 1 indicates a directed edge from the corresponding row vertex to the column vertex.

***Output Format***

The output prints the topological order of the vertices as space-separated 1-indexed integers on a single line.

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 3
1 0 0
0 1 0
1 0 0

Output: 3 1 2

***Answer***

```
#include <stdio.h>

int main() {
   int n;
   scanf("%d", &n);

   int adj[10][10];
   int indegree[10] = {0};

   // read matrix + compute indegree
   for (int i = 0; i < n; i++) {
      for (int j = 0; j < n; j++) {
         scanf("%d", &adj[i][j]);
         if (adj[i][j] == 1) {
```

```
            indegree[j]++;
        }
    }
}

int result[10];
int visited[10] = {0};
int idx = 0;

for (int count = 0; count < n; count++) {

    int u = -1;

    // ALWAYS pick smallest 0-indegree node
    for (int i = 0; i < n; i++) {
        if (!visited[i] && indegree[i] == 0) {
            u = i;
            break;
        }
    }

    visited[u] = 1;
    result[idx++] = u + 1;   // 1-indexed output

    // remove outgoing edges
    for (int v = 0; v < n; v++) {
        if (adj[u][v] == 1) {
            indegree[v]--;
        }
    }
}

// print result
for (int i = 0; i < n; i++) {
    printf("%d ", result[i]);
}

return 0;
}
```

***Status :*** Wrong ***Marks : 0/10***

# Vel Tech Group of Institutions

Name: NIKHIL  KUMAR
Email: vtu28682@veltech.edu.in
Roll no: vtu28682
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

## 2028_NeoColab_Design and Analysis of AlgorithmsS1L10

### VTU_DAA_Week 8_PAH

Attempt : 1
Total Mark : 50
Marks Obtained : 23

### Section 1 : Coding

1. Problem Statement

Sophia, a robotics engineer, is designing a task execution system where tasks have dependencies. Each task is represented as a node in a directed acyclic graph (DAG), and a directed edge (u  v) means task u must be completed before task v.

To determine the correct execution order, Sophia needs to perform a topological sort on the graph.

Your task is to help Sophia implement topological sorting and print the order in which tasks should be executed.

***Input Format***

The first line contains an integer N, representing the number of vertices (tasks)

in the DAG.

The second line contains an integer E, representing the number of directed edges (dependencies) in the graph.

The next E lines each contain two space-separated integers:

- src  The starting vertex of a directed edge.
- dest  The ending vertex of a directed edge.

***Output Format***

The output prints N space-separated integers in a single line, representing the vertices in topological order.

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 6
6
5 2
5 0
4 0
4 1
2 3
3 1
Output: 4 5 0 2 3 1

***Answer***

```
import java.util.*;

public class Main {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      int n = sc.nextInt();
      int e = sc.nextInt();

      int[][] adj = new int[n][n];
      int[] indegree = new int[n];
```

```
        for (int i = 0; i < e; i++) {
            int u = sc.nextInt();
            int v = sc.nextInt();

            adj[u][v] = 1;
            indegree[v]++;
        }

        ArrayList<Integer> result = new ArrayList<>();
        boolean[] visited = new boolean[n];

        // ? repeat N times to guarantee all nodes are picked
        for (int count = 0; count < n; count++) {

            int u = -1;

            // pick smallest available indegree 0 node
            for (int i = 0; i < n; i++) {
                if (!visited[i] && indegree[i] == 0) {
                    u = i;
                    break;
                }
            }

            visited[u] = true;
            result.add(u);

            for (int v = 0; v < n; v++) {
                if (adj[u][v] == 1) {
                    indegree[v]--;
                }
            }
        }

        for (int i = 0; i < n; i++) {
            System.out.print(result.get(i) + " ");
        }
    }
}
```

***Status :*** Partially correct

***Marks : 1/10***

2. Problem Statement

Write a program to find the topological order of vertices in a directed acyclic graph (DAG) represented as an adjacency list. This involves establishing a linear arrangement of vertices such that, for every directed edge (u, v), vertex u precedes vertex v in the sequence.

Your task specifically requires printing the topological order in the reverse order of their respective end times obtained through a depth-first search (DFS) traversal.

***Input Format***

The first line consists of an integer N, representing the number of vertices.

The second line consists of an integer E, representing the number of directed edges.

The next E lines each contain two space-separated integers x and y, representing a directed edge from vertex x to vertex y.

***Output Format***

The output prints Topological Order: followed by N space-separated integers representing the vertices (0-indexed) in topological order, in reverse order of DFS end times.

Refer to the sample output for the formatting specifications.

***Sample Test Case***

Input: 5
5
1 2
2 3
3 4
3 5
4 5
Output: Topological Order: 1 2 3 4 5

***Answer***

vtu28682

```
#include <stdio.h>

#define MAX 100

int adj[MAX][MAX];
int visited[MAX];
int stack[MAX];
int top = -1;

void dfs(int v, int n) {
    visited[v] = 1;

    for (int i = 0; i < n; i++) {
        if (adj[v][i] == 1 && !visited[i]) {
            dfs(i, n);
        }
    }

    stack[++top] = v;
}

int main() {
    int n, e;
    scanf("%d", &n);
    scanf("%d", &e);

    for (int i = 0; i < n; i++) {
        visited[i] = 0;
        for (int j = 0; j < n; j++) {
            adj[i][j] = 0;
        }
    }

    for (int i = 0; i < e; i++) {
        int x, y;
        scanf("%d %d", &x, &y);
        adj[x][y] = 1;
    }

    for (int i = 0; i < n; i++) {
        if (!visited[i]) {
            dfs(i, n);
```

```
    }
  }

  printf("Topological Order: ");

  for (int i = top; i >= 0; i--) {
    printf("%d ", stack[i]);
  }

  return 0;
}
```

***Status :*** Wrong **Marks : 0/10**

3. Problem Statement

Given a total of N tasks labeled from 0 to N-1. Some tasks may have prerequisites. A prerequisite pair [u, v] means task u must be completed before task v. Given the total number of tasks and a list of prerequisite pairs, return the ordering of tasks you should complete to finish all tasks. There may be multiple correct orders — return any one of them. If it is impossible to finish all tasks (due to a cycle), print nothing.

Examples

Input: N=2, Prerequisite_Pairs=[[1, 0]]

Output: [0, 1]

Explanation: There are a total of 2 tasks to pick. To pick task 1 you should have finished task 0. So the correct task order is [0, 1] .

Input: N=4, Prerequisite_Pairs=[[1, 0], [2, 0], [3, 1], [3, 2]]

Output: [0, 1, 2, 3] or [0, 2, 1, 3]

Explanation: There are a total of 4 tasks to pick. To pick task 3 you should have finished both tasks 1 and 2. Both tasks 1 and 2 should be pick after you finished task 0. So one correct task order is [0, 1, 2, 3]. Another correct ordering is [0, 2, 1, 3].

***Input Format***

The first line of input consists of an integer n, representing the number of tasks.

The second line consists of an integer p, representing the number of prerequisite pairs.

The next p lines each consist of two space-separated integers u and v, representing that task u must be completed before task v.

***Output Format***

The output prints the n tasks as space-separated integers on a single line, representing a valid task execution order. If no valid ordering exists, print nothing.

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 3
0
Output: 0 1 2

***Answer***

```
// You are using GCC
#include <stdio.h>

int main() {
    int n, p;
    scanf("%d", &n);
    scanf("%d", &p);

    int adj[101][101] = {0};
    int indegree[101] = {0};

    // build graph + indegree
    for (int i = 0; i < p; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        adj[u][v] = 1;
```

```
    indegree[v]++;
}

int queue[101];
int front = 0, rear = 0;

// push all 0 indegree nodes
for (int i = 0; i < n; i++) {
    if (indegree[i] == 0) {
        queue[rear++] = i;
    }
}

int topo[101];
int k = 0;

while (front < rear) {
    int node = queue[front++];
    topo[k++] = node;

    for (int i = 0; i < n; i++) {
        if (adj[node][i]) {
            indegree[i]--;
            if (indegree[i] == 0) {
                queue[rear++] = i;
            }
        }
    }
}

// cycle check
if (k != n) {
    // print nothing if cycle exists
    return 0;
}

// print result
for (int i = 0; i < k; i++) {
    printf("%d ", topo[i]);
}

return 0;
```

}

***Status :*** Correct ***Marks : 10/10***

4. Problem Statement

Raj is given a weighted directed graph with N vertices and the weights of the edges between the vertices.

His task is to implement Dijkstra's algorithm to find the shortest distance from a given source vertex to all other vertices in the graph.

***Input Format***

The first line contains an integer N, the number of vertices in the graph.

The next N lines each contain N space-separated integers, representing the adjacency matrix of the graph. The value at the i-th row and j-th column represents the weight of the edge from vertex i to vertex j. A value of 0 indicates no direct edge between the vertices.

The last line contains an integer src, representing the source vertex.

***Output Format***

The first line of output prints the heading: Vertex Distance from Source

The next N lines each print the vertex number and its shortest distance from the source vertex, separated by two tab characters. Vertices are numbered from 0 to N-1.

Refer to the sample output for formatting specifications

***Sample Test Case***

Input: 4
0 2 0 5
2 0 4 0
0 4 0 1
5 0 1 0

0
Output: Vertex Distance from Source
0 0
1 2
2 6
3 5

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <limits.h>

#define MAX 100

int minDistance(int dist[], int visited[], int n) {
    int min = INT_MAX, index = -1;

    for (int i = 0; i < n; i++) {
        if (!visited[i] && dist[i] <= min) {
            min = dist[i];
            index = i;
        }
    }
    return index;
}

void dijkstra(int graph[MAX][MAX], int n, int src) {
    int dist[MAX];
    int visited[MAX] = {0};

    // initialize distances
    for (int i = 0; i < n; i++)
        dist[i] = INT_MAX;

    dist[src] = 0;

    for (int count = 0; count < n - 1; count++) {
        int u = minDistance(dist, visited, n);
        visited[u] = 1;

        for (int v = 0; v < n; v++) {
            if (!visited[v] &&
```

```
            graph[u][v] != 0 &&
            dist[u] != INT_MAX &&
            dist[u] + graph[u][v] < dist[v]) {

            dist[v] = dist[u] + graph[u][v];
        }
    }
  }

  printf("Vertex Distance from Source\n");
  for (int i = 0; i < n; i++) {
    printf("%d\t\t%d\n", i, dist[i]);
  }
}

int main() {
  int n, src;
  int graph[MAX][MAX];

  scanf("%d", &n);

  for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
      scanf("%d", &graph[i][j]);
    }
  }

  scanf("%d", &src);

  dijkstra(graph, n, src);

  return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

5. Problem Statement

Anu is given a weighted, directed, and connected graph with V vertices and E edges represented as an adjacency list. They are also given a source vertex S.

The task is to find the shortest distance from the source vertex S to all other vertices in the graph using Dijkstra's algorithm.

***Input Format***

The first line contains an integer V representing the number of vertices in the graph.

The second line contains an integer E representing the number of edges in the graph.

The next E lines contain three integers each: from, to, and weight, denoting an edge between vertices from and to with the given weight.

The last line contains an integer S representing the source vertex.

***Output Format***

The output prints a single line in the format:

Shortest distances from vertex S to all other vertices: d0 d1 d2 ... dV-1

where S is the source vertex number and d0, d1, ..., dV-1 are the shortest distances from S to vertices 0 through V-1 respectively, each followed by a space.

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 2
1
0 1 9
0
Output: Shortest distances from vertex 0 to all other vertices: 0 9

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <limits.h>
```

```
#define MAX 100
#define INF 1000000000

typedef struct {
    int v, w;
} Edge;

Edge adj[MAX][MAX];
int adjSize[MAX];

int minDistance(int dist[], int visited[], int V) {
    int min = INF, index = -1;

    for (int i = 0; i < V; i++) {
        if (!visited[i] && dist[i] < min) {
            min = dist[i];
            index = i;
        }
    }
    return index;
}

void dijkstra(int V, int S) {
    int dist[MAX];
    int visited[MAX] = {0};

    for (int i = 0; i < V; i++)
        dist[i] = INF;

    dist[S] = 0;

    for (int count = 0; count < V - 1; count++) {
        int u = minDistance(dist, visited, V);
        if (u == -1) break;

        visited[u] = 1;

        for (int i = 0; i < adjSize[u]; i++) {
            int v = adj[u][i].v;
            int w = adj[u][i].w;

            if (!visited[v] && dist[u] != INF &&
```

```
            dist[u] + w < dist[v]) {
            dist[v] = dist[u] + w;
        }
    }
  }

  printf("Shortest distances from vertex %d to all other vertices: ", S);
  for (int i = 0; i < V; i++) {
    printf("%d ", dist[i]);
  }
}

int main() {
  int V, E;
  scanf("%d", &V);
  scanf("%d", &E);

  for (int i = 0; i < E; i++) {
    int u, v, w;
    scanf("%d %d %d", &u, &v, &w);

    adj[u][adjSize[u]].v = v;
    adj[u][adjSize[u]].w = w;
    adjSize[u]++;
  }

  int S;
  scanf("%d", &S);

  dijkstra(V, S);

  return 0;
}
```

***Status :*** Partially correct ***Marks : 2/10***

# Vel Tech Group of Institutions

Name: NIKHIL  KUMAR
Email: vtu28682@veltech.edu.in
Roll no: vtu28682
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

## 2028_NeoColab_Design and Analysis of AlgorithmsS1L10

### VTU_DAA_Week 8_CY

Attempt : 1
Total Mark : 50
Marks Obtained : 31

### Section 1 : Coding

1. Problem Statement

You are tasked with finding the shortest communication paths from a source router to all other routers in a network using Dijkstra's algorithm.

The network is represented as a graph, where routers are nodes and communication links between them are edges with weights representing the time or cost of communication. A value of 0 in the adjacency matrix indicates no direct link between two routers.

Example

Explanation:

The minimum distance from 0 to 1 = 4. Path: 0  1

The minimum distance from 0 to 2 = 12. Path: 0 1 2

The minimum distance from 0 to 3 = 19. Path: 0 1 2 3

The minimum distance from 0 to 4 = 21. Path: 0 7 6 5 4

The minimum distance from 0 to 5 = 11. Path: 0 7 6 5

The minimum distance from 0 to 6 = 9. Path: 0 7 6

The minimum distance from 0 to 7 = 8. Path: 0 7

The minimum distance from 0 to 8 = 14. Path: 0 1 2 8"

***Input Format***

The first line contains an integer V, representing the number of vertices in the graph.

The next V lines each contain V space-separated integers, representing the adjacency matrix of the graph. A value of 0 indicates no direct edge between the vertices.

The last line contains an integer src, representing the source vertex.

***Output Format***

The first line of output prints: Vertex followed by a tab character followed by Distance from Source

The next V lines each print the vertex number followed by three tab characters, a space, and the shortest distance from the source. Vertices are printed in ascending order from 0 to V-1.

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 4
0 1 3 1
1 0 4 2
3 4 0 5
1 2 5 0

vtu28682

1

Output: Vertex  Distance from Source

0  1

1  0

2  4

3  2

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <limits.h>

#define MAX 10
#define INF 1000000000

int minDistance(int dist[], int visited[], int V) {
    int min = INF, index = -1;

    for (int i = 0; i < V; i++) {
        if (!visited[i] && dist[i] < min) {
            min = dist[i];
            index = i;
        }
    }
    return index;
}

void dijkstra(int graph[MAX][MAX], int V, int src) {
    int dist[MAX];
    int visited[MAX] = {0};

    for (int i = 0; i < V; i++)
        dist[i] = INF;

    dist[src] = 0;

    for (int count = 0; count < V - 1; count++) {
        int u = minDistance(dist, visited, V);
        visited[u] = 1;

        for (int v = 0; v < V; v++) {
            if (!visited[v] &&
```

```
            graph[u][v] != 0 &&
            dist[u] != INF &&
            dist[u] + graph[u][v] < dist[v]) {

            dist[v] = dist[u] + graph[u][v];
        }
    }
  }

  printf("Vertex \tDistance from Source\n");

  for (int i = 0; i < V; i++) {
    printf("%d\t\t\t %d\n", i, dist[i]);
  }
}

int main() {
  int V;
  int graph[MAX][MAX];

  scanf("%d", &V);

  for (int i = 0; i < V; i++) {
    for (int j = 0; j < V; j++) {
      scanf("%d", &graph[i][j]);
    }
  }

  int src;
  scanf("%d", &src);

  dijkstra(graph, V, src);

  return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

2. Problem Statement

Given a network of cities represented as an unweighted graph, find the

shortest path between a source city and a destination city in terms of the minimum number of roads to be traversed. Implement a program that utilizes the Dijkstra algorithm to solve this problem.

The program should take the city network, source city, and destination city as inputs, and provide the shortest path length and the sequence of cities to be visited along the path as outputs.

If no path exists between the source and destination, print 'No path found'. Note: Since the graph is unweighted, all edges have an implicit weight of 1.

***Input Format***

The first line contains an integer n, representing the number of cities in the network.

The second line contains an integer m, representing the number of roads in the city network.

The following m lines contain pairs of integers u and v, representing a road between cities u and v.

The next line contains an integer s, representing the source city.

The last line contains an integer d, representing the destination city.

***Output Format***

The first line prints Shortest path length: followed by the number of edges in the shortest path.

The second line prints Path: followed by the city indices along the shortest path, each followed by a space.

If no path exists, print No path found.

Refer to the sample output for formatting specifications.

***Sample Test Case***

vtu28682

Input: 8
10
0 1
0 3
1 2
3 4
3 7
4 5
4 6
4 7
5 6
6 7
0
7
Output: Shortest path length: 2
Path: 0 3 7

***Answer***

```
// You are using GCC
#include <stdio.h>

int main() {
    int n, m;
    scanf("%d", &n);
    scanf("%d", &m);

    int adj[101][101] = {0};

    for (int i = 0; i < m; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        adj[u][v] = 1;
        adj[v][u] = 1;
    }

    int s, d;
    scanf("%d", &s);
    scanf("%d", &d);

    int queue[101], front = 0, rear = 0;
    int visited[101] = {0};
    int parent[101];
```

```
    for (int i = 0; i < n; i++)
        parent[i] = -1;

    queue[rear++] = s;
    visited[s] = 1;

    while (front < rear) {
        int u = queue[front++];

        for (int v = 0; v < n; v++) {
            if (adj[u][v] && !visited[v]) {
                visited[v] = 1;
                parent[v] = u;
                queue[rear++] = v;
            }
        }
    }

    if (!visited[d]) {
        printf("No path found");
        return 0;
    }

    int path[101], len = 0;
    int cur = d;

    while (cur != -1) {
        path[len++] = cur;
        cur = parent[cur];
    }

    printf("Shortest path length: %d\n", len - 1);
    printf("Path: ");

    for (int i = len - 1; i >= 0; i--) {
        printf("%d ", path[i]);
    }

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

3. Problem Statement

Liam, a compiler designer, is optimizing task scheduling in a dependency-based system. He is working with a Directed Acyclic Graph (DAG), where each node represents a task, and a directed edge from one node to another signifies a dependency.

To schedule the tasks optimally, Liam must perform a topological sort using Depth-First Search (DFS), sorting the tasks based on their departure times in descending order.

Your task is to help Liam implement this topological sorting algorithm.

***Input Format***

The first line contains an integer N, representing the number of vertices (tasks) in the DAG.

The second line contains an integer E, representing the number of directed edges (dependencies) in the graph.

The next E lines each contain two space-separated integers:

- src The starting vertex of a directed edge.
- dest The ending vertex of a directed edge.

***Output Format***

The output prints N space-separated integers in a single line, representing the 0-indexed vertices in descending order of their DFS departure times, each followed by a space.

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 6

6
5 2
5 0
4 0
4 1
2 3
3 1

Output: 5 4 2 3 1 0

***Answer***

```
// You are using GCC
#include <stdio.h>

int adj[101][101];
int visited[101];
int stack[101];
int top = -1;
int n;

void dfs(int u) {
    visited[u] = 1;

    for (int v = 0; v < n; v++) {
        if (adj[u][v] && !visited[v]) {
            dfs(v);
        }
    }

    stack[++top] = u;   // push after finishing (departure time)
}

int main() {
    int e;
    scanf("%d", &n);
    scanf("%d", &e);

    for (int i = 0; i < e; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        adj[u][v] = 1;
    }
```

```
    for (int i = 0; i < n; i++) {
        visited[i] = 0;
    }

    for (int i = 0; i < n; i++) {
        if (!visited[i]) {
            dfs(i);
        }
    }

    for (int i = top; i >= 0; i--) {
        printf("%d ", stack[i]);
    }

    return 0;
}
```

***Status :*** Partially correct **Marks : 1/10**

4. Problem Statement:

You are given a directed acyclic graph (DAG) G with N vertices and M edges. From G, a new graph G^ is constructed using the same vertex set and the following rules:

If there exists an edge from vertex u to vertex v in G, then there is no edge from v to u in G^.If there exists an edge from u to v in G, and from v to w in G, then there exists an edge from u to w in G^.In G^, find the maximum possible size of a set S of vertices such that for every pair of vertices (u, v) in S, there exists a directed edge between them (either u  v or v  u).

Example:

Input: N=3, M=2, edges: 1  2, 2  3

G^ contains edges: 1  2, 2  3, and 1  3 (by rule 2).

Set S = {1, 2, 3}: edge exists between every pair (1  2, 2  3, 1  3). This is valid with size 3.

Output: 3

***Input Format***

The first line contains two space-separated integers N and M, representing the number of vertices and edges in graph G.

The next M lines each contain two space-separated integers u and v (1 ≤ u, v ≤ N), representing a directed edge from vertex u to vertex v.

***Output Format***

Print a single integer representing the maximum possible size of set S.

Refer to the sample output for formatting specifications

***Sample Test Case***

Input: 3 2
1 2
2 3
Output: 3

***Answer***

```
// You are using GCC
#include <stdio.h>

int adj[101][101];
int visited[101];
int order[101];
int dp[101];
int n, m;
int idx = 0;

void dfs(int u) {
   visited[u] = 1;

   for (int v = 1; v <= n; v++) {
      if (adj[u][v] && !visited[v]) {
         dfs(v);
      }
   }
```

```
    order[idx++] = u;
}

int main() {
    scanf("%d %d", &n, &m);

    for (int i = 0; i < m; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        adj[u][v] = 1;
    }

    // topological sort (DFS)
    for (int i = 1; i <= n; i++) {
        if (!visited[i]) dfs(i);
    }

    // reverse order processing
    for (int i = 1; i <= n; i++) {
        dp[i] = 1;
    }

    for (int i = idx - 1; i >= 0; i--) {
        int u = order[i];

        for (int v = 1; v <= n; v++) {
            if (adj[u][v]) {
                if (dp[v] < dp[u] + 1) {
                    dp[v] = dp[u] + 1;
                }
            }
        }
    }

    int ans = 0;
    for (int i = 1; i <= n; i++) {
        if (dp[i] > ans)
            ans = dp[i];
    }

    printf("%d", ans);
```

```
    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

5. Problem Statement

You are given a directed acyclic graph (DAG) represented as an adjacency list.

Your task is to perform a topological sort on the graph and print the vertices in topological order. The input graph is guaranteed to be a directed acyclic graph.

***Input Format***

The first line of input consists of an integer N, representing the number of vertices in the directed acyclic graph.

The second line consists of an integer E, representing the number of directed edges in the graph.

The following E lines consist of two space-separated integers, src and dest, representing a directed edge from vertex src to vertex dest.

***Output Format***

The output consists of the vertices in topological order separated by spaces.

Refer to the sample output for the formatting specifications.

***Sample Test Case***

Input: 6
6
5 2
5 0
4 0
4 1
2 3

3 1
Output: 4 5 0 2 3 1

***Answer***

```
// You are using GCC
#include <stdio.h>

#define MAX 100

int adj[MAX][MAX];
int indegree[MAX];
int queue[MAX];
int front = 0, rear = 0;

void enqueue(int x) {
    queue[rear++] = x;
}

int dequeue() {
    return queue[front++];
}

int isEmpty() {
    return front == rear;
}

int main() {
    int n, e;
    scanf("%d", &n);
    scanf("%d", &e);

    // initialize
    for (int i = 0; i < n; i++) {
        indegree[i] = 0;
        for (int j = 0; j < n; j++) {
            adj[i][j] = 0;
        }
    }

    // build graph
    for (int i = 0; i < e; i++) {
        int u, v;
```

```
        scanf("%d %d", &u, &v);
        adj[u][v] = 1;
        indegree[v]++;
    }

    // push all 0 indegree nodes
    for (int i = 0; i < n; i++) {
        if (indegree[i] == 0) {
            enqueue(i);
        }
    }

    int count = 0;

    while (!isEmpty()) {
        int u = dequeue();
        printf("%d ", u);
        count++;

        for (int v = 0; v < n; v++) {
            if (adj[u][v]) {
                indegree[v]--;
                if (indegree[v] == 0) {
                    enqueue(v);
                }
            }
        }
    }

    return 0;
}
```

***Status :*** Wrong ***Marks : 0/10***

# Vel Tech Group of Institutions

Name: NIKHIL  KUMAR
Email: vtu28682@veltech.edu.in
Roll no: vtu28682
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

## 2028_NeoColab_Design and Analysis of AlgorithmsS1L10

### VTU_DAA_Week 9_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 15

### Section 1 : MCQ

1. The problem of finding a path in a graph that visits every vertex exactly once is called?

***Answer***

Hamiltonian path problem

***Status :*** Correct ***Marks : 1/1***

2. In the Hamiltonian Cycle problem, if a graph contains a Hamiltonian cycle, it is called a

***Answer***

Hamiltonian graph

***Status :*** Correct ***Marks : 1/1***

vtu28682

3. What is the primary difficulty in finding a Hamiltonian Path in a general graph?

***Answer***

It is an NP-complete problem.

***Status :*** Correct ***Marks : 1/1***

4. Which of the following statements is true regarding the backtracking algorithm for the Hamiltonian Cycle problem?

***Answer***

It explores different paths in the graph to find a Hamiltonian cycle.

***Status :*** Correct ***Marks : 1/1***

5. Which algorithmic paradigm is commonly used to solve the Hamiltonian Cycle problem?

***Answer***

Backtracking

***Status :*** Correct ***Marks : 1/1***

6. The following paradigm can be used to find the solution to the problem in minimum time: Given a set of non-negative integers, and a value K, determine if there is a subset of the given set with a sum equal to K:

***Answer***

Dynamic Programming

***Status :*** Correct ***Marks : 1/1***

7. What is a subset sum problem?

***Answer***

vtu28682

Checking for the presence of a subset that has the sum of elements equal to a given number and printing true or false based on the result

***Status :*** Correct ***Marks : 1/1***

8. Which of the following is not true about the subset sum problem?

***Answer***

the dynamic programming solution has a time complexity of O(n log n)

***Status :*** Correct ***Marks : 1/1***

9. What is the worst case time complexity of dynamic programming solution of the subset sum problem(sum=given subset sum)?

***Answer***

O(sum*n)

***Status :*** Correct ***Marks : 1/1***

10. Placing N-queens so that no two queens attack each other is called _____.

***Answer***

N-queen’s problem

***Status :*** Correct ***Marks : 1/1***

11. Where is the n-queens problem implemented?

***Answer***

Chess

***Status :*** Correct ***Marks : 1/1***

12. Which of the following methods can be used to solve N-Queen's problem?

vtu28682

***Answer***

Backtracking

***Status :*** Correct ***Marks : 1/1***

13. Which one of the following is an application of the backtracking algorithm?

***Answer***

Crossword

***Status :*** Correct ***Marks : 1/1***

14. For N Queens problem, the time complexity is

***Answer***

O(N!)

***Status :*** Correct ***Marks : 1/1***

15. How many unique solutions are there to the 4 Queens problem, where solutions that are reflections or rotations of each other are considered the same?

***Answer***

2

***Status :*** Correct ***Marks : 1/1***

vtu28682

# Vel Tech Group of Institutions

Name: NIKHIL  KUMAR
Email: vtu28682@veltech.edu.in
Roll no: vtu28682
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

## 2028_NeoColab_Design and Analysis of AlgorithmsS1L10

### VTU_DAA_Week 10_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 15

### Section 1 : MCQ

1. Which of the following methods is commonly used to solve assignment problems?

***Answer***

Hungarian method

***Status :*** Correct ***Marks : 1/1***

2. The application of assignment problems is to obtain _____

***Answer***

Minimum cost or maximum profit

***Status :*** Correct ***Marks : 1/1***

vtu28682

3. The assignment problem is said to be balanced if _____.

***Answer***

the number of rows is equal to the number of columns

***Status :*** Correct ***Marks : 1/1***

4. The minimum number of lines covering all zeros in a reduced cost matrix of order n can be _____.

***Answer***

at the most n

***Status :*** Correct ***Marks : 1/1***

5. The assignment problem _____.

***Answer***

All of the mentioned options

***Status :*** Correct ***Marks : 1/1***

6. You are given a knapsack that can carry a maximum weight of 60. There are 4 items with weights of {20, 30, 40, and 70} and values {70, 80, 90, and 200}.

What is the maximum value of the items you can carry in the knapsack?

***Answer***

160

***Status :*** Correct ***Marks : 1/1***

7. What is the objective of the knapsack problem?

***Answer***

vtu28682

To Get Maximum Total Value In The Knapsack

***Status :*** Correct ***Marks : 1/1***

8. Which of the following methods is used to prune the search space in the Branch and Bound method for the Knapsack problem?

***Answer***

Ignoring nodes with a weight exceeding the capacity.

***Status :*** Correct ***Marks : 1/1***

9. What is the key difference between Backtracking and Branch and Bound in solving the Knapsack problem?

***Answer***

Backtracking does not use any bounding function, while Branch and Bound does.

***Status :*** Correct ***Marks : 1/1***

10. What is the primary advantage of using the Branch and Bound approach for the Knapsack problem?

***Answer***

It systematically explores possible solutions while pruning non-promising ones.

***Status :*** Correct ***Marks : 1/1***

11. The travelling salesman problem can be solved using _____.

***Answer***

a minimum spanning tree

***Status :*** Correct ***Marks : 1/1***

12. In a traveling salesman problem, the elements of diagonal from left-top to right-bottom are_____.

***Answer***

all infinity

***Status :*** Correct ***Marks : 1/1***

13. Which type of graph is used to model the TSP problem?

***Answer***

Undirected Weighted Graph

***Status :*** Correct ***Marks : 1/1***

14. What is the main goal of using Branch and Bound in TSP?

***Answer***

To minimize search space by pruning unpromising paths

***Status :*** Correct ***Marks : 1/1***

15. Which value helps in deciding whether a node should be pruned in Branch and Bound?

***Answer***

Bound (lower bound of cost)

***Status :*** Correct ***Marks : 1/1***

vtu28682

# Vel Tech Group of Institutions

Name: NIKHIL  KUMAR
Email: vtu28682@veltech.edu.in
Roll no: vtu28682
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

## 2028_NeoColab_Design and Analysis of AlgorithmsS1L10

### VTU_DAA_Week 11_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 15

### Section 1 : Coding

1. _____ is a matching with the largest number of edges.

***Answer***

Maximum bipartite matching

***Status :*** Correct ***Marks : 1/1***

2. How many colors are used in a bipartite graph?

***Answer***

2

***Status :*** Correct ***Marks : 1/1***

vtu28682

3. What is the simplest method to prove that a graph is bipartite?

***Answer***

It does not have a cycle of an odd length

***Status :*** Correct ***Marks : 1/1***

4. A matching that matches all the vertices of a graph is called?

***Answer***

Perfect matching

***Status :*** Correct ***Marks : 1/1***

5. In a bipartite graph G=(V,U,E), the matching of a free vertex in V to a free vertex in U called?

***Answer***

Augmenting

***Status :*** Correct ***Marks : 1/1***

6. There are four students in a class namely A, B, C and D. A tells that a triangle is a bipartite graph. B tells pentagon is a bipartite graph. C tells square is a bipartite graph. D tells heptagon is a bipartite graph. Who among the following is correct?

***Answer***

C

***Status :*** Correct ***Marks : 1/1***

7. Which of the following is not a property of the bipartite graph?

***Answer***

Asymmetric spectrum

vtu28682

***Status :*** Correct ***Marks : 1/1***

8. The maximum number of edges in a bipartite graph with 14 vertices is _____.

***Answer***

49

***Status :*** Correct ***Marks : 1/1***

9. What does the Halting Problem attempt to determine?

***Answer***

Whether a program will eventually stop running or continue forever

***Status :*** Correct ***Marks : 1/1***

10. The Halting Problem was introduced by which computer scientist?

***Answer***

Alan Turing

***Status :*** Correct ***Marks : 1/1***

11. Why is the Halting Problem considered unsolvable?

***Answer***

Because no algorithm can determine for every program whether it halts or runs forever

***Status :*** Correct ***Marks : 1/1***

12. In computation theory, what does an undecidable problem mean?

***Answer***

A problem that cannot be solved by any algorithm for all inputs

vtu28682

***Status :*** Correct ***Marks : 1/1***

13. In the context of the Halting Problem, what does the term halting refer to?

***Answer***

A program stopping its execution after completing its task

***Status :*** Correct ***Marks : 1/1***

14. What is the main implication of the Halting Problem for computer science?

***Answer***

Some computational problems cannot be solved by any algorithm

***Status :*** Correct ***Marks : 1/1***

15. The Halting Problem generally takes which of the following as input?

***Answer***

A program and its input data

***Status :*** Correct ***Marks : 1/1***

vtu28682

# Vel Tech Group of Institutions

Name: NIKHIL  KUMAR
Email: vtu28682@veltech.edu.in
Roll no: vtu28682
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

## 2028_NeoColab_Design and Analysis of AlgorithmsS1L10

### VTU_DAA_Week 9_COD

Attempt : 1
Total Mark : 40
Marks Obtained : 39

### Section 1 : Coding

1. Problem Statement

Bob has a task to find a Hamiltonian cycle in a given undirected simple graph (no self-loops) represented as an adjacency matrix, where a value of 1 indicates an edge exists between two vertices and a value of 0 indicates no edge. A Hamiltonian cycle is a cycle that visits every vertex exactly once and returns to the starting vertex. Bob needs to write a program that will determine if such a cycle exists and, if so, print the cycle starting and ending at vertex 0. If no cycle is found, the program should output "No solution".

Your task is to help Bob implement this.

***Input Format***

The first line contains an integer V, representing the number of vertices.

The following V lines each contain V space-separated integers (0 or 1), representing the adjacency matrix. The matrix is symmetric (undirected graph) and has zeros on the diagonal (no self-loops)

***Output Format***

If a Hamiltonian cycle is found, the first line prints "Solution found" and the second line prints the cycle as a series of space-separated integers, representing the vertices starting and ending at vertex 0.

If no Hamiltonian cycle is found, print "No solution".

Refer to the sample output for the formatting specifications.

***Sample Test Case***

Input: 8
0 1 0 1 1 0 0 0
1 0 1 0 0 1 0 0
0 1 0 1 0 0 1 0
1 0 1 0 0 0 0 1
1 0 0 0 0 1 0 1
0 1 0 0 1 0 1 0
0 0 1 0 0 1 0 1
0 0 0 1 1 0 1 0
Output: Solution found
0 1 2 3 7 6 5 4 0

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <stdbool.h>

#define MAX 8

int V;
int graph[MAX][MAX];
int path[MAX];

bool isSafe(int v, int pos) {
```

```
    // Check if vertex is adjacent to previous vertex
    if (graph[path[pos - 1]][v] == 0)
        return false;

    // Check if already included
    for (int i = 0; i < pos; i++)
        if (path[i] == v)
            return false;

    return true;
}

bool hamiltonianUtil(int pos) {
    if (pos == V) {
        // Check last vertex connects to first
        return graph[path[pos - 1]][path[0]] == 1;
    }

    for (int v = 1; v < V; v++) {
        if (isSafe(v, pos)) {
            path[pos] = v;

            if (hamiltonianUtil(pos + 1))
                return true;

            path[pos] = -1; // backtrack
        }
    }

    return false;
}

bool findCycle() {
    for (int i = 0; i < V; i++)
        path[i] = -1;

    path[0] = 0;

    if (!hamiltonianUtil(1))
        return false;

    return true;
```

```
}

int main() {
    scanf("%d", &V);

    for (int i = 0; i < V; i++)
        for (int j = 0; j < V; j++)
            scanf("%d", &graph[i][j]);

    if (findCycle()) {
        printf("Solution found\n");
        for (int i = 0; i < V; i++)
            printf("%d ", path[i]);
        printf("0");
    } else {
        printf("No solution");
    }

    return 0;
}
```

***Status :*** Correct **Marks : 10/10**

2. Problem Statement

Given an undirected complete graph with N vertices (N ≥ 3), find the number of distinct Hamiltonian cycles in the graph using backtracking. A Hamiltonian cycle is a cycle that visits each vertex exactly once and returns to the starting vertex.

Two Hamiltonian cycles are considered identical if one can be obtained from the other by rotation (changing the starting vertex) or reversal (traversing in the opposite direction).

Example:

For N=4, the vertices are 0, 1, 2, 3. The 3 distinct Hamiltonian cycles are:

0 1 2 3 00 1 3 2 00 2 1 3 0

Output: 3

***Input Format***

The input consists of a single integer N, representing the number of vertices in the complete graph.

***Output Format***

Print a single integer on its own line representing the number of distinct Hamiltonian cycles in the complete graph with N vertices.

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 6

Output: 60

***Answer***

```
// You are using GCC
#include <stdio.h>

int N;
int count = 0;
int visited[10];
int path[10];

void dfs(int pos) {
    if (pos == N) {
        count++;
        return;
    }

    for (int i = 1; i < N; i++) {
        if (!visited[i]) {
            visited[i] = 1;
            path[pos] = i;

            dfs(pos + 1);

            visited[i] = 0;
        }
```

```
    }
}

int main() {
    scanf("%d", &N);

    // fix starting vertex = 0
    visited[0] = 1;
    path[0] = 0;

    dfs(1);

    // divide by 2 because reverse cycles are same
    printf("%d", count / 2);

    return 0;
}
```

***Status :*** Correct **Marks : 10/10**

3. Problem Statement

A college is organizing an event and has collected donations from N sponsors. Each sponsor contributes a fixed amount. The event coordinator wants to know all possible groups of sponsors whose donation sum matches exactly the required event budget S.

Write a program using Backtracking to find all subsets of sponsor contributions whose sum is exactly equal to S.

A subset may contain any number of sponsors, but each sponsor amount can be used only once.

Formula:

A subset is valid if: a1 + a2 + ... + ak = S

***Input Format***

The first line contains an integer N, representing the number of sponsors.

The second line contains N space-separated integers in ascending order,

representing the donation amounts.

The third line contains an integer S, representing the required budget.

***Output Format***

Print all subsets whose sum is exactly S, one subset per line. Within each subset, print the elements in the order they appear in the input, each followed by a space. Subsets are printed in the order they are discovered during backtracking (subsets starting with smaller elements appear first).

If no subset exists, print: No subset found

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 4
10 20 30 40
50
Output: 10 40
20 30

***Answer***

```
// You are using GCC
#include <stdio.h>

int arr[20], subset[20];
int N, S;
int found = 0;

void printSubset(int size) {
    for (int i = 0; i < size; i++) {
        printf("%d ", subset[i]);
    }
    printf("\n");
}
```

```
void backtrack(int index, int sum, int size) {
    if (sum == S) {
        printSubset(size);
        found = 1;
        return;
    }

    if (index == N || sum > S)
        return;

    // include current element
    subset[size] = arr[index];
    backtrack(index + 1, sum + arr[index], size + 1);

    // exclude current element
    backtrack(index + 1, sum, size);
}

int main() {
    scanf("%d", &N);

    for (int i = 0; i < N; i++)
        scanf("%d", &arr[i]);

    scanf("%d", &S);

    backtrack(0, 0, 0);

    if (!found)
        printf("No subset found");

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

4. Problem Statement

Emily is working on arranging queens on a chessboard where one queen must already be fixed at a given position. She wants to find all possible configurations of placing N queens such that no two queens attack each

other, with the condition that the queen at the specified position cannot be moved.

A queen can attack any cell in its row, column, or diagonal. Row and column indices are 0-based.

Write a program to ensure that the chessboard solutions are displayed correctly, or state "No solution" if none exist.

***Input Format***

The first line contains an integer N representing the size of the chessboard.

The second line contains an integer i, representing the row index of the fixed queen.

The third line contains an integer j, representing the column index of the fixed queen.

***Output Format***

The first line of output consists of a single floating-point number representing the maximum total value, formatted to two decimal places.

The next n lines each consist of a single floating-point number representing the fraction x of each item selected, formatted to two decimal places. x represents the fraction of the i-th item taken (1.00 means the item is fully taken, a value between 0.00 and 1.00 means a fraction is taken). The fractions should be printed in the same order as the input items.

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 4
0
1
Output: . Q . .
. . . Q
Q . . .
. . Q .

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <stdbool.h>

int N;
int board[10];
int fixedRow, fixedCol;

bool isSafe(int row, int col) {
    for (int i = 0; i < row; i++) {
        if (board[i] == col ||
            board[i] - i == col - row ||
            board[i] + i == col + row)
            return false;
    }
    return true;
}

void printBoard() {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            if (board[i] == j)
                printf("Q ");
            else
                printf(". ");
        }
        printf("\n");
    }
    printf("\n");
}

void solve(int row) {
    if (row == N) {
        printBoard();
        return;
    }

    // fixed queen row
    if (row == fixedRow) {
```

```
        if (isSafe(row, fixedCol)) {
            board[row] = fixedCol;
            solve(row + 1);
        }
        return;
    }

    for (int col = 0; col < N; col++) {
        if (isSafe(row, col)) {
            board[row] = col;
            solve(row + 1);
        }
    }
}

int main() {
    scanf("%d", &N);
    scanf("%d", &fixedRow);
    scanf("%d", &fixedCol);

    solve(0);

    return 0;
}
```

***Status :*** Partially correct ***Marks : 9/10***

# Vel Tech Group of Institutions

Name: NIKHIL  KUMAR
Email: vtu28682@veltech.edu.in
Roll no: vtu28682
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

## 2028_NeoColab_Design and Analysis of AlgorithmsS1L10

### VTU_DAA_Week 9_PAH

Attempt : 1
Total Mark : 50
Marks Obtained : 41

### Section 1 : Coding

1. Problem Statement

Diya is given a weighted, undirected graph with N vertices. She wants to find the minimum weight Hamiltonian cycle in the graph, if one exists. A Hamiltonian cycle is a cycle that visits every vertex exactly once, except for the starting and ending vertices, which are the same. The weight of a Hamiltonian cycle is the sum of the weights of all edges in the cycle:

W = w(v  , v  ) + w(v  , v  ) + ... + w(v  , v  )

Write a program using Backtracking to find the minimum weight Hamiltonian cycle.

Example: For a graph with 4 vertices and the adjacency matrix:

0 10 15 20

10  0 35 25

vtu28682

15 35  0 30

20 25 30  0

The algorithm explores all Hamiltonian cycles starting from vertex 0. Among all valid cycles, 0 1 3 2 0 has the minimum weight: 10 + 25 + 30 + 15 = 80.

***Input Format***

The first line contains an integer N representing the number of vertices in the graph.

The next N lines each contain N space-separated integers representing the adjacency matrix of the weighted graph. The value at graph[i][j] is the weight of the edge between vertices i and j. A value of 0 means no edge exists. The matrix is symmetric (graph[i][j] == graph[j][i]) and the diagonal is always 0 (graph[i][i] == 0).

***Output Format***

If a minimum weight Hamiltonian cycle exists, print Minimum Weight Hamiltonian Cycle: on the first line, then print the vertices of the cycle in square brackets separated by a comma and a space, including the starting vertex at both the beginning and the end of the cycle. On the next line, print Total Weight: followed by a space and the integer total weight.

If no Hamiltonian cycle exists, print No Hamiltonian Cycle found with minimum weight.

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 4
0 10 15 20
10 0 35 25
15 35 0 30
20 25 30 0

Output: Minimum Weight Hamiltonian Cycle:
[0, 1, 3, 2, 0]
Total Weight: 80

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <limits.h>

#define MAX 10

int N;
int graph[MAX][MAX];
int visited[MAX];
int path[MAX];
int bestPath[MAX];

int minCost = INT_MAX;

void copyPath() {
    for (int i = 0; i < N; i++)
        bestPath[i] = path[i];
    bestPath[N] = 0;
}

void tsp(int pos, int count, int cost) {
    if (count == N) {
        if (graph[path[pos - 1]][0] != 0) {
            int totalCost = cost + graph[path[pos - 1]][0];

            if (totalCost < minCost) {
                minCost = totalCost;
                copyPath();
            }
        }
        return;
    }

    for (int i = 0; i < N; i++) {
        if (!visited[i] && graph[path[pos - 1]][i] != 0) {
            visited[i] = 1;
            path[pos] = i;
```

```
            tsp(pos + 1, count + 1, cost + graph[path[pos - 1]][i]);

            visited[i] = 0;
        }
    }
}

int main() {
    scanf("%d", &N);

    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            scanf("%d", &graph[i][j]);

    visited[0] = 1;
    path[0] = 0;

    tsp(1, 1, 0);

    if (minCost == INT_MAX) {
        printf("No Hamiltonian Cycle found with minimum weight.");
    } else {
        printf("Minimum Weight Hamiltonian Cycle:\n");

        printf("[");
        for (int i = 0; i < N; i++)
            printf("%d, ", bestPath[i]);
        printf("0]\n");

        printf("Total Weight: %d", minCost);
    }

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

2. Problem Statement

Alice is tasked with determining whether a given graph contains a

Hamiltonian Cycle and, if it does, listing all such cycles. A Hamiltonian Cycle in a graph is a cycle that visits each vertex exactly once and returns to the starting vertex.

Write a program using Backtracking that takes a graph represented by an adjacency matrix and finds all Hamiltonian Cycles starting from the first vertex (vertex 0), if any exist.

***Input Format***

The first line contains an integer N representing the number of vertices in the graph.

The next N lines each contain a single string — the name of each vertex (no surrounding spaces).

The following N lines each contain N space-separated integers (0 or 1) representing the adjacency matrix. A value of 1 at position [i][j] means an edge exists between vertex i and vertex j; 0 means no edge.

***Output Format***

For each Hamiltonian Cycle found, print a line in the format: ['v0', 'v1', ..., 'vn', 'v0'] where vertex names are enclosed in single quotes, separated by a comma and a space, with the starting vertex repeated at the end. Print cycles in the order they are discovered by the backtracking algorithm.

If no Hamiltonian Cycle exists, print: No Hamiltonian Cycle

Refer to the sample output for the formatting specifications.

***Sample Test Case***

Input: 8
a
b
c
d

vtu28682

e
f
g
h
0 1 0 0 0 0 0 1
1 0 1 0 0 0 0 0
0 1 0 1 0 0 0 1
0 0 1 0 1 0 1 0
0 0 0 1 0 1 0 0
0 0 0 0 1 0 1 0
0 0 0 1 0 1 0 1
1 0 1 0 0 0 1 0

Output: ['a', 'b ', 'c ', 'd ', 'e ', 'f ', 'g ', 'h', 'a']
['a', 'h', 'g ', 'f ', 'e ', 'd ', 'c ', 'b ', 'a']

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <string.h>
#include <stdbool.h>

#define MAX 9

int N;
int graph[MAX][MAX];
char names[MAX][20];

int path[MAX];
bool visited[MAX];
int found = 0;

void printCycle() {
    printf("['%s'", names[path[0]]);

    for (int i = 1; i < N; i++) {
        printf(", '%s'", names[path[i]]);
    }

    printf(", '%s']\n", names[path[0]]);
}

void backtrack(int pos) {
```

```
    if (pos == N) {
        if (graph[path[pos - 1]][path[0]] == 1) {
            printCycle();
            found = 1;
        }
        return;
    }

    for (int v = 1; v < N; v++) {
        if (!visited[v] && graph[path[pos - 1]][v] == 1) {
            visited[v] = true;
            path[pos] = v;

            backtrack(pos + 1);

            visited[v] = false;
        }
    }
}

int main() {
    scanf("%d", &N);

    // Read vertex names
    for (int i = 0; i < N; i++) {
        scanf(" %[^\n]", names[i]);
    }

    // Read adjacency matrix
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    // start from vertex 0
    path[0] = 0;
    visited[0] = true;

    backtrack(1);

    if (!found) {
```

```
    printf("No Hamiltonian Cycle");
  }

  return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

3. Problem Statement

A company has N employees, and each employee can contribute a fixed number of training hours.

The manager wants to form groups of employees such that the total training hours of a selected group is exactly equal to the required training goal S.

Write a program using Backtracking to find all distinct subsets of employee training hours whose sum is exactly equal to S.

Each employee's training hours can be used only once.

If the input contains duplicate hour values, the output must not contain repeated subsets.

Condition

A subset is valid if:

h1 + h2 + ... + hk = S

***Input Format***

The first line contains an integer N representing the number of employees.

The second line contains N space-separated integers representing the training hours of each employee. The input may contain duplicate values.

The third line contains an integer S representing the required training goal.

***Output Format***

Print each distinct valid subset on a separate line. Within each subset, print elements in ascending order, each followed by a space (including the last

element). Print subsets in the order they are found during backtracking on the sorted input.

If no valid subset exists, print: No subset found

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 6
2 2 3 5 6 8
10
Output: 2 2 6
2 3 5
2 8

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

int arr[20], subset[20];
int N, S;
int found = 0;

int cmp(const void *a, const void *b) {
    return (*(int*)a - *(int*)b);
}

void printSubset(int size) {
    for (int i = 0; i < size; i++) {
        printf("%d ", subset[i]);
    }
    printf("\n");
}

void backtrack(int start, int sum, int size) {
    if (sum == S) {
        printSubset(size);
        found = 1;
        return;
```

```
    }
    if (sum > S)
        return;

    for (int i = start; i < N; i++) {

        // skip duplicates
        if (i > start && arr[i] == arr[i - 1])
            continue;

        subset[size] = arr[i];

        backtrack(i + 1, sum + arr[i], size + 1);
    }
}

int main() {
    scanf("%d", &N);

    for (int i = 0; i < N; i++)
        scanf("%d", &arr[i]);

    scanf("%d", &S);

    qsort(arr, N, sizeof(int), cmp);

    backtrack(0, 0, 0);

    if (!found)
        printf("No subset found");

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

4. Problem Statement

Nikila is exploring the N Queens Problem. This classic problem involves placing N queens on an N×N chessboard such that no two queens threaten

each other. In chess, queens can move horizontally, vertically, and diagonally, so the challenge is to arrange the queens to ensure they do not share the same row, column, or diagonal.

Write a program using Backtracking to find and print all valid solutions to the N Queens Problem.

***Input Format***

The input consists of a single integer N representing both the size of the chessboard (N×N) and the number of queens to be placed.

***Output Format***

Print all valid solutions in the order they are found by the backtracking algorithm. Each solution is represented as an N×N matrix where 1 indicates the position of a queen and 0 represents an empty cell. Each value is followed by a space. Each row is printed on a new line. Print a blank line after each solution including the last one. If no valid solution exists, print nothing.

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 4

Output: 0 0 1 0
1 0 0 0
0 0 0 1
0 1 0 0

0 1 0 0
0 0 0 1
1 0 0 0
0 0 1 0

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <stdbool.h>
```

```
int N;
int board[10];

bool isSafe(int row, int col) {
    for (int i = 0; i < row; i++) {
        if (board[i] == col ||
            board[i] - i == col - row ||
            board[i] + i == col + row)
            return false;
    }
    return true;
}

void printBoard() {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            if (board[i] == j)
                printf("1 ");
            else
                printf("0 ");
        }
        printf("\n");
    }
    printf("\n");
}

void solve(int row) {
    if (row == N) {
        printBoard();
        return;
    }

    for (int col = 0; col < N; col++) {
        if (isSafe(row, col)) {
            board[row] = col;
            solve(row + 1);
        }
    }
}

int main() {
    scanf("%d", &N);
```

```
    solve(0);

    return 0;
}
```

***Status :*** Partially correct ***Marks : 1/10***

5. Problem Statement

Sarah wants to verify whether a given arrangement of queens on a 4×4 chessboard constitutes a valid solution to the 4 Queens Problem. The objective is to position four queens on the board in a way that prevents any queen from threatening another, considering horizontal, vertical, and diagonal movements.

To determine the validity of the solution, Sarah needs to ensure that there are exactly four queens on the board, each row and column contains only one queen, and no two queens share the same diagonal.

Your task is to assist Sarah by implementing a program that checks these criteria for the given chessboard arrangement.

***Input Format***

The input consists of 4 lines, each containing 4 space-separated integers representing one row of the chessboard. Each integer is either 1 (queen present) or 0 (empty cell).

***Output Format***

Print a single line indicating whether the placement is valid, checking conditions in the following priority order:

1. If the total number of queens is not exactly 4: False - Invalid number of queens: X where X is the count of queens on the board.
2. If a row does not contain exactly one queen: False - Row Y has Z queens where Y is the 0-based row index and Z is the number of queens in that row.
3. If a column does not contain exactly one queen: False - Column Y has Z queens where Y is the 0-based column index and Z is the number of queens in that column.
4. If a main diagonal (top-left to bottom-right) contains more than one queen:

False - Main diagonal (d=X) has Y queens where X is the diagonal identifier computed as column - row (range −3 to 3) and Y is the number of queens on that diagonal.
5. If an anti-diagonal (top-right to bottom-left) contains more than one queen: False - Anti-diagonal (d=X) has Y queens where X is the diagonal identifier computed as column + row (range 0 to 6) and Y is the number of queens on that diagonal.
6. If all conditions are satisfied: True - Valid 4 Queens solution: All conditions satisfied

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 0 1 0 0
0 0 0 1
1 0 0 0
0 0 1 0

Output: True - Valid 4 Queens solution: All conditions satisfied

***Answer***

```
// You are using GCC
#include <stdio.h>

int main() {
    int board[4][4];
    int total = 0;

    int rowCount[4] = {0};
    int colCount[4] = {0};
    int mainDiag[7] = {0};   // col - row + 3 shifted to 0-6
    int antiDiag[7] = {0};   // row + col

    // Input + counting
    for (int i = 0; i < 4; i++) {
        for (int j = 0; j < 4; j++) {
            scanf("%d", &board[i][j]);
            if (board[i][j] == 1) {
                total++;
                rowCount[i]++;
```

```
            colCount[j]++;
            mainDiag[j - i + 3]++;
            antiDiag[i + j]++;
        }
    }
}

// 1. total queens check
if (total != 4) {
    printf("False - Invalid number of queens: %d", total);
    return 0;
}

// 2. row check
for (int i = 0; i < 4; i++) {
    if (rowCount[i] != 1) {
        printf("False - Row %d has %d queens", i, rowCount[i]);
        return 0;
    }
}

// 3. column check
for (int j = 0; j < 4; j++) {
    if (colCount[j] != 1) {
        printf("False - Column %d has %d queens", j, colCount[j]);
        return 0;
    }
}

// 4. main diagonal check
for (int d = 0; d < 7; d++) {
    if (mainDiag[d] > 1) {
        printf("False - Main diagonal (d=%d) has %d queens", d - 3, mainDiag[d]);
        return 0;
    }
}

// 5. anti diagonal check
for (int d = 0; d < 7; d++) {
    if (antiDiag[d] > 1) {
        printf("False - Anti-diagonal (d=%d) has %d queens", d, antiDiag[d]);
        return 0;
```

```
    }
    }

    // valid case
    printf("True - Valid 4 Queens solution: All conditions satisfied");

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

# Vel Tech Group of Institutions

Name: NIKHIL  KUMAR
Email: vtu28682@veltech.edu.in
Roll no: vtu28682
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

## 2028_NeoColab_Design and Analysis of AlgorithmsS1L10

### VTU_DAA_Week 9_CY

Attempt : 1
Total Mark : 50
Marks Obtained : 30

### Section 1 : Coding

1. Problem Statement

Given an adjacency matrix adj[][] of an undirected graph consisting of N vertices, find a Hamiltonian Path in the graph if one exists. If found, print the path as a sequence of vertex indices separated by ->. Otherwise, print No Hamiltonian Path found.

A Hamiltonian path visits each and every vertex of the graph exactly once. The adjacency matrix is symmetric with zeros on the diagonal (no self-loops).

Write a program using Backtracking to find the Hamiltonian Path.

Examples:

Input: adj[][] = {{0, 1, 1, 1, 0}, {1, 0, 1, 0, 1}, {1, 1, 0, 1, 1}, {1, 0, 1, 0, 0}}

Output: Yes

Explanation:

There exists a Hamiltonian Path for the given graph, as shown in the image below:

Input: adj[][] = {{0, 1, 0, 0}, {1, 0, 1, 1}, {0, 1, 0, 0}, {0, 1, 0, 0}}

Output: No

***Input Format***

The first line contains an integer N, representing the number of vertices in the graph.

The next N lines each contain N space-separated integers (0 or 1), representing the adjacency matrix of the graph.

***Output Format***

If a Hamiltonian path exists, print the vertices of the path separated by -> on a single line.

If no Hamiltonian path exists, print: No Hamiltonian Path found

Refer to the sample output for formatting specifications.

Refer to the sample output for the formatting specifications.

***Sample Test Case***

Input: 5
0 1 1 1 0
1 0 1 0 1
1 1 0 1 1
1 0 1 0 0
0 1 1 0 0
Output: 0 -> 1 -> 4 -> 2 -> 3

***Answer***

// You are using GCC

```
#include <stdio.h>
#include <stdbool.h>

#define MAX 6

int N;
int adj[MAX][MAX];
int path[MAX];
bool visited[MAX];
int found = 0;

bool dfs(int pos) {
    if (pos == N) {
        found = 1;
        return true;
    }

    for (int v = 0; v < N; v++) {
        if (!visited[v] && adj[path[pos - 1]][v] == 1) {
            visited[v] = true;
            path[pos] = v;

            if (dfs(pos + 1))
                return true;

            visited[v] = false;
        }
    }

    return false;
}

int main() {
    scanf("%d", &N);

    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            scanf("%d", &adj[i][j]);

    // try starting from each vertex
    for (int start = 0; start < N; start++) {
```

```
        for (int i = 0; i < N; i++)
            visited[i] = false;

        path[0] = start;
        visited[start] = true;

        if (dfs(1))
            break;
    }

    if (found) {
        for (int i = 0; i < N; i++) {
            printf("%d", path[i]);
            if (i != N - 1)
                printf(" -> ");
        }
    } else {
        printf("No Hamiltonian Path found");
    }

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

2. Problem Statement

You are given an undirected graph with N vertices. Implement a function longest_hamiltonian_cycle to find the longest Hamiltonian cycle in the graph. A Hamiltonian cycle is a cycle that visits every vertex exactly once, except for the starting and ending vertices, which are the same.

***Input Format***

The first line of input consists of an integer N representing the number of vertices in the graph.

The next N lines each contain N space-separated integers, representing the adjacency matrix of the graph. The value at graph[i][j] is 1 if there is an edge between vertex i and vertex j, and 0 otherwise.

***Output Format***

If a Hamiltonian cycle exists, print "Longest Hamiltonian Cycle:" followed by the vertices of the longest cycle found in order within square bracket separated by a comma.

If no Hamiltonian cycle exists, print "No Hamiltonian Cycle found".

Refer to the sample output for the formatting specifications.

***Sample Test Case***

Input: 4
0 1 1 0
1 0 1 1
1 1 0 1
0 1 1 0
Output: Longest Hamiltonian Cycle: [0, 1, 3, 2]

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <stdbool.h>

#define MAX 7

int N;
int graph[MAX][MAX];
int path[MAX];
bool visited[MAX];
int found = 0;

bool dfs(int pos) {
   if (pos == N) {
      // check cycle back to start
      if (graph[path[N - 1]][path[0]] == 1) {
         found = 1;
         return true;
      }
      return false;
```

```
    }

    for (int v = 0; v < N; v++) {
        if (!visited[v] && graph[path[pos - 1]][v] == 1) {
            visited[v] = true;
            path[pos] = v;

            if (dfs(pos + 1))
                return true;

            visited[v] = false;
        }
    }

    return false;
}

int main() {
    scanf("%d", &N);

    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            scanf("%d", &graph[i][j]);

    // start from vertex 0
    path[0] = 0;
    visited[0] = true;

    dfs(1);

    if (found) {
        printf("Longest Hamiltonian Cycle: [");
        for (int i = 0; i < N; i++) {
            printf("%d", path[i]);
            if (i != N - 1)
                printf(", ");
        }
        printf("]");
    } else {
        printf("No Hamiltonian Cycle found");
    }
```

```
    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

3. Problem Statement

Nira is studying the N-Queens problem and needs to write a program to solve it. The program should find a valid arrangement of N queens on an N×N chessboard, ensuring that no two queens can attack each other.

Two queens attack each other if they are in the same row, the same column, or the same diagonal. Once a solution is found, display the board with queens placed correctly. A solution is guaranteed to exist for all valid values of N.

Write a program using Backtracking to solve the N-Queens problem.

Example

Input:

4 // Value of N

Output:

0 0 1 0

1 0 0 0

0 0 0 1

0 1 0 0

Explanation: The output represents a valid arrangement of queens on the 4x4 chessboard, where 1 indicates the position of a queen and 0 indicates an empty cell.

In the first row, a queen is placed in the third column.In the second row, a queen is placed in the first column.In the third row, a queen is placed in the fourth column.In the fourth row, a queen is placed in the second column.This arrangement of queens on the board ensures that no two

queens threaten each other.

***Input Format***

The input consists of a single integer N, representing the size of the chessboard (N×N).

***Output Format***

Print the N×N board as N rows of N space-separated integers. Each integer is followed by a space (including the last integer on each row). Print 1 where a queen is placed and 0 for an empty cell. Print any one valid solution.

Refer to the sample output for the formatting specifications.

***Sample Test Case***

Input: 4

Output: 0 0 1 0
1 0 0 0
0 0 0 1
0 1 0 0

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <stdbool.h>

#define MAX 20

int N;
int board[MAX];
bool found = false;

bool isSafe(int row, int col) {
    for (int i = 0; i < row; i++) {
        if (board[i] == col ||
            board[i] - i == col - row ||
            board[i] + i == col + row)
            return false;
    }
```

```
    return true;
}

void printBoard() {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            if (board[i] == j)
                printf("1 ");
            else
                printf("0 ");
        }
        printf("\n");
    }
}

void solve(int row) {
    if (found) return;

    if (row == N) {
        printBoard();
        found = true;
        return;
    }

    for (int col = 0; col < N; col++) {
        if (isSafe(row, col)) {
            board[row] = col;
            solve(row + 1);
        }
    }
}

int main() {
    scanf("%d", &N);

    solve(0);

    return 0;
}
```

***Status :*** Wrong ***Marks : 0/10***

4. Problem Statement

In chess, a knight moves in an "L" shape: two squares in one direction and one square perpendicular to it. From position (0, 0) on an infinite chessboard, a knight can move to one of the following eight positions in a single step:

(1, 2), (−1, 2), (1, −2), (−1, −2), (2, 1), (−2, 1), (2, −1), (−2, −1).

Given a target position (x, y) where both x and y are positive integers, find the minimum number of steps needed for the knight to travel from (0, 0) to (x, y) using BFS (Breadth-First Search).

Beginning from location (0,0), what is the minimum number of steps needed for a knight to get to some other arbitrary location (x,y)?

***Input Format***

The first line contains two space-separated integers x and y, representing the target position of the knight on the chessboard.

***Output Format***

Print a single integer representing the minimum number of steps needed for the knight to move from (0, 0) to (x, y).

Refer to the sample output for formatting specifications

***Sample Test Case***

Input: 1 2
Output: 1

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <stdbool.h>

#define MAX 1000
```

```
int dx[8] = {1, 2, 2, 1, -1, -2, -2, -1};
int dy[8] = {2, 1, -1, -2, -2, -1, 1, 2};

typedef struct {
    int x, y, dist;
} Node;

bool visited[MAX][MAX];

int bfs(int tx, int ty) {
    Node queue[MAX * MAX];
    int front = 0, rear = 0;

    queue[rear++] = (Node){0, 0, 0};
    visited[0][0] = true;

    while (front < rear) {
        Node curr = queue[front++];

        if (curr.x == tx && curr.y == ty)
            return curr.dist;

        for (int i = 0; i < 8; i++) {
            int nx = curr.x + dx[i];
            int ny = curr.y + dy[i];

            if (nx >= 0 && ny >= 0 && nx < MAX && ny < MAX && !visited[nx][ny]) {
                visited[nx][ny] = true;
                queue[rear++] = (Node){nx, ny, curr.dist + 1};
            }
        }
    }

    return -1;
}

int main() {
    int x, y;
    scanf("%d %d", &x, &y);

    printf("%d", bfs(x, y));
```

```
return 0;
}
```

***Status :*** Wrong ***Marks : 0/10***

5. Problem Statement

A customer is shopping in a supermarket and has selected N items, each with a fixed price.

The customer has a discount coupon that can be applied only if the total price of some selected items is exactly equal to a target amount S.

Write a program using Backtracking to find all distinct subsets of item prices whose sum is exactly equal to S.

Each item price can be used only once. If the input contains duplicate prices, the output must not contain repeated subsets.

Formula: A subset is valid if: p1 + p2 + ... + pk = S

Print each valid subset with its elements in ascending order. Print subsets in the order they are discovered by the backtracking algorithm (which processes the sorted array from left to right).

***Input Format***

The first line contains an integer N representing the number of items.

The second line contains N space-separated integers representing item prices. The prices may include duplicate values.

The third line contains an integer S representing the target bill amount.

***Output Format***

Print each distinct valid subset on a separate line. Within each subset, print elements in ascending order, each followed by a space (including the last element).

Print subsets in the order they are found during backtracking on the sorted input.

If no valid subset exists, print: No subset found

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 5
5 5 10 15 20
25

Output: 5 5 15
5 20
10 15

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

int arr[20], subset[20];
int N, S;
int found = 0;

int cmp(const void *a, const void *b) {
    return (*(int*)a - *(int*)b);
}

void printSubset(int size) {
    for (int i = 0; i < size; i++) {
        printf("%d ", subset[i]);
    }
    printf("\n");
}

void backtrack(int start, int sum, int size) {
    if (sum == S) {
        printSubset(size);
        found = 1;
        return;
    }
```

```
    if (sum > S)
        return;

    for (int i = start; i < N; i++) {

        // skip duplicates
        if (i > start && arr[i] == arr[i - 1])
            continue;

        subset[size] = arr[i];

        backtrack(i + 1, sum + arr[i], size + 1);
    }
}

int main() {
    scanf("%d", &N);

    for (int i = 0; i < N; i++)
        scanf("%d", &arr[i]);

    scanf("%d", &S);

    qsort(arr, N, sizeof(int), cmp);

    backtrack(0, 0, 0);

    if (!found)
        printf("No subset found");

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

# Vel Tech Group of Institutions

Name: NIKHIL  KUMAR
Email: vtu28682@veltech.edu.in
Roll no: vtu28682
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

## 2028_NeoColab_Design and Analysis of AlgorithmsS1L10

### VTU_DAA_Week 10_COD

Attempt : 1
Total Mark : 30
Marks Obtained : 30

### Section 1 : Coding

1. Problem Statement

Riya, the manager of a software development company, needs to assign three developers to three different projects.

Each developer has a different cost (in terms of time and resource usage) for completing each project. Riya wants to assign exactly one developer to each project such that the total development cost is minimized.

To solve this efficiently, she decides to use the Branch and Bound technique for the Assignment Problem.

Your task is to help Riya determine the minimum total cost of assigning developers to projects.

***Input Format***

vtu28682

The input consists of three lines with three space-separated integers each, representing a 3×3 cost matrix.

Each cell (i, j) represents the cost of assigning the ith developer to the jth project.

***Output Format***

The output should display the minimum total cost of the assignment in the following exact format: "Minimum total cost: X" , Where X is the minimum total cost obtained after assigning each worker to exactly one task.

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 10 12 19
10 5 7
15 14 13

Output: Minimum total cost: 28

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <limits.h>
#include <stdbool.h>

int cost[3][3];
bool used[3];
int minCost = INT_MAX;

void assign(int worker, int currCost) {
    if (worker == 3) {
        if (currCost < minCost)
            minCost = currCost;
        return;
    }

    for (int job = 0; job < 3; job++) {
        if (!used[job]) {
            used[job] = true;
```

```
            assign(worker + 1, currCost + cost[worker][job]);

            used[job] = false;
        }
    }
}

int main() {
    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 3; j++)
            scanf("%d", &cost[i][j]);

    assign(0, 0);

    printf("Minimum total cost: %d", minCost);

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

2. Problem Statement

Arjun is the manager of a disaster relief center responsible for sending essential supplies to a flood-affected area using a single rescue truck. The truck has a limited carrying capacity, so Arjun must carefully select which supply packages to load.

Each package has a specific weight and an importance score representing how critical it is for the affected people. Arjun wants to maximize the total importance score of the packages loaded onto the truck without exceeding its weight capacity.

To solve this efficiently, Arjun uses the Branch and Bound technique, which systematically explores possible combinations of packages while pruning choices that cannot produce better results.

Your task is to help Arjun determine the maximum total importance score that can be transported.

***Input Format***

The first line contains an integer N, representing the number of available packages.

The next N lines each contain two integers: weight[i] value[i]

- weight[i] – weight of the package
- value[i] – importance score of the package

The last line contains an integer W, representing the maximum capacity of the rescue truck.

***Output Format***

The output prints a single integer representing the maximum total importance score that can be carried without exceeding the truck's weight capacity.

Refer to the sample input and output for formatting specifications.

***Sample Test Case***

Input: 5
2 40
3 50
1 100
5 95
3 30
10

Output: 245

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int w, v;
    double ratio;
} Item;
```

```
Item items[1000];
int N, W;
int best = 0;

int cmp(const void *a, const void *b) {
    Item *i1 = (Item *)a;
    Item *i2 = (Item *)b;
    if (i2->ratio > i1->ratio) return 1;
    else if (i2->ratio < i1->ratio) return -1;
    return 0;
}

// upper bound using fractional knapsack
double bound(int idx, int currW, int currV) {
    double profit = currV;
    int weight = currW;

    for (int i = idx; i < N; i++) {
        if (weight + items[i].w <= W) {
            weight += items[i].w;
            profit += items[i].v;
        } else {
            int remain = W - weight;
            profit += items[i].ratio * remain;
            break;
        }
    }
    return profit;
}

void dfs(int idx, int currW, int currV) {
    if (currW > W)
        return;

    if (currV > best)
        best = currV;

    if (idx == N)
        return;

    if (bound(idx, currW, currV) <= best)
        return;
```

```
    // include item
    dfs(idx + 1,
        currW + items[idx].w,
        currV + items[idx].v);

    // exclude item
    dfs(idx + 1,
        currW,
        currV);
}

int main() {
    scanf("%d", &N);

    for (int i = 0; i < N; i++) {
        scanf("%d %d", &items[i].w, &items[i].v);
        items[i].ratio = (double)items[i].v / items[i].w;
    }

    scanf("%d", &W);

    qsort(items, N, sizeof(Item), cmp);

    dfs(0, 0, 0);

    printf("%d", best);

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

3. Problem Statement

You are an art collector who wants to visit a set of art galleries in a city. Each gallery is at a different location, and the cost of traveling between galleries is known.

Your goal is to find the shortest route that allows you to visit each gallery

exactly once and return to your starting point, using the traveling salesman problem (TSP). Write a program to calculate the minimum cost of this route.

Note: Solve it using Branch and Bound technique.

***Input Format***

The first line contains an integer n, the number of galleries.

The next n lines contain n integers each, representing the adjacency matrix.

The ith integer on the jth line represents the cost of traveling from gallery i to gallery j.

***Output Format***

The output displays "Minimum Cost is: " followed by the minimum cost of the route that visits each gallery exactly once and returns to the starting gallery.

Refer to the sample output for the formatting specifications.

***Sample Test Case***

Input: 4
0 10 15 20
10 0 35 25
15 35 0 30
20 25 30 0
Output: Minimum Cost is: 80

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <limits.h>

#define MAX 10

int n;
int cost[MAX][MAX];
int visited[MAX];
```

```
int bestCost = INT_MAX;
int path[MAX];

int minEdge = INT_MAX;

// get minimum non-zero edge (for bound estimation)
void findMinEdge() {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (i != j && cost[i][j] > 0 && cost[i][j] < minEdge) {
                minEdge = cost[i][j];
            }
        }
    }
}

// lower bound estimate
int bound(int currCost, int level) {
    return currCost + (n - level) * minEdge;
}

void tsp(int level, int currNode, int currCost) {
    if (level == n) {
        if (cost[currNode][0] > 0) {
            int total = currCost + cost[currNode][0];
            if (total < bestCost)
                bestCost = total;
        }
        return;
    }

    if (bound(currCost, level) >= bestCost)
        return;

    for (int i = 0; i < n; i++) {
        if (!visited[i] && cost[currNode][i] > 0) {
            visited[i] = 1;

            tsp(level + 1, i, currCost + cost[currNode][i]);

            visited[i] = 0;
```

```
        }
    }
}

int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &cost[i][j]);

    findMinEdge();

    visited[0] = 1;
    tsp(1, 0, 0);

    printf("Minimum Cost is: %d", bestCost);

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

# Vel Tech Group of Institutions

Name: NIKHIL  KUMAR
Email: vtu28682@veltech.edu.in
Roll no: vtu28682
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

## 2028_NeoColab_Design and Analysis of AlgorithmsS1L10

### VTU_DAA_Week 10_PAH

Attempt : 1
Total Mark : 50
Marks Obtained : 44.5

### Section 1 : Coding

1. Problem Statement

Arun, the operations manager of a logistics company, needs to assign three delivery drivers to three delivery zones.

Each driver requires a different amount of fuel (in liters) to complete deliveries in each zone due to varying distances and traffic conditions. Arun wants to assign exactly one driver to each zone such that the total fuel consumption is minimized.

To efficiently explore possible assignments and avoid unnecessary computations, Arun decides to use the Branch and Bound technique.

Your task is to help Arun determine the optimal assignment of drivers to zones that results in the least total fuel consumption.

***Input Format***

The input consists of three lines with three space-separated integers each, representing a 3 × 3 matrix.

Each cell (i, j) represents the fuel required (in liters) if the ith driver is assigned to the jth zone.

***Output Format***

The output should display the optimal assignment in the following format:

Optimal Assignment:

Driver 1 -> Zone X

Driver 2 -> Zone Y

Driver 3 -> Zone Z

Where:

Each driver is assigned to exactly one zone.

Each zone is assigned to exactly one driver.

The assignment must produce the least total fuel consumption.

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 8 6 10
9 7 4
3 12 5
Output: Optimal Assignment:
Driver 1 -> Zone 2
Driver 2 -> Zone 3
Driver 3 -> Zone 1

***Answer***

vtu28682

```
// You are using GCC
#include <stdio.h>
#include <limits.h>
#include <stdbool.h>

int cost[3][3];
bool used[3];
int assignment[3];

int bestCost = INT_MAX;
int bestAssign[3];

void solve(int driver, int currCost) {
    if (driver == 3) {
        if (currCost < bestCost) {
            bestCost = currCost;
            for (int i = 0; i < 3; i++)
                bestAssign[i] = assignment[i];
        }
        return;
    }

    for (int zone = 0; zone < 3; zone++) {
        if (!used[zone]) {
            used[zone] = true;
            assignment[driver] = zone;

            solve(driver + 1, currCost + cost[driver][zone]);

            used[zone] = false;
        }
    }
}

int main() {
    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 3; j++)
            scanf("%d", &cost[i][j]);

    solve(0, 0);

    printf("Optimal Assignment:\n");
```

```
    for (int i = 0; i < 3; i++) {
        printf("Driver %d -> Zone %d\n", i + 1, bestAssign[i] + 1);
    }

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

2. Problem Statement

Captain Arvind has discovered a hidden island filled with valuable treasures. Before leaving the island, he must load the treasures onto his small ship. However, the ship has a strict weight capacity, so he cannot carry everything he finds.

Each treasure chest has a specific value and weight. Captain Arvind wants to select the best combination of treasure chests to load onto the ship so that the total value of the treasures is maximized without exceeding the ship's weight limit.

To solve this efficiently, Captain Arvind decides to use the Branch and Bound technique applied to the 0/1 Knapsack problem, which systematically explores possible combinations and eliminates choices that cannot produce better results.

Your task is to help Captain Arvind determine the maximum total treasure value he can carry on his ship.

***Input Format***

The first line contains an integer n, representing the total number of treasure chests.

The next n lines contain two space-separated integers: val[i] w[i]

where

- val[i] represents the value of the i-th treasure chest

- w[i] represents the weight of the i-th treasure chest

The last line contains an integer W, representing the maximum weight capacity of the ship.

***Output Format***

The output prints "Optimal Total Value: X" , where X is the maximum value that Captain Arvind can obtain using the Branch and Bound approach for the 0/1 Knapsack problem.

Refer to the sample output for the formatting specifications.

***Sample Test Case***

Input: 3
60 10
100 20
120 30
50

Output: Optimal Total Value: 220

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int val, wt;
    double ratio;
} Item;

Item items[10];
int n, W;
int best = 0;

int cmp(const void *a, const void *b) {
    Item *x = (Item *)a;
    Item *y = (Item *)b;
    if (y->ratio > x->ratio) return 1;
    else if (y->ratio < x->ratio) return -1;
```

```
    return 0;
}

// fractional upper bound
double bound(int idx, int currW, int currV) {
    double profit = currV;
    int weight = currW;

    for (int i = idx; i < n; i++) {
        if (weight + items[i].wt <= W) {
            weight += items[i].wt;
            profit += items[i].val;
        } else {
            int rem = W - weight;
            profit += items[i].ratio * rem;
            break;
        }
    }
    return profit;
}

void dfs(int idx, int currW, int currV) {
    if (currW > W)
        return;

    if (currV > best)
        best = currV;

    if (idx == n)
        return;

    if (bound(idx, currW, currV) <= best)
        return;

    // include item
    dfs(idx + 1,
        currW + items[idx].wt,
        currV + items[idx].val);

    // exclude item
    dfs(idx + 1,
        currW,
```

```
    currV);
}

int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf("%d %d", &items[i].val, &items[i].wt);
        items[i].ratio = (double)items[i].val / items[i].wt;
    }

    scanf("%d", &W);

    qsort(items, n, sizeof(Item), cmp);

    dfs(0, 0, 0);

    printf("Optimal Total Value: %d", best);

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

3. Problem Statement

Dr. Aarav is a field scientist preparing for an important research expedition to a remote mountain region. Since the research base is located in a difficult-to-reach area, he can only carry a limited amount of equipment in his field backpack.

Each piece of equipment has a scientific importance value and a specific weight. Dr. Aarav wants to choose the best combination of equipment so that the total scientific value is maximized without exceeding the backpack's weight capacity.

To solve this efficiently, Dr. Aarav applies the Branch and Bound technique using the 0/1 Knapsack algorithm, which systematically explores possible selections and eliminates combinations that cannot yield better results.

Your task is to help Dr. Aarav determine which equipment weights should be selected to achieve the maximum total value.

Note: The weights must be printed in the same order as they appear in the input.

***Input Format***

The first line of input contains an integer n, representing the number of available equipment items.

The second line contains n space-separated integers val[i], representing the scientific value of each item.

The third line contains n space-separated integers w[i], representing the weight of each item.

The fourth line contains an integer W, representing the maximum carrying capacity of the backpack.

***Output Format***

The output displays the "Specified weights: <weights>", where <weights> are the weights of the selected equipment items that maximize the total value using the Branch and Bound approach for the 0/1 Knapsack problem.

Refer to the sample outputs for the exact format.

***Sample Test Case***

Input: 4
10 20 30 40
1 2 3 4
8
Output: Specified weights: 1 3 4

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>
```

```
#include <stdbool.h>

typedef struct {
    int val, wt, idx;
    double ratio;
} Item;

Item items[10];
int n, W;

int bestValue = 0;
bool bestTaken[10];

bool taken[10];

int cmp(const void *a, const void *b) {
    Item *x = (Item *)a;
    Item *y = (Item *)b;
    if (y->ratio > x->ratio) return 1;
    else if (y->ratio < x->ratio) return -1;
    return 0;
}

// fractional bound
double bound(int i, int currW, int currV) {
    double profit = currV;
    int weight = currW;

    for (int j = i; j < n; j++) {
        if (weight + items[j].wt <= W) {
            weight += items[j].wt;
            profit += items[j].val;
        } else {
            int rem = W - weight;
            profit += items[j].ratio * rem;
            break;
        }
    }
    return profit;
}

void dfs(int i, int currW, int currV) {
```

```
    if (currW > W)
        return;

    if (currV > bestValue) {
        bestValue = currV;
        for (int k = 0; k < n; k++)
            bestTaken[k] = taken[k];
    }

    if (i == n)
        return;

    if (bound(i, currW, currV) <= bestValue)
        return;

    // include
    taken[i] = true;
    dfs(i + 1, currW + items[i].wt, currV + items[i].val);

    // exclude
    taken[i] = false;
    dfs(i + 1, currW, currV);
}

int main() {
    scanf("%d", &n);

    int val[10], wt[10];

    for (int i = 0; i < n; i++)
        scanf("%d", &val[i]);

    for (int i = 0; i < n; i++)
        scanf("%d", &wt[i]);

    scanf("%d", &W);

    for (int i = 0; i < n; i++) {
        items[i].val = val[i];
        items[i].wt = wt[i];
        items[i].idx = i;
        items[i].ratio = (double)val[i] / wt[i];
```

```
    }
    qsort(items, n, sizeof(Item), cmp);

    for (int i = 0; i < n; i++)
        taken[i] = false;

    dfs(0, 0, 0);

    printf("Specified weights: ");

    for (int i = 0; i < n; i++) {
        if (bestTaken[i]) {
            printf("%d ", items[i].wt);
        }
    }

    return 0;
}
```

***Status :*** Partially correct ***Marks : 6/10***

4. Problem Statement

Janu is tasked with creating a program that solves the Traveling Salesman Problem (TSP). The goal is to find the shortest possible route that visits a set of cities and returns to the starting city.

You are required to implement a program that takes input regarding the number of cities and the distance matrix between cities and outputs the optimal route and its associated minimum cost.

The diagonal values will be 0, as the distance from a city to itself is 0.

Note: Solve it using Branch and Bound approach.

***Input Format***

The first line of input consists of an integer n, representing the number of cities.

The next n lines contain n space-separated integers each, representing the distance matrix between the cities.

The cell (i, j) represents the distance from city i to city j.

***Output Format***

The first line of output displays "The Path is: " followed by the path of the route using ---> in between.

The second line displays "Minimum cost is " followed by the total minimum cost associated with the shortest route.

Refer to the sample output for the formatting specifications.

***Sample Test Case***

Input: 4
0 4 1 3
4 0 2 1
1 2 0 5
3 1 5 0
Output: The Path is: 1--->3--->2--->4--->1
Minimum cost is 7

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <limits.h>
#include <stdbool.h>

#define MAX 10

int n;
int cost[MAX][MAX];

bool visited[MAX];
int path[MAX];
int bestPath[MAX];

int bestCost = INT_MAX;
```

```
int minEdge = INT_MAX;

// find minimum edge in graph (for bound)
void findMinEdge() {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (i != j && cost[i][j] > 0 && cost[i][j] < minEdge) {
                minEdge = cost[i][j];
            }
        }
    }
}

// lower bound
int bound(int currCost, int level) {
    return currCost + (n - level + 1) * minEdge;
}

void tsp(int level, int currCity, int currCost) {
    if (level == n) {
        if (cost[currCity][0] > 0) {
            int total = currCost + cost[currCity][0];
            if (total < bestCost) {
                bestCost = total;
                for (int i = 0; i < n; i++)
                    bestPath[i] = path[i];
            }
        }
        return;
    }

    if (bound(currCost, level) >= bestCost)
        return;

    for (int i = 0; i < n; i++) {
        if (!visited[i] && cost[currCity][i] > 0) {
            visited[i] = true;
            path[level] = i;

            tsp(level + 1, i, currCost + cost[currCity][i]);
```

```
            visited[i] = false;
        }
    }
}

int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &cost[i][j]);

    findMinEdge();

    visited[0] = true;
    path[0] = 0;

    tsp(1, 0, 0);

    printf("The Path is: ");

    for (int i = 0; i < n; i++) {
        printf("%d--->", bestPath[i] + 1);
    }
    printf("1\n");

    printf("Minimum cost is %d", bestCost);

    return 0;
}
```

***Status :*** Partially correct **Marks : 8.5/10**

5. Problem Statement

A traveling salesman is tasked with finding the shortest route to visit a set of cities and return to the starting city. The goal is to minimize the total distance traveled by the salesman while visiting each city exactly once.

***Input Format***

The first line contains an integer n, the number of cities.

The next n lines contain n space-separated integers each, representing the distances between cities.

The ith line represents the distances from city i to all other cities.

The last line contains an integer s, representing the starting city.

***Output Format***

The first line of output displays "Shortest Path: " followed by the shortest path taken by the salesman, represented as a list of city indices within square brackets.

The second line of output displays "Minimum Cost: " followed by the minimum cost (total distance) of the shortest path.

Refer to the sample output for the formatting specifications.

***Sample Test Case***

Input: 4
0 10 15 20
10 0 35 25
15 35 0 30
20 25 30 0
0
Output: Shortest Path: [0, 1, 3, 2, 0]
Minimum Cost: 80

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <limits.h>
#include <stdbool.h>

#define MAX 10

int n, start;
int cost[MAX][MAX];
```

```
bool visited[MAX];
int path[MAX], bestPath[MAX];

int bestCost = INT_MAX;

int minEdge = INT_MAX;

// find smallest non-zero edge
void findMinEdge() {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (i != j && cost[i][j] > 0 && cost[i][j] < minEdge) {
                minEdge = cost[i][j];
            }
        }
    }
}

// simple lower bound
int bound(int currCost, int level) {
    return currCost + (n - level + 1) * minEdge;
}

void tsp(int level, int currCity, int currCost) {
    if (level == n) {
        if (cost[currCity][start] > 0) {
            int total = currCost + cost[currCity][start];
            if (total < bestCost) {
                bestCost = total;

                for (int i = 0; i < n; i++)
                    bestPath[i] = path[i];
            }
        }
        return;
    }

    if (bound(currCost, level) >= bestCost)
        return;

    for (int i = 0; i < n; i++) {
```

```
        if (!visited[i] && cost[currCity][i] > 0) {
            visited[i] = true;
            path[level] = i;

            tsp(level + 1, i, currCost + cost[currCity][i]);

            visited[i] = false;
        }
    }
}

int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &cost[i][j]);

    scanf("%d", &start);

    findMinEdge();

    visited[start] = true;
    path[0] = start;

    tsp(1, start, 0);

    printf("Shortest Path: [");

    for (int i = 0; i < n; i++) {
        printf("%d", bestPath[i]);
        if (i != n - 1) printf(", ");
    }

    printf(", %d]\n", start);

    printf("Minimum Cost: %d", bestCost);

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

# Vel Tech Group of Institutions

Name: NIKHIL  KUMAR
Email: vtu28682@veltech.edu.in
Roll no: vtu28682
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

## 2028_NeoColab_Design and Analysis of AlgorithmsS1L10

### VTU_DAA_Week 10_CY

Attempt : 1
Total Mark : 50
Marks Obtained : 45

### Section 1 : Coding

1. Problem Statement

Meera, the director of a national disaster response team, must deploy three specialized rescue units to three critical disaster sites. Each rescue unit has a different response time (in minutes) to reach each site due to distance, terrain, and accessibility constraints.

However, Meera has an additional operational rule:

Rescue Unit 1 cannot be assigned to Site 3 due to equipment limitations.Rescue Unit 2 must not be assigned to Site 1 because of route restrictions.

Meera decides to use the Branch and Bound technique to explore only valid and efficient assignments while respecting all constraints.

Your task is to determine the total response time of the best valid

deployment plan that satisfies all given restrictions.

***Input Format***

The input consists of three lines with three space-separated integers each, representing a 3 × 3 response time matrix.

Each cell (i, j) represents the response time (in minutes) if the ith rescue unit is deployed to the jth disaster site.

***Output Format***

The output should display the total response time in the following exact format: "Total Response Time: X" ,where X is the total response time of the best valid deployment plan.

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 12 25 40
30 14 28
16 18 11
Output: Total Response Time: 37

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <limits.h>

int min(int a, int b) {
    return (a < b) ? a : b;
}

int main() {
    int cost[3][3];

    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            scanf("%d", &cost[i][j]);
        }
```

```
    }
    int perm[3] = {0, 1, 2};
    int best = INT_MAX;

    // try all permutations manually (3! = 6)
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            if (j == i) continue;
            for (int k = 0; k < 3; k++) {
                if (k == i || k == j) continue;

                int assign[3] = {perm[i], perm[j], perm[k]};

                // constraints
                if (assign[0] == 2) continue; // Unit 1 cannot go to Site 3
                if (assign[1] == 0) continue; // Unit 2 cannot go to Site 1

                int sum = 0;
                for (int x = 0; x < 3; x++) {
                    sum += cost[x][assign[x]];
                }

                best = min(best, sum);
            }
        }
    }

    printf("Total Response Time: %d", best);

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

2. Problem Statement

Captain Aidan, a daring pirate, has discovered an ancient treasure cave filled with valuable relics. However, the cave is protected by an ancient spell that limits the weight of the treasure he can carry. Aidan has a magic knapsack with a maximum weight capacity of W. Inside the cave, he finds

n different treasures, each with a specific weight and value.

Since he can only take each treasure once, Aidan must carefully choose which items to take to maximize the total value without exceeding the weight limit of his knapsack.

Help Captain Aidan pick the most valuable treasures while staying within the weight limit using branch and bound technique

***Input Format***

The first line contains two integers n (the number of items) and W (the capacity of the knapsack), separated by a space.

The second line contains n integers, where the i-th integer represents the weight of the i-th item.

The third line contains n integers, where the i-th integer represents the value of the i-th item

***Output Format***

The output prints the single integer representing the maximum value obtained by filling the knapsack with the given set of items.

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 5 60
10 20 30 40 50
60 100 120 140 180

Output: 280

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

int n, W;
```

```
typedef struct {
    int weight;
    int value;
    float ratio;
} Item;

typedef struct {
    int level;
    int profit;
    int weight;
    float bound;
} Node;

Item items[105];

// descending sort by value/weight ratio
int compare(const void *a, const void *b) {
    Item *i1 = (Item *)a;
    Item *i2 = (Item *)b;
    if (i2->ratio > i1->ratio) return 1;
    else return -1;
}

// compute upper bound
float bound(Node u) {
    if (u.weight >= W) return 0;

    float profit_bound = u.profit;
    int j = u.level + 1;
    int total_weight = u.weight;

    while (j < n && total_weight + items[j].weight <= W) {
        total_weight += items[j].weight;
        profit_bound += items[j].value;
        j++;
    }

    if (j < n) {
        profit_bound += (W - total_weight) * items[j].ratio;
    }
```

```
    return profit_bound;
}

int max(int a, int b) {
    return (a > b) ? a : b;
}

// simple queue for BFS-like exploration
Node queue[10000];
int front = 0, rear = 0;

void push(Node x) {
    queue[rear++] = x;
}

Node pop() {
    return queue[front++];
}

int empty() {
    return front == rear;
}

int main() {
    scanf("%d %d", &n, &W);

    int w[105], v[105];

    for (int i = 0; i < n; i++) scanf("%d", &w[i]);
    for (int i = 0; i < n; i++) scanf("%d", &v[i]);

    for (int i = 0; i < n; i++) {
        items[i].weight = w[i];
        items[i].value = v[i];
        items[i].ratio = (float)v[i] / w[i];
    }

    qsort(items, n, sizeof(Item), compare);

    Node u, vnode;
    int maxProfit = 0;
```

```
    u.level = -1;
    u.profit = 0;
    u.weight = 0;
    u.bound = bound(u);

    push(u);

    while (!empty()) {
        u = pop();

        if (u.level == n - 1) continue;

        // take next item
        vnode.level = u.level + 1;
        vnode.weight = u.weight + items[vnode.level].weight;
        vnode.profit = u.profit + items[vnode.level].value;

        if (vnode.weight <= W && vnode.profit > maxProfit)
            maxProfit = vnode.profit;

        vnode.bound = bound(vnode);

        if (vnode.bound > maxProfit)
            push(vnode);

        // not take item
        vnode.weight = u.weight;
        vnode.profit = u.profit;
        vnode.bound = bound(vnode);

        if (vnode.bound > maxProfit)
            push(vnode);
    }

    printf("%d", maxProfit);

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

3. Problem Statement

Commander Vikram is preparing a spacecraft for an interplanetary research mission. The spacecraft has a limited cargo capacity, so he must carefully decide which scientific equipment modules to load.

Each module has a scientific importance value and a specific weight. Commander Vikram wants to load the spacecraft in such a way that the total scientific value of the selected modules is maximized without exceeding the cargo weight limit.

To solve this efficiently, he uses the Branch and Bound technique applied to the 0/1 Knapsack problem, which explores possible combinations of equipment and eliminates options that cannot lead to better solutions.

Your task is to help Commander Vikram determine the maximum achievable value and the selected module weights that should be loaded into the spacecraft.

Note: The weights must be printed in reverse order of the input, meaning the algorithm starts evaluating items from the last item and prints the selected weights first.

***Input Format***

The first line contains an integer n, representing the number of equipment modules available.

The second line contains n space-separated integers val[i], representing the scientific value of each module.

The third line contains n space-separated integers wt[i], representing the weight of each module.

The fourth line contains an integer W, representing the maximum cargo capacity of the spacecraft.

***Output Format***

The first line output displays "Maximum value: " followed by an integer, representing the maximum value that can be achieved.

vtu28682

The second line output displays "Selected weights: " followed by space-separated integers, printed in reverse order of the input.

Refer to the sample outputs for the exact format.

***Sample Test Case***

Input: 3
60 100 120
10 20 30
50
Output: Maximum value: 220
Selected weights: 30 20

***Answer***

```
// You are using GCC
#include <stdio.h>

int n, W;
int val[15], wt[15];
int bestValue = 0;
int bestSelected[15];
int currentSelected[15];

// compare function for sorting by value/weight ratio (not strictly required but helps bounding)
void sortByRatio() {
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            float r1 = (float)val[i] / wt[i];
            float r2 = (float)val[j] / wt[j];
            if (r2 > r1) {
                int tv = val[i]; val[i] = val[j]; val[j] = tv;
                int tw = wt[i]; wt[i] = wt[j]; wt[j] = tw;
            }
        }
    }
}

// upper bound (fractional knapsack idea)
```

```
float bound(int level, int weight, int profit) {
    if (weight >= W) return 0;

    float result = profit;
    int totalWeight = weight;

    for (int i = level + 1; i < n; i++) {
        if (totalWeight + wt[i] <= W) {
            totalWeight += wt[i];
            result += val[i];
        } else {
            result += (W - totalWeight) * ((float)val[i] / wt[i]);
            break;
        }
    }

    return result;
}

void knapsack(int level, int weight, int profit) {
    if (weight <= W && profit > bestValue) {
        bestValue = profit;

        for (int i = 0; i < n; i++)
            bestSelected[i] = currentSelected[i];
    }

    if (level == n - 1) return;

    if (bound(level, weight, profit) <= bestValue)
        return;

    // include item
    currentSelected[level + 1] = 1;
    knapsack(level + 1,
             weight + wt[level + 1],
             profit + val[level + 1]);

    // exclude item
    currentSelected[level + 1] = 0;
    knapsack(level + 1,
             weight,
```

```
        profit);
}

int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
        scanf("%d", &val[i]);

    for (int i = 0; i < n; i++)
        scanf("%d", &wt[i]);

    scanf("%d", &W);

    // optional sorting for better pruning
    sortByRatio();

    for (int i = 0; i < n; i++)
        currentSelected[i] = 0;

    knapsack(-1, 0, 0);

    printf("Maximum value: %d\n", bestValue);

    printf("Selected weights: ");

    // print in reverse order of input
    for (int i = n - 1; i >= 0; i--) {
        if (bestSelected[i]) {
            printf("%d ", wt[i]);
        }
    }

    return 0;
}
```

***Status :*** Partially correct **Marks : 5/10**

4. Problem Statement

You are tasked with solving the Travelling Salesman Problem (TSP) for a

given set of cities.

Given the distances between pairs of cities, your goal is to find the shortest route that visits each city exactly once and returns to the starting city.

Note: Solve it using Branch and Bound approach.

***Input Format***

The first line of input contains an integer n, representing the number of cities.

The following n lines each contain n space-separated integers, representing the distances between pairs of cities.

The distance between city i and city j is given at the ith row and jth column of the matrix.

***Output Format***

The first line displays "The Path is: " followed by the sequence of cities visited in the shortest path,

separated by "--->".

The second line displays "Minimum cost is X", where X is the minimum cost of the shortest path found.

Refer to the sample output for the formatting specifications.

***Sample Test Case***

Input: 4
0 4 1 3
4 0 2 1
1 2 0 5
3 1 5 0
Output: The Path is: 1--->3--->2--->4--->1
Minimum cost is 7

***Answer***

// You are using GCC

```
#include <stdio.h>
#include <limits.h>

#define N 10

int n;
int cost[N][N];
int visited[N];

int final_path[N];
int temp_path[N];

int final_res = INT_MAX;

// copy solution
void copyPath() {
    for (int i = 0; i < n; i++)
        final_path[i] = temp_path[i];
}

// calculate current cost
int calculateCost(int path[]) {
    int total = 0;
    for (int i = 0; i < n - 1; i++) {
        total += cost[path[i]][path[i + 1]];
    }
    total += cost[path[n - 1]][path[0]];
    return total;
}

// simple branch and bound (DFS with pruning)
void tsp(int level, int current_cost) {
    if (level == n) {
        int total = current_cost + cost[temp_path[n - 1]][temp_path[0]];
        if (total < final_res) {
            final_res = total;
            copyPath();
        }
        return;
    }

    for (int i = 0; i < n; i++) {
```

```
        if (!visited[i]) {

            int prev = temp_path[level - 1];
            int new_cost = current_cost + cost[prev][i];

            if (new_cost >= final_res)
                continue; // pruning

            temp_path[level] = i;
            visited[i] = 1;

            tsp(level + 1, new_cost);

            visited[i] = 0;
        }
    }
}

int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &cost[i][j]);
        }
    }

    for (int i = 0; i < n; i++)
        visited[i] = 0;

    // start from city 0 (city 1 in output)
    temp_path[0] = 0;
    visited[0] = 1;

    tsp(1, 0);

    printf("The Path is: ");
    for (int i = 0; i < n; i++) {
        printf("%d--->", final_path[i] + 1);
    }
    printf("%d\n", final_path[0] + 1);
```

```
    printf("Minimum cost is %d", final_res);

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

5. Problem Statement

You are a logistics manager responsible for planning delivery routes for a fleet of vehicles. Each vehicle needs to visit a set of locations to deliver goods, and the cost of traveling between locations is known.

Your goal is to find the shortest route for each vehicle, ensuring that each location is visited exactly once before returning to the starting point.

Note: Solve it using Branch and Bound approach.

***Input Format***

The first line contains an integer n, the number of locations.

The next n lines contain n integers each, representing the cost matrix of traveling between locations.

The ith integer on the jth line represents the cost of traveling from location i to location j.

***Output Format***

The first line of output displays "Shortest Path: " followed by the shortest path that visits all cities.

The second line of output displays "Minimum cost is " followed by the minimum cost.

Refer to the sample output for the formatting specifications.

***Sample Test Case***

Input: 5
0 10 8 9 7
10 0 10 5 6
8 10 0 8 9
9 5 8 0 6
7 6 9 6 0

Output: Shortest Path: 1 3 4 2 5 1
Minimum cost is 34

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <limits.h>

#define MAX 10

int n;
int cost[MAX][MAX];
int visited[MAX];

int bestPath[MAX];
int currPath[MAX];

int bestCost = INT_MAX;

void copyPath() {
    for (int i = 0; i < n; i++) {
        bestPath[i] = currPath[i];
    }
}

// compute complete tour cost
int getCost() {
    int total = 0;
    for (int i = 0; i < n - 1; i++) {
        total += cost[currPath[i]][currPath[i + 1]];
    }
    total += cost[currPath[n - 1]][currPath[0]];
    return total;
}

// DFS with pruning (Branch & Bound)
```

```
void tsp(int level, int currentCost) {
    if (level == n) {
        int total = currentCost + cost[currPath[n - 1]][currPath[0]];

        if (total < bestCost) {
            bestCost = total;
            copyPath();
        }
        return;
    }

    for (int i = 0; i < n; i++) {
        if (!visited[i]) {

            int prev = currPath[level - 1];
            int newCost = currentCost + cost[prev][i];

            if (newCost >= bestCost)
                continue; // pruning

            currPath[level] = i;
            visited[i] = 1;

            tsp(level + 1, newCost);

            visited[i] = 0;
        }
    }
}

int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &cost[i][j]);
        }
    }

    for (int i = 0; i < n; i++)
        visited[i] = 0;
```

```
    // start from city 0 (city 1 in output)
    currPath[0] = 0;
    visited[0] = 1;

    tsp(1, 0);

    printf("Shortest Path: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", bestPath[i] + 1);
    }
    printf("%d\n", bestPath[0] + 1);

    printf("Minimum cost is %d", bestCost);

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

# Vel Tech Group of Institutions

Name: NIKHIL  KUMAR
Email: vtu28682@veltech.edu.in
Roll no: vtu28682
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

## 2028_NeoColab_Design and Analysis of AlgorithmsS1L10

### VTU_DAA_Week 11_COD

Attempt : 1
Total Mark : 30
Marks Obtained : 30

### Section 1 : Coding

1. Problem Statement

Alice needs to assign works to employees in a company. Each employee can be assigned at most one work, and each work can be assigned to at most one employee. Her goal is to maximize the number of employees who are assigned works, ensuring that each work is assigned to exactly one employee and each employee is assigned at most one work.

Help Alice to complete the task.

***Input Format***

The first line consists of an integer N, representing the number of connections between employees and works.

The next N lines consists of two space-separated integers X and Y, representing a connection where X is the index of an employee (0-based indexing), and Y is

the index of a work (0-based indexing), indicating that employee X can perform work Y.

***Output Format***

The first line of the output displays "Maximum number of employees that could get works are: a", where a is an integer representing the maximum number of employees that could be assigned works according to the given conditions.

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 9
0 0
0 1
0 2
1 0
2 1
3 4
4 3
4 4
5 5
Output: Maximum number of employees that could get works are: 6

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <string.h>

#define MAX 100

int graph[MAX][MAX];
int match[MAX];
int visited[MAX];

int nEdges;
int maxNode = 0;

int dfs(int u) {
    for (int v = 0; v <= maxNode; v++) {
```

```
        if (graph[u][v] && !visited[v]) {
            visited[v] = 1;

            if (match[v] == -1 || dfs(match[v])) {
                match[v] = u;
                return 1;
            }
        }
    }
    return 0;
}

int main() {
    scanf("%d", &nEdges);

    memset(graph, 0, sizeof(graph));
    memset(match, -1, sizeof(match));

    int x, y;

    for (int i = 0; i < nEdges; i++) {
        scanf("%d %d", &x, &y);
        graph[x][y] = 1;

        if (x > maxNode) maxNode = x;
        if (y > maxNode) maxNode = y;
    }

    int result = 0;

    for (int i = 0; i <= maxNode; i++) {
        memset(visited, 0, sizeof(visited));
        if (dfs(i)) {
            result++;
        }
    }

    printf("Maximum number of employees that could get works are: %d", result);

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

2. Problem Statement

Given an undirected bipartite graph with X vertices on the left side and Y vertices on the right side, the graph may contain multiple edges between the same pair of vertices.

Your task is to determine the minimum vertex cover for this graph.

A vertex cover of a graph is a set of vertices such that every edge in the graph is incident to at least one vertex in the set.

***Input Format***

The first line contains two space-separated integers X and Y, representing the number of vertices in the left set and right set, respectively.

The second line contains an integer e, representing the number of edges in the bipartite graph.

The next e lines each contain two space-separated integers u and v, representing an edge between: vertex u in the left set and vertex v in the right set

***Output Format***

The output prints the vertices in the minimum vertex cover, separated by a space.

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 4 4
4
0 1
0 2
1 3
2 3

Output: 0 3

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <string.h>

#define MAX 105

int X, Y, e;
int graph[MAX][MAX];

int matchR[MAX];
int visR[MAX];
int visL[MAX];

int queue[MAX];
int front, rear;

// DFS for maximum matching
int bpm(int u) {
    for (int v = 0; v < Y; v++) {
        if (graph[u][v] && !visR[v]) {
            visR[v] = 1;

            if (matchR[v] == -1 || bpm(matchR[v])) {
                matchR[v] = u;
                return 1;
            }
        }
    }
    return 0;
}

// BFS/DFS to find reachable left nodes in alternating graph
void mark(int u) {
    visL[u] = 1;

    for (int v = 0; v < Y; v++) {
        if (graph[u][v]) {
            if (!visR[v]) {
                visR[v] = 1;
```

```
            if (matchR[v] != -1 && !visL[matchR[v]]) {
                mark(matchR[v]);
            }
        }
    }
  }
}

int main() {
    scanf("%d %d", &X, &Y);
    scanf("%d", &e);

    memset(graph, 0, sizeof(graph));
    memset(matchR, -1, sizeof(matchR));

    for (int i = 0; i < e; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        graph[u][v] = 1;
    }

    // Step 1: Maximum matching
    int maxMatch = 0;
    for (int i = 0; i < X; i++) {
        memset(visR, 0, sizeof(visR));
        if (bpm(i)) maxMatch++;
    }

    // Step 2: Find minimum vertex cover using Kőnig's theorem
    memset(visL, 0, sizeof(visL));
    memset(visR, 0, sizeof(visR));

    for (int i = 0; i < X; i++) {
        int isMatched = 0;
        for (int j = 0; j < Y; j++) {
            if (graph[i][j] && matchR[j] == i) {
                isMatched = 1;
                break;
            }
        }
        if (!isMatched) {
```

```
            mark(i);
        }
    }

    // Output vertex cover:
    // Left side: not visited
    // Right side: visited
    for (int i = 0; i < X; i++) {
        if (!visL[i])
            printf("%d ", i);
    }

    for (int j = 0; j < Y; j++) {
        if (visR[j])
            printf("%d ", j);
    }

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

3. Problem Statement

You are given an undirected graph with 'num_vertices' vertices and 'num_edges' edges. Your task is to determine whether the given graph is bipartite or not. A graph is bipartite if it can be divided into two disjoint sets of vertices such that each edge connects a vertex from one set to a vertex from the other set.

***Input Format***

The first line contains two integers separated by a space: 'num_vertices' and 'num_edges'. These represent the number of vertices and edges in the graph, respectively.

The next 'num_edges' lines each contain two integers 'u' and 'v' separated by a space, representing an edge between vertex 'u' and vertex 'v'.

***Output Format***

The output displays a single line indicating whether the given graph is bipartite

or not.

If the graph is bipartite, print "The graph is bipartite."

If the graph is not bipartite, print "The graph is not bipartite."

***Sample Test Case***

Input: 4 4
0 1
1 2
2 3
3 0
Output: The graph is bipartite.

***Answer***

```
// You are using GCC
#include <stdio.h>

#define MAX 105

int graph[MAX][MAX];
int color[MAX];
int queue[MAX];

int isBipartite(int start, int n) {
    int front = 0, rear = 0;

    queue[rear++] = start;
    color[start] = 0;

    while (front < rear) {
        int u = queue[front++];

        for (int v = 0; v < n; v++) {
            if (graph[u][v]) {

                if (color[v] == -1) {
                    color[v] = 1 - color[u];
                    queue[rear++] = v;
                }
                else if (color[v] == color[u]) {
                    return 0;
```

```
            }
        }
    }
}

    return 1;
}

int main() {
    int n, e;
    scanf("%d %d", &n, &e);

    for (int i = 0; i < n; i++)
        color[i] = -1;

    for (int i = 0; i < e; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        graph[u][v] = 1;
        graph[v][u] = 1;
    }

    for (int i = 0; i < n; i++) {
        if (color[i] == -1) {
            if (!isBipartite(i, n)) {
                printf("The graph is not bipartite.");
                return 0;
            }
        }
    }

    printf("The graph is bipartite.");

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

# Vel Tech Group of Institutions

Name: NIKHIL  KUMAR
Email: vtu28682@veltech.edu.in
Roll no: vtu28682
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

## 2028_NeoColab_Design and Analysis of AlgorithmsS1L10

### VTU_DAA_Week 11_PAH

Attempt : 1
Total Mark : 50
Marks Obtained : 42.5

### Section 1 : Coding

1. Problem Statement

You are given a bipartite graph represented using an edge list. The graph contains two sets of nodes tasks and processors. Each edge connects a task node to a processor node, indicating that the processor can execute that task.

Your goal is to determine the maximum matching in the bipartite graph. A matching is a set of edges such that:

Each task is assigned to at most one processor.Each processor is assigned to at most one task.

Write a program to compute the maximum number of task–processor assignments possible.

***Input Format***

The first line contains an integer T, representing the number of tasks.

The second line contains an integer P, representing the number of processors.

The third line contains an integer E, representing the number of edges in the bipartite graph.

The next E lines each contain two space-separated integers task and processor where this indicates that the given task can be assigned to the specified processor.

***Output Format***

The output prints the result in the following format: "The maximum matching is: X", where X is the maximum number of matches that can be made in the bipartite graph.

Refer to the sample output for the formatting specifications.

***Sample Test Case***

Input: 4
3
5
1 1
2 2
4 3
3 3
4 1
Output: The maximum matching is: 3

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <string.h>

#define MAX 15

int graph[MAX][MAX];
int match[MAX];
int visited[MAX];
```

```
int T, P, E;

// DFS to find augmenting path
int dfs(int task) {
    for (int proc = 1; proc <= P; proc++) {
        if (graph[task][proc] && !visited[proc]) {
            visited[proc] = 1;

            if (match[proc] == -1 || dfs(match[proc])) {
                match[proc] = task;
                return 1;
            }
        }
    }
    return 0;
}

int main() {
    scanf("%d", &T);
    scanf("%d", &P);
    scanf("%d", &E);

    memset(graph, 0, sizeof(graph));
    memset(match, -1, sizeof(match));

    for (int i = 0; i < E; i++) {
        int t, p;
        scanf("%d %d", &t, &p);
        graph[t][p] = 1;
    }

    int maxMatching = 0;

    for (int i = 1; i <= T; i++) {
        memset(visited, 0, sizeof(visited));
        if (dfs(i)) {
            maxMatching++;
        }
    }

    printf("The maximum matching is: %d", maxMatching);
```

```
    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

2. Problem Statement

Irfan is working on a project that involves managing relationships between two sets of entities, set U and set V. He wants to determine the maximum matching in a bipartite graph to optimize connections without overlap.Irfan also wants to explore different scenarios by adding a new connection (edge) between two entities and observing how it affects the maximum matching.

Your task is to compute the size of the maximum matching before and after adding a specified edge.

Note: Vertices in set U are numbered from 0 to n-1, and vertices in set V are numbered from 1 to m.

***Input Format***

The first line contains an integer n, representing the number of vertices in set U.

The second line contains an integer m, representing the number of vertices in set V.

The third line contains an integer e, representing the number of edges.

The next e lines each contain two integers u and v, representing an edge between vertex u from set U and vertex v from set V.

The last line contains two integers u and v, representing the edge to be added.

***Output Format***

The output prints the results in the following format:

Size of maximum matching before adding edges: X

Size of maximum matching after adding edges: Y

Refer to the sample output for the formatting specifications.

***Sample Test Case***

Input: 3
4
3
0 1
0 2
1 1
1 2
Output: Size of maximum matching before adding edges: 1
Size of maximum matching after adding edges: 1

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <string.h>

#define MAX 105

int graph[MAX][MAX];
int match[MAX];
int visited[MAX];

int n, m, e;

// DFS for augmenting path
int dfs(int u) {
    for (int v = 1; v <= m; v++) {
        if (graph[u][v] && !visited[v]) {
            visited[v] = 1;

            if (match[v] == -1 || dfs(match[v])) {
                match[v] = u;
                return 1;
            }
        }
```

```
    }
    return 0;
}

// compute max matching
int maxMatching() {
    memset(match, -1, sizeof(match));
    int result = 0;

    for (int i = 0; i < n; i++) {
        memset(visited, 0, sizeof(visited));
        if (dfs(i)) {
            result++;
        }
    }

    return result;
}

int main() {
    scanf("%d", &n);
    scanf("%d", &m);
    scanf("%d", &e);

    memset(graph, 0, sizeof(graph));

    for (int i = 0; i < e; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        graph[u][v] = 1;
    }

    int u, v;
    scanf("%d %d", &u, &v);

    int before = maxMatching();

    // add new edge
    graph[u][v] = 1;

    int after = maxMatching();
```

```
    printf("Size of maximum matching before adding edges: %d\n", before);
    printf("Size of maximum matching after adding edges: %d", after);

    return 0;
}
```

***Status :*** Partially correct ***Marks : 2.5/10***

3. Problem Statement

Jack wants to create a program to manage a bipartite graph, which contains two sets of vertices.He needs the program to:

Initialize the graph with a specific size,Add edges between the two sets of vertices, andFind pairings where each vertex from the first set is matched with at most one vertex from the second set, and no two vertices from the first set are matched with the same vertex from the second set.

The goal is to determine the maximum matching pairs in the bipartite graph.

***Input Format***

The first line contains an integer m, representing the number of vertices in the first set.

The second line contains an integer n, representing the number of vertices in the second set.

The third line contains an integer e, representing the number of edges in the graph.

The next e lines each contain two integers u and v, representing an edge between vertex u in the first set and vertex v in the second set.

***Output Format***

The output prints the maximum matching pairs in increasing order of u, where each pair is printed in the format: (u, v)

If there are no matching pairs, print: "No matching found"

Refer to the sample output for the formatting specifications.

***Sample Test Case***

Input: 4
3
5
1 1
2 2
4 3
3 3
4 1

Output: (1, 1)
(2, 2)
(3, 3)

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <string.h>

#define MAX 15

int graph[MAX][MAX];
int match[MAX];
int visited[MAX];

int m, n, e;

// DFS for augmenting path
int dfs(int u) {
    for (int v = 1; v <= n; v++) {
        if (graph[u][v] && !visited[v]) {
            visited[v] = 1;

            if (match[v] == -1 || dfs(match[v])) {
                match[v] = u;
                return 1;
            }
        }
    }
```

```
    return 0;
}

typedef struct {
    int u, v;
} Pair;

int main() {
    scanf("%d", &m);
    scanf("%d", &n);
    scanf("%d", &e);

    memset(graph, 0, sizeof(graph));
    memset(match, -1, sizeof(match));

    for (int i = 0; i < e; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        graph[u][v] = 1;
    }

    // maximum matching
    for (int i = 1; i <= m; i++) {
        memset(visited, 0, sizeof(visited));
        dfs(i);
    }

    Pair res[MAX];
    int count = 0;

    for (int v = 1; v <= n; v++) {
        if (match[v] != -1) {
            res[count].u = match[v];
            res[count].v = v;
            count++;
        }
    }

    if (count == 0) {
        printf("No matching found");
        return 0;
    }
```

```
    // sort by u (increasing order)
    for (int i = 0; i < count - 1; i++) {
        for (int j = i + 1; j < count; j++) {
            if (res[i].u > res[j].u) {
                Pair temp = res[i];
                res[i] = res[j];
                res[j] = temp;
            }
        }
    }

    for (int i = 0; i < count; i++) {
        printf("(%d, %d)\n", res[i].u, res[i].v);
    }

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

4. Problem Statement

Alex needs to pair students from two different classes for a project, aiming to maximize the number of compatible pairs. Each student can be paired with only one student from the other class. The compatibility between students is represented as a binary adjacency matrix.

Alex wants to write a program to find the maximum number of compatible student pairs that can be formed.

***Input Format***

The first line contains an integer n, representing the number of students in each group.

The second line contains an integer m, representing the number of students in the second group.

The next n lines contain m space-separated integers each, representing the adjacency matrix of student compatibility.

***Output Format***

The single line containing the phrase "Maximum Bipartite Matching: x" where x is an integer representing the maximum number of compatible pairs that can be formed.

Refer to the sample output for the formatting specifications.

***Sample Test Case***

Input: 4
3
1 1 0
1 0 1
0 1 0
1 0 0
Output: Maximum Bipartite Matching: 3

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <string.h>

#define MAX 105

int graph[MAX][MAX];
int match[MAX];
int visited[MAX];

int n, m;

// DFS for finding augmenting path
int dfs(int u) {
    for (int v = 0; v < m; v++) {
        if (graph[u][v] && !visited[v]) {
            visited[v] = 1;

            if (match[v] == -1 || dfs(match[v])) {
                match[v] = u;
                return 1;
```

```
        }
    }
    }
    return 0;
}

int main() {
    scanf("%d", &n);
    scanf("%d", &m);

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    memset(match, -1, sizeof(match));

    int result = 0;

    for (int i = 0; i < n; i++) {
        memset(visited, 0, sizeof(visited));
        if (dfs(i)) {
            result++;
        }
    }

    printf("Maximum Bipartite Matching: %d", result);

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

5. Problem Statement:

John is developing a recommendation system for a social networking platform. The system suggests potential friends to users based on their interests and mutual connections. To enhance the user experience, you want to ensure that the recommendation algorithm considers the removal

of certain users from the network due to various reasons such as account deletion or user activity.

However, he still wants to maintain effective friend suggestions even after removing these users. John plan to integrate a functionality that calculates the maximum matching in the bipartite graph representing the user network after removing specific users and their connections.

Help him to achieve the task.

***Input Format***

Enter the number of vertices in partition

Enter the number of vertices in partition

Enter the adjacency matrix representation of the bipartite graph

Enter the vertex to remove

***Output Format***

Maximum matching after removing vertex

***Sample Test Case***

Input: 3
2
0 1
1 0
0 0
1
Output: Maximum matching after removing vertex 1: 1

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <string.h>

#define MAX 15

int graph[MAX][MAX];
int match[MAX];
```

```
int visited[MAX];

int n, m;

// DFS for augmenting path
int dfs(int u) {
    for (int v = 0; v < m; v++) {
        if (graph[u][v] && !visited[v]) {
            visited[v] = 1;

            if (match[v] == -1 || dfs(match[v])) {
                match[v] = u;
                return 1;
            }
        }
    }
    return 0;
}

int main() {
    scanf("%d", &n);
    scanf("%d", &m);

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    int removeVertex;
    scanf("%d", &removeVertex);

    // remove vertex (disable its edges)
    if (removeVertex >= 0 && removeVertex < n) {
        for (int j = 0; j < m; j++) {
            graph[removeVertex][j] = 0;
        }
    }

    memset(match, -1, sizeof(match));

    int result = 0;
```

```
    for (int i = 0; i < n; i++) {
        memset(visited, 0, sizeof(visited));
        if (dfs(i)) {
            result++;
        }
    }

    printf("Maximum matching after removing vertex %d: %d", removeVertex,
result);

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

# Vel Tech Group of Institutions

Name: NIKHIL  KUMAR
Email: vtu28682@veltech.edu.in
Roll no: vtu28682
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

## 2028_NeoColab_Design and Analysis of AlgorithmsS1L10

### VTU_DAA_Week 11_CY

Attempt : 1
Total Mark : 50
Marks Obtained : 50

### Section 1 : Coding

1. Problem Statement:

John is developing a recommendation system for a social networking platform. The system suggests potential friends to users based on their interests and mutual connections. To enhance the user experience, you want to ensure that the recommendation algorithm considers the removal of certain users from the network due to various reasons such as account deletion or user activity.

However, he still wants to maintain effective friend suggestions even after removing these users. John plan to integrate a functionality that calculates the maximum matching in the bipartite graph representing the user network after removing specific users and their connections.

Help him to achieve the task.

vtu28682

***Input Format***

Enter the number of vertices in partition

Enter the number of vertices in partition

Enter the adjacency matrix representation of the bipartite graph

Enter the vertex to remove

***Output Format***

Maximum matching after removing vertex

***Sample Test Case***

Input: 3
2
0 1
1 0
0 0
1
Output: Maximum matching after removing vertex 1: 1

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <string.h>

#define MAX 15

int graph[MAX][MAX];
int match[MAX];
int visited[MAX];

int n, m;

// DFS for augmenting path
int dfs(int u) {
   for (int v = 0; v < m; v++) {
      if (graph[u][v] && !visited[v]) {
         visited[v] = 1;

         if (match[v] == -1 || dfs(match[v])) {
```

```
            match[v] = u;
            return 1;
        }
    }
  }
  return 0;
}

int main() {
  scanf("%d", &n);
  scanf("%d", &m);

  for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        scanf("%d", &graph[i][j]);
    }
  }

  int remove;
  scanf("%d", &remove);

  // remove vertex from partition 1 (row removal)
  if (remove >= 0 && remove < n) {
    for (int j = 0; j < m; j++) {
        graph[remove][j] = 0;
    }
  }

  memset(match, -1, sizeof(match));

  int result = 0;

  for (int i = 0; i < n; i++) {
    if (i == remove) continue;   // skip removed vertex

    memset(visited, 0, sizeof(visited));
    if (dfs(i)) {
        result++;
    }
  }

  printf("Maximum matching after removing vertex %d: %d", remove, result);
```

```
    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

2. Problem Statement

Charlie is working on optimizing the hiring process for interns at a tech startup. There are 4 open positions and 4 applicants, each having different skill sets.

Charlie wants to design a system that matches applicants to positions based on their suitability. The suitability is represented using a binary matrix, where:

1 indicates the applicant is suitable for the position.0 indicates the applicant is not suitable for the position.

Each applicant can be assigned to only one position, and each position can accept only one applicant.

The goal is to determine the maximum number of positions that can be filled by assigning suitable applicants.

***Input Format***

The input consists of 4 lines.

Each line contains 4 space-separated integers (0 or 1).

Each row represents an applicant.

Each column represents a job position.

***Output Format***

The output prints the result in the following format: "The job can hire up to X applicants" where X represents the maximum number of applicants that can be assigned to positions.

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 1 1 0 0
1 0 0 0
0 0 1 1
0 0 0 1
Output: The job can hire up to 4 applicants

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <string.h>

#define N 4

int graph[N][N];
int match[N];
int visited[N];

// DFS for finding augmenting path
int dfs(int u) {
   for (int v = 0; v < N; v++) {
      if (graph[u][v] && !visited[v]) {
         visited[v] = 1;

         if (match[v] == -1 || dfs(match[v])) {
            match[v] = u;
            return 1;
         }
      }
   }
   return 0;
}

int main() {
   for (int i = 0; i < N; i++) {
      for (int j = 0; j < N; j++) {
         scanf("%d", &graph[i][j]);
      }
```

```
    }
    memset(match, -1, sizeof(match));

    int result = 0;

    for (int i = 0; i < N; i++) {
        memset(visited, 0, sizeof(visited));
        if (dfs(i)) {
            result++;
        }
    }

    printf("The job can hire up to %d applicants", result);

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

3. Problem Statement

Emily is working on a project that involves managing relationships between two sets of entities, set U and set V. She wants to determine the maximum matching in a bipartite graph to optimize connections without overlap.

Emily also needs the ability to remove a specific connection (edge) between two entities in order to analyze different scenarios and measure the impact on the size of the maximum matching.

Your task is to compute the size of the maximum matching before and after removing a given edge.

Note: Vertices in set U are numbered from 1 to n, and vertices in set V are numbered from 1 to m.

***Input Format***

The first line contains an integer n, representing the number of vertices in set U.

The second line contains an integer m, representing the number of vertices in set V.

The third line contains an integer e, representing the number of edges.

The next e lines each contain two integers u and v, representing an edge between vertex u (from set U) and vertex v (from set V).

The last line contains two integers u and v, representing the edge that must be removed.

***Output Format***

The first line displays: "Size of maximum matching before edge removal: X"

The second line displays: "Size of maximum matching after edge removal: Y"

Refer to the sample output for the formatting specifications

***Sample Test Case***

Input: 3
3
3
0 1
0 2
1 2
1 2
Output: Size of maximum matching before edge removal: 1
Size of maximum matching after edge removal: 0

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <string.h>

#define MAX 105

int graph[MAX][MAX];
int match[MAX];
```

```
int visited[MAX];

int n, m, e;

// DFS for bipartite matching
int dfs(int u) {
    for (int v = 1; v <= m; v++) {
        if (graph[u][v] && !visited[v]) {
            visited[v] = 1;

            if (match[v] == -1 || dfs(match[v])) {
                match[v] = u;
                return 1;
            }
        }
    }
    return 0;
}

// function to compute maximum matching
int maxMatching() {
    memset(match, -1, sizeof(match));
    int result = 0;

    for (int i = 1; i <= n; i++) {
        memset(visited, 0, sizeof(visited));
        if (dfs(i)) {
            result++;
        }
    }

    return result;
}

int main() {
    scanf("%d", &n);
    scanf("%d", &m);
    scanf("%d", &e);

    memset(graph, 0, sizeof(graph));

    for (int i = 0; i < e; i++) {
```

```
        int u, v;
        scanf("%d %d", &u, &v);
        graph[u][v] = 1;
    }

    int ru, rv;
    scanf("%d %d", &ru, &rv);

    // before removal
    int before = maxMatching();

    // remove edge
    graph[ru][rv] = 0;

    // after removal
    int after = maxMatching();

    printf("Size of maximum matching before edge removal: %d\n", before);
    printf("Size of maximum matching after edge removal: %d", after);

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

4. Problem Statement

In a computer lab, Alex is experimenting with task scheduling on a cluster of processors. The lab has 7 tasks that need to be executed and 5 processors available.

Each task can run only on certain processors due to compatibility constraints. These compatibilities are represented using a binary matrix, where:

1 indicates the task can run on that processor.0 indicates the task cannot run on that processor.

Each processor can execute only one task at a time, and each task can be assigned to only one processor. Alex wants to determine the maximum number of tasks that can be executed simultaneously.

Write a program to compute the maximum number of compatible task–processor assignments.

***Input Format***

The input consists of 7 lines.

Each line contains 5 space-separated integers (0 or 1).

Each row represents a task.

Each column represents a processor.

***Output Format***

The output prints the result in the format: "The processors can run a maximum of x tasks at one time." , where x is the maximum number of tasks that can be executed simultaneously.

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 1 1 0 0 0
0 1 0 0 0
0 0 1 1 0
0 0 0 1 1
1 0 0 0 0
0 0 1 0 0
0 0 0 0 1
Output: The processors can run a maximum of 5 tasks at one time.

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <string.h>

#define T 7
#define P 5
```

```
int graph[T][P];
int match[P];
int visited[P];

// DFS for finding augmenting path
int dfs(int task) {
    for (int proc = 0; proc < P; proc++) {
        if (graph[task][proc] && !visited[proc]) {
            visited[proc] = 1;

            if (match[proc] == -1 || dfs(match[proc])) {
                match[proc] = task;
                return 1;
            }
        }
    }
    return 0;
}

int main() {
    for (int i = 0; i < T; i++) {
        for (int j = 0; j < P; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    memset(match, -1, sizeof(match));

    int result = 0;

    for (int i = 0; i < T; i++) {
        memset(visited, 0, sizeof(visited));
        if (dfs(i)) {
            result++;
        }
    }

    printf("The processors can run a maximum of %d tasks at one time.", result);

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

5. Problem Statement

Alex is working on a project that involves pairing students from two different classes for a team project. The project's success depends on maximizing the number of compatible pairs between these two groups. Each student from one group can be paired with a student from the other group, but no student can belong to more than one pair.

Alex represents the compatibility between students using a binary adjacency matrix, where:

1 indicates that the students are compatible.0 indicates that the students are not compatible.

Write a program to determine the maximum number of compatible student pairs that can be formed.

***Input Format***

The first line contains two integers n and m, representing the number of students in the first group and second group, respectively.

The next n lines each contain m space-separated integers, representing the adjacency matrix of student compatibility.

***Output Format***

The output prints a single line in the following format: "Maximum Bipartite Matching: X" , where X is the maximum number of compatible student pairs.

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 4 3
1 1 0
1 0 1
0 1 0

1 0 0
Output: Maximum Bipartite Matching: 3

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <string.h>

#define MAX 105

int graph[MAX][MAX];
int match[MAX];
int visited[MAX];

int n, m;

// DFS to find augmenting path
int dfs(int u) {
    for (int v = 0; v < m; v++) {
        if (graph[u][v] && !visited[v]) {
            visited[v] = 1;

            if (match[v] == -1 || dfs(match[v])) {
                match[v] = u;
                return 1;
            }
        }
    }
    return 0;
}

int main() {
    scanf("%d %d", &n, &m);

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    memset(match, -1, sizeof(match));
```

```
    int result = 0;

    for (int i = 0; i < n; i++) {
        memset(visited, 0, sizeof(visited));
        if (dfs(i)) {
            result++;
        }
    }

    printf("Maximum Bipartite Matching: %d", result);

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

# Vel Tech Group of Institutions

Name: NIKHIL  KUMAR
Email: vtu28682@veltech.edu.in
Roll no: vtu28682
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

## 2028_NeoColab_Design and Analysis of AlgorithmsS1L10

### VTU_DAA_Week 12_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 13

### Section 1 : MCQ

1. How many conditions have to be met if an NP-complete problem is polynomially reducible?

***Answer***

2

***Status :*** Correct ***Marks : 1/1***

2. _____ is the class of decision problems that can be solved by non-deterministic polynomial algorithms.

***Answer***

NP

***Status :*** Correct ***Marks : 1/1***

vtu28682

3. To which of the following classes does a CNF-satisfiability problem belong?

***Answer***

NP class

***Status :*** Wrong ***Marks : 0/1***

4. For problems X and Y, Y is NP-complete and X reduces to Y in polynomial time. Which of the following is TRUE?

***Answer***

X is NP-hard

***Status :*** Wrong ***Marks : 0/1***

5. A problem in NP is NP-complete if

***Answer***

The 3-SAT problem can be reduced to it in polynomial time

***Status :*** Correct ***Marks : 1/1***

6. Which class contains problems that can be solved in polynomial time using a deterministic machine?

***Answer***

P

***Status :*** Correct ***Marks : 1/1***

7. What is the key property of problems in NP?

***Answer***

Their solutions can be verified in polynomial time

***Status :*** Correct ***Marks : 1/1***

vtu28682

8. What does it mean if a problem is in both NP and NP-Complete?

***Answer***

It is the hardest among NP problems

***Status :*** Correct ***Marks : 1/1***

9. If P = NP is ever proven, what would it imply?

***Answer***

All problems in NP can be solved in polynomial time

***Status :*** Correct ***Marks : 1/1***

10. What is the class NP named after?

***Answer***

Nondeterministic Polynomial time

***Status :*** Correct ***Marks : 1/1***

11. Which complexity class includes problems that are at least as hard as the hardest problems in NP?

***Answer***

NP-hard

***Status :*** Correct ***Marks : 1/1***

12. What is the defining property of NP-hard problems?

***Answer***

They are at least as hard as the hardest problems in NP.

***Status :*** Correct ***Marks : 1/1***

vtu28682

13. In computational complexity theory, what does "NP" stand for?

***Answer***

Non-Deterministic Polynomial Time

***Status :*** Correct ***Marks : 1/1***

14. What is the main characteristic that distinguishes NP-complete problems from NP-hard problems?

***Answer***

NP-complete problems are the hardest problems in NP.

***Status :*** Correct ***Marks : 1/1***

15. The concept of "polynomial-time reduction" is commonly used to establish which of the following relationships between problems?

***Answer***

NP-hardness

***Status :*** Correct ***Marks : 1/1***

vtu28682

# Vel Tech Group of Institutions

Name: NIKHIL  KUMAR
Email: vtu28682@veltech.edu.in
Roll no: vtu28682
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

## 2028_NeoColab_Design and Analysis of AlgorithmsS1L10

### VTU_DAA_Week 12_COD

Attempt : 1
Total Mark : 30
Marks Obtained : 30

### Section 1 : Coding

1. Problem Statement

Raj is developing financial software that must identify all non-empty subsets of a given list of expenses whose total sum equals a specified target value.

Each expense can either be included or excluded from a subset. The same expense cannot be used more than once in a subset.

Your task is to print all such valid subsets. If no valid non-empty subset exists whose sum equals the target value, print:

No subset matches the target sum

This problem demonstrates how the number of possible subsets increases exponentially when using brute-force approaches.

vtu28682

Formula

Subset Sum = sum of selected elements in the subset

A subset is considered valid if:

sum(subset elements) = target

***Input Format***

The first line contains an integer n, representing the number of expenses.

The second line contains n space-separated integers representing the expense values.

The third line contains an integer t, representing the target sum.

***Output Format***

Print each valid non-empty subset whose elements sum to t, one subset per line.

Elements of a subset must be printed as space-separated integers.

If no such subset exists, print:

No subset matches the target sum

Refer to the sample output for formatting.

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 3
1 2 3
3
Output: 1 2
3

vtu28682

***Answer***

```
// You are using GCC
#include <stdio.h>

#define MAX 25

int arr[MAX];
int subset[MAX];
int n, target;
int found = 0;

void printSubset(int size) {
    for (int i = 0; i < size; i++) {
        printf("%d ", subset[i]);
    }
    printf("\n");
}

void solve(int index, int size, int sum) {
    if (index == n) {
        if (sum == target && size > 0) {
            printSubset(size);
            found = 1;
        }
        return;
    }

    // include current element
    subset[size] = arr[index];
    solve(index + 1, size + 1, sum + arr[index]);

    // exclude current element
    solve(index + 1, size, sum);
}

int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
```

```
    scanf("%d", &target);

    solve(0, 0, 0);

    if (!found) {
        printf("No subset matches the target sum");
    }

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

2. Problem Statement

Sanjana is a cybersecurity analyst responsible for securing communication in a corporate network. The network consists of multiple computers (nodes) connected by communication links (edges).

To ensure secure communication, Sanjana wants to select a group of computers such that every pair of selected computers is directly connected to each other. This ensures a fully trusted group where all members can communicate securely without intermediaries. Such a group is called a clique.

Sanjana is given a list of connections in the network, and she needs to determine whether there exists a clique of size k.

***Input Format***

The first line contains two integers n and m, representing the number of computers (nodes) and connections (edges).

The next m lines each contain two integers u and v, representing a connection between nodes u and v.

The last line contains an integer k, representing the required clique size.

***Output Format***

If a clique of size k exists, the output prints: "Clique Found: v1 v2 v3 ..."

Otherwise, the output prints: "No clique of given size exists"

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 5 6
0 1
0 2
1 2
1 3
2 3
3 4
3
Output: Clique Found: 0 1 2

***Answer***

```
#include <stdio.h>
#include <string.h>

#define MAX 25

int graph[MAX][MAX];
int n, m, k;
int combination[MAX];
int found = 0;

// Check if selected vertices form a clique
int isClique(int size) {
    for (int i = 0; i < size; i++) {
        for (int j = i + 1; j < size; j++) {
            if (!graph[combination[i]][combination[j]]) {
                return 0;
            }
        }
    }
    return 1;
}
```

```
// Backtracking to choose k vertices
void solve(int start, int idx) {
    if (found) return;

    if (idx == k) {
        if (isClique(k)) {
            printf("Clique Found: ");
            for (int i = 0; i < k; i++) {
                printf("%d ", combination[i]);
            }
            found = 1;
        }
        return;
    }

    for (int i = start; i < n; i++) {
        combination[idx] = i;
        solve(i + 1, idx + 1);
        if (found) return;
    }
}

int main() {
    scanf("%d %d", &n, &m);

    memset(graph, 0, sizeof(graph));

    for (int i = 0; i < m; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        graph[u][v] = graph[v][u] = 1;
    }

    scanf("%d", &k);

    solve(0, 0);

    if (!found) {
        printf("No clique of given size exists");
    }

    return 0;
```

}

***Status :*** Correct ***Marks : 10/10***

3. Problem Statement

Rehan, a logistics manager, is packing valuable items into two separate containers. Each item has a specific value, and he wants both containers to carry the same total value to ensure balance and fairness. He needs to decide if it's possible to divide the items this way and print one such valid division if it exists.

This problem is based on a classic NP-complete problem known as the Partition Problem, where finding a solution may require checking all combinations in the worst case.

Help him write a program to solve this problem.

***Input Format***

The first line of input consists of an integer n, representing the number of items.

The second line contains n space-separated integers, where each integer represents the value of an item.

***Output Format***

If a valid equal partition exists, print:

- Subset 1: x1 x2 x3 ...
- Subset 2: y1 y2 y3 ...

If no such partition exists, it prints "No valid equal partition exists."

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 4

1 5 11 5
Output: Subset 1: 1 5 5
Subset 2: 11

***Answer***

```
// You are using GCC
#include <stdio.h>

#define MAX 20

int arr[MAX], n;
int selected[MAX];
int found = 0;
int target;

// print subsets
void printResult() {
    printf("Subset 1: ");
    for (int i = 0; i < n; i++) {
        if (selected[i])
            printf("%d ", arr[i]);
    }
    printf("\nSubset 2: ");
    for (int i = 0; i < n; i++) {
        if (!selected[i])
            printf("%d ", arr[i]);
    }
}

// recursive subset generation
void solve(int idx, int sum) {
    if (found) return;

    if (sum == target) {
        found = 1;
        printResult();
        return;
    }

    if (idx == n || sum > target) return;

    // include element
```

```
    selected[idx] = 1;
    solve(idx + 1, sum + arr[idx]);

    // exclude element
    selected[idx] = 0;
    solve(idx + 1, sum);
}

int main() {
    scanf("%d", &n);

    int total = 0;
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
        total += arr[i];
    }

    if (total % 2 != 0) {
        printf("No valid equal partition exists.");
        return 0;
    }

    target = total / 2;

    solve(0, 0);

    if (!found) {
        printf("No valid equal partition exists.");
    }

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

# Vel Tech Group of Institutions

Name: NIKHIL  KUMAR
Email: vtu28682@veltech.edu.in
Roll no: vtu28682
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

## 2028_NeoColab_Design and Analysis of AlgorithmsS1L10

### VTU_DAA_Week 12_PAH

Attempt : 1
Total Mark : 50
Marks Obtained : 45

### Section 1 : Coding

1. Problem Statement

Meera is preparing for a science exhibition and needs to select a group of items such that their total weight is exactly equal to a given target weight. Each item has a known weight, and every item can be selected at most once.

Given a list of item weights and a target weight k, determine whether there exists a subset of items whose total weight is exactly k.

If such a subset exists, print any one valid subset.

If no such subset exists, print -1.

The solution must be implemented using Dynamic Programming.

vtu28682

Formula

Let

dp[i][j] = true

if a subset of the first i items can form a sum j.

Transition:

If arr[i-1] > j

dp[i][j] = dp[i-1][j]

Else

dp[i][j] = dp[i-1][j] OR dp[i-1][j - arr[i-1]]

***Input Format***

The first line contains two integers n and k, where n is the number of items and k is the target weight.

The second line contains n space-separated integers representing item weights.

***Output Format***

If a valid subset exists, print the elements of one such subset in a single line separated by spaces.

If no subset exists, print -1.

Refer to the sample output for formatting specifications.

***Sample Test Case***

vtu28682

Input: 5 9
3 34 4 12 5
Output: 5 4

***Answer***

```
// You are using GCC
#include <stdio.h>

#define MAX 105

int dp[MAX][1005];
int arr[MAX];
int n, k;

// store chosen elements
int subset[MAX];
int idx = 0;

int main() {
    scanf("%d %d", &n, &k);

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    // DP initialization
    for (int i = 0; i <= n; i++) {
        dp[i][0] = 1;
    }

    // build DP table
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= k; j++) {
            dp[i][j] = dp[i - 1][j];

            if (arr[i - 1] <= j) {
                dp[i][j] = dp[i][j] || dp[i - 1][j - arr[i - 1]];
            }
        }
    }

    // if not possible
```

```
    if (!dp[n][k]) {
        printf("-1");
        return 0;
    }

    // reconstruct subset
    int i = n, j = k;

    while (i > 0 && j > 0) {
        if (!dp[i - 1][j]) {
            subset[idx++] = arr[i - 1];
            j -= arr[i - 1];
        }
        i--;
    }

    // print subset
    for (int t = 0; t < idx; t++) {
        printf("%d ", subset[t]);
    }

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

2. Problem Statement

Ramya is working as a logistics coordinator for a delivery company. Every day, she receives a list of delivery locations arranged along a straight highway. Each location has a certain delivery priority value (which may be positive or negative depending on profit or loss).

Ramya wants to choose a continuous sequence of delivery locations such that the total delivery priority is maximized, ensuring maximum profit for that day.

Since the company handles a large number of deliveries, Ramya needs an efficient solution that runs in polynomial time.

***Input Format***

The first line contains an integer n, representing the number of delivery locations.

The second line contains n space-separated integers, representing the priority values.

***Output Format***

The output prints a single integer representing the maximum total priority of any contiguous sequence of locations.

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 8
-2 3 5 -1 4 -5 2 6

Output: 14

***Answer***

```
// You are using GCC
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int arr[n];

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    int current_sum = arr[0];
    int max_sum = arr[0];

    for (int i = 1; i < n; i++) {
        if (current_sum + arr[i] > arr[i]) {
            current_sum = current_sum + arr[i];
        } else {
```

```
        current_sum = arr[i];
    }

    if (current_sum > max_sum) {
        max_sum = current_sum;
    }
  }

  printf("%d", max_sum);

  return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

3. Problem Statement

Ragav is managing a chain of warehouses located along a long highway. Each warehouse is positioned at a specific coordinate on the road.

To improve efficiency, Ragav wants to choose a central hub location such that the total travel distance from all warehouses to this hub is minimized.

He can place the hub at any one of the existing warehouse locations.

Since the number of warehouses can be very large, Ragav needs an efficient algorithm that runs in polynomial time to compute the optimal location.

***Input Format***

The first line contains an integer n, representing the number of warehouses.

The second line contains n space-separated integers, representing the positions of warehouses on the highway.

***Output Format***

The output prints a single integer representing the minimum total distance from all warehouses to the chosen hub.

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 5
1 2 3 6 10

Output: 13

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

int cmp(const void *a, const void *b) {
    long long x = *(long long*)a;
    long long y = *(long long*)b;
    if (x < y) return -1;
    if (x > y) return 1;
    return 0;
}

int main() {
    int n;
    scanf("%d", &n);

    long long arr[n];

    for (int i = 0; i < n; i++) {
        scanf("%lld", &arr[i]);
    }

    qsort(arr, n, sizeof(long long), cmp);

    long long median = arr[n / 2];

    long long total = 0;

    for (int i = 0; i < n; i++) {
        total += llabs(arr[i] - median);
    }
```

```
    printf("%lld", total);

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

4. Problem Statement

Peter is a project manager in a software company. He has a list of employees, and each employee has a certain skill contribution value (which can be positive or negative depending on their impact on the project).

Peter wants to form a team such that the total skill contribution is exactly equal to a given target value. Each employee can be selected at most once, and the team must contain at least one employee.

Since the number of possible teams grows rapidly with the number of employees, Peter needs to determine whether such a team exists.

***Input Format***

The first line contains an integer n, representing the number of employees.

The second line contains n space-separated integers, representing the skill contribution of each employee.

The third line contains an integer target, representing the required total skill.

***Output Format***

If a valid team exists, the output prints: "Team Found: x1 x2 x3 ..."

If no such team exists, the output prints: "No valid team found"

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 5
3 7 -2 5 1
6

Output: Team Found: 3 -2 5

***Answer***

```
// You are using GCC
#include <stdio.h>

#define MAX 25

int arr[MAX];
int n, target;
int subset[MAX];
int found = 0;

void printSubset(int size) {
    printf("Team Found: ");
    for (int i = 0; i < size; i++) {
        printf("%d ", subset[i]);
    }
}

void solve(int idx, int sum, int size) {
    if (found) return;

    if (idx == n) {
        if (sum == target && size > 0) {
            printSubset(size);
            found = 1;
        }
        return;
    }

    // include current element
    subset[size] = arr[idx];
    solve(idx + 1, sum + arr[idx], size + 1);

    // exclude current element
    solve(idx + 1, sum, size);
```

```
}
int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    scanf("%d", &target);

    solve(0, 0, 0);

    if (!found) {
        printf("No valid team found");
    }

    return 0;
}
```

***Status :*** Partially correct ***Marks : 5/10***

5. Problem Statement

Viyana is an event coordinator organizing multiple sessions for a tech conference. Each session has a start time and an end time.

Due to limited resources, Viyana wants to select a group of sessions such that:

No two selected sessions overlap in time.The total number of selected sessions is maximized.

However, to make the event more impactful, each session also has a priority score, and Viyana wants to maximize the total priority score instead of just the count.

Your task is to help Viyana select the optimal set of non-overlapping sessions that yields the maximum total priority score.

***Input Format***

The first line contains an integer n, representing the number of sessions.

The next n lines each contain three integers:start time, end time and priority score.

***Output Format***

The first line of output prints the maximum total priority score.

In the next line, print the selected sessions (indices starting from 1) in ascending order.

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 4
1 3 5
2 5 6
4 6 5
6 7 4

Output: 14
1 3 4

***Answer***

```
// You are using GCC
#include <stdio.h>

#define MAX 25

int start[MAX], endt[MAX], val[MAX];
int n;

int best = 0;
int bestMask = 0;

int isValid(int mask) {
    int lastEnd = -1;
```

```
    for (int i = 0; i < n; i++) {
        if (mask & (1 << i)) {
            if (start[i] < lastEnd) {
                return 0;
            }
            lastEnd = endt[i];
        }
    }
    return 1;
}

int calcValue(int mask) {
    int sum = 0;
    for (int i = 0; i < n; i++) {
        if (mask & (1 << i)) {
            sum += val[i];
        }
    }
    return sum;
}

int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf("%d %d %d", &start[i], &endt[i], &val[i]);
    }

    for (int mask = 0; mask < (1 << n); mask++) {

        // check validity (non-overlapping)
        int valid = 1;

        for (int i = 0; i < n && valid; i++) {
            if (!(mask & (1 << i))) continue;

            for (int j = i + 1; j < n; j++) {
                if (!(mask & (1 << j))) continue;

                if (!(endt[i] <= start[j] || endt[j] <= start[i])) {
                    valid = 0;
```

```
                break;
            }
        }
    }

    if (valid) {
        int sum = calcValue(mask);

        if (sum > best) {
            best = sum;
            bestMask = mask;
        }
    }
}

printf("%d\n", best);

for (int i = 0; i < n; i++) {
    if (bestMask & (1 << i)) {
        printf("%d ", i + 1);
    }
}

return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

# Vel Tech Group of Institutions

Name: NIKHIL  KUMAR
Email: vtu28682@veltech.edu.in
Roll no: vtu28682
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

## 2028_NeoColab_Design and Analysis of AlgorithmsS1L10

## VTU_DAA_Week 12_CY

Attempt : 1
Total Mark : 50
Marks Obtained : 50

## Section 1 : Coding

1. Problem Statement

You are given a graph, and your task is to find the minimum vertex cover for the given graph. A vertex cover is a subset of vertices in the graph such that every edge in the graph is incident to at least one vertex in the cover.

***Input Format***

The first line contains an integer 'vertices' representing the number of vertices in the graph.

The second line contains an integer 'edges' representing the number of edges in the graph.

The following 'edges' lines each contain two integers 'u' and 'v' representing an edge between vertices 'u' and 'v'.

***Output Format***

The output prints the minimum vertex cover for the given graph, where each vertex in the cover is separated by a space.

Refer to the sample output for the formatting specifications.

***Sample Test Case***

Input: 7
6
0 1
0 2
1 3
3 4
4 5
5 6

Output: 0 1 3 4 5 6

***Answer***

```
// You are using GCC
#include <stdio.h>

int edges[505][2];
int removed[505];
int covered[105];

int main() {
    int vertices, e;
    scanf("%d", &vertices);
    scanf("%d", &e);

    for (int i = 0; i < e; i++) {
        scanf("%d %d", &edges[i][0], &edges[i][1]);
        removed[i] = 0;
    }

    int remaining = e;

    while (1) {
```

```
        int found = -1;

        // find first uncovered edge
        for (int i = 0; i < e; i++) {
            if (!removed[i]) {
                found = i;
                break;
            }
        }

        if (found == -1) break;

        int u = edges[found][0];
        int v = edges[found][1];

        covered[u] = 1;
        covered[v] = 1;

        // remove all edges touching u or v
        for (int i = 0; i < e; i++) {
            if (edges[i][0] == u || edges[i][1] == u ||
                edges[i][0] == v || edges[i][1] == v) {
                removed[i] = 1;
            }
        }
    }

    for (int i = 0; i < vertices; i++) {
        if (covered[i]) {
            printf("%d ", i);
        }
    }

    return 0;
}
```

***Status :*** Correct **Marks : 10/10**

2. Problem Statement

A financial analyst is studying the daily profit and loss values of a company

over a period of time. Each day's profit or loss is recorded as an integer in an array, where positive values indicate profit and negative values indicate loss.

The analyst wants to identify a continuous period of days during which the company achieved the maximum total profit. This period must consist of consecutive days only, and at least one day must be selected.

Given an array of integers representing daily profit and loss values, determine the maximum possible sum of any contiguous subarray.

To solve this efficiently, you must use a linear-time algorithm.

***Input Format***

The first line contains an integer n, representing the number of days.

The second line contains n space-separated integers representing daily profit or loss values.

***Output Format***

Print a single integer representing the maximum sum of a contiguous subarray.

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 9
-2 1 -3 4 -1 2 1 -5 4

Output: 6

***Answer***

```
// You are using GCC
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);
```

```
    int arr[n];

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    int current_sum = arr[0];
    int max_sum = arr[0];

    for (int i = 1; i < n; i++) {
        if (current_sum + arr[i] > arr[i]) {
            current_sum = current_sum + arr[i];
        } else {
            current_sum = arr[i];
        }

        if (current_sum > max_sum) {
            max_sum = current_sum;
        }
    }

    printf("%d", max_sum);

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

3. Problem Statement

Given an array of integers, determine whether there exists a majority element in the array.

A majority element is defined as an element that appears more than n/2 times in the array.

Your task is to implement a function majorityElement() that takes the input array and returns the majority element if it exists. If no such element exists, return -1.

Formula

An element is considered a majority element if:

frequency(element) > n/2

where n is the total number of elements in the array.

***Input Format***

The first line of input contains an integer n, representing the number of elements in the array.

The second line contains n space-separated integers a1 a2 a3 ... an representing the array elements.

***Output Format***

Print the majority element if it exists. Otherwise, print -1.

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 9
3 3 4 2 4 4 2 4 4
Output: 4

***Answer***

```
// You are using GCC
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int arr[n];

    for (int i = 0; i < n; i++) {
```

vtu28682

```
        scanf("%d", &arr[i]);
    }

    // Step 1: Find candidate
    int candidate = arr[0];
    int count = 1;

    for (int i = 1; i < n; i++) {
        if (arr[i] == candidate) {
            count++;
        } else {
            count--;
        }

        if (count == 0) {
            candidate = arr[i];
            count = 1;
        }
    }

    // Step 2: Verify candidate
    count = 0;
    for (int i = 0; i < n; i++) {
        if (arr[i] == candidate) {
            count++;
        }
    }

    if (count > n / 2) {
        printf("%d", candidate);
    } else {
        printf("-1");
    }

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

4. Problem Statement

Aarav is analyzing temperature fluctuations recorded over several days. The temperature changes are stored in an array of integers. He wants to identify a contiguous period of days such that the product of the temperature changes during that period is maximized.

Given an array of integers, determine the maximum product that can be obtained from any contiguous subarray containing at least one element.

Your task is to compute this value efficiently.

Formula

To correctly handle both positive and negative values, track both the maximum and minimum product ending at each index.

If the current element is negative, swap the previous maximum and minimum values before updating.

At each index i:

currentMax = max(arr[i], arr[i] × currentMax)

currentMin = min(arr[i], arr[i] × currentMin)

The final answer is the maximum value obtained among all currentMax values.

***Input Format***

The first line contains an integer n, representing the number of days.

The second line contains n space-separated integers representing the temperature change factors.

***Output Format***

The first line contains an integer n, representing the number of days.

The second line contains n space-separated integers representing the temperature change factors.

vtu28682

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 6
2 3 -2 4 -1 5

Output: 240

***Answer***

```
// You are using GCC
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int arr[n];

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    int max_ending_here = arr[0];
    int min_ending_here = arr[0];
    int result = arr[0];

    for (int i = 1; i < n; i++) {
        int x = arr[i];

        if (x < 0) {
            int temp = max_ending_here;
            max_ending_here = min_ending_here;
            min_ending_here = temp;
        }

        if (x > max_ending_here * x)
            max_ending_here = x;
        else
            max_ending_here = max_ending_here * x;

        if (x < min_ending_here * x)
            min_ending_here = x;
```

```
    else
        min_ending_here = min_ending_here * x;

    if (max_ending_here > result)
        result = max_ending_here;
  }

  printf("%d", result);

  return 0;
}
```

***Status :*** Correct ***Marks : 10/10***

5. Problem Statement

Sidhant is a logistics manager responsible for planning delivery routes for a fleet of trucks. He has a list of cities, and the distances between every pair of cities are given in the form of a matrix.

Each truck must start from a central warehouse (city 0), visit every other city exactly once, and return back to the warehouse. The goal is to minimize the total travel distance.

Sidhant needs to determine the minimum distance required to complete the route and also print the optimal path taken.

***Input Format***

The first line contains an integer n, representing the number of cities.

The next n lines contain n space-separated integers, representing the distance matrix.

***Output Format***

The first line of output prints the minimum travel distance.

In the next line, print the optimal path starting and ending at city 0.

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 4
0 10 15 20
10 0 35 25
15 35 0 30
20 25 30 0

Output: 80
0 1 3 2 0

***Answer***

```
// You are using GCC
#include <stdio.h>
#include <limits.h>

#define MAX 15

int n;
int cost[MAX][MAX];
int visited[MAX];

int min_cost = INT_MAX;
int best_path[MAX];
int curr_path[MAX];

void tsp(int level, int curr_cost, int last) {
    if (level == n) {
        if (cost[last][0] != 0) {
            int total = curr_cost + cost[last][0];
            if (total < min_cost) {
                min_cost = total;
                for (int i = 0; i < n; i++) {
                    best_path[i] = curr_path[i];
                }
            }
        }
        return;
    }
```

```
    for (int i = 1; i < n; i++) {
        if (!visited[i] && cost[last][i] != 0) {
            visited[i] = 1;
            curr_path[level] = i;

            tsp(level + 1, curr_cost + cost[last][i], i);

            visited[i] = 0;
        }
    }
}

int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &cost[i][j]);
        }
    }

    visited[0] = 1;
    curr_path[0] = 0;

    tsp(1, 0, 0);

    printf("%d\n", min_cost);

    printf("0 ");
    for (int i = 1; i < n; i++) {
        printf("%d ", best_path[i]);
    }
    printf("0");

    return 0;
}
```

***Status :*** Correct ***Marks : 10/10***